# MDN JavaScript Documentation

## Volume 6 of 9 — Built-in objects: JSON – Proxy

**Unofficial print compilation.** This volume contains 162 MDN articles, reproduced without changes to their wording.

---

### Source, attribution and licence

All articles in this volume are from **MDN Web Docs**, `https://developer.mozilla.org/en-US/docs/Web/JavaScript`, retrieved on **2026-11-10** from MDN's rendered pages. They are written **by Mozilla Contributors** and licensed under the Creative Commons Attribution-ShareAlike licence, **CC-BY-SA 2.5** or any later version (`https://creativecommons.org/licenses/by-sa/2.5/`). The title and the URL of the specific MDN page are printed at the head of every article; the list of authors of a page is available through that page on MDN. Code samples added to MDN on or after 20 August 2010 are in the public domain (**CC0**, `https://creativecommons.org/publicdomain/zero/1.0/`); older samples are under the MIT licence. Browser compatibility data is from `@mdn/browser-compat-data` 8.1.8 (CC0).

**Modifications.** This is not an official MDN or Mozilla publication and is not endorsed by Mozilla. The content was re-laid-out for A4 paper: website navigation, banners and interactive widgets were removed or replaced by short print notes; compatibility and specification tables, which the website generates from data, were re-rendered from the same data; in-volume links were turned into PDF links. Wording, code and section order were not edited. Like the original, this compilation is distributed under CC-BY-SA 2.5 or later. “MDN” and “Mozilla” are trademarks of the Mozilla Foundation; its logos and website design are not part of the licensed content and are not used here.

### How to read this book

- **Text.** Article wording, headings, section order, notes, tables and code are MDN's, taken programmatically from MDN's rendered pages. Only the layout was changed for paper.
- **Dashed boxes in italics starting with ▷** and the small grey lines under each article title (breadcrumb, source URL, availability) are additions of this print edition, not MDN text.
- **Code** is printed character for character in a monospaced font. A line longer than about 92 characters wraps at the right edge and continues at the left edge of the code box; no characters are added. Labels above a box give the language; **✗ bad example** and **✓ good example** replace MDN's red/green colouring. “code continues” / “code continued” in the page margin mark a code box that runs over a page break.
- **Interactive examples and live samples** cannot run on paper. Their source code is printed; output is shown only where MDN's source states it (for example in `// Expected output` comments).
- **Links.** Dotted underlines mark links. In the PDF, links to articles in this volume jump to them; all other links open the MDN page. Web API, DOM, HTML/CSS, Glossary and Learn pages are outside the scope of this collection (only `/en-US/docs/Web/JavaScript/` is included) and remain links to MDN.
- **Badges** such as Deprecated Experimental Non-standard Optional reproduce MDN's status icons and labels as text.
- **Browser compatibility tables** are rebuilt from MDN's browser-compat-data (the data the website's tables are generated from). Each cell gives the first browser version that supports the feature. **No** (grey cell) = not supported; **?** = unknown; **a–b** = supported from version a, removed in version b (grey cell); **Prev.** = only in a preview release of that browser; ≤ = that version or earlier; small raised numbers refer to the numbered notes below the table (flags, prefixes, partial support and other remarks).
- **“Baseline” availability lines** are condensed from the Baseline status MDN shows at the top of a page.

# Contents


**JSON** 5
- JSON.isRawJSON() 10
- JSON.parse() 13
- JSON.rawJSON() 17
- JSON.stringify() 20

**Map** 30
- Map() constructor 39
- Map.groupBy() 40
- Map[Symbol.species] 44
- Map.prototype.clear() 45
- Map.prototype.delete() 47
- Map.prototype.entries() 48
- Map.prototype.forEach() 50
- Map.prototype.get() 52
- Map.prototype.getOrInsert() 54
- Map.prototype.getOrInsertComputed() 56
- Map.prototype.has() 58
- Map.prototype.keys() 59
- Map.prototype.set() 61
- Map.prototype.values() 63
- Map.prototype[Symbol.iterator]() 64
- Map.prototype.size 67

**Math** 69
- Math.abs() 74
- Math.acos() 76
- Math.acosh() 78
- Math.asin() 79
- Math.asinh() 81
- Math.atan() 83
- Math.atan2() 85
- Math.atanh() 89
- Math.cbrt() 91
- Math.ceil() 92
- Math.clz32() 94
- Math.cos() 98
- Math.cosh() 99
- Math.exp() 101
- Math.expm1() 103
- Math.f16round() 105
- Math.floor() 107
- Math.fround() 110
- Math.hypot() 112
- Math.imul() 114
- Math.log() 116
- Math.log10() 118
- Math.log1p() 119
- Math.log2() 122
- Math.max() 123
- Math.min() 126
- Math.pow() 128
- Math.random() 130
- Math.round() 133
- Math.sign() 135
- Math.sin() 137
- Math.sinh() 138
- Math.sqrt() 140
- Math.sumPrecise() 142
- Math.tan() 144
- Math.tanh() 146
- Math.trunc() 148
- Math.E 150
- Math.LN10 151
- Math.LN2 153
- Math.LOG10E 154
- Math.LOG2E 156
- Math.PI 157
- Math.SQRT1_2 159
- Math.SQRT2 160

**NaN** 163

**Number** 167
- Number() constructor 173
- Number.isFinite() 175
- Number.isInteger() 177
- Number.isNaN() 179
- Number.isSafeInteger() 181
- Number.parseFloat() 183
- Number.parseInt() 185
- Number.EPSILON 186
- Number.MAX_SAFE_INTEGER 189
- Number.MAX_VALUE 191
- Number.MIN_SAFE_INTEGER 193
- Number.MIN_VALUE 195
- Number.NaN 196
- Number.NEGATIVE_INFINITY 198
- Number.POSITIVE_INFINITY 200
- Number.prototype.toExponential() 202
- Number.prototype.toFixed() 204
- Number.prototype.toLocaleString() 207
- Number.prototype.toPrecision() 210
- Number.prototype.toString() 213
- Number.prototype.valueOf() 216

**Object** 218
- Object() constructor 227
- Object.assign() 229
- Object.create() 234
- Object.defineProperties() 238
- Object.defineProperty() 240
- Object.entries() 250
- Object.freeze() 253
- Object.fromEntries() 258
- Object.getOwnPropertyDescriptor() 261
- Object.getOwnPropertyDescriptors() 264
- Object.getOwnPropertyNames() 266
- Object.getOwnPropertySymbols() 269
- Object.getPrototypeOf() 271
- Object.groupBy() 273
- Object.hasOwn() 276
- Object.is() 280
- Object.isExtensible() 282
- Object.isFrozen() 284
- Object.isSealed() 288
- Object.keys() 290
- Object.preventExtensions() 293
- Object.seal() 295
- Object.setPrototypeOf() 298
- Object.values() 302

Object.prototype.__defineGetter__() 304
Object.prototype.__defineSetter__() 307
Object.prototype.__lookupGetter__() 309
Object.prototype.__lookupSetter__() 312
Object.prototype.hasOwnProperty() 314
Object.prototype.isPrototypeOf() 318
Object.prototype.propertyIsEnumerable() 321
Object.prototype.toLocaleString() 325
Object.prototype.toString() 328
Object.prototype.valueOf() 331
Object.prototype.__proto__ 334
Object.prototype.constructor 337

**parseFloat()** **344**

**parseInt()** **348**

**Promise** **353**
Promise() constructor 366
Promise.all() 372
Promise.allKeyed() 379
Promise.allSettled() 382
Promise.allSettledKeyed() 384
Promise.any() 387
Promise.race() 390
Promise.reject() 396
Promise.resolve() 398
Promise.try() 403
Promise.withResolvers() 407
Promise[Symbol.species] 410
Promise.prototype.catch() 412
Promise.prototype.finally() 416
Promise.prototype.then() 419

**Proxy** **427**
Proxy() constructor 436
handler.apply() 438
handler.construct() 440
handler.defineProperty() 443
handler.deleteProperty() 447
handler.get() 449
handler.getOwnPropertyDescriptor() 453
handler.getPrototypeOf() 456
handler.has() 459
handler.isExtensible() 462
handler.ownKeys() 464
handler.preventExtensions() 467
handler.set() 470
handler.setPrototypeOf() 473
Proxy.revocable() 476

JavaScript › Reference › Standard built-in objects

# JSON

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON · MDN page last modified 2025-11-20

**Baseline: Widely available** — available across major browsers since July 2015.

The **`JSON`** namespace object contains static methods for parsing values from and converting values to JavaScript Object Notation (JSON).

## Description

Unlike most global objects, `JSON` is not a constructor. You cannot use it with the `new` operator or invoke the `JSON` object as a function. All properties and methods of `JSON` are static (just like the `Math` object).

### JavaScript and JSON differences

JSON is a syntax for serializing objects, arrays, numbers, strings, booleans, and `null`. It is based upon JavaScript syntax, but is distinct from JavaScript: most of JavaScript is *not* JSON. For example:

**Objects and Arrays**

Property names must be double-quoted strings; trailing commas are forbidden.

**Numbers**

Leading zeros are prohibited. A decimal point must be followed by at least one digit. `NaN` and `Infinity` are unsupported.

Any JSON text is a valid JavaScript expression, but only after the JSON superset revision. Before the revision, U+2028 LINE SEPARATOR and U+2029 PARAGRAPH SEPARATOR are allowed in string literals and property keys in JSON; but the same use in JavaScript string literals is a `SyntaxError`.

Other differences include allowing only double-quoted strings and no support for `undefined` or comments. For those who wish to use a more human-friendly configuration format based on JSON, there is JSON5, used by the Babel compiler, and the more commonly used YAML.

The same text may represent different values in JavaScript object literals vs. JSON as well. For more information, see Object literal syntax vs. JSON.

### Full JSON grammar

Valid JSON syntax is formally defined by the following grammar, expressed in ABNF, and copied from IETF JSON standard (RFC):

```
JSON-text = ws value ws
begin-array     = ws %x5B ws  ; [ left square bracket
begin-object    = ws %x7B ws  ; { left curly bracket
end-array       = ws %x5D ws  ; ] right square bracket
end-object      = ws %x7D ws  ; } right curly bracket
name-separator  = ws %x3A ws  ; : colon
value-separator = ws %x2C ws  ; , comma
ws = *(
     %x20 /              ; Space
     %x09 /              ; Horizontal tab
     %x0A /              ; Line feed or New line
     %x0D                ; Carriage return
     )
value = false / null / true / object / array / number / string
false = %x66.61.6c.73.65   ; false
null  = %x6e.75.6c.6c      ; null
```

*(code continues on the next page)*

*(code continued from the previous page)*

```
true  = %x74.72.75.65      ; true
object = begin-object [ member *( value-separator member ) ]
         end-object
member = string name-separator value
array = begin-array [ value *( value-separator value ) ] end-array
number = [ minus ] int [ frac ] [ exp ]
decimal-point = %x2E       ; .
digit1-9 = %x31-39         ; 1-9
e = %x65 / %x45            ; e E
exp = e [ minus / plus ] 1*DIGIT
frac = decimal-point 1*DIGIT
int = zero / ( digit1-9 *DIGIT )
minus = %x2D               ; -
plus = %x2B                ; +
zero = %x30                ; 0
string = quotation-mark *char quotation-mark
char = unescaped /
    escape (
        %x22 /          ; "    quotation mark  U+0022
        %x5C /          ; \    reverse solidus U+005C
        %x2F /          ; /    solidus         U+002F
        %x62 /          ; b    backspace       U+0008
        %x66 /          ; f    form feed       U+000C
        %x6E /          ; n    line feed       U+000A
        %x72 /          ; r    carriage return U+000D
        %x74 /          ; t    tab             U+0009
        %x75 4HEXDIG )  ; uXXXX                U+XXXX
escape = %x5C              ; \
quotation-mark = %x22      ; "
unescaped = %x20-21 / %x23-5B / %x5D-10FFFF
HEXDIG = DIGIT / %x41-46 / %x61-66   ; 0-9, A-F, or a-f
       ; HEXDIG equivalent to HEXDIG rule in [RFC5234]
DIGIT = %x30-39            ; 0-9
      ; DIGIT equivalent to DIGIT rule in [RFC5234]
```

Insignificant whitespace may be present anywhere except within a `JSONNumber` (numbers must contain no whitespace) or `JSONString` (where it is interpreted as the corresponding character in the string, or would cause an error). The tab (U+0009), carriage return (U+000D), line feed (U+000A), and space (U+0020) characters are the only valid whitespace characters.

## Static properties

**`JSON[Symbol.toStringTag]`**

The initial value of the `[Symbol.toStringTag]` property is the string `"JSON"`. This property is used in `Object.prototype.toString()`.

## Static methods

**`JSON.isRawJSON()`**

Tests whether a value is an object returned by `JSON.rawJSON()`.

**`JSON.parse()`**

Parse a piece of string text as JSON, optionally transforming the produced value and its properties, and return the value.

**`JSON.rawJSON()`**

Creates a "raw JSON" object containing a piece of JSON text. When serialized to JSON, the raw JSON object is treated as if it is already a piece of JSON. This text is required to be valid JSON.

**`JSON.stringify()`**

Return a JSON string corresponding to the specified value, optionally including only certain properties or replacing property values in a user-defined manner.

## Examples

### Example JSON

json

```
{
  "browsers": {
    "firefox": {
      "name": "Firefox",
      "pref_url": "about:config",
      "releases": {
        "1": {
          "release_date": "2004-11-09",
          "status": "retired",
          "engine": "Gecko",
          "engine_version": "1.7"
        }
      }
    }
  }
}
```

You can use the `JSON.parse()` method to convert the above JSON string into a JavaScript object:

js

```
const jsonText = `{
  "browsers": {
    "firefox": {
      "name": "Firefox",
      "pref_url": "about:config",
      "releases": {
        "1": {
          "release_date": "2004-11-09",
          "status": "retired",
          "engine": "Gecko",
          "engine_version": "1.7"
        }
      }
    }
  }
}`;

console.log(JSON.parse(jsonText));
```

### Lossless number serialization

JSON can contain number literals of arbitrary precision. However, it is not possible to represent all JSON numbers exactly in JavaScript, because JavaScript uses floating point representation which has a fixed precision. For example, `12345678901234567890 === 12345678901234567000` in JavaScript because they

have the same floating point representation. This means there is no JavaScript number that corresponds precisely to the `12345678901234567890` JSON number.

Let's assume you have an exact representation of some number (either via `BigInt` or a custom library):

```js
const data = {
  // Using a BigInt here to store the exact value,
  // but it can also be a custom high-precision number library,
  // if the number might not be an integer.
  gross_gdp: 12345678901234567890n,
};
```

You want to serialize it and then parse to the same exact number. There are several difficulties:

- On the serialization side, in order to obtain a number in JSON, you have to pass a number to `JSON.stringify`, either via the `replacer` function or via the `toJSON` method. But, in either case, you have already lost precision during number conversion. If you pass a string to `JSON.stringify`, it will be serialized as a string, not a number.
- On the parsing side, not all numbers can be represented exactly. For example, `JSON.parse("12345678901234567890")` returns `12345678901234568000` because the number is rounded to the nearest representable number. Even if you use a `reviver` function, the number will already be rounded before the `reviver` function is called.

There are, in general, two ways to ensure that numbers are losslessly converted to JSON and parsed back: one involves a JSON number, another involves a JSON string. JSON is a *communication format*, so if you use JSON, you are likely communicating with another system (HTTP request, storing in database, etc.). The best solution to choose depends on the recipient system.

### Using JSON strings

If the recipient system does not have same JSON-handling capabilities as JavaScript, and does not support high precision numbers, you may want to serialize the number as a string, and then handle it as a string on the recipient side. This is also the only option in older JavaScript.

To specify how custom data types (including `BigInt`) should be serialized to JSON, either add a `toJSON` method to your data type, or use the `replacer` function of `JSON.stringify()`.

```js
// Using toJSON() method
BigInt.prototype.toJSON = function () {
  return this.toString();
};
const str1 = JSON.stringify(data);

// Using JSON.stringify() with replacer
const str2 = JSON.stringify(data, (key, value) => {
  if (key === "gross_gdp") {
    return value.toString();
  }
  return value;
});
```

In either case, the JSON text will look like `{"gross_gdp":"12345678901234567890"}`, where the value is a string, not a number. Then, on the recipient side, you can parse the JSON and handle the string.

### Using JSON numbers

If the recipient of this message natively supports high precision numbers (such as Python integers), passing numbers as JSON numbers is obviously better, because they can directly parse to the high precision type instead of parsing a string from JSON, and then parsing a number from the string. In JavaScript, you can serialize arbitrary data types to JSON numbers without producing a number value first (resulting in loss of precision) by using `JSON.rawJSON()` to precisely specify what the JSON source text should be.

```js
// Using toJSON() method
BigInt.prototype.toJSON = function () {
  return JSON.rawJSON(this.toString());
};
const str1 = JSON.stringify(data);

// Using JSON.stringify() with replacer
const str2 = JSON.stringify(data, (key, value) => {
  if (key === "gross_gdp") {
    return JSON.rawJSON(value.toString());
  }
  return value;
});
```

The text passed to `JSON.rawJSON` is treated as if it is already a piece of JSON, so it won't be serialized again as a string. Therefore, the JSON text will look like `{"gross_gdp":12345678901234567890}`, where the value is a number. This JSON can then be parsed by the recipient without any extra processing, provided that the recipient system does not have the same precision limitations as JavaScript.

When parsing JSON containing high-precision numbers in JavaScript, take extra care because when `JSON.parse()` invokes the `reviver` function, the value you receive is already parsed (and has lost precision). You can use the `context.source` parameter of the `JSON.parse()` `reviver` function to re-parse the number yourself.

```js
const parsedData = JSON.parse(str, (key, value, context) => {
  if (key === "gross_gdp") {
    // Or use the constructor of your custom high-precision number library
    return BigInt(context.source);
  }
  return value;
});
// { gross_gdp: 12345678901234567890n }
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/structured-data.html#sec-json-object |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `JSON` | 3 | 12 | 3.5 | 10.5 | 4 | 18 | 4 | 11 | 4 | 1.0 | 4.4 | 4 | 1.0.0 | 1.0 | 0.10.0 |
| `isRawJSON` | 114 | 114 | 135 | 100 | 18.4 | 114 | 135 | 76 | 18.4 | 23.0 | 114 | 18.4 | 1.1.43 | 1.33 | 21.0.0 |
| JavaScript is a superset of JSON | 66 | 79 | 62 | 53 | 12 | 66 | 62 | 47 | 12 | 9.0 | 66 | 12 | 1.0.0 | 1.0 | 10.0.0 |
| `parse` | 3 | 12 | 3.5 | 10.5 | 4 | 18 | 4 | 11 | 4 | 1.0 | 4.4 | 4 | 1.0.0 | 1.0 | 0.10.0 |
| Reviver has `context` parameter | 114 | 114 | 135 | 100 | 18.4 | 114 | 135 | 76 | 18.4 | 23.0 | 114 | 18.4 | 1.1.43 | 1.33 | 21.0.0 |
| `rawJSON` | 114 | 114 | 135 | 100 | 18.4 | 114 | 135 | 76 | 18.4 | 23.0 | 114 | 18.4 | 1.1.43 | 1.33 | 21.0.0 |
| `stringify` | 3 | 12 | 3.5 | 10.5 | 4 | 18 | 4 | 11 | 4 | 1.0 | 4.4 | 4 | 1.0.0 | 1.0 | 0.10.0 |
| Strings are escaped to well-formed UTF-8 | 72 | 79 | 64 | 60 | 12.1 | 72 | 64 | 50 | 12.2 | 11.0 | 72 | 12.2 | 1.0.0 | 1.0 | 12.0.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.JSON`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- JSON Diff
- JSON Beautifier/editor
- JSON Parser
- JSON Validator

---

JavaScript › Reference › Standard built-in objects › JSON

# JSON.isRawJSON()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON/isRawJSON · MDN page last modified 2026-10-26

**Baseline 2025: Newly available** — works across the latest browser versions since March 2025; may not work in older ones.

The **`JSON.isRawJSON()`** static method tests whether a value is an object returned by `JSON.rawJSON()`.

## Syntax

```js
JSON.isRawJSON(value)
```

### Parameters

**`value`**

The value to test.

### Return value

`true` if `value` is created by `JSON.rawJSON()`; otherwise, `false`.

## Description

"Raw JSON" objects, when serialized to JSON, are treated as if they are already a piece of JSON. Furthermore, because of the way `JSON.rawJSON()` works, the raw JSON is guaranteed to be syntactically valid JSON. For more information on the shape and behavior of raw JSON objects, see `JSON.rawJSON()`. This method exists to allow other serialization libraries to implement similar behavior to `JSON.stringify()` for raw JSON objects.

## Examples

### Using JSON.isRawJSON()

The following example demonstrates how to use `JSON.isRawJSON()` to test whether an object was returned by `JSON.rawJSON()`. It implements a custom serializer that serializes data to a YAML-like format.

js

```
function mySerializer(value, indent = "") {
  if (typeof value !== "object" || value === null) {
    return JSON.stringify(value);
  }
  if (JSON.isRawJSON(value)) {
    return value.rawJSON;
  }
  const subIndent = `${indent}  `;
  if (Array.isArray(value)) {
    return `- ${value.map((v) => mySerializer(v, subIndent)).join(`\n${indent}- `)}`;
  }
  return Object.entries(value)
    .map(([key, value]) => {
      const subValue = mySerializer(value, subIndent);
      if (subValue.includes("\n")) {
        return `${key}:\n${subIndent}${subValue}`;
      }
      return `${key}: ${subValue}`;
    })
    .join(`\n${indent}`);
}

console.log(
  mySerializer({
    name: "Josh",
    userId: JSON.rawJSON("12345678901234567890"),
    friends: [
      { name: "Alice", userId: JSON.rawJSON("9876543210987654321") },
      { name: "Bob", userId: JSON.rawJSON("56789012345678901234") },
    ],
  }),
);

// name: "Josh"
// userId: 12345678901234567890
// friends:
//   - name: "Alice"
//     userId: 9876543210987654321
//   - name: "Bob"
//     userId: 56789012345678901234
```

If in the above example, the `userId` values were not created by `JSON.rawJSON()`, but passed as numbers directly, then we will get loss of precision upfront because of JS floating point precision limitations.

js

```
console.log(
  mySerializer({
    name: "Josh",
    userId: 12345678901234567890,
    friends: [
      { name: "Alice", userId: 9876543210987654321 },
      { name: "Bob", userId: 56789012345678901234 },
    ],
  }),
);

// name: "Josh"
// userId: 12345678901234567000
// friends:
//   - name: "Alice"
//     userId: 9876543210987655000
//   - name: "Bob"
//     userId: 56789012345678900000
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/structured-data.html#sec-json.israwjson |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `isRawJSON` | 114 | 114 | 135 | 100 | 18.4 | 114 | 135 | 76 | 18.4 | 23.0 | 114 | 18.4 | 1.1.43 | 1.33 | 21.0.0 |

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.JSON.isRawJSON`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `JSON.isRawJSON` in `core-js`
- `JSON`
- `JSON.stringify()`
- `JSON.rawJSON()`

JavaScript › Reference › Standard built-in objects › JSON

# JSON.parse()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON/parse · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since July 2015.

The **`JSON.parse()`** static method parses a JSON string, constructing the JavaScript value or object described by the string. An optional *reviver* function can be provided to perform a transformation on the resulting object before it is returned.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: JSON.parse()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const json = '{"result":true, "count":42}';
const obj = JSON.parse(json);

console.log(obj.count);
// Expected output: 42

console.log(obj.result);
// Expected output: true
```

## Syntax

js

```
JSON.parse(text)
JSON.parse(text, reviver)
```

### Parameters

**`text`**

The string to parse as JSON. See the `JSON` object for a description of JSON syntax.

**`reviver`** Optional

If a function, this prescribes how each value originally produced by parsing is transformed before being returned. Non-callable values are ignored. The function is called with the following arguments:

**`key`**

The key associated with the value.

**`value`**

The value produced by parsing.

**`context`**

A context object that holds state relevant to the current expression being revived. It is a new object for each invocation of the reviver function.

**`source`** Optional

The original JSON string representing this value. This property is only present when reviving primitive values, but not when `value` is an object or array.

### Return value

The `Object`, `Array`, string, number, boolean, or `null` value corresponding to the given JSON `text`.

### Exceptions

**`SyntaxError`**

Thrown if the string to parse is not valid JSON.

## Description

`JSON.parse()` parses a JSON string according to the JSON grammar, then evaluates the string as if it's a JavaScript expression. The only instance where a piece of JSON text represents a different value from the same JavaScript expression is when dealing with the `"__proto__"` key — see Object literal syntax vs. JSON.

### The reviver parameter

If a `reviver` is specified, the value computed by parsing is *transformed* before being returned. Specifically, the computed value and all its properties (in a depth-first fashion, beginning with the most nested properties and proceeding to the original value itself) are individually run through the `reviver`.

The `reviver` is called with the object containing the property being processed as `this` (unless you define the `reviver` as an arrow function, in which case there's no separate `this` binding) and three arguments: `key`, `value`, and `context`. The key is a string even for arrays. For primitive values, the `context` argument has a field called `source` containing the source text of this value. If the `reviver` function returns `undefined` (or returns no value — for example, if execution falls off the end of the function), the property is deleted from the object. Otherwise, the property is redefined to be the return value. If the `reviver` only transforms some values and not others, be certain to return all untransformed values as-is — otherwise, they will be deleted from the resulting object.

Similar to the `replacer` parameter of `JSON.stringify()`, for arrays and objects, `reviver` will be last called on the root value with an empty string as the `key` and the root object as the `value`. For other valid JSON values, `reviver` works similarly and is called once with an empty string as the `key` and the value itself as the `value`.

If you return another value from `reviver`, that value will completely replace the originally parsed value. This even applies to the root value. For example:

```js
const transformedObj = JSON.parse('[1,5,{"s":1}]', (key, value) =>
  typeof value === "object" ? undefined : value,
);

console.log(transformedObj); // undefined
```

There is no way to work around this generically. You cannot specially handle the case where `key` is an empty string, because JSON objects can also contain keys that are empty strings. You need to know very precisely what kind of transformation is needed for each key when implementing the reviver.

Note that `reviver` is run after the value is parsed. So, for example, numbers in JSON text will have already been converted to JavaScript numbers, and may lose precision in the process. One way to transfer large numbers without loss of precision is to serialize them as strings, and revive them to BigInts, or other appropriate arbitrary precision formats.

You can also use the `context.source` property to access the original JSON source text representing the value, as shown below:

```js
const bigJSON = '{"gross_gdp": 12345678901234567890}';
const bigObj = JSON.parse(bigJSON, (key, value, context) => {
  if (key === "gross_gdp") {
    // Ignore the value because it has already lost precision
    return BigInt(context.source);
  }
  return value;
});
```

## Examples

### Using JSON.parse()

```js
JSON.parse("{}"); // {}
JSON.parse("true"); // true
JSON.parse('"foo"'); // "foo"
JSON.parse('[1, 5, "false"]'); // [1, 5, "false"]
JSON.parse("null"); // null
```

### Using the reviver parameter

```js
JSON.parse(
  '{"p": 5}',
  (key, value) =>
    typeof value === "number"
      ? value * 2 // return value * 2 for numbers
      : value, // return everything else unchanged
);
// { p: 10 }

JSON.parse('{"1": 1, "2": 2, "3": {"4": 4, "5": {"6": 6}}}', (key, value) => {
  console.log(key);
  return value;
});
// 1
// 2
// 4
// 6
// 5
// 3
// ""
```

### Using reviver when paired with the replacer of JSON.stringify()

In order for a value to properly round-trip (that is, it gets deserialized to the same original object), the serialization process must preserve the type information. For example, you can use the `replacer` parameter of `JSON.stringify()` for this purpose:

js

```
// Maps are normally serialized as objects with no properties.
// We can use the replacer to specify the entries to be serialized.
const map = new Map([
  [1, "one"],
  [2, "two"],
  [3, "three"],
]);

const jsonText = JSON.stringify(map, (key, value) =>
  value instanceof Map ? Array.from(value.entries()) : value,
);

console.log(jsonText);
// [[1,"one"],[2,"two"],[3,"three"]]

const map2 = JSON.parse(jsonText, (key, value) =>
  Array.isArray(value) && value.every(Array.isArray) ? new Map(value) : value,
);

console.log(map2);
// Map { 1 => "one", 2 => "two", 3 => "three" }
```

Because JSON has no syntax space for annotating type metadata, in order to revive values that are not plain objects, you have to consider one of the following:

- Serialize the entire object to a string and prefix it with a type tag.
- "Guess" based on the structure of the data (for example, an array of two-member arrays)
- If the shape of the payload is fixed, based on the property name (for example, all properties called `registry` hold `Map` objects).

### Illegal JSON

When `JSON.parse` receives a string that does not conform to the JSON grammar, it throws a `SyntaxError`.

Arrays and objects cannot have trailing commas in JSON:

js · ✗ bad example

```
JSON.parse("[1, 2, 3, 4, ]");
// SyntaxError: Unexpected token ] in JSON at position 13

JSON.parse('{"foo": 1, }');
// SyntaxError: Unexpected token } in JSON at position 12
```

JSON strings must be delimited by double (not single) quotes:

js · ✗ bad example

```
JSON.parse("{'foo': 1}");
// SyntaxError: Unexpected token ' in JSON at position 1

JSON.parse("'string'");
// SyntaxError: Unexpected token ' in JSON at position 0
```

If you are writing JSON inside a JavaScript string literal, you should either use single quotes to delimit the JavaScript string literal, or escape the double quotes that delimit the JSON string:

js · ✓ good example

```
JSON.parse('{"foo": 1}'); // OK
JSON.parse("{\"foo\": 1}"); // OK
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/structured-data.html#sec-json.parse |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| parse | 3 | 12 | 3.5 | 10.5 | 4 | 18 | 4 | 11 | 4 | 1.0 | 4.4 | 4 | 1.0.0 | 1.0 | 0.10.0 |
| Reviver has context parameter | 114 | 114 | 135 | 100 | 18.4 | 114 | 135 | 76 | 18.4 | 23.0 | 114 | 18.4 | 1.1.43 | 1.33 | 21.0.0 |

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.JSON.parse`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of modern `JSON.parse` behavior (reviver's `context` parameter) in `core-js`
- `JSON.stringify()`

---

JavaScript › Reference › Standard built-in objects › JSON

# JSON.rawJSON()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON/rawJSON · MDN page last modified 2026-10-26

**Baseline 2025: Newly available** — works across the latest browser versions since March 2025; may not work in older ones.

The **`JSON.rawJSON()`** static method creates a "raw JSON" object containing a piece of JSON text. When serialized to JSON, the raw JSON object is treated as if it is already a piece of JSON. This text is required to be valid JSON.

## Syntax

js

```
JSON.rawJSON(string)
```

### Parameters

**`string`**

The JSON text. Must be valid JSON **representing a primitive value**.

### Return value

An object that can be used to create JSON text with the exact same content as the `string` provided, without quotes around the string itself. This object has `null` prototype and is frozen (so it never gets accidentally serialized as a regular object by any kind of primitive conversion), and the following property:

**`rawJSON`**

The original JSON `string` provided.

Furthermore, it has a private field that marks itself as a raw JSON object. This allows it to be identified by `JSON.stringify()` and `JSON.isRawJSON()`.

### Exceptions

**`SyntaxError`**

Thrown if the `string` is not valid JSON, or if it represents an object or array.

## Description

A raw JSON object can be seen as an immutable, atomic data structure like any kind of primitive. It is not a regular object and it contains no data other than the raw JSON text. It is used to "pre-serialize" data to formats that `JSON.stringify` itself cannot produce for various reasons. The most typical use case is the floating point number loss of precision problem. For example:

```js
JSON.stringify({ value: 12345678901234567890 });
// {"value":12345678901234567000}
```

The value is not exactly equivalent to the original number any more! This is because JavaScript uses floating point representation for all numbers, so it cannot represent all integers exactly. The number literal `12345678901234567890` itself is already rounded to the nearest representable number when it is parsed by JavaScript.

Without `JSON.rawJSON`, there is no way to tell `JSON.stringify` to produce the number literal `12345678901234567890`, because there is simply no corresponding JavaScript number value. With raw JSON, you can directly tell `JSON.stringify()` what a particular value should be stringified as:

```js
const rawJSON = JSON.rawJSON("12345678901234567890");
JSON.stringify({ value: rawJSON });
// {"value":12345678901234567890}
```

For a more complete example of this, see Lossless number serialization.

Note that although we passed a string to `JSON.rawJSON()`, it still becomes a number in the final JSON. This is because the string represents the verbatim JSON text. If you want to serialize a string, you should use `JSON.rawJSON()` with a quotes-enclosed string value:

```js
const rawJSON = JSON.rawJSON('"Hello world"');
JSON.stringify({ value: rawJSON });
// {"value":"Hello world"}
```

`JSON.rawJSON` allows you to insert arbitrary JSON text, but does not allow you to create invalid JSON. Anything that was not permitted by the JSON syntax is not permitted by `JSON.rawJSON()` either:

js · ✗ bad example

```
const rawJSON = JSON.rawJSON('"Hello\nworld"'); // Syntax error, because line breaks are not
allowed in JSON strings
```

Furthermore, you cannot use `JSON.rawJSON()` to create JSON objects or arrays.

## Examples

### Using JSON.rawJSON() to create JSON expressions of different types

js

```
const numJSON = JSON.rawJSON("123");
const strJSON = JSON.rawJSON('"Hello world"');
const boolJSON = JSON.rawJSON("true");
const nullJSON = JSON.rawJSON("null");

console.log(
  JSON.stringify({
    age: numJSON,
    message: strJSON,
    isActive: boolJSON,
    nothing: nullJSON,
  }),
);

// {"age":123,"message":"Hello world","isActive":true,"nothing":null}
```

However, you cannot use `JSON.rawJSON()` to create JSON objects or arrays:

js · ✗ bad example

```
const arrJSON = JSON.rawJSON("[1, 2, 3]");
const objJSON = JSON.rawJSON('{"a": 1, "b": 2}');
// SyntaxError
```

### Using JSON.rawJSON() to create escaped string literals

Apart from numbers, there is only one other type that does not have a one-to-one correspondence between JavaScript values and JSON text: strings. When strings are serialized to JSON, all code points, other than those that are not legal inside JSON string literals (such as line breaks), are printed literally:

js

```
console.log(JSON.stringify({ value: "\ud83d\ude04" })); // {"value":"😄"}
```

This may not be desirable, because the receiver of this string may handle Unicode differently. To improve interoperability, you can explicitly specify the string to be serialized with escape sequences:

js

```
const rawJSON = JSON.rawJSON('"\\ud83d\\ude04"');
const objStr = JSON.stringify({ value: rawJSON });
console.log(objStr); // {"value":"\ud83d\ude04"}
console.log(JSON.parse(objStr).value); // 😄
```

Note that the double backslashes in the `rawJSON` actually represents a single slash character.

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/structured-data.html#sec-json.rawjson` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Nodejs |
| `rawJSON` | 114 | 114 | 135 | 100 | 18.4 | 114 | 135 | 76 | 18.4 | 23.0 | 114 | 18.4 | 1.1.43 | 1.33 | 21.0.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.JSON.rawJSON`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `JSON.rawJSON` in `core-js`
- `JSON`
- `JSON.isRawJSON()`
- `JSON.stringify()`

---

JavaScript › Reference › Standard built-in objects › JSON

# JSON.stringify()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON/stringify · MDN page last modified 2026-10-26
**Baseline: Widely available** — available across major browsers since July 2015.

The **`JSON.stringify()`** static method converts a JavaScript value to a JSON string, optionally replacing values if a replacer function is specified or optionally including only the specified properties if a replacer array is specified.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: JSON.stringify()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(JSON.stringify({ x: 5, y: 6 }));
// Expected output: '{"x":5,"y":6}'

console.log(
  JSON.stringify([new Number(3), new String("false"), new Boolean(false)]),
);
// Expected output: '[3,"false",false]'

console.log(JSON.stringify({ x: [10, undefined, function () {}, Symbol("")] }));
// Expected output: '{"x":[10,null,null,null]}'

console.log(JSON.stringify(new Date(2006, 0, 2, 15, 4, 5)));
// Expected output: '"2006-01-02T15:04:05.000Z"'
```

## Syntax

js

```
JSON.stringify(value)
JSON.stringify(value, replacer)
JSON.stringify(value, replacer, space)
```

### Parameters

**`value`**

The value to convert to a JSON string.

**`replacer`** Optional

A function that alters the behavior of the stringification process, or an array of strings and numbers that specifies properties of `value` to be included in the output. If `replacer` is an array, all elements in this array that are not strings or numbers (either primitives or wrapper objects), including `Symbol` values, are completely ignored. If `replacer` is anything other than a function or an array (e.g., `null` or not provided), all string-keyed properties of the object are included in the resulting JSON string.

**`space`** Optional

A string or number that's used to insert white space (including indentation, line break characters, etc.) into the output JSON string for readability purposes.

If this is a number, it indicates the number of space characters to be used as indentation, clamped to 10 (that is, any number greater than `10` is treated as if it were `10`). Values less than 1 indicate that no space should be used.

If this is a string, the string (or the first 10 characters of the string, if it's longer than that) is inserted before every nested object or array.

If `space` is anything other than a string or number (can be either a primitive or a wrapper object) — for example, is `null` or not provided — no white space is used.

### Return value

A JSON string representing the given value, or undefined.

### Exceptions

**`TypeError`**

Thrown in one of the following cases:

- `value` contains a circular reference.
- A `BigInt` value is encountered.

## Description

`JSON.stringify()` converts a value to the JSON notation that the value represents. Values are stringified in the following manner:

- `Boolean`, `Number`, `String`, and `BigInt` (obtainable via `Object()`) objects are converted to the corresponding primitive values during stringification, in accordance with the traditional conversion semantics. `Symbol` objects (obtainable via `Object()`) are treated as plain objects.
- Attempting to serialize `BigInt` values will throw. However, if the BigInt has a `toJSON()` method (through monkey patching: `BigInt.prototype.toJSON = ...`), that method can provide the serialization result. This constraint ensures that a proper serialization (and, very likely, its accompanying deserialization) behavior is always explicitly provided by the user.
- `undefined`, `Function`, and `Symbol` values are not valid JSON values. If any such values are encountered during conversion, they are either omitted (when found in an object) or changed to `null` (when found in an array). `JSON.stringify()` can return `undefined` when passing in "pure" values like `JSON.stringify(() => {})` or `JSON.stringify(undefined)`.
- The numbers `Infinity` and `NaN`, as well as the value `null`, are all considered `null`. (But unlike the values in the previous point, they would never be omitted.)
- Arrays are serialized as arrays (enclosed by square brackets). Only array indices between 0 and `length - 1` (inclusive) are serialized; other properties are ignored.
- The special raw JSON object created with `JSON.rawJSON()` is serialized as the raw JSON text it contains (by accessing its `rawJSON` property).
- For other objects:
  - All `Symbol`-keyed properties will be completely ignored, even when using the `replacer` parameter.
  - If the value has a `toJSON()` method, it's responsible for defining what data will be serialized. Instead of the object being serialized, the value returned by the `toJSON()` method when called will be serialized. `JSON.stringify()` calls `toJSON` with one parameter, the `key`, which has the same semantic as the `key` parameter of the `replacer` function:
    - if this object is a property value, the property name
    - if it is in an array, the index in the array, as a string
    - if `JSON.stringify()` was directly called on this object, an empty string

    All `Temporal` objects implement the `toJSON()` method, which returns a string (the same as calling `toString()`). Thus, they will be serialized as strings. Similarly, `Date` objects implement `toJSON()`, which returns the same as `toISOString()`.
  - Only enumerable own properties are visited. This means `Map`, `Set`, etc. will become `"{}"`. You can use the `replacer` parameter to serialize them to something more useful.

    Properties are visited using the same algorithm as `Object.keys()`, which has a well-defined order and is stable across implementations. For example, `JSON.stringify` on the same object

will always produce the same string, and `JSON.parse(JSON.stringify(obj))` would produce an object with the same key ordering as the original (assuming the object is completely JSON-serializable).

### The replacer parameter

The `replacer` parameter can be either a function or an array.

As an array, its elements indicate the names of the properties in the object that should be included in the resulting JSON string. Only string and number values are taken into account; symbol keys are ignored.

As a function, it takes two parameters: the `key` and the `value` being stringified. The object in which the key was found is provided as the `replacer`'s `this` context.

The `replacer` function is called for the initial object being stringified as well, in which case the `key` is an empty string (`""`). It is then called for each property on the object or array being stringified. Array indices will be provided in its string form as `key`. The current property value will be replaced with the `replacer`'s return value for stringification. This means:

- If you return a number, string, boolean, or `null`, that value is directly serialized and used as the property's value. (Returning a BigInt will throw as well.)
- If you return a `Function`, `Symbol`, or `undefined`, the property is not included in the output.
- If you return any other object, the object is recursively stringified, calling the `replacer` function on each property.

> **Note:** When parsing JSON generated with `replacer` functions, you would likely want to use the `reviver` parameter to perform the reverse operation.

Typically, array elements' index would never shift (even when the element is an invalid value like a function, it will become `null` instead of omitted). Using the `replacer` function allows you to control the order of the array elements by returning a different array.

### The space parameter

The `space` parameter may be used to control spacing in the final string.

- If it is a number, successive levels in the stringification will each be indented by this many space characters.
- If it is a string, successive levels will be indented by this string.

Each level of indentation will never be longer than 10. Number values of `space` are clamped to 10, and string values are truncated to 10 characters.

## Examples

## Using JSON.stringify

js

```
JSON.stringify({}); // '{}'
JSON.stringify(true); // 'true'
JSON.stringify("foo"); // '"foo"'
JSON.stringify([1, "false", false]); // '[1,"false",false]'
JSON.stringify([NaN, null, Infinity]); // '[null,null,null]'
JSON.stringify({ x: 5 }); // '{"x":5}'

JSON.stringify(new Date(1906, 0, 2, 15, 4, 5));
// '"1906-01-02T15:04:05.000Z"'

JSON.stringify({ x: 5, y: 6 });
// '{"x":5,"y":6}'
JSON.stringify([new Number(3), new String("false"), new Boolean(false)]);
// '[3,"false",false]'

// String-keyed array elements are not enumerable and make no sense in JSON
const a = ["foo", "bar"];
a["baz"] = "quux"; // a: [ 0: 'foo', 1: 'bar', baz: 'quux' ]
JSON.stringify(a);
// '["foo","bar"]'

JSON.stringify({ x: [10, undefined, function () {}, Symbol("")] });
// '{"x":[10,null,null,null]}'

// Standard data structures
JSON.stringify([
  new Set([1]),
  new Map([[1, 2]]),
  new WeakSet([{ a: 1 }]),
  new WeakMap([[{ a: 1 }, 2]]),
]);
// '[{},{},{},{}]'

// TypedArray
JSON.stringify([new Int8Array([1]), new Int16Array([1]), new Int32Array([1])]);
// '[{"0":1},{"0":1},{"0":1}]'
JSON.stringify([
  new Uint8Array([1]),
  new Uint8ClampedArray([1]),
  new Uint16Array([1]),
  new Uint32Array([1]),
]);
// '[{"0":1},{"0":1},{"0":1},{"0":1}]'
JSON.stringify([new Float32Array([1]), new Float64Array([1])]);
// '[{"0":1},{"0":1}]'

// toJSON()
JSON.stringify({
  x: 5,
  y: 6,
  toJSON() {
    return this.x + this.y;
  },
});
// '11'
```

*(code continues on the next page)*

(code continued from the previous page)

```
// Symbols:
JSON.stringify({ x: undefined, y: Object, z: Symbol("") });
// '{}'
JSON.stringify({ [Symbol("foo")]: "foo" });
// '{}'
JSON.stringify({ [Symbol.for("foo")]: "foo" }, [Symbol.for("foo")]);
// '{}'
JSON.stringify({ [Symbol.for("foo")]: "foo" }, (k, v) => {
  if (typeof k === "symbol") {
    return "a symbol";
  }
});
// undefined

// Non-enumerable properties:
JSON.stringify(
  Object.create(null, {
    x: { value: "x", enumerable: false },
    y: { value: "y", enumerable: true },
  }),
);
// '{"y":"y"}'

// BigInt values throw
JSON.stringify({ x: 2n });
// TypeError: BigInt value can't be serialized in JSON
```

## Using a function as replacer

js

```
function replacer(key, value) {
  // Filtering out properties
  if (typeof value === "string") {
    return undefined;
  }
  return value;
}

const foo = {
  foundation: "Mozilla",
  model: "box",
  week: 45,
  transport: "car",
  month: 7,
};
JSON.stringify(foo, replacer);
// '{"week":45,"month":7}'
```

If you wish the `replacer` to distinguish an initial object from a key with an empty string property (since both would give the empty string as key and potentially an object as value), you will have to keep track of the iteration count (if it is beyond the first iteration, it is a genuine empty string key).

js

```
function makeReplacer() {
  let isInitial = true;

  return (key, value) => {
    if (isInitial) {
      isInitial = false;
```

(code continues on the next page)

*(code continued from the previous page)*

```
      return value;
    }
    if (key === "") {
      // Omit all properties with name "" (except the initial object)
      return undefined;
    }
    return value;
  };
}

const replacer = makeReplacer();
console.log(JSON.stringify({ "": 1, b: 2 }, replacer)); // "{"b":2}"
```

### Using an array as replacer

js

```
const foo = {
  foundation: "Mozilla",
  model: "box",
  week: 45,
  transport: "car",
  month: 7,
};

JSON.stringify(foo, ["week", "month"]);
// '{"week":45,"month":7}', only keep "week" and "month" properties
```

### Using the space parameter

Indent the output with one space:

js

```
console.log(JSON.stringify({ a: 2 }, null, " "));
/*
{
 "a": 2
}
*/
```

Using a tab character mimics standard pretty-print appearance:

js

```
console.log(JSON.stringify({ uno: 1, dos: 2 }, null, "\t"));
/*
{
    "uno": 1,
    "dos": 2
}
*/
```

### toJSON() behavior

Defining `toJSON()` for an object allows overriding its serialization behavior.

```js
const obj = {
  data: "data",

  toJSON(key) {
    return key ? `Now I am a nested object under key '${key}'` : this;
  },
};

JSON.stringify(obj);
// '{"data":"data"}'

JSON.stringify({ obj });
// '{"obj":"Now I am a nested object under key 'obj'"}'

JSON.stringify([obj]);
// '["Now I am a nested object under key '0'"]'
```

### Issue with serializing circular references

Since the JSON format doesn't support object references (although an IETF draft exists), a `TypeError` will be thrown if one attempts to encode an object with circular references.

**js · ✗ bad example**

```js
const circularReference = {};
circularReference.myself = circularReference;

// Serializing circular references throws "TypeError: cyclic object value"
JSON.stringify(circularReference);
```

To serialize circular references, you can use a library that supports them (e.g., cycle.js by Douglas Crockford) or implement a solution yourself, which will require finding and replacing (or removing) the cyclic references by serializable values.

If you are using `JSON.stringify()` to deep-copy an object, you may instead want to use `structuredClone()`, which supports circular references. JavaScript engine APIs for binary serialization, such as `v8.serialize()`, also support circular references.

### Using JSON.stringify() with localStorage

In a case where you want to store an object created by your user and allow it to be restored even after the browser has been closed, the following example is a model for the applicability of `JSON.stringify()`:

```js
// Creating an example of JSON
const session = {
  screens: [],
  state: true,
};
session.screens.push({ name: "screenA", width: 450, height: 250 });
session.screens.push({ name: "screenB", width: 650, height: 350 });
session.screens.push({ name: "screenC", width: 750, height: 120 });
session.screens.push({ name: "screenD", width: 250, height: 60 });
session.screens.push({ name: "screenE", width: 390, height: 120 });
session.screens.push({ name: "screenF", width: 1240, height: 650 });
```

(code continues on the next page)

*(code continued from the previous page)*

```
// Converting the JSON string with JSON.stringify()
// then saving with localStorage in the name of session
localStorage.setItem("session", JSON.stringify(session));

// Example of how to transform the String generated through
// JSON.stringify() and saved in localStorage in JSON object again
const restoredSession = JSON.parse(localStorage.getItem("session"));

// Now restoredSession variable contains the object that was saved
// in localStorage
console.log(restoredSession);
```

### Well-formed JSON.stringify()

Engines implementing the well-formed JSON.stringify specification will stringify lone surrogates (any code point from U+D800 to U+DFFF) using Unicode escape sequences rather than literally (outputting lone surrogates). Before this change, such strings could not be encoded in valid UTF-8 or UTF-16:

```
JSON.stringify("\uD800"); // '"\ud800"'
```

But with this change `JSON.stringify()` represents lone surrogates using JSON escape sequences that *can* be encoded in valid UTF-8 or UTF-16:

```
JSON.stringify("\uD800"); // '"\\ud800"'
```

This change should be backwards-compatible as long as you pass the result of `JSON.stringify()` to APIs such as `JSON.parse()` that will accept any valid JSON text, because they will treat Unicode escapes of lone surrogates as identical to the lone surrogates themselves. *Only* if you are directly interpreting the result of `JSON.stringify()` do you need to carefully handle `JSON.stringify()`'s two possible encodings of these code points.

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/structured-data.html#sec-json.stringify` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `stringify` | 3 | 12 | 3.5 | 10.5 | 4 | 18 | 4 | 11 | 4 | 1.0 | 4.4 | 4 | 1.0.0 | 1.0 | 0.10.0 |
| Strings are escaped to well-formed UTF-8 | 72 | 79 | 64 | 60 | 12.1 | 72 | 64 | 50 | 12.2 | 11.0 | 72 | 12.2 | 1.0.0 | 1.0 | 12.0.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.JSON.stringify`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of modern `JSON.stringify` behavior (symbol, well-formed unicode, raw JSON) in `core-js`
- `JSON.parse()`
- `JSON.rawJSON()`

JavaScript › Reference › Standard built-in objects

# Map

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Map · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Map`** object holds key-value pairs and remembers the original insertion order of the keys. Any value (both objects and primitive values) may be used as either a key or a value.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Map". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const map = new Map();

map.set("a", 1);
map.set("b", 2);
map.set("c", 3);

console.log(map.get("a"));
// Expected output: 1

map.set("a", 97);

console.log(map.get("a"));
// Expected output: 97

console.log(map.size);
// Expected output: 3

map.delete("b");

console.log(map.size);
// Expected output: 2
```

## Description

`Map` objects are collections of key-value pairs. A key in the `Map` **may only occur once**; it is unique in the `Map`'s collection. A `Map` object is iterated by key-value pairs — a `for...of` loop returns a 2-member array of `[key, value]` for each iteration. Iteration happens in *insertion order*, which corresponds to the order in which each key-value pair was first inserted into the map by the `set()` method (that is, there wasn't a key with the same value already in the map when `set()` was called).

The specification requires maps to be implemented "that, on average, provide access times that are sublinear on the number of elements in the collection". Therefore, it could be represented internally as a hash table (with O(1) lookup), a search tree (with O(log(N)) lookup), or any other data structure, as long as the complexity is better than O(N).

### Key equality

Value equality is based on the SameValueZero algorithm. (It used to use SameValue, which treated `0` and `-0` as different. Check browser compatibility.) This means `NaN` is considered the same as `NaN` (even though `NaN !== NaN`) and all other values are considered equal according to the semantics of the `===` operator. Also, for object keys, equality is based on object identity. They are compared by reference, not by value. See Using the Map object for examples.

### Objects vs. Maps

`Object` is similar to `Map`—both let you set keys to values, retrieve those values, delete keys, and detect whether something is stored at a key. For this reason (and because there were no built-in alternatives), `Object` has been used as `Map` historically.

However, there are important differences that make `Map` preferable in some cases:

| | Map | Object |
|---|---|---|
| **Accidental Keys** | A `Map` does not contain any keys by default. It only contains what is explicitly put into it. | An `Object` has a prototype, so it contains default keys that could collide with your own keys if you're not careful.<br><br>**Note:** This can be bypassed by using `Object.create(null)`, but this is seldom done. |
| **Security** | A `Map` is safe to use with user-provided keys and values. | Setting user-provided key-value pairs on an `Object` may allow an attacker to override the object's prototype, which can lead to object injection attacks or prototype pollution attacks. Like the accidental keys issue, this can also be mitigated by using a `null`-prototype object. |
| **Key Types** | A `Map`'s keys can be any value (including functions, objects, or any primitive). | The keys of an `Object` must be either a `String` or a `Symbol`. |
| **Key Order** | The keys in `Map` are ordered in a straightforward way: A `Map` object iterates entries, keys, and values in the order of entry insertion. | Although the keys of an ordinary `Object` are ordered now, this was not always the case, and the order is complex. As a result, it's best not to rely on property order.<br><br>The order was first defined for own properties only in ECMAScript 2015; ECMAScript 2020 defines order for inherited properties as well. But note that no single mechanism iterates **all** of an object's properties; the various mechanisms each include different subsets of properties. (`for-in` includes only enumerable string-keyed properties; `Object.keys` includes only own, enumerable, string-keyed properties; `Object.getOwnPropertyNames` includes own, string-keyed properties even if non-enumerable; `Object.getOwnPropertySymbols` does the same for just `Symbol`-keyed properties, etc.) |
| **Size** | The number of items in a `Map` is easily retrieved from its `size` property. | Determining the number of items in an `Object` is more roundabout and less efficient. A common way to do it is through the `length` of the array returned from `Object.keys()`. |
| **Iteration** | A `Map` is an iterable, so it can be directly iterated. | `Object` does not implement an iteration protocol, and so objects are not directly iterable using the JavaScript for..of statement (by default).<br><br>**Note:**<br>• An object can implement the iteration protocol, or you can get an iterable for an object using `Object.keys` or `Object.entries`.<br>• The for..in statement allows you to iterate over the *enumerable* properties of an object. |

| | Map | Object |
|---|---|---|
| **Performance** | Performs better in scenarios involving frequent additions and removals of key-value pairs. | Not optimized for frequent additions and removals of key-value pairs. |
| **Serialization and parsing** | No native support for serialization or parsing.<br>(But you can build your own serialization and parsing support for `Map` by using `JSON.stringify()` with its *replacer* argument, and by using `JSON.parse()` with its *reviver* argument. See the Stack Overflow question How do you JSON.stringify an ES6 Map?). | Native support for serialization from `Object` to JSON, using `JSON.stringify()`.<br>Native support for parsing from JSON to `Object`, using `JSON.parse()`. |

### Setting object properties

Setting Object properties works for Map objects as well, and can cause considerable confusion.

Therefore, this appears to work in a way:

js · ✗ bad example

```
const wrongMap = new Map();
wrongMap["bla"] = "blaa";
wrongMap["bla2"] = "blaaa2";

console.log(wrongMap); // Map { bla: 'blaa', bla2: 'blaaa2' }
```

But that way of setting a property does not interact with the Map data structure. It uses the feature of the generic object. The value of 'bla' is not stored in the Map for queries. Other operations on the data fail:

js · ✗ bad example

```
wrongMap.has("bla"); // false
wrongMap.delete("bla"); // false
console.log(wrongMap); // Map { bla: 'blaa', bla2: 'blaaa2' }
```

The correct usage for storing data in the Map is through the `set(key, value)` method.

js · ✓ good example

```
const contacts = new Map();
contacts.set("Jessie", { phone: "213-555-1234", address: "123 N 1st Ave" });
contacts.has("Jessie"); // true
contacts.get("Hilary"); // undefined
contacts.set("Hilary", { phone: "617-555-4321", address: "321 S 2nd St" });
contacts.get("Jessie"); // {phone: "213-555-1234", address: "123 N 1st Ave"}
contacts.delete("Raymond"); // false
contacts.delete("Jessie"); // true
console.log(contacts.size); // 1
```

### Map-like browser APIs

**Browser `Map`-like objects** (or "maplike objects") are Web API interfaces that behave in many ways like a `Map`.

Just like `Map`, entries can be iterated in the same order that they were added to the object. `Map`-like objects and `Map` also have properties and methods that share the same name and behavior. However unlike `Map` they only allow specific predefined types for the keys and values of each entry.

The allowed types are set in the specification IDL definition. For example, `RTCStatsReport` is a `Map`-like object that must use strings for keys and objects for values. This is defined in the specification IDL below:

webidl

```
interface RTCStatsReport {
  readonly maplike<DOMString, object>;
};
```

`Map`-like objects are either read-only or read-writable (see the `readonly` keyword in the IDL above).

- Read-only `Map`-like objects have the property `size`, and the methods: `entries()`, `forEach()`, `get()`, `has()`, `keys()`, `values()`, and `[Symbol.iterator]()`.
- Writeable `Map`-like objects additionally have the methods: `clear()`, `delete()`, and `set()`.

The methods and properties have the same behavior as the equivalent entities in `Map`, except for the restriction on the types of the keys and values.

The following are examples of read-only `Map`-like browser objects:

- `AudioParamMap`
- `CSSFontFeatureValuesMap`
- `EventCounts`
- `KeyboardLayoutMap`
- `MIDIInputMap`
- `MIDIOutputMap`
- `RTCStatsReport`

## Constructor

**`Map()`**

Creates a new `Map` object.

## Static properties

**`Map[Symbol.species]`**

The constructor function that is used to create derived objects.

## Static methods

**`Map.groupBy()`**

Groups the elements of a given iterable using the values returned by a provided callback function. The final returned `Map` uses the unique values from the test function as keys, which can be used to get the array of elements in each group.

## Instance properties

These properties are defined on `Map.prototype` and shared by all `Map` instances.

**`Map.prototype.constructor`**

The constructor function that created the instance object. For `Map` instances, the initial value is the `Map` constructor.

**`Map.prototype.size`**

Returns the number of key/value pairs in the `Map` object.

**`Map.prototype[Symbol.toStringTag]`**

The initial value of the `[Symbol.toStringTag]` property is the string `"Map"`. This property is used in `Object.prototype.toString()`.

## Instance methods

**`Map.prototype.clear()`**

Removes all key-value pairs from the `Map` object.

**`Map.prototype.delete()`**

Removes the entry specified by the key from this `Map`.

**`Map.prototype.entries()`**

Returns a new Iterator object that contains a two-member array of `[key, value]` for each element in the `Map` object in insertion order.

**`Map.prototype.forEach()`**

Calls `callbackFn` once for each key-value pair present in the `Map` object, in insertion order. If a `thisArg` parameter is provided to `forEach`, it will be used as the `this` value for each callback.

**`Map.prototype.get()`**

Returns the value corresponding to the key in this `Map`, or `undefined` if there is none.

**`Map.prototype.getOrInsert()`**

Returns the value corresponding to the specified key in this `Map`. If the key is not present, it inserts a new entry with the key and a given default value, and returns the inserted value.

**`Map.prototype.getOrInsertComputed()`**

Returns the value corresponding to the specified key in this `Map`. If the key is not present, it inserts a new entry with the key and a default value computed from a given callback, and returns the inserted value.

**`Map.prototype.has()`**

Returns a boolean indicating whether an entry with the specified key exists in this `Map` or not.

**`Map.prototype.keys()`**

Returns a new Iterator object that contains the keys for each element in the `Map` object in insertion order.

**`Map.prototype.set()`**

Adds a new entry with a specified key and value to this `Map`, or updates an existing entry if the key already exists.

**`Map.prototype.values()`**

Returns a new Iterator object that contains the values for each element in the `Map` object in insertion order.

**`Map.prototype[Symbol.iterator]()`**

Returns a new Iterator object that contains a two-member array of `[key, value]` for each element in the `Map` object in insertion order.

## Examples

### Using the Map object

```js
const myMap = new Map();

const keyString = "a string";
const keyObj = {};
const keyFunc = () => {};

// setting the values
myMap.set(keyString, "value associated with 'a string'");
myMap.set(keyObj, "value associated with keyObj");
myMap.set(keyFunc, "value associated with keyFunc");

console.log(myMap.size); // 3

// getting the values
console.log(myMap.get(keyString)); // "value associated with 'a string'"
console.log(myMap.get(keyObj)); // "value associated with keyObj"
console.log(myMap.get(keyFunc)); // "value associated with keyFunc"

console.log(myMap.get("a string")); // "value associated with 'a string'", because keyString
=== 'a string'
console.log(myMap.get({})); // undefined, because keyObj !== {}
console.log(myMap.get(() => {})); // undefined, because keyFunc !== () => {}
```

### Using NaN as Map keys

`NaN` can also be used as a key. Even though every `NaN` is not equal to itself (`NaN !== NaN` is true), the following example works because `NaN`s are indistinguishable from each other:

```js
const myMap = new Map();
myMap.set(NaN, "not a number");

myMap.get(NaN);
// "not a number"

const otherNaN = Number("foo");
myMap.get(otherNaN);
// "not a number"
```

### Iterating Map with for...of

Maps can be iterated using a `for...of` loop:

```js
const myMap = new Map();
myMap.set(0, "zero");
myMap.set(1, "one");

for (const [key, value] of myMap) {
  console.log(`${key} = ${value}`);
}
// 0 = zero
// 1 = one

for (const key of myMap.keys()) {
  console.log(key);
}
// 0
// 1

for (const value of myMap.values()) {
  console.log(value);
}
// zero
// one

for (const [key, value] of myMap.entries()) {
  console.log(`${key} = ${value}`);
}
// 0 = zero
// 1 = one
```

### Iterating Map with forEach()

Maps can be iterated using the `forEach()` method:

```js
myMap.forEach((value, key) => {
  console.log(`${key} = ${value}`);
});
// 0 = zero
// 1 = one
```

### Relation with Array objects

```js
const kvArray = [
  ["key1", "value1"],
  ["key2", "value2"],
];

// Use the regular Map constructor to transform a 2D key-value Array into a map
const myMap = new Map(kvArray);

console.log(myMap.get("key1")); // "value1"

// Use Array.from() to transform a map into a 2D key-value Array
console.log(Array.from(myMap)); // Will show you exactly the same Array as kvArray

// A succinct way to do the same, using the spread syntax
console.log([...myMap]);
```

*(code continues on the next page)*

(code continued from the previous page)

```
// Or use the keys() or values() iterators, and convert them to an array
console.log(Array.from(myMap.keys())); // ["key1", "key2"]
```

## Cloning and merging Maps

Just like `Arrays`, `Maps` can be cloned:

js

```
const original = new Map([[1, "one"]]);

const clone = new Map(original);

console.log(clone.get(1)); // one
console.log(original === clone); // false (useful for shallow comparison)
```

> **Note:** Keep in mind that *the data itself* is not cloned. In other words, it is only a shallow copy of the `Map`.

Maps can be merged, maintaining key uniqueness:

js

```
const first = new Map([
  [1, "one"],
  [2, "two"],
  [3, "three"],
]);

const second = new Map([
  [1, "uno"],
  [2, "dos"],
]);

// Merge two maps. The last repeated key wins.
// Spread syntax essentially converts a Map to an Array
const merged = new Map([...first, ...second]);

console.log(merged.get(1)); // uno
console.log(merged.get(2)); // dos
console.log(merged.get(3)); // three
```

Maps can be merged with Arrays, too:

```js
const first = new Map([
  [1, "one"],
  [2, "two"],
  [3, "three"],
]);

const second = new Map([
  [1, "uno"],
  [2, "dos"],
]);

// Merge maps with an array. The last repeated key wins.
const merged = new Map([...first, ...second, [1, "un"]]);

console.log(merged.get(1)); // un
console.log(merged.get(2)); // dos
console.log(merged.get(3)); // three
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/keyed-collections.html#sec-map-objects |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| Map | 38 | 12 | 13 | 25 | 8 | 38 | 14 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |
| [Symbol.iterator] | 38 | 12 | 36[1,2,3,4] | 25 | 10 | 38 | 36[1,2,3,4] | 25 | 10 | 3.0 | 38 | 10 | 1.0.0 | 1.0 | 0.12.0 |
| [Symbol.species] | 51 | 13 | 41 | 38 | 10 | 51 | 41 | 41 | 10 | 5.0 | 51 | 10 | 1.0.0 | 1.0 | 6.5.0 |
| Map() constructor | 38 | 12 | 13 | 25 | 8 | 38 | 14 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |
| new Map(iterable) | 38 | 12 | 13 | 25 | 9 | 38 | 14 | 25 | 9 | 3.0 | 38 | 9 | 1.0.0 | 1.0 | 0.12.0 |
| new Map(null) | 38 | 12 | 37 | 25 | 9 | 38 | 37 | 25 | 9 | 3.0 | 38 | 9 | 1.0.0 | 1.0 | 0.12.0 |
| clear | 38 | 12 | 19 | 25 | 8 | 38 | 19 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |
| delete | 38 | 12 | 13 | 25 | 8 | 38 | 14 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |
| entries | 38 | 12 | 20 | 25 | 8 | 38 | 20 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |
| forEach | 38 | 12 | 25 | 25 | 8 | 38 | 25 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |
| get | 38 | 12 | 13 | 25 | 8 | 38 | 14 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |
| getOrInsert | 145 | 145 | 144 | 129 | 26.2 | 145 | 144 | 96 | 26.2 | **No** | 145 | 26.2 | 1.2.20 | 2.6.7 | 26.0.0 |
| getOrInsertComputed | 145 | 145 | 144 | 129 | 26.2 | 145 | 144 | 96 | 26.2 | **No** | 145 | 26.2 | 1.2.20 | 2.6.7 | 26.0.0 |
| groupBy | 117 | 117 | 119 | 103 | 17.4[5] | 117 | 119 | 78 | 17.4[5] | 24.0 | 117 | 17.4[5] | 1.0.19 | 1.37 | 21.0.0 |
| has | 38 | 12 | 13 | 25 | 8 | 38 | 14 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |
| Key equality for -0 and 0 | 38 | 12 | 29 | 25 | 9 | 38 | 29 | 25 | 9 | 3.0 | 38 | 9 | 1.0.0 | 1.0 | 4.0.0 |
| keys | 38 | 12 | 20 | 25 | 8 | 38 | 20 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |
| set | 38 | 12 | 13 | 25 | 8 | 38 | 14 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `size` | 38 | 12 | 19[6] | 25 | 8 | 38 | 19[7] | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |
| `values` | 38 | 12 | 20 | 25 | 8 | 38 | 20 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |

1. Also from version 27–36: uses the non-standard name `@@iterator`.
2. Also from version 27–36: A placeholder property named `@@iterator` is used.
3. Also from version 17–27: uses the non-standard name `iterator`.
4. Also from version 17–27: A placeholder property named `iterator` is used.
5. Also from version 16.4–17.4: uses the non-standard name `Array.prototype.groupToMap`.
6. From Firefox 13 to Firefox 18, the `size` property was implemented as a `Map.prototype.size()` method, this has been changed to a property in later versions conform to the ECMAScript 2015 specification.
7. From Firefox for Android 14 to Firefox for Android 18, the `size` property was implemented as a `Map.prototype.size()` method, this has been changed to a property in later versions conform to the ECMAScript 2015 specification.

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Map`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill for `Map` in `core-js`
- es-shims polyfill of `Map`
- `Set`
- `WeakMap`
- `WeakSet`

---

JavaScript › Reference › Standard built-in objects › Map

# Map() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Map/Map · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Map()`** constructor creates `Map` objects.

## Syntax

```js
new Map()
new Map(iterable)
```

> **Note:** `Map()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

### Parameters

**`iterable`** Optional

If an iterable object (such as an array) is passed, all of its elements will be added to the new `Map`. Each element must be an object with two properties: `0` and `1`, which correspond to the key and value (for example, `[[1, "one"],[2, "two"]]`). If you don't specify this parameter, or its value is `null` or `undefined`, the new `Map` is empty.

## Examples

### Creating a new Map

```js
const myMap = new Map([
  [1, "one"],
  [2, "two"],
  [3, "three"],
]);
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/keyed-collections.html#sec-map-constructor |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Nodejs |
| `Map()` constructor | 38 | 12 | 13 | 25 | 8 | 38 | 14 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |
| `new Map(iterable)` | 38 | 12 | 13 | 25 | 9 | 38 | 14 | 25 | 9 | 3.0 | 38 | 9 | 1.0.0 | 1.0 | 0.12.0 |
| `new Map(null)` | 38 | 12 | 37 | 25 | 9 | 38 | 37 | 25 | 9 | 3.0 | 38 | 9 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Map.Map`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill for `Map` in `core-js`
- es-shims polyfill of `Map`
- `Set`
- `WeakMap`
- `WeakSet`

JavaScript › Reference › Standard built-in objects › Map

# Map.groupBy()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Map/groupBy · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since March 2024.

> **Note:** In some versions of some browsers, this method was implemented as the method `Array.prototype.groupToMap()`. Due to web compatibility issues, it is now implemented as a static method. Check the browser compatibility table for details.

The **`Map.groupBy()`** static method groups the elements of a given iterable using the values returned by a provided callback function. The final returned `Map` uses the unique values from the test function as keys, which can be used to get the array of elements in each group.

The method is primarily useful when grouping elements that are associated with an object, and in particular when that object might change over time. If the object is invariant, you might instead represent it using a string, and group elements with `Object.groupBy()`.

## Try it

*▷ Interactive example on MDN: "JavaScript Demo: Map.groupBy()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const inventory = [
  { name: "asparagus", type: "vegetables", quantity: 9 },
  { name: "bananas", type: "fruit", quantity: 5 },
  { name: "goat", type: "meat", quantity: 23 },
  { name: "cherries", type: "fruit", quantity: 12 },
  { name: "fish", type: "meat", quantity: 22 },
];

const restock = { restock: true };
const sufficient = { restock: false };
const result = Map.groupBy(inventory, ({ quantity }) =>
  quantity < 6 ? restock : sufficient,
);
console.log(result.get(restock));
// [{ name: "bananas", type: "fruit", quantity: 5 }]
```

## Syntax

js

```
Map.groupBy(items, callbackFn)
```

### Parameters

**`items`**

An iterable (such as an `Array`) whose elements will be grouped.

**`callbackFn`**

A function to execute for each element in the iterable. It should return a value (object or primitive) indicating the group of the current element. The function is called with the following arguments:

**`element`**

The current element being processed.

**`index`**

The index of the current element being processed.

### Return value

A `Map` object with keys for each group, each assigned to an array containing the elements of the associated group.

## Description

`Map.groupBy()` calls a provided `callbackFn` function once for each element in an iterable. The callback function should return a value indicating the group of the associated element. The values returned by `callbackFn` are used as keys for the `Map` returned by `Map.groupBy()`. Each key has an associated array containing all the elements for which the callback returned the same value.

The elements in the returned `Map` and the original iterable are the same (not deep copies). Changing the internal structure of the elements will be reflected in both the original iterable and the returned `Map`.

This method is useful when you need to group information that is related to a particular object that might potentially change over time. This is because even if the object is modified, it will continue to work as a key to the returned `Map`. If you instead create a string representation for the object and use that as a grouping key in `Object.groupBy()`, you must maintain the mapping between the original object and its representation as the object changes.

> **Note:** To access the groups in the returned `Map`, you must use the same object that was originally used as a key in the `Map` (although you may modify its properties). You can't use another object that just happens to have the same name and properties.

`Map.groupBy` does not read the value of `this`. It can be called on any object and a new `Map` instance will be returned.

## Examples

### Using Map.groupBy()

First we define an array containing objects representing an inventory of different foodstuffs. Each food has a `type` and a `quantity`.

```js
const inventory = [
  { name: "asparagus", type: "vegetables", quantity: 9 },
  { name: "bananas", type: "fruit", quantity: 5 },
  { name: "goat", type: "meat", quantity: 23 },
  { name: "cherries", type: "fruit", quantity: 12 },
  { name: "fish", type: "meat", quantity: 22 },
];
```

The code below uses `Map.groupBy()` with an arrow function that returns the object keys named `restock` or `sufficient`, depending on whether the element has `quantity < 6`. The returned `result` object is a `Map` so we need to call `get()` with the key to obtain the array.

```js
const restock = { restock: true };
const sufficient = { restock: false };
const result = Map.groupBy(inventory, ({ quantity }) =>
  quantity < 6 ? restock : sufficient,
);
console.log(result.get(restock));
// [{ name: "bananas", type: "fruit", quantity: 5 }]
```

Note that the function argument `{ quantity }` is a basic example of object destructuring syntax for function arguments. This unpacks the `quantity` property of an object passed as a parameter, and

assigns it to a variable named `quantity` in the body of the function. This is a very succinct way to access the relevant values of elements within a function.

The key to a `Map` can be modified and still used. However you can't recreate the key and still use it. For this reason it is important that anything that needs to use the map keeps a reference to its keys.

js

```
// The key can be modified and still used
restock["fast"] = true;
console.log(result.get(restock));
// [{ name: "bananas", type: "fruit", quantity: 5 }]

// A new key can't be used, even if it has the same structure!
const restock2 = { restock: true };
console.log(result.get(restock2)); // undefined
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/keyed-collections.html#sec-map.groupby |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| groupBy | 117 | 117 | 119 | 103 | 17.4[1] | 117 | 119 | 78 | 17.4[1] | 24.0 | 117 | 17.4[1] | 1.0.19 | 1.37 | 21.0.0 |

1. Also from version 16.4–17.4: uses the non-standard name `Array.prototype.groupToMap`.

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Map.groupBy`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Map.groupBy` in `core-js`
- es-shims polyfill of `Map.groupBy`
- Indexed collections guide
- `Array.prototype.reduce()`
- `Map()`
- `Object.groupBy()`

JavaScript › Reference › Standard built-in objects › Map

# Map[Symbol.species]

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Map/Symbol.species · MDN page last modified 2025-07-10

**Baseline: Widely available** — available across major browsers since September 2016.

The **`Map[Symbol.species]`** static accessor property is an unused accessor property specifying how to copy `Map` objects.

## Syntax

```
Map[Symbol.species]
```

### Return value

The value of the constructor (`this`) on which `get [Symbol.species]` was called. The return value is used to construct copied `Map` instances.

## Description

The `[Symbol.species]` accessor property returns the default constructor for `Map` objects. Subclass constructors may override it to change the constructor assignment.

> **Note:** This property is currently unused by all `Map` methods.

## Examples

### Species in ordinary objects

The `[Symbol.species]` property returns the default constructor function, which is the `Map` constructor for `Map`.

```
Map[Symbol.species]; // function Map()
```

### Species in derived objects

In an instance of a custom `Map` subclass, such as `MyMap`, the `MyMap` species is the `MyMap` constructor. However, you might want to overwrite this, in order to return parent `Map` objects in your derived class methods:

```
class MyMap extends Map {
  // Overwrite MyMap species to the parent Map constructor
  static get [Symbol.species]() {
    return Map;
  }
}
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/keyed-collections.html#sec-get-map-%symbol.species%` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| [Symbol.species] | 51 | 13 | 41 | 38 | 10 | 51 | 41 | 41 | 10 | 5.0 | 51 | 10 | 1.0.0 | 1.0 | 6.5.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Map.@@species`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Map`
- `Symbol.species`

---

JavaScript › Reference › Standard built-in objects › Map

# Map.prototype.clear()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Map/clear · MDN page last modified 2025-07-20

**Baseline: Widely available** — available across major browsers since July 2015.

The **`clear()`** method of `Map` instances removes all elements from this map.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Map.prototype.clear()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const map = new Map();

map.set("bar", "baz");
map.set(1, "foo");

console.log(map.size);
// Expected output: 2

map.clear();

console.log(map.size);
// Expected output: 0
```

## Syntax

```js
clear()
```

### Parameters

None.

### Return value

None (`undefined`).

## Examples

### Using clear()

```js
const myMap = new Map();
myMap.set("bar", "baz");
myMap.set(1, "foo");

console.log(myMap.size); // 2
console.log(myMap.has("bar")); // true

myMap.clear();

console.log(myMap.size); // 0
console.log(myMap.has("bar")); // false
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/keyed-collections.html#sec-map.prototype.clear` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `clear` | 38 | 12 | 19 | 25 | 8 | 38 | 19 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Map.clear`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Map

JavaScript › Reference › Standard built-in objects › Map

# Map.prototype.delete()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Map/delete · MDN page last modified 2025-09-22

**Baseline: Widely available** — available across major browsers since July 2015.

The **`delete()`** method of `Map` instances removes the entry specified by the key from this `Map`.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Map.prototype.delete()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const map = new Map();
map.set("bar", "foo");

console.log(map.delete("bar"));
// Expected result: true
// True indicates successful removal

console.log(map.has("bar"));
// Expected result: false
```

## Syntax

js

```
mapInstance.delete(key)
```

### Parameters

**key**

The key of the entry to remove from the `Map` object. Object keys are compared by reference, not by value.

### Return value

`true` if an entry in the `Map` object has been removed successfully. `false` if the key is not found in the `Map`.

## Examples

### Using delete()

js

```
const myMap = new Map();
myMap.set("bar", "foo");

console.log(myMap.delete("bar")); // Returns true. Successfully removed.
console.log(myMap.has("bar")); // Returns false. The "bar" element is no longer present.
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/keyed-collections.html#sec-map.prototype.delete` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `delete` | 38 | 12 | 13 | 25 | 8 | 38 | 14 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Map.delete`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Map`
- `Map.prototype.clear()`
- `Map.prototype.get()`
- `Map.prototype.set()`
- `Map.prototype.has()`

---

JavaScript › Reference › Standard built-in objects › Map

# Map.prototype.entries()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Map/entries · MDN page last modified 2025-07-20

**Baseline: Widely available** — available across major browsers since July 2015.

The **`entries()`** method of `Map` instances returns a new *map iterator* object that contains the `[key, value]` pairs for each element in this map in insertion order.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Map.prototype.entries()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const map = new Map();

map.set("0", "foo");
map.set(1, "bar");

const iterator = map.entries();

console.log(iterator.next().value);
// Expected output: Array ["0", "foo"]

console.log(iterator.next().value);
// Expected output: Array [1, "bar"]
```

## Syntax

js

```
entries()
```

### Parameters

None.

### Return value

A new iterable iterator object.

## Examples

### Using entries()

```js
const myMap = new Map();
myMap.set("0", "foo");
myMap.set(1, "bar");
myMap.set({}, "baz");

const mapIter = myMap.entries();

console.log(mapIter.next().value); // ["0", "foo"]
console.log(mapIter.next().value); // [1, "bar"]
console.log(mapIter.next().value); // [Object, "baz"]
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/keyed-collections.html#sec-map.prototype.entries |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| entries | 38 | 12 | 20 | 25 | 8 | 38 | 20 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Map.entries`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Map.prototype.keys()`
- `Map.prototype.values()`

JavaScript › Reference › Standard built-in objects › Map

# Map.prototype.forEach()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Map/forEach · MDN page last modified 2025-10-30

**Baseline: Widely available** — available across major browsers since July 2015.

The **`forEach()`** method of `Map` instances executes a provided function once per each key/value pair in this map, in insertion order.

## Try it

*▷ Interactive example on MDN: "JavaScript Demo: Map.prototype.forEach()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function logMapElements(value, key, map) {
  console.log(`m[${key}] = ${value}`);
}

new Map([
  ["foo", 3],
  ["bar", {}],
  ["baz", undefined],
]).forEach(logMapElements);

// Expected output: "m[foo] = 3"
// Expected output: "m[bar] = [object Object]"
// Expected output: "m[baz] = undefined"
```

## Syntax

js

```
forEach(callbackFn)
forEach(callbackFn, thisArg)
```

### Parameters

**`callbackFn`**

A function to execute for each entry in the map. The function is called with the following arguments:

**`value`**

Value of each iteration.

**`key`**

Key of each iteration.

**`map`**

The map being iterated.

**`thisArg`** Optional

A value to use as `this` when executing `callbackFn`.

### Return value

None (`undefined`).

## Description

The `forEach` method executes the provided `callback` once for each key of the map which actually exist. It is not invoked for keys which have been deleted. However, it is executed for values which are present but have the value `undefined`.

`callback` is invoked with **three arguments**:

- the entry's `value`
- the entry's `key`
- the **`Map` object** being traversed

If a `thisArg` parameter is provided to `forEach`, it will be passed to `callback` when invoked, for use as its `this` value. Otherwise, the value `undefined` will be passed for use as its `this` value. The `this` value ultimately observable by `callback` is determined according to the usual rules for determining the `this` seen by a function.

Each value is visited once, except in the case when it was deleted and re-added before `forEach` has finished. `callback` is not invoked for values deleted before being visited. New values added before `forEach` has finished will be visited.

## Examples

### Printing the contents of a Map object

The following code logs a line for each element in a `Map` object:

```js
function logMapElements(value, key, map) {
  console.log(`map.get('${key}') = ${value}`);
}
new Map([
  ["foo", 3],
  ["bar", {}],
  ["baz", undefined],
]).forEach(logMapElements);
// Logs:
// "map.get('foo') = 3"
// "map.get('bar') = [object Object]"
// "map.get('baz') = undefined"
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/keyed-collections.html#sec-map.prototype.foreach |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `forEach` | 38 | 12 | 25 | 25 | 8 | 38 | 25 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Map.forEach`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Array.prototype.forEach()`
- `Set.prototype.forEach()`

---

JavaScript › Reference › Standard built-in objects › Map

# Map.prototype.get()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Map/get · MDN page last modified 2025-09-22

**Baseline: Widely available** — available across major browsers since July 2015.

The **`get()`** method of `Map` instances returns the value corresponding to the key in this `Map`, or `undefined` if there is none. Object values are returned as the same reference that was originally stored, not as a copy, so mutations to the returned object will be reflected anywhere that reference is held, including inside the `Map`.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Map.prototype.get()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const map = new Map();
map.set("bar", "foo");

console.log(map.get("bar"));
// Expected output: "foo"

console.log(map.get("baz"));
// Expected output: undefined
```

## Syntax

js

```
get(key)
```

### Parameters

**key**

The key of the value to return from the `Map` object. Object keys are compared by reference, not by value.

### Return value

The value associated with the specified key in the `Map` object. If the key can't be found, `undefined` is returned.

## Examples

### Using get()

```js
const myMap = new Map();
myMap.set("bar", "foo");

console.log(myMap.get("bar")); // Returns "foo"
console.log(myMap.get("baz")); // Returns undefined
```

### Using get() to retrieve a reference to an object

```js
const arr = [];
const myMap = new Map();
myMap.set("bar", arr);

myMap.get("bar").push("foo");

console.log(arr); // ["foo"]
console.log(myMap.get("bar")); // ["foo"]
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/keyed-collections.html#sec-map.prototype.get |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| get | 38 | 12 | 13 | 25 | 8 | 38 | 14 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Map.get`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Map`
- `Map.prototype.delete()`
- `Map.prototype.set()`
- `Map.prototype.has()`

JavaScript › Reference › Standard built-in objects › Map

# Map.prototype.getOrInsert()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Map/getOrInsert · MDN page last modified 2026-10-26

**Baseline 2026: Newly available** — works across the latest browser versions since February 2026; may not work in older ones.

The **`getOrInsert()`** method of `Map` instances returns the value corresponding to the specified key in this `Map`. If the key is not present, it inserts a new entry with the key and a given default value, and returns the inserted value.

If the computation of the default value is expensive, consider using `Map.prototype.getOrInsertComputed()` instead, which takes a callback to compute the default value only if it's actually needed.

## Try it

*▷ Interactive example on MDN: "JavaScript Demo: Map.prototype.getOrInsert()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const map = new Map([["bar", "foo"]]);
console.log(map.getOrInsert("bar", "default"));
// Expected output: "foo"

console.log(map.getOrInsert("baz", "default"));
// Expected output: "default"
```

## Syntax

js

```
getOrInsert(key, defaultValue)
```

### Parameters

**`key`**

The key of the value to return from the `Map` object. Object keys are compared by reference, not by value.

**`defaultValue`**

The value to insert and return if the key is not already present in the `Map` object.

### Return value

The value associated with the specified key in the `Map` object. If the key can't be found, `defaultValue` is inserted and returned.

## Description

The `getOrInsert()` method is equivalent to the following:

```js
if (map.has(key)) {
  return map.get(key);
}
map.set(key, defaultValue);
return defaultValue;
```

It is also similar to the following pattern (which is slightly less reliable if `null` or `undefined` are valid values in your map):

```js
map.set(key, map.get(key) ?? defaultValue);
```

## Examples

### Multimap

In a map where each key is mapped to an array of values, you can use `getOrInsert()` to ensure that the array exists for a given key before attempting to push a new value to the array.

```js
map.getOrInsert(key, []).push(value);
```

### Applying default values

You can use `getOrInsert()` to ensure that a key exists in a map, even if you currently don't need its value. This is usually to normalize user input.

Imagine you have a map of user preferences, and you want to ensure that a certain preference is always set to a default value if the user hasn't specified it:

```js
const options = readConfig();
options.getOrInsert("theme", "light");
options.getOrInsert("fontSize", 14);

// Later in your code, you can safely assume these options exist
document.body.dataset.theme = options.get("theme");
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/keyed-collections.html#sec-map.prototype.getorinsert |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `getOrInsert` | 145 | 145 | 144 | 129 | 26.2 | 145 | 144 | 96 | 26.2 | **No** | 145 | 26.2 | 1.2.20 | 2.6.7 | 26.0.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Map.getOrInsert`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Map.prototype.getOrInsert` in `core-js`
- es-shims polyfill of `Map.prototype.getOrInsert`
- `Map`
- `Map.prototype.get()`
- `Map.prototype.set()`
- `Map.prototype.has()`
- `Map.prototype.getOrInsertComputed()`

---

JavaScript › Reference › Standard built-in objects › Map

# Map.prototype.getOrInsertComputed()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Map/getOrInsertComputed · MDN page last modified 2026-10-26

**Baseline 2026: Newly available** — works across the latest browser versions since February 2026; may not work in older ones.

The **`getOrInsertComputed()`** method of `Map` instances returns the value corresponding to the specified key in this `Map`. If the key is not present, it inserts a new entry with the key and a default value computed from a given callback, and returns the inserted value.

Use this method instead of `Map.prototype.getOrInsert()` when the default value is expensive to compute, and you want to avoid computing it unless it's actually needed.

## Try it

*▷ Interactive example on MDN: "JavaScript Demo: Map.prototype.getOrInsertComputed()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const map = new Map([["bar", "foo"]]);
const defaultCreator = (key) => `default for ${key}`;

console.log(map.getOrInsertComputed("bar", defaultCreator));
// Expected output: "foo"

console.log(map.getOrInsertComputed("baz", defaultCreator));
// Expected output: "default for baz"
```

## Syntax

js

```
getOrInsertComputed(key, callback)
```

### Parameters

**key**

The key of the element to return from the `Map` object. Object keys are compared by reference, not by value.

**`callback`**

A function that returns the value to insert and return if the key is not already present in the `Map` object. The function is called with the following argument:

**`key`**

The same key that was passed to `getOrInsertComputed()`.

### Return value

The value associated with the specified key in the `Map` object. If the key can't be found, the result of `callback(key)` is inserted and returned.

### Exceptions

**`TypeError`**

Thrown if `callback` isn't callable.

## Examples

### Avoiding unnecessary default computation

When using `Map.prototype.getOrInsert()`, the default value is computed every time, even if it is not needed. With `getOrInsertComputed()`, the default value is only computed when necessary.

```js
const map = new Map([["bar", "foo"]]);
const defaultCreator = (key) => {
  console.log(`Creating default for ${key}`);
  return `default for ${key}`;
};

map.getOrInsert("bar", defaultCreator("bar")); // Logs "Creating default for bar"
map.getOrInsertComputed("bar", defaultCreator); // No log
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/keyed-collections.html#sec-map.prototype.getorinsertcomputed |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `getOrInsertComputed` | 145 | 145 | 144 | 129 | 26.2 | 145 | 144 | 96 | 26.2 | **No** | 145 | 26.2 | 1.2.20 | 2.6.7 | 26.0.0 |

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Map.getOrInsertComputed`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Map.prototype.getOrInsertComputed` in `core-js`
- es-shims polyfill of `Map.prototype.getOrInsertComputed`
- `Map`
- `Map.prototype.get()`
- `Map.prototype.set()`
- `Map.prototype.has()`
- `Map.prototype.getOrInsert()`

---

JavaScript › Reference › Standard built-in objects › Map

# Map.prototype.has()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Map/has · MDN page last modified 2025-10-28

**Baseline: Widely available** — available across major browsers since July 2015.

The **`has()`** method of `Map` instances returns a boolean indicating whether an entry with the specified key exists in this `Map` or not.

## Try it

*▷ Interactive example on MDN: "JavaScript Demo: Map.prototype.has()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const map = new Map();
map.set("bar", "foo");

console.log(map.has("bar"));
// Expected output: true

console.log(map.has("baz"));
// Expected output: false
```

## Syntax

js

```
has(key)
```

### Parameters

**key**

The key of the entry to test for presence in the `Map` object. Object keys are compared by reference, not by value.

### Return value

Returns `true` if an entry with the specified key exists in the `Map` object; otherwise `false`.

## Examples

### Using has()

```js
const myMap = new Map();
myMap.set("bar", "foo");

console.log(myMap.has("bar")); // true
console.log(myMap.has("baz")); // false
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/keyed-collections.html#sec-map.prototype.has |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| has | 38 | 12 | 13 | 25 | 8 | 38 | 14 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Map.has`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Map`
- `Map.prototype.delete()`
- `Map.prototype.get()`
- `Map.prototype.set()`

---

JavaScript › Reference › Standard built-in objects › Map

# Map.prototype.keys()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Map/keys · MDN page last modified 2025-07-20

**Baseline: Widely available** — available across major browsers since July 2015.

The **`keys()`** method of `Map` instances returns a new *map iterator* object that contains the keys for each element in this map in insertion order.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Map.prototype.keys()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const map = new Map();

map.set("0", "foo");
map.set(1, "bar");

const iterator = map.keys();

console.log(iterator.next().value);
// Expected output: "0"

console.log(iterator.next().value);
// Expected output: 1
```

## Syntax

js

```
keys()
```

### Parameters

None.

### Return value

A new iterable iterator object.

## Examples

### Using keys()

js

```
const myMap = new Map();
myMap.set("0", "foo");
myMap.set(1, "bar");
myMap.set({}, "baz");

const mapIter = myMap.keys();

console.log(mapIter.next().value); // "0"
console.log(mapIter.next().value); // 1
console.log(mapIter.next().value); // {}
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/keyed-collections.html#sec-map.prototype.keys |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| keys | 38 | 12 | 20 | 25 | 8 | 38 | 20 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Map.keys`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Map.prototype.entries()`
- `Map.prototype.values()`

---

JavaScript › Reference › Standard built-in objects › Map

# Map.prototype.set()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Map/set · MDN page last modified 2026-09-07

**Baseline: Widely available** — available across major browsers since July 2015.

The **`set()`** method of `Map` instances adds a new entry with a specified key and value to this `Map`, or updates an existing entry if the key already exists.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Map.prototype.set()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const map = new Map();
map.set("bar", "foo");

console.log(map.get("bar"));
// Expected output: "foo"

console.log(map.get("baz"));
// Expected output: undefined
```

## Syntax

js

```
set(key, value)
```

### Parameters

**`key`**

The key of the entry to add to or modify within the `Map` object. Can be any value. Object keys are compared by reference, not by value.

**`value`**

The value of the entry to add to or modify within the `Map` object. Can be any value.

### Return value

The `Map` object.

## Examples

### Using set()

```js
const myMap = new Map();

// Add new elements to the map
myMap.set("bar", "foo");
myMap.set(1, "foobar");

// Update an element in the map
myMap.set("bar", "baz");
```

### Using the set() with chaining

Since the `set()` method returns the same `Map` object, you can chain the method call like below:

```js
// Add new elements to the map with chaining.
myMap.set("bar", "foo").set(1, "foobar").set(2, "baz");
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/keyed-collections.html#sec-map.prototype.set |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| set | 38 | 12 | 13 | 25 | 8 | 38 | 14 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Map.set`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Map`
- `Map.prototype.delete()`
- `Map.prototype.get()`
- `Map.prototype.has()`

JavaScript › Reference › Standard built-in objects › Map

# Map.prototype.values()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Map/values · MDN page last modified 2025-07-20

**Baseline: Widely available** — available across major browsers since July 2015.

The **`values()`** method of `Map` instances returns a new *map iterator* object that contains the values for each element in this map in insertion order.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Map.prototype.values()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const map = new Map();

map.set("0", "foo");
map.set(1, "bar");

const iterator = map.values();

console.log(iterator.next().value);
// Expected output: "foo"

console.log(iterator.next().value);
// Expected output: "bar"
```

## Syntax

js

```
values()
```

### Parameters

None.

### Return value

A new iterable iterator object.

## Examples

### Using values()

js

```
const myMap = new Map();
myMap.set("0", "foo");
myMap.set(1, "bar");
myMap.set({}, "baz");

const mapIter = myMap.values();

console.log(mapIter.next().value); // "foo"
console.log(mapIter.next().value); // "bar"
console.log(mapIter.next().value); // "baz"
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/keyed-collections.html#sec-map.prototype.values` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Nodejs |
| `values` | 38 | 12 | 20 | 25 | 8 | 38 | 20 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Map.values`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Map.prototype.entries()`
- `Map.prototype.keys()`

---

JavaScript › Reference › Standard built-in objects › Map

# Map.prototype[Symbol.iterator]()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Map/Symbol.iterator · MDN page last modified 2025-07-20

**Baseline: Widely available** — available across major browsers since September 2016.

The **`[Symbol.iterator]()`** method of `Map` instances implements the iterable protocol and allows `Map` objects to be consumed by most syntaxes expecting iterables, such as the spread syntax and `for...of` loops. It returns a map iterator object that yields the key-value pairs of the map in insertion order.

The initial value of this property is the same function object as the initial value of the `Map.prototype.entries` property.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Map.prototype[Symbol.iterator]()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const map = new Map();

map.set("0", "foo");
map.set(1, "bar");

const iterator = map[Symbol.iterator]();

for (const item of iterator) {
  console.log(item);
}
// Expected output: Array ["0", "foo"]
// Expected output: Array [1, "bar"]
```

## Syntax

js

```
map[Symbol.iterator]()
```

### Parameters

None.

### Return value

The same return value as `Map.prototype.entries()`: a new iterable iterator object that yields the key-value pairs of the map.

## Examples

### Iteration using for...of loop

Note that you seldom need to call this method directly. The existence of the `[Symbol.iterator]()` method makes `Map` objects iterable, and iterating syntaxes like the `for...of` loop automatically call this method to obtain the iterator to loop over.

js

```
const myMap = new Map();
myMap.set("0", "foo");
myMap.set(1, "bar");
myMap.set({}, "baz");

for (const entry of myMap) {
  console.log(entry);
}
// ["0", "foo"]
// [1, "bar"]
// [{}, "baz"]
```

*(code continues on the next page)*

*(code continued from the previous page)*

```
for (const [key, value] of myMap) {
  console.log(`${key}: ${value}`);
}
// 0: foo
// 1: bar
// [Object]: baz
```

### Manually hand-rolling the iterator

You may still manually call the `next()` method of the returned iterator object to achieve maximum control over the iteration process.

js

```
const myMap = new Map();
myMap.set("0", "foo");
myMap.set(1, "bar");
myMap.set({}, "baz");

const mapIter = myMap[Symbol.iterator]();

console.log(mapIter.next().value); // ["0", "foo"]
console.log(mapIter.next().value); // [1, "bar"]
console.log(mapIter.next().value); // [Object, "baz"]
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/keyed-collections.html#sec-map.prototype-%symbol.iterator% |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| [Symbol.iterator] | 38 | 12 | 36[1,2,3,4] | 25 | 10 | 38 | 36[1,2,3,4] | 25 | 10 | 3.0 | 38 | 10 | 1.0.0 | 1.0 | 0.12.0 |

1. Also from version 27–36: uses the non-standard name `@@iterator`.
2. Also from version 27–36: A placeholder property named `@@iterator` is used.
3. Also from version 17–27: uses the non-standard name `iterator`.
4. Also from version 17–27: A placeholder property named `iterator` is used.

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Map.@@iterator`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Map`
- `Map.prototype.entries()`
- `Map.prototype.keys()`
- `Map.prototype.values()`
- `Symbol.iterator`

- Iteration protocols

---

JavaScript › Reference › Standard built-in objects › Map

# Map.prototype.size

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Map/size · MDN page last modified 2026-09-05

**Baseline: Widely available** — available across major browsers since July 2015.

The **`size`** accessor property of `Map` instances returns the number of elements in this map.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Map.prototype.size". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const map = new Map();

map.set("a", "alpha");
map.set("b", "beta");
map.set("g", "gamma");

console.log(map.size);
// Expected output: 3
```

## Description

The value of `size` is an integer representing how many entries the `Map` object has. A set accessor function for `size` is `undefined`; you cannot change this property.

## Examples

### Using size

js

```
const myMap = new Map();
myMap.set("a", "alpha");
myMap.set("b", "beta");
myMap.set("g", "gamma");

console.log(myMap.size); // 3
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/keyed-collections.html#sec-get-map.prototype.size` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `size` | 38 | 12 | 19[1] | 25 | 8 | 38 | 19[2] | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |

1. From Firefox 13 to Firefox 18, the `size` property was implemented as a `Map.prototype.size()` method, this has been changed to a property in later versions conform to the ECMAScript 2015 specification.
2. From Firefox for Android 14 to Firefox for Android 18, the `size` property was implemented as a `Map.prototype.size()` method, this has been changed to a property in later versions conform to the ECMAScript 2015 specification.

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Map.size`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Map`

JavaScript › Reference › Standard built-in objects

# Math

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math · MDN page last modified 2025-07-10

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math`** namespace object contains static properties and methods for mathematical constants and functions.

`Math` works with the `Number` type. It doesn't work with `BigInt`.

## Description

Unlike most global objects, `Math` is not a constructor. You cannot use it with the new operator or invoke the `Math` object as a function. All properties and methods of `Math` are static.

> **Note:** Many `Math` functions have a precision that's *implementation-dependent*.
>
> This means that different browsers can give a different result. Even the same JavaScript engine on a different OS or architecture can give different results!

## Static properties

**`Math.E`**

Euler's number and the base of natural logarithms; approximately `2.718`.

**`Math.LN10`**

Natural logarithm of `10`; approximately `2.303`.

**`Math.LN2`**

Natural logarithm of `2`; approximately `0.693`.

**`Math.LOG10E`**

Base-10 logarithm of `E`; approximately `0.434`.

**`Math.LOG2E`**

Base-2 logarithm of `E`; approximately `1.443`.

**`Math.PI`**

Ratio of a circle's circumference to its diameter; approximately `3.14159`.

**`Math.SQRT1_2`**

Square root of ½; approximately `0.707`.

**`Math.SQRT2`**

Square root of `2`; approximately `1.414`.

**`Math[Symbol.toStringTag]`**

The initial value of the `[Symbol.toStringTag]` property is the string `"Math"`. This property is used in `Object.prototype.toString()`.

## Static methods

**`Math.abs()`**

Returns the absolute value of the input.

**`Math.acos()`**
Returns the arccosine of the input.

**`Math.acosh()`**
Returns the hyperbolic arccosine of the input.

**`Math.asin()`**
Returns the arcsine of the input.

**`Math.asinh()`**
Returns the hyperbolic arcsine of a number.

**`Math.atan()`**
Returns the arctangent of the input.

**`Math.atan2()`**
Returns the arctangent of the quotient of its arguments.

**`Math.atanh()`**
Returns the hyperbolic arctangent of the input.

**`Math.cbrt()`**
Returns the cube root of the input.

**`Math.ceil()`**
Returns the smallest integer greater than or equal to the input.

**`Math.clz32()`**
Returns the number of leading zero bits of the 32-bit integer input.

**`Math.cos()`**
Returns the cosine of the input.

**`Math.cosh()`**
Returns the hyperbolic cosine of the input.

**`Math.exp()`**
Returns $e^x$, where x is the argument, and e is Euler's number (`2.718`..., the base of the natural logarithm).

**`Math.expm1()`**
Returns subtracting `1` from `exp(x)`.

**`Math.floor()`**
Returns the largest integer less than or equal to the input.

**`Math.f16round()`**
Returns the nearest half precision float representation of the input.

**`Math.fround()`**
Returns the nearest single precision float representation of the input.

**`Math.hypot()`**
Returns the square root of the sum of squares of its arguments.

**`Math.imul()`**
Returns the result of the 32-bit integer multiplication of the inputs.

**`Math.log()`**
Returns the natural logarithm ($\log_e$; also, $\ln$) of the input.

**`Math.log10()`**
Returns the base-10 logarithm of the input.

**`Math.log1p()`**
Returns the natural logarithm ($\log_e$; also $\ln$) of `1 + x` for the number `x`.

**`Math.log2()`**
Returns the base-2 logarithm of the input.

**`Math.max()`**
Returns the largest of zero or more numbers.

**`Math.min()`**
Returns the smallest of zero or more numbers.

**`Math.pow()`**
Returns base `x` to the exponent power `y` (that is, $x^y$).

**`Math.random()`**
Returns a pseudo-random number between `0` and `1`.

**`Math.round()`**
Returns the value of the input rounded to the nearest integer.

**`Math.sign()`**
Returns the sign of the input, indicating whether it is positive, negative, or zero.

**`Math.sin()`**
Returns the sine of the input.

**`Math.sinh()`**
Returns the hyperbolic sine of the input.

**`Math.sqrt()`**
Returns the positive square root of the input.

**`Math.sumPrecise()`**
Returns the sum of a passed iterable of numbers, avoiding floating point precision loss in intermediate results.

**`Math.tan()`**
Returns the tangent of the input.

**`Math.tanh()`**
Returns the hyperbolic tangent of the input.

**`Math.trunc()`**
Returns the integer portion of the input, removing any fractional digits.

## Examples

### Converting between degrees and radians

The trigonometric functions `sin()`, `cos()`, `tan()`, `asin()`, `acos()`, `atan()`, and `atan2()` expect (and return) angles in *radians*.

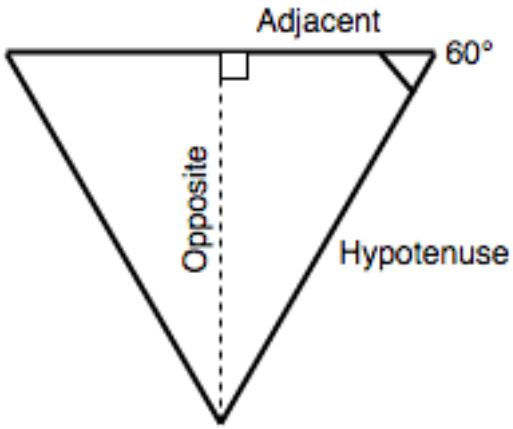
Since humans tend to think in degrees, and some functions (such as CSS transforms) can accept degrees, it is a good idea to keep functions handy that convert between the two:

```js
function degToRad(degrees) {
  return degrees * (Math.PI / 180);
}

function radToDeg(rad) {
  return rad / (Math.PI / 180);
}
```

### Calculating the height of an equilateral triangle

If we want to calculate the height of an equilateral triangle, and we know its side length is 100, we can use the formulae *length of the adjacent multiplied by the tangent of the angle is equal to the opposite.*




In JavaScript, we can do this with the following:

```js
50 * Math.tan(degToRad(60));
```

We use our `degToRad()` function to convert 60 degrees to radians, as `Math.tan()` expects an input value in radians.

### Returning a random integer between two bounds

This can be achieved with a combination of `Math.random()` and `Math.floor()`:

```js
function random(min, max) {
  const num = Math.floor(Math.random() * (max - min + 1)) + min;
  return num;
}

random(1, 10);
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math-object |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| Math | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| E | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| LN10 | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| LN2 | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| LOG10E | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| LOG2E | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| PI | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| SQRT1_2 | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| SQRT2 | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| abs | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| acos | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| acosh | 38 | 12 | 25 | 25 | 8 | 38 | 25 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |
| asin | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| asinh | 38 | 12 | 25 | 25 | 8 | 38 | 25 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |
| atan | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| atan2 | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| atanh | 38 | 12 | 25 | 25 | 8 | 38 | 25 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |
| cbrt | 38 | 12 | 25 | 25 | 8 | 38 | 25 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |
| ceil | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| clz32 | 38 | 12 | 31 | 25 | 7 | 38 | 31 | 25 | 7 | 3.0 | 38 | 7 | 1.0.0 | 1.0 | 0.12.0 |
| cos | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| cosh | 38 | 12 | 25 | 25 | 8 | 38 | 25 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |
| exp | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| expm1 | 38 | 12 | 25 | 25 | 8 | 38 | 25 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |
| f16round | 135 | 135 | 129 | 120 | 18.2 | 135 | 129 | 89 | 18.2 | 29.0 | 135 | 18.2 | 1.1.23 | 1.43 | 24.0.0 |
| floor | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| fround | 38 | 12 | 26 | 25 | 8 | 38 | 26 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |
| hypot | 38 | 12 | 27 | 25 | 8 | 38 | 27 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |
| imul | 28 | 12 | 20 | 16 | 7 | 28 | 20 | 15 | 7 | 1.5 | 4.4 | 7 | 1.0.0 | 1.0 | 0.12.0 |
| log | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| log10 | 38 | 12 | 25 | 25 | 8 | 38 | 25 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |
| log1p | 38 | 12 | 25 | 25 | 8 | 38 | 25 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |
| log2 | 38 | 12 | 25 | 25 | 8 | 38 | 25 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |
| max | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| min | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| pow | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| random | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| round | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| sign | 38 | 12 | 25 | 25 | 9 | 38 | 25 | 25 | 9 | 3.0 | 38 | 9 | 1.0.0 | 1.0 | 0.12.0 |
| sin | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Nodejs |
| `sinh` | 38 | 12 | 25 | 25 | 8 | 38 | 25 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |
| `sqrt` | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| `sumPrecise` | 147 | 147 | 137 | 131 | 26.2 | 147 | 137 | 98 | 26.2 | **No** | 147 | 26.2 | 1.2.18 | 2.7.8 | **No** |
| `tan` | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| `tanh` | 38 | 12 | 25 | 25 | 8 | 38 | 25 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |
| `trunc` | 38 | 12 | 25 | 25 | 8 | 38 | 25 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Math`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Number`

---

JavaScript › Reference › Standard built-in objects › Math

# Math.abs()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/abs · MDN page last modified 2025-07-10

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math.abs()`** static method returns the absolute value of a number.

## Try it

*▷ Interactive example on MDN: "JavaScript Demo: Math.abs()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function difference(a, b) {
  return Math.abs(a - b);
}

console.log(difference(3, 5));
// Expected output: 2

console.log(difference(5, 3));
// Expected output: 2

console.log(difference(1.23456, 7.89012));
// Expected output: 6.6555599999999995
```

## Syntax

js

```
Math.abs(x)
```

### Parameters

**x**

A number.

### Return value

The absolute value of x. If x is negative or `-0`, returns its opposite number `-x` (which is non-negative). Otherwise, returns x itself. The result is therefore always a positive number or `0`.

## Description

Because `abs()` is a static method of `Math`, you always use it as `Math.abs()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

## Examples

### Using Math.abs()

```js
Math.abs(-Infinity); // Infinity
Math.abs(-1); // 1
Math.abs(-0); // 0
Math.abs(0); // 0
Math.abs(1); // 1
Math.abs(Infinity); // Infinity
```

### Coercion of parameter

`Math.abs()` coerces its parameter to a number. Non-coercible values will become `NaN`, making `Math.abs()` also return `NaN`.

```js
Math.abs("-1"); // 1
Math.abs(-2); // 2
Math.abs(null); // 0
Math.abs(""); // 0
Math.abs([]); // 0
Math.abs([2]); // 2
Math.abs([1, 2]); // NaN
Math.abs({}); // NaN
Math.abs("string"); // NaN
Math.abs(); // NaN
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.abs |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| abs | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Math.abs`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Math.ceil()`
- `Math.floor()`
- `Math.round()`
- `Math.sign()`
- `Math.trunc()`

---

JavaScript › Reference › Standard built-in objects › Math

# Math.acos()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/acos · MDN page last modified 2025-12-29

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math.acos()`** static method returns the inverse cosine (in radians) of a number. That is,

$$\forall x \in [-1, 1],\ \texttt{Math.acos}\,(\mathrm{x}) = \arccos(x) = \text{the unique } y \in [0, \pi] \text{ such that } \cos(y) = x$$

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.acos()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
// Calculates angle of a right-angle triangle in radians
function calcAngle(adjacent, hypotenuse) {
  return Math.acos(adjacent / hypotenuse);
}

console.log(calcAngle(8, 10));
// Expected output: 0.6435011087932843

console.log(calcAngle(5, 3));
// Expected output: NaN
```

## Syntax

js

```
Math.acos(x)
```

### Parameters

**x**

A number between -1 and 1, inclusive, representing the angle's cosine value.

### Return value

The inverse cosine (angle in radians between 0 and π, inclusive) of x. If x is less than -1 or greater than 1, returns `NaN`.

## Description

Because `acos()` is a static method of `Math`, you always use it as `Math.acos()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

## Examples

### Using Math.acos()

```js
Math.acos(-2); // NaN
Math.acos(-1); // 3.141592653589793 (π)
Math.acos(0); // 1.5707963267948966 (π/2)
Math.acos(0.5); // 1.0471975511965979 (π/3)
Math.acos(1); // 0
Math.acos(2); // NaN
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.acos |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| acos | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Math.acos`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Math.asin()`
- `Math.atan()`
- `Math.atan2()`
- `Math.cos()`
- `Math.sin()`

- `Math.tan()`
- CSS `acos()` function

---

JavaScript › Reference › Standard built-in objects › Math

# Math.acosh()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/acosh · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math.acosh()`** static method returns the inverse hyperbolic cosine of a number. That is,

$$\forall x \geq 1,\ \texttt{Math.acosh}(x) = \operatorname{arcosh}(x) = \text{the unique } y \geq 0 \text{ such that } \cosh(y) = x = \ln\left(x + \sqrt{x^2 - 1}\right)$$

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.acosh()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(Math.acosh(0.999999999999));
// Expected output: NaN

console.log(Math.acosh(1));
// Expected output: 0

console.log(Math.acosh(2));
// Expected output: 1.3169578969248166

console.log(Math.acosh(2.5));
// Expected output: 1.566799236972411
```

## Syntax

js

```
Math.acosh(x)
```

### Parameters

**x**

A number greater than or equal to 1.

### Return value

The inverse hyperbolic cosine of `x`. If `x` is less than 1, returns `NaN`.

## Description

Because `acosh()` is a static method of `Math`, you always use it as `Math.acosh()`, rather than as a method of a `Math` object you created (`Math` is no constructor).

## Examples

### Using Math.acosh()

```js
Math.acosh(0); // NaN
Math.acosh(1); // 0
Math.acosh(2); // 1.3169578969248166
Math.acosh(Infinity); // Infinity
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.acosh |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| acosh | 38 | 12 | 25 | 25 | 8 | 38 | 25 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Math.acosh`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Math.acosh` in `core-js`
- es-shims polyfill of `Math.acosh`
- `Math.asinh()`
- `Math.atanh()`
- `Math.cosh()`
- `Math.sinh()`
- `Math.tanh()`

---

JavaScript › Reference › Standard built-in objects › Math

# Math.asin()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/asin · MDN page last modified 2025-12-29

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math.asin()`** static method returns the inverse sine (in radians) of a number. That is,

$$\forall x \in [-1, 1],\ \texttt{Math.asin}\,(\mathrm{x}) = \arcsin(x) = \text{the unique } y \in \left[-\frac{\pi}{2}, \frac{\pi}{2}\right] \text{ such that } \sin(y) = x$$

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.asin()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
// Calculates angle of a right-angle triangle in radians
function calcAngle(opposite, hypotenuse) {
  return Math.asin(opposite / hypotenuse);
}

console.log(calcAngle(6, 10));
// Expected output: 0.6435011087932844

console.log(calcAngle(5, 3));
// Expected output: NaN
```

## Syntax

js

```
Math.asin(x)
```

### Parameters

**x**

A number between -1 and 1, inclusive, representing the angle's sine value.

### Return value

The inverse sine (angle in radians between $-\frac{\pi}{2}$ and $\frac{\pi}{2}$, inclusive) of x. If x is less than -1 or greater than 1, returns NaN.

## Description

Because `asin()` is a static method of `Math`, you always use it as `Math.asin()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

## Examples

### Using Math.asin()

js

```
Math.asin(-2); // NaN
Math.asin(-1); // -1.5707963267948966 (-π/2)
Math.asin(-0); // -0
Math.asin(0); // 0
Math.asin(0.5); // 0.5235987755982989 (π/6)
Math.asin(1); // 1.5707963267948966 (π/2)
Math.asin(2); // NaN
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.asin |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| asin | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Math.asin`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Math.acos()`
- `Math.atan()`
- `Math.atan2()`
- `Math.cos()`
- `Math.sin()`
- `Math.tan()`
- CSS `asin()` function

---

JavaScript › Reference › Standard built-in objects › Math

# Math.asinh()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/asinh · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math.asinh()`** static method returns the inverse hyperbolic sine of a number. That is,

$$\begin{aligned}\texttt{Math.asinh}(x) &= \operatorname{arsinh}(x) = \text{the unique } y \text{ such that } \sinh(y) = x \\ &= \ln\left(x + \sqrt{x^2 + 1}\right)\end{aligned}$$

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.asinh()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(Math.asinh(1));
// Expected output: 0.881373587019543

console.log(Math.asinh(0));
// Expected output: 0

console.log(Math.asinh(-1));
// Expected output: -0.881373587019543

console.log(Math.asinh(2));
// Expected output: 1.4436354751788103
```

## Syntax

```js
Math.asinh(x)
```

### Parameters

**x**

A number.

### Return value

The inverse hyperbolic sine of `x`.

## Description

Because `asinh()` is a static method of `Math`, you always use it as `Math.asinh()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

## Examples

### Using Math.asinh()

```js
Math.asinh(-Infinity); // -Infinity
Math.asinh(-1); // -0.881373587019543
Math.asinh(-0); // -0
Math.asinh(0); // 0
Math.asinh(1); // 0.881373587019543
Math.asinh(Infinity); // Infinity
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.asinh |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| asinh | 38 | 12 | 25 | 25 | 8 | 38 | 25 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Math.asinh`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Math.asinh` in `core-js`
- es-shims polyfill of `Math.asinh`
- `Math.acosh()`
- `Math.atanh()`
- `Math.cosh()`
- `Math.sinh()`
- `Math.tanh()`

---



# Math.atan()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/atan · MDN page last modified 2025-12-29

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math.atan()`** static method returns the inverse tangent (in radians) of a number, that is

$$\texttt{Math.atan}(\mathrm{x}) = \arctan(x) = \text{the unique } y \in \left[-\frac{\pi}{2}, \frac{\pi}{2}\right] \text{ such that } \tan(y) = x$$

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.atan()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
// Calculates angle of a right-angle triangle in radians
function calcAngle(opposite, adjacent) {
  return Math.atan(opposite / adjacent);
}

console.log(calcAngle(8, 10));
// Expected output: 0.6747409422235527

console.log(calcAngle(5, 3));
// Expected output: 1.0303768265243125
```

## Syntax

js

```
Math.atan(x)
```

### Parameters

**x**

A number.

### Return value

The inverse tangent (angle in radians between $-\frac{\pi}{2}$ and $\frac{\pi}{2}$, inclusive) of x. If x is `Infinity`, it returns $\frac{\pi}{2}$. If x is `-Infinity`, it returns $-\frac{\pi}{2}$.

## Description

Because `atan()` is a static method of `Math`, you always use it as `Math.atan()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

## Examples

### Using Math.atan()

```js
Math.atan(-Infinity); // -1.5707963267948966 (-π/2)
Math.atan(-0); // -0
Math.atan(0); // 0
Math.atan(1); // 0.7853981633974483  (π/4)
Math.atan(Infinity); // 1.5707963267948966  (π/2)

// The angle that the line (0,0) -- (x,y) forms with the x-axis in a Cartesian coordinate system
const theta = (x, y) => Math.atan(y / x);
```

Note that you may want to avoid the `theta` function and use `Math.atan2()` instead, which has a wider range (between -π and π) and avoids outputting `NaN` for cases such as when `x` is `0`.

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.atan |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| atan | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Math.atan`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Math.acos()`
- `Math.asin()`
- `Math.atan2()`
- `Math.cos()`
- `Math.sin()`
- `Math.tan()`
- CSS `atan()` function

JavaScript › Reference › Standard built-in objects › Math

# Math.atan2()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/atan2 · MDN page last modified 2025-12-29

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math.atan2()`** static method returns the angle in the plane (in radians) between the positive x-axis and the ray from (0, 0) to the point (x, y), for `Math.atan2(y, x)`.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.atan2()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function calcAngleDegrees(x, y) {
  return (Math.atan2(y, x) * 180) / Math.PI;
}

console.log(calcAngleDegrees(5, 5));
// Expected output: 45

console.log(calcAngleDegrees(10, 10));
// Expected output: 45

console.log(calcAngleDegrees(0, 10));
// Expected output: 90
```

## Syntax

js

```
Math.atan2(y, x)
```

### Parameters

**y**

The y coordinate of the point.

**x**

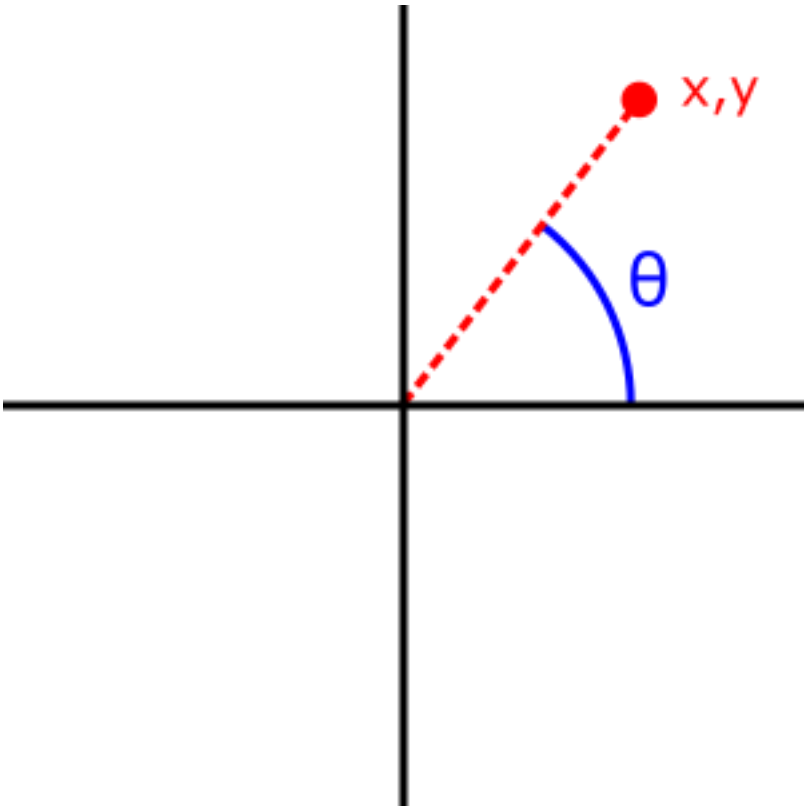
The x coordinate of the point.

### Return value

The angle in radians (between -π and π, inclusive) between the positive x-axis and the ray from (0, 0) to the point (x, y).

## Description

The `Math.atan2()` method measures the counterclockwise angle θ, in radians, between the positive x-axis and the point `(x, y)`. Note that the arguments to this function pass the y-coordinate first and the x-coordinate second.




`Math.atan2()` is passed separate `x` and `y` arguments, while `Math.atan()` is passed the ratio of those two arguments. `Math.atan2(y, x)` differs from `Math.atan(y / x)` in the following cases:

| `x` | `y` | `Math.atan2(y, x)` | `Math.atan(y / x)` |
|---|---|---|---|
| `Infinity` | `Infinity` | π / 4 | `NaN` |
| `Infinity` | `-Infinity` | -π / 4 | `NaN` |
| `-Infinity` | `Infinity` | 3π / 4 | `NaN` |
| `-Infinity` | `-Infinity` | -3π / 4 | `NaN` |
| 0 | 0 | 0 | `NaN` |
| 0 | -0 | -0 | `NaN` |
| < 0 (including `-0`) | 0 | π | 0 |
| < 0 (including `-0`) | -0 | -π | 0 |
| `-Infinity` | > 0 | π | -0 |
| -0 | > 0 | π / 2 | -π / 2 |
| `-Infinity` | < 0 | -π | 0 |
| -0 | < 0 | -π / 2 | π / 2 |

In addition, for points in the second and third quadrants (`x < 0`), `Math.atan2()` would output an angle less than $-\frac{\pi}{2}$ or greater than $\frac{\pi}{2}$.

Because `atan2()` is a static method of `Math`, you always use it as `Math.atan2()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

## Examples

### Using Math.atan2()

```js
Math.atan2(90, 15); // 1.4056476493802699
Math.atan2(15, 90); // 0.16514867741462683
```

### Difference between Math.atan2(y, x) and Math.atan(y / x)

The following script prints all inputs that produce a difference between `Math.atan2(y, x)` and `Math.atan(y / x)`.

```js
const formattedNumbers = new Map([
  [-Math.PI, "-π"],
  [(-3 * Math.PI) / 4, "-3π/4"],
  [-Math.PI / 2, "-π/2"],
  [-Math.PI / 4, "-π/4"],
  [Math.PI / 4, "π/4"],
  [Math.PI / 2, "π/2"],
  [(3 * Math.PI) / 4, "3π/4"],
  [Math.PI, "π"],
  [-Infinity, "-∞"],
  [Infinity, "∞"],
]);

function format(template, ...args) {
  return String.raw(
    { raw: template },
    ...args.map((num) =>
      (Object.is(num, -0)
        ? "-0"
        : (formattedNumbers.get(num) ?? String(num))
      ).padEnd(5),
    ),
  );
}

console.log(`| x     | y     | atan2 | atan  |
|-------|-------|-------|-------|`);

for (const x of [-Infinity, -1, -0, 0, 1, Infinity]) {
  for (const y of [-Infinity, -1, -0, 0, 1, Infinity]) {
    const atan2 = Math.atan2(y, x);
    const atan = Math.atan(y / x);
    if (!Object.is(atan2, atan)) {
      console.log(format`| ${x} | ${y} | ${atan2} | ${atan} |`);
    }
  }
}
```

The output is:

```
| x     | y     | atan2 | atan  |
|-------|-------|-------|-------|
| -∞    | -∞    | -3π/4 | NaN   |
| -∞    | -1    | -π    | 0     |
```

(code continues on the next page)

*(code continued from the previous page)*

```
| -∞    | -0    | -π    | 0    |
| -∞    | 0     | π     | -0   |
| -∞    | 1     | π     | -0   |
| -∞    | ∞     | 3π/4  | NaN  |
| -1    | -∞    | -π/2  | π/2  |
| -1    | -1    | -3π/4 | π/4  |
| -1    | -0    | -π    | 0    |
| -1    | 0     | π     | -0   |
| -1    | 1     | 3π/4  | -π/4 |
| -1    | ∞     | π/2   | -π/2 |
| -0    | -∞    | -π/2  | π/2  |
| -0    | -1    | -π/2  | π/2  |
| -0    | -0    | -π    | NaN  |
| -0    | 0     | π     | NaN  |
| -0    | 1     | π/2   | -π/2 |
| -0    | ∞     | π/2   | -π/2 |
| 0     | -0    | -0    | NaN  |
| 0     | 0     | 0     | NaN  |
| ∞     | -∞    | -π/4  | NaN  |
| ∞     | ∞     | π/4   | NaN  |
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.atan2` |

## Browser compatibility

| | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| Feature | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `atan2` | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Math.atan2`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Math.acos()`
- `Math.asin()`
- `Math.atan()`
- `Math.cos()`
- `Math.sin()`
- `Math.tan()`
- CSS `atan2()` function

JavaScript › Reference › Standard built-in objects › Math

# Math.atanh()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/atanh · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math.atanh()`** static method returns the inverse hyperbolic tangent of a number. That is,

$$\forall x \in (-1, 1),\ \texttt{Math.atanh}(x) = \operatorname{artanh}(x) = \text{the unique } y \text{ such that } \tanh(y) = x = \frac{1}{2}\ln\left(\frac{1+x}{1-x}\right)$$

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.atanh()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(Math.atanh(-1));
// Expected output: -Infinity

console.log(Math.atanh(0));
// Expected output: 0

console.log(Math.atanh(0.5));
// Expected output: 0.549306144334055 (approximately)

console.log(Math.atanh(1));
// Expected output: Infinity
```

## Syntax

js

```
Math.atanh(x)
```

### Parameters

**x**

A number between -1 and 1, inclusive.

### Return value

The inverse hyperbolic tangent of x. If x is 1, returns `Infinity`. If x is -1, returns `-Infinity`. If x is less than -1 or greater than 1, returns `NaN`.

## Description

Because `atanh()` is a static method of `Math`, you always use it as `Math.atanh()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

## Examples

### Using Math.atanh()

```js
Math.atanh(-2); // NaN
Math.atanh(-1); // -Infinity
Math.atanh(-0); // -0
Math.atanh(0); // 0
Math.atanh(0.5); // 0.5493061443340548
Math.atanh(1); // Infinity
Math.atanh(2); // NaN
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.atanh |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| atanh | 38 | 12 | 25 | 25 | 8 | 38 | 25 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Math.atanh`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Math.atanh` in `core-js`
- es-shims polyfill of `Math.atanh`
- `Math.acosh()`
- `Math.asinh()`
- `Math.cosh()`
- `Math.sinh()`
- `Math.tanh()`

JavaScript › Reference › Standard built-in objects › Math

# Math.cbrt()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/cbrt · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math.cbrt()`** static method returns the cube root of a number. That is

$$\texttt{Math.cbrt}\,(\texttt{x}) = \sqrt[3]{x} = \text{the unique } y \text{ such that } y^3 = x$$

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.cbrt()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(Math.cbrt(-1));
// Expected output: -1

console.log(Math.cbrt(1));
// Expected output: 1

console.log(Math.cbrt(Infinity));
// Expected output: Infinity

console.log(Math.cbrt(64));
// Expected output: 4
```

## Syntax

js

```
Math.cbrt(x)
```

### Parameters

**x**

A number.

### Return value

The cube root of `x`.

## Description

Because `cbrt()` is a static method of `Math`, you always use it as `Math.cbrt()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

## Examples

### Using Math.cbrt()

```js
Math.cbrt(-Infinity); // -Infinity
Math.cbrt(-1); // -1
Math.cbrt(-0); // -0
Math.cbrt(0); // 0
Math.cbrt(1); // 1
Math.cbrt(2); // 1.2599210498948732
Math.cbrt(Infinity); // Infinity
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.cbrt |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| cbrt | 38 | 12 | 25 | 25 | 8 | 38 | 25 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Math.cbrt`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Math.cbrt` in `core-js`
- es-shims polyfill of `Math.cbrt`
- `Math.pow()`
- `Math.sqrt()`

---

JavaScript › Reference › Standard built-in objects › Math

# Math.ceil()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/ceil · MDN page last modified 2025-07-10

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math.ceil()`** static method always rounds up and returns the smallest integer greater than or equal to a given number.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.ceil()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(Math.ceil(0.95));
// Expected output: 1

console.log(Math.ceil(4));
// Expected output: 4

console.log(Math.ceil(7.004));
// Expected output: 8

console.log(Math.ceil(-7.004));
// Expected output: -7
```

## Syntax

js

```
Math.ceil(x)
```

### Parameters

**x**

A number.

### Return value

The smallest integer greater than or equal to `x`. It's the same value as `-Math.floor(-x)`.

## Description

Because `ceil()` is a static method of `Math`, you always use it as `Math.ceil()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

## Examples

### Using Math.ceil()

js

```
Math.ceil(-Infinity); // -Infinity
Math.ceil(-7.004); // -7
Math.ceil(-4); // -4
Math.ceil(-0.95); // -0
Math.ceil(-0); // -0
Math.ceil(0); // 0
Math.ceil(0.95); // 1
Math.ceil(4); // 4
Math.ceil(7.004); // 8
Math.ceil(Infinity); // Infinity
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.ceil` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `ceil` | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Math.ceil`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Math.abs()`
- `Math.floor()`
- `Math.round()`
- `Math.sign()`
- `Math.trunc()`

---

JavaScript › Reference › Standard built-in objects › Math

# Math.clz32()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/clz32 · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math.clz32()`** static method returns the number of leading zero bits in the 32-bit binary representation of a number.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.clz32()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
// 00000000000000000000000000000001
console.log(Math.clz32(1));
// Expected output: 31

// 00000000000000000000000000000100
console.log(Math.clz32(4));
// Expected output: 29

// 00000000000000000000001111101000
console.log(Math.clz32(1000));
// Expected output: 22
```

## Syntax

```js
Math.clz32(x)
```

### Parameters

**x**

A number.

### Return value

The number of leading zero bits in the 32-bit binary representation of `x`.

## Description

`clz32` is short for **C**ount**L**eading**Z**eros**32**.

If `x` is not a number, it will be converted to a number first, then converted to a 32-bit unsigned integer.

If the converted 32-bit unsigned integer is `0`, `32` is returned, because all bits are `0`. If the most significant bit is `1` (i.e., the number is greater than or equal to $2^{31}$), `0` is returned.

This function is particularly useful for systems that compile to JS, like Emscripten.

## Examples

### Using Math.clz32()

```js
Math.clz32(1); // 31
Math.clz32(1000); // 22
Math.clz32(); // 32

const stuff = [
  NaN,
  Infinity,
  -Infinity,
  0,
  -0,
  false,
  null,
  undefined,
  "foo",
  {},
  [],
];
stuff.every((n) => Math.clz32(n) === 32); // true

Math.clz32(true); // 31
Math.clz32(3.5); // 30
```

### Implementing Count Leading Ones and beyond

At present, there is no `Math.clon` for "Count Leading Ones" (named "clon", not "clo", because "clo" and "clz" are too similar especially for non-English-speaking people). However, a `clon` function can easily be created by inverting the bits of a number and passing the result to `Math.clz32`. Doing this will work

because the inverse of 1 is 0 and vice versa. Thus, inverting the bits will inverse the measured quantity of 0's (from `Math.clz32`), thereby making `Math.clz32` count the number of ones instead of counting the number of zeros.

Consider the following 32-bit word:

```js
const a = 32776; // 00000000000000001000000000001000 (16 leading zeros)
Math.clz32(a); // 16

const b = ~32776; // 11111111111111110111111111110111 (32776 inverted, 0 leading zeros)
Math.clz32(b); // 0 (this is equal to how many leading one's there are in a)
```

Using this logic, a `clon` function can be created as follows:

```js
const clz = Math.clz32;

function clon(integer) {
  return clz(~integer);
}
```

Further, this technique could be extended to create a jumpless "Count Trailing Zeros" function, as seen below. The `ctrz` function takes a bitwise AND of the integer with its two's complement. By how two's complement works, all trailing zeros will be converted to ones, and then when adding 1, it would be carried over until the first `0` (which was originally a `1`) is reached. All bits higher than this one stay the same and are inverses of the original integer's bits. Therefore, when doing bitwise AND with the original integer, all higher bits become `0`, which can be counted with `clz`. The number of trailing zeros, plus the first `1` bit, plus the leading bits that were counted by `clz`, total to 32.

```js
function ctrz(integer) {
  integer >>>= 0; // coerce to Uint32
  if (integer === 0) {
    // skipping this step would make it return -1
    return 32;
  }
  integer &= -integer; // equivalent to `int = int & (~int + 1)`
  return 31 - clz(integer);
}
```

Then we can define a "Count Trailing Ones" function like so:

```js
function ctron(integer) {
  return ctrz(~integer);
}
```

These helper functions can be made into an asm.js module for a potential performance improvement.

```js
const countTrailsMethods = (function (stdlib, foreign, heap) {
  "use asm";
  const clz = stdlib.Math.clz32;

  // count trailing zeros
  function ctrz(integer) {
    integer = integer | 0; // coerce to an integer
```

*(code continues on the next page)*

*(code continued from the previous page)*

```
    if ((integer | 0) == 0) {
      // skipping this step would make it return -1
      return 32;
    }
    // Note: asm.js doesn't have compound assignment operators such as &=
    integer = integer & -integer; // equivalent to `int = int & (~int + 1)`
    return (31 - clz(integer)) | 0;
  }

  // count trailing ones
  function ctron(integer) {
    integer = integer | 0; // coerce to an integer
    return ctrz(~integer) | 0;
  }

  // asm.js demands plain objects:
  return { ctrz: ctrz, ctron: ctron };
})(window, null, null);

const { ctrz, ctron } = countTrailsMethods;
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.clz32` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `clz32` | 38 | 12 | 31 | 25 | 7 | 38 | 31 | 25 | 7 | 3.0 | 38 | 7 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Math.clz32`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Math.clz32` in `core-js`
- es-shims polyfill of `Math.clz32`
- `Math`
- `Math.imul`

JavaScript › Reference › Standard built-in objects › Math

# Math.cos()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/cos · MDN page last modified 2025-12-29

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math.cos()`** static method returns the cosine of a number in radians.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.cos()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function getCircleX(radians, radius) {
  return Math.cos(radians) * radius;
}

console.log(getCircleX(1, 10));
// Expected output: 5.403023058681398

console.log(getCircleX(2, 10));
// Expected output: -4.161468365471424

console.log(getCircleX(Math.PI, 10));
// Expected output: -10
```

## Syntax

js

```
Math.cos(x)
```

### Parameters

**x**

A number representing an angle in radians.

### Return value

The cosine of x, between -1 and 1, inclusive. If x is `Infinity`, `-Infinity`, or `NaN`, returns `NaN`.

## Description

Because `cos()` is a static method of `Math`, you always use it as `Math.cos()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

## Examples

### Using Math.cos()

```js
Math.cos(-Infinity); // NaN
Math.cos(-0); // 1
Math.cos(0); // 1
Math.cos(1); // 0.5403023058681398
Math.cos(Math.PI); // -1
Math.cos(2 * Math.PI); // 1
Math.cos(Infinity); // NaN
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.cos |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| cos | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Math.cos`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Math.acos()`
- `Math.asin()`
- `Math.atan()`
- `Math.atan2()`
- `Math.sin()`
- `Math.tan()`
- CSS `cos()` function

---

JavaScript › Reference › Standard built-in objects › Math

# Math.cosh()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/cosh · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math.cosh()`** static method returns the hyperbolic cosine of a number. That is,

$$\texttt{Math.cosh}\,(\texttt{x}) = \cosh(x) = \frac{\mathrm{e}^{x} + \mathrm{e}^{-x}}{2}$$

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.cosh()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(Math.cosh(0));
// Expected output: 1

console.log(Math.cosh(1));
// Expected output: 1.543080634815244 (approximately)

console.log(Math.cosh(-1));
// Expected output: 1.543080634815244 (approximately)

console.log(Math.cosh(2));
// Expected output: 3.7621956910836314
```

## Syntax

js

```
Math.cosh(x)
```

### Parameters

**x**

A number.

### Return value

The hyperbolic cosine of x.

## Description

Because `cosh()` is a static method of `Math`, you always use it as `Math.cosh()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

## Examples

### Using Math.cosh()

js

```
Math.cosh(-Infinity); // Infinity
Math.cosh(-1); // 1.5430806348152437
Math.cosh(-0); // 1
Math.cosh(0); // 1
Math.cosh(1); // 1.5430806348152437
Math.cosh(Infinity); // Infinity
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.cosh |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| cosh | 38 | 12 | 25 | 25 | 8 | 38 | 25 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Math.cosh`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Math.cosh` in `core-js`
- es-shims polyfill of `Math.cosh`
- `Math.acosh()`
- `Math.asinh()`
- `Math.atanh()`
- `Math.sinh()`
- `Math.tanh()`

---

JavaScript › Reference › Standard built-in objects › Math

# Math.exp()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/exp · MDN page last modified 2025-07-10

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math.exp()`** static method returns e raised to the power of a number. That is

$$\texttt{Math.exp}(x) = e^x$$

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.exp()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(Math.exp(0));
// Expected output: 1

console.log(Math.exp(1));
// Expected output: 2.718281828459 (approximately)

console.log(Math.exp(-1));
// Expected output: 0.36787944117144233

console.log(Math.exp(2));
// Expected output: 7.38905609893065
```

## Syntax

```js
Math.exp(x)
```

### Parameters

**x**

A number.

### Return value

A nonnegative number representing $e^x$, where e is the base of the natural logarithm.

## Description

Because `exp()` is a static method of `Math`, you always use it as `Math.exp()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

Beware that e to the power of a number very close to 0 will be very close to 1 and suffer from loss of precision. In this case, you may want to use `Math.expm1` instead, and obtain a much higher-precision fractional part of the answer.

## Examples

### Using Math.exp()

```js
Math.exp(-Infinity); // 0
Math.exp(-1); // 0.36787944117144233
Math.exp(0); // 1
Math.exp(1); // 2.718281828459045
Math.exp(Infinity); // Infinity
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.exp |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| exp | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Math.exp`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Math.E`
- `Math.expm1()`
- `Math.log()`
- `Math.log10()`
- `Math.log1p()`
- `Math.log2()`
- `Math.pow()`

---



# Math.expm1()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/expm1 · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math.expm1()`** static method returns e raised to the power of a number, subtracted by 1. That is

$$\texttt{Math.expm1}\,(x) = e^x - 1$$

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.expm1()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(Math.expm1(0));
// Expected output: 0

console.log(Math.expm1(1));
// Expected output: 1.718281828459045

console.log(Math.expm1(-1));
// Expected output: -0.6321205588285577

console.log(Math.expm1(2));
// Expected output: 6.38905609893065
```

## Syntax

js

```
Math.expm1(x)
```

### Parameters

**x**

A number.

### Return value

A number representing $e^x$ - 1, where e is the base of the natural logarithm.

## Description

For very small values of *x*, adding 1 can reduce or eliminate precision. The double floats used in JS give you about 15 digits of precision. 1 + 1e-15 = 1.000000000000001, but 1 + 1e-16 = 1.000000000000000 and therefore exactly 1.0 in that arithmetic, because digits past 15 are rounded off.

When you calculate $e^x$, where x is a number very close to 0, you should get an answer very close to 1 + x because: $\lim_{x \to 0} \frac{e^x - 1}{x} = 1$. If you calculate `Math.exp(1.1111111111e-15) - 1`, you should get an answer close to `1.1111111111e-15`. Instead, due to the highest significant figure in the result of `Math.exp` being the units digit `1`, the final value ends up being `1.1102230246251565e-15`, with only 3 correct digits. If you calculate `Math.expm1(1.1111111111e-15)` instead, you will get a much more accurate answer, `1.1111111111000007e-15`, with 11 correct digits of precision.

Because `expm1()` is a static method of `Math`, you always use it as `Math.expm1()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

## Examples

### Using Math.expm1()

```js
Math.expm1(-Infinity); // -1
Math.expm1(-1); // -0.6321205588285577
Math.expm1(-0); // -0
Math.expm1(0); // 0
Math.expm1(1); // 1.718281828459045
Math.expm1(Infinity); // Infinity
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.expm1 |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| expm1 | 38 | 12 | 25 | 25 | 8 | 38 | 25 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Math.expm1`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Math.expm1` in `core-js`
- `Math.E`
- `Math.exp()`

- Math.log()
- Math.log10()
- Math.log1p()
- Math.log2()
- Math.pow()

---

JavaScript › Reference › Standard built-in objects › Math

# Math.f16round()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/f16round · MDN page last modified 2026-10-26

**Baseline 2025: Newly available** — works across the latest browser versions since April 2025; may not work in older ones.

The **`Math.f16round()`** static method returns the nearest 16-bit half precision float representation of a number.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.f16round()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(Math.f16round(5.5));
// Expected output: 5.5

console.log(Math.f16round(5.05));
// Expected output: 5.05078125

console.log(Math.f16round(5));
// Expected output: 5

console.log(Math.f16round(-5.05));
// Expected output: -5.05078125
```

## Syntax

js

```
Math.f16round(doubleFloat)
```

### Parameters

**`doubleFloat`**

A number.

### Return value

The nearest 16-bit half precision float representation of `doubleFloat`.

## Description

`Math.f16round` is the 16-bit counterpart of `Math.fround()`. It is intended to smooth some rough edges when interacting with float16 numbers, such as when reading from a `Float16Array`. Internally, JavaScript continues to treat the number as a 64-bit float, it just performs a "round to even" on the 10th bit of the mantissa, and sets all following mantissa bits to `0`. If the number is outside the range of a 16-bit float, `Infinity` or `-Infinity` is returned.

Because `f16round()` is a static method of `Math`, you always use it as `Math.f16round()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

## Examples

### Using Math.f16round()

The number 1.5 can be precisely represented in the binary numeral system, and is identical in 16-bit and 64-bit:

```js
Math.f16round(1.5); // 1.5
Math.f16round(1.5) === 1.5; // true
```

However, the number 1.337 cannot be precisely represented in the binary numeral system, so it differs in 16-bit and 64-bit:

```js
Math.f16round(1.337); // 1.3369140625
Math.f16round(1.337) === 1.337; // false
```

100000 is too big for a 16-bit float, so `Infinity` is returned:

```js
Math.f16round(100000); // Infinity
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.f16round` |

## Browser compatibility

| | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| Feature | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `f16round` | 135 | 135 | 129 | 120 | 18.2 | 135 | 129 | 89 | 18.2 | 29.0 | 135 | 18.2 | 1.1.23 | 1.43 | 24.0.0 |

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Math.f16round`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Math.f16round` in `core-js`
- es-shims polyfill of `Math.f16round`
- `Math.fround()`
- `Math.round()`

JavaScript › Reference › Standard built-in objects › Math

# Math.floor()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/floor · MDN page last modified 2025-07-10

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math.floor()`** static method always rounds down and returns the largest integer less than or equal to a given number.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.floor()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(Math.floor(5.95));
// Expected output: 5

console.log(Math.floor(5.05));
// Expected output: 5

console.log(Math.floor(5));
// Expected output: 5

console.log(Math.floor(-5.05));
// Expected output: -6
```

## Syntax

js

```
Math.floor(x)
```

### Parameters

**x**

A number.

### Return value

The largest integer smaller than or equal to `x`. It's the same value as `-Math.ceil(-x)`.

## Description

Because `floor()` is a static method of `Math`, you always use it as `Math.floor()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

## Examples

### Using Math.floor()

```js
Math.floor(-Infinity); // -Infinity
Math.floor(-45.95); // -46
Math.floor(-45.05); // -46
Math.floor(-0); // -0
Math.floor(0); // 0
Math.floor(4); // 4
Math.floor(45.05); // 45
Math.floor(45.95); // 45
Math.floor(Infinity); // Infinity
```

### Decimal adjustment

In this example, we implement a method called `decimalAdjust()` that is an enhancement method of `Math.floor()`, `Math.ceil()`, and `Math.round()`. While the three `Math` functions always adjust the input to the units digit, `decimalAdjust` accepts an `exp` parameter that specifies the number of digits to the left of the decimal point to which the number should be adjusted. For example, `-1` means it would leave one digit after the decimal point (as in "$\times 10^{-1}$"). In addition, it allows you to select the means of adjustment — `round`, `floor`, or `ceil` — through the `type` parameter.

It does so by multiplying the number by a power of 10, then rounding the result to the nearest integer, then dividing by the power of 10. To better preserve precision, it takes advantage of Number's `toString()` method, which represents large or small numbers in scientific notation (like `6.02e23`).

```js
/**
 * Adjusts a number to the specified digit.
 *
 * @param {"round" | "floor" | "ceil"} type The type of adjustment.
 * @param {number} value The number.
 * @param {number} exp The exponent (the 10 logarithm of the adjustment base).
 * @returns {number} The adjusted value.
 */
function decimalAdjust(type, value, exp) {
  type = String(type);
  if (!["round", "floor", "ceil"].includes(type)) {
    throw new TypeError(
      "The type of decimal adjustment must be one of 'round', 'floor', or 'ceil'.",
    );
  }
  exp = Number(exp);
  value = Number(value);
  if (exp % 1 !== 0 || Number.isNaN(value)) {
    return NaN;
  } else if (exp === 0) {
    return Math[type](value);
  }
  const [magnitude, exponent = 0] = value.toString().split("e");
  const adjustedValue = Math[type](`${magnitude}e${exponent - exp}`);
  // Shift back
  const [newMagnitude, newExponent = 0] = adjustedValue.toString().split("e");
  return Number(`${newMagnitude}e${Number(newExponent) + exp}`);
}
```

*(code continues on the next page)*

(code continued from the previous page)

```
// Decimal round
const round10 = (value, exp) => decimalAdjust("round", value, exp);
// Decimal floor
const floor10 = (value, exp) => decimalAdjust("floor", value, exp);
// Decimal ceil
const ceil10 = (value, exp) => decimalAdjust("ceil", value, exp);

// Round
round10(55.55, -1); // 55.6
round10(55.549, -1); // 55.5
round10(55, 1); // 60
round10(54.9, 1); // 50
round10(-55.55, -1); // -55.5
round10(-55.551, -1); // -55.6
round10(-55, 1); // -50
round10(-55.1, 1); // -60
// Floor
floor10(55.59, -1); // 55.5
floor10(59, 1); // 50
floor10(-55.51, -1); // -55.6
floor10(-51, 1); // -60
// Ceil
ceil10(55.51, -1); // 55.6
ceil10(51, 1); // 60
ceil10(-55.59, -1); // -55.5
ceil10(-59, 1); // -50
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.floor` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `floor` | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Math.floor`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Math.abs()`
- `Math.ceil()`
- `Math.round()`
- `Math.sign()`
- `Math.trunc()`

JavaScript › Reference › Standard built-in objects › Math

# Math.fround()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/fround · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math.fround()`** static method returns the nearest 32-bit single precision float representation of a number.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.fround()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(Math.fround(5.5));
// Expected output: 5.5

console.log(Math.fround(5.05));
// Expected output: 5.050000190734863

console.log(Math.fround(5));
// Expected output: 5

console.log(Math.fround(-5.05));
// Expected output: -5.050000190734863
```

## Syntax

js

```
Math.fround(doubleFloat)
```

### Parameters

**`doubleFloat`**

A number.

### Return value

The nearest 32-bit single precision float representation of `doubleFloat`.

## Description

JavaScript uses 64-bit double floating-point numbers internally, which offer a very high precision. However, sometimes you may be working with 32-bit floating-point numbers, for example if you are reading values from a `Float32Array`. This can create confusion: checking a 64-bit float and a 32-bit float for equality may fail even though the numbers are seemingly identical.

To solve this, `Math.fround()` can be used to cast the 64-bit float to a 32-bit float. Internally, JavaScript continues to treat the number as a 64-bit float, it just performs a "round to even" on the 23rd bit of the mantissa, and sets all following mantissa bits to `0`. If the number is outside the range of a 32-bit float, `Infinity` or `-Infinity` is returned.

Because `fround()` is a static method of `Math`, you always use it as `Math.fround()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

## Examples

### Using Math.fround()

The number 1.5 can be precisely represented in the binary numeral system, and is identical in 32-bit and 64-bit:

```js
Math.fround(1.5); // 1.5
Math.fround(1.5) === 1.5; // true
```

However, the number 1.337 cannot be precisely represented in the binary numeral system, so it differs in 32-bit and 64-bit:

```js
Math.fround(1.337); // 1.3370000123977661
Math.fround(1.337) === 1.337; // false
```

$2^{150}$ is too big for a 32-bit float, so `Infinity` is returned:

```js
2 ** 150; // 1.42724769270596e+45
Math.fround(2 ** 150); // Infinity
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.fround |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `fround` | 38 | 12 | 26 | 25 | 8 | 38 | 26 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Math.fround`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Math.fround` in `core-js`
- es-shims polyfill of `Math.fround`
- `Math.round()`

JavaScript › Reference › Standard built-in objects › Math

# Math.hypot()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/hypot · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math.hypot()`** static method returns the square root of the sum of squares of its arguments. That is,

$$\texttt{Math.hypot}\,(v_1, v_2, \ \ldots, v_n) = \sqrt{\sum_{i=1}^{n} v_i^2} = \sqrt{v_1^2 + v_2^2 + \ \ldots \ + v_n^2}$$

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.hypot()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(Math.hypot(3, 4));
// Expected output: 5

console.log(Math.hypot(5, 12));
// Expected output: 13

console.log(Math.hypot(3, 4, 5));
// Expected output: 7.0710678118654755

console.log(Math.hypot(-5));
// Expected output: 5
```

## Syntax

js

```
Math.hypot()
Math.hypot(value1)
Math.hypot(value1, value2)
Math.hypot(value1, value2, /* …, */ valueN)
```

### Parameters

**`value1, ..., valueN`**

Numbers.

### Return value

The square root of the sum of squares of the given arguments. Returns `Infinity` if any of the arguments is ±Infinity. Otherwise, if at least one of the arguments is or is converted to `NaN`, returns `NaN`. Returns `0` if no arguments are given or all arguments are ±0.

## Description

Calculating the hypotenuse of a right triangle, or the magnitude of a complex number, uses the formula `Math.sqrt(v1*v1 + v2*v2)`, where v1 and v2 are the lengths of the triangle's legs, or the complex number's real and complex components. The corresponding distance in 2 or more dimensions can be

calculated by adding more squares under the square root: `Math.sqrt(v1*v1 + v2*v2 + v3*v3 + v4*v4)`.

This function makes this calculation easier and faster; you call `Math.hypot(v1, v2)`, or `Math.hypot(v1, /* …, */, vN)`.

`Math.hypot` also avoids overflow/underflow problems if the magnitude of your numbers is very large. The largest number you can represent in JS is `Number.MAX_VALUE`, which is around $10^{308}$. If your numbers are larger than about $10^{154}$, taking the square of them will result in Infinity. For example, `Math.sqrt(1e200*1e200 + 1e200*1e200) = Infinity`. If you use `hypot()` instead, you get a better answer: `Math.hypot(1e200, 1e200) = 1.4142...e+200`. This is also true with very small numbers. `Math.sqrt(1e-200*1e-200 + 1e-200*1e-200) = 0`, but `Math.hypot(1e-200, 1e-200) = 1.4142...e-200`.

With one argument, `Math.hypot()` is equivalent to `Math.abs()`. `Math.hypot.length` is 2, which weakly signals that it's designed to handle at least two parameters.

Because `hypot()` is a static method of `Math`, you always use it as `Math.hypot()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

## Examples

### Using Math.hypot()

```js
Math.hypot(3, 4); // 5
Math.hypot(3, 4, 5); // 7.0710678118654755
Math.hypot(); // 0
Math.hypot(NaN); // NaN
Math.hypot(NaN, Infinity); // Infinity
Math.hypot(3, 4, "foo"); // NaN, since +'foo' => NaN
Math.hypot(3, 4, "5"); // 7.0710678118654755, +'5' => 5
Math.hypot(-3); // 3, the same as Math.abs(-3)
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.hypot |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| hypot | 38 | 12 | 27 | 25 | 8 | 38 | 27 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Math.hypot`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Math.hypot` in `core-js`
- `Math.abs()`
- `Math.pow()`
- `Math.sqrt()`

---

JavaScript › Reference › Standard built-in objects › Math

# Math.imul()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/imul · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math.imul()`** static method returns the result of the C-like 32-bit multiplication of the two parameters.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.imul()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(Math.imul(3, 4));
// Expected output: 12

console.log(Math.imul(-5, 12));
// Expected output: -60

console.log(Math.imul(0xffffffff, 5));
// Expected output: -5

console.log(Math.imul(0xfffffffe, 5));
// Expected output: -10
```

## Syntax

js

```
Math.imul(a, b)
```

### Parameters

**a**

First number.

**b**

Second number.

### Return value

The result of the C-like 32-bit multiplication of the given arguments.

## Description

`Math.imul()` allows for 32-bit integer multiplication with C-like semantics. This feature is useful for projects like Emscripten.

Because `imul()` is a static method of `Math`, you always use it as `Math.imul()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

If you use normal JavaScript floating point numbers in `imul()`, you will experience a degrade in performance. This is because of the costly conversion from a floating point to an integer for multiplication, and then converting the multiplied integer back into a floating point. However, with asm.js, which allows JIT-optimizers to more confidently use integers in JavaScript, multiplying two numbers stored internally as integers (which is only possible with asm.js) with `imul()` could be potentially more performant.

## Examples

### Using Math.imul()

```js
Math.imul(2, 4); // 8
Math.imul(-1, 8); // -8
Math.imul(-2, -2); // 4
Math.imul(0xffffffff, 5); // -5
Math.imul(0xfffffffe, 5); // -10
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.imul` |

## Browser compatibility

| | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| Feature | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `imul` | 28 | 12 | 20 | 16 | 7 | 28 | 20 | 15 | 7 | 1.5 | 4.4 | 7 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Math.imul`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Math.imul` in `core-js`
- es-shims polyfill of `Math.imul`
- Emscripten on Wikipedia

JavaScript › Reference › Standard built-in objects › Math

# Math.log()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/log · MDN page last modified 2025-07-10

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math.log()`** static method returns the natural logarithm (base e) of a number. That is

$$\forall x > 0,\ \texttt{Math.log}(x) = \ln(x) = \text{the unique } y \text{ such that } e^y = x$$

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.log()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function getBaseLog(x, y) {
  return Math.log(y) / Math.log(x);
}

// 2 x 2 x 2 = 8
console.log(getBaseLog(2, 8));
// Expected output: 3

// 5 x 5 x 5 x 5 = 625
console.log(getBaseLog(5, 625));
// Expected output: 4
```

## Syntax

js

```
Math.log(x)
```

### Parameters

**x**

A number greater than or equal to 0.

### Return value

The natural logarithm (base e) of `x`. If `x` is ±0, returns `-Infinity`. If `x < 0`, returns `NaN`.

## Description

Because `log()` is a static method of `Math`, you always use it as `Math.log()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

If you need the natural log of 2 or 10, use the constants `Math.LN2` or `Math.LN10`. If you need a logarithm to base 2 or 10, use `Math.log2()` or `Math.log10()`. If you need a logarithm to other bases, use `Math.log(x) / Math.log(otherBase)` as in the example below; you might want to precalculate `1 / Math.log(otherBase)` since multiplication in `Math.log(x) * constant` is much faster.

Beware that positive numbers very close to 1 can suffer from loss of precision and make its natural logarithm less accurate. In this case, you may want to use `Math.log1p` instead.

## Examples

### Using Math.log()

```js
Math.log(-1); // NaN
Math.log(-0); // -Infinity
Math.log(0); // -Infinity
Math.log(1); // 0
Math.log(10); // 2.302585092994046
Math.log(Infinity); // Infinity
```

### Using Math.log() with a different base

The following function returns the logarithm of y with base x (i.e., $\log_x y$):

```js
function getBaseLog(x, y) {
  return Math.log(y) / Math.log(x);
}
```

If you run `getBaseLog(10, 1000)`, it returns `2.9999999999999996` due to floating-point rounding, but still very close to the actual answer of 3.

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.log` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| log | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Math.log`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Math.exp()`
- `Math.log1p()`
- `Math.log10()`
- `Math.log2()`
- `Math.pow()`

JavaScript › Reference › Standard built-in objects › Math

# Math.log10()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/log10 · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math.log10()`** static method returns the base 10 logarithm of a number. That is

$$\forall x > 0,\ \texttt{Math.log10}\,(\mathrm{x}) = \log_{10}(x) = \text{the unique } y \text{ such that } 10^y = x$$

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.log10()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(Math.log10(100000));
// Expected output: 5

console.log(Math.log10(2));
// Expected output: 0.3010299956639812

console.log(Math.log10(1));
// Expected output: 0

console.log(Math.log10(0));
// Expected output: -Infinity
```

## Syntax

js

```
Math.log10(x)
```

### Parameters

**x**

A number greater than or equal to 0.

### Return value

The base 10 logarithm of `x`. If `x < 0`, returns `NaN`.

## Description

Because `log10()` is a static method of `Math`, you always use it as `Math.log10()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

This function is the equivalent of `Math.log(x) / Math.log(10)`. For `log10(e)`, use the constant `Math.LOG10E`, which is 1 / `Math.LN10`.

## Examples

### Using Math.log10()

```js
Math.log10(-2); // NaN
Math.log10(-0); // -Infinity
Math.log10(0); // -Infinity
Math.log10(1); // 0
Math.log10(2); // 0.3010299956639812
Math.log10(100000); // 5
Math.log10(Infinity); // Infinity
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.log10 |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| log10 | 38 | 12 | 25 | 25 | 8 | 38 | 25 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8, javascript.builtins.Math.log10. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Math.log10` in `core-js`
- es-shims polyfill of `Math.log10`
- `Math.exp()`
- `Math.log()`
- `Math.log1p()`
- `Math.log2()`
- `Math.pow()`

---

JavaScript › Reference › Standard built-in objects › Math

# Math.log1p()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/log1p · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math.log1p()`** static method returns the natural logarithm (base e) of `1 + x`, where x is the argument. That is:

$$\forall x > -1,\ \texttt{Math.log1p}\,(x) = \ln(1 + x)$$

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.log1p()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(Math.log1p(1));
// Expected output: 0.6931471805599453

console.log(Math.log1p(0));
// Expected output: 0

console.log(Math.log1p(-1));
// Expected output: -Infinity

console.log(Math.log1p(-2));
// Expected output: NaN
```

## Syntax

js

```
Math.log1p(x)
```

### Parameters

**x**

A number greater than or equal to -1.

### Return value

The natural logarithm (base e) of `x + 1`. If `x` is -1, returns `-Infinity`. If `x < -1`, returns `NaN`.

## Description

For very small values of *x*, adding 1 can reduce or eliminate precision. The double floats used in JS give you about 15 digits of precision. 1 + 1e-15 = 1.000000000000001, but 1 + 1e-16 = 1.000000000000000 and therefore exactly 1.0 in that arithmetic, because digits past 15 are rounded off.

When you calculate log(1 + *x*), where *x* is a small positive number, you should get an answer very close to *x* because: $\lim_{x \to 0} \frac{\log(1+x)}{x} = 1$. If you calculate `Math.log(1 + 1.1111111111e-15)`, you should get an answer close to `1.1111111111e-15`. Instead, you will end up taking the logarithm of `1.0000000000000000111022` (the roundoff is in binary, so sometimes it gets ugly), and get the answer 1.11022…e-15, with only 3 correct digits. If you calculate `Math.log1p(1.1111111111e-15)` instead, you will get a much more accurate answer, `1.1111111110999995e-15`, with 15 correct digits of precision (actually 16 in this case).

If the value of `x` is less than -1, the return value is always `NaN`.

Because `log1p()` is a static method of `Math`, you always use it as `Math.log1p()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

## Examples

### Using Math.log1p()

```js
Math.log1p(-2); // NaN
Math.log1p(-1); // -Infinity
Math.log1p(-0); // -0
Math.log1p(0); // 0
Math.log1p(1); // 0.6931471805599453
Math.log1p(Infinity); // Infinity
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.log1p |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| log1p | 38 | 12 | 25 | 25 | 8 | 38 | 25 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Math.log1p`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Math.log1p` in `core-js`
- es-shims polyfill of `Math.log1p`
- `Math.exp()`
- `Math.log()`
- `Math.expm1()`
- `Math.log10()`
- `Math.log2()`
- `Math.pow()`

JavaScript › Reference › Standard built-in objects › Math

# Math.log2()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/log2 · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math.log2()`** static method returns the base 2 logarithm of a number. That is

$$\forall x > 0,\ \texttt{Math.log2}\,(\texttt{x}) = \log_2(x) = \text{the unique } y \text{ such that } 2^y = x$$

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.log2()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(Math.log2(3));
// Expected output: 1.584962500721156

console.log(Math.log2(2));
// Expected output: 1

console.log(Math.log2(1));
// Expected output: 0

console.log(Math.log2(0));
// Expected output: -Infinity
```

## Syntax

js

```
Math.log2(x)
```

### Parameters

**x**

A number greater than or equal to 0.

### Return value

The base 2 logarithm of `x`. If `x < 0`, returns `NaN`.

## Description

Because `log2()` is a static method of `Math`, you always use it as `Math.log2()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

This function is the equivalent of `Math.log(x) / Math.log(2)`. For `log2(e)`, use the constant `Math.LOG2E`, which is 1 / `Math.LN2`.

## Examples

### Using Math.log2()

```js
Math.log2(-2); // NaN
Math.log2(-0); // -Infinity
Math.log2(0); // -Infinity
Math.log2(1); // 0
Math.log2(2); // 1
Math.log2(3); // 1.584962500721156
Math.log2(1024); // 10
Math.log2(Infinity); // Infinity
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.log2 |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| log2 | 38 | 12 | 25 | 25 | 8 | 38 | 25 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Math.log2`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Math.log2` in `core-js`
- `Math.exp()`
- `Math.log()`
- `Math.log10()`
- `Math.log1p()`
- `Math.pow()`

---

JavaScript › Reference › Standard built-in objects › Math

# Math.max()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/max · MDN page last modified 2025-07-20

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math.max()`** static method returns the largest of the numbers given as input parameters, or `-Infinity` if there are no parameters.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.max()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(Math.max(1, 3, 2));
// Expected output: 3

console.log(Math.max(-1, -3, -2));
// Expected output: -1

const array = [1, 3, 2];

console.log(Math.max(...array));
// Expected output: 3
```

## Syntax

js

```
Math.max()
Math.max(value1)
Math.max(value1, value2)
Math.max(value1, value2, /* …, */ valueN)
```

### Parameters

**`value1, ..., valueN`**

Zero or more numbers among which the largest value will be selected and returned.

### Return value

The largest of the given numbers. Returns `NaN` if any of the parameters is or is converted into `NaN`. Returns `-Infinity` if no parameters are provided.

## Description

Because `max()` is a static method of `Math`, you always use it as `Math.max()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

`Math.max.length` is 2, which weakly signals that it's designed to handle at least two parameters.

## Examples

### Using Math.max()

js

```
Math.max(10, 20); // 20
Math.max(-10, -20); // -10
Math.max(-10, 20); // 20
```

### Getting the maximum element of an array

`Array.prototype.reduce()` can be used to find the maximum element in a numeric array, by comparing each value:

```js
const arr = [1, 2, 3];
const max = arr.reduce((a, b) => Math.max(a, b), -Infinity);
```

The following function uses `Function.prototype.apply()` to get the maximum of an array. `getMaxOfArray([1, 2, 3])` is equivalent to `Math.max(1, 2, 3)`, but you can use `getMaxOfArray()` on programmatically constructed arrays. This should only be used for arrays with relatively few elements.

```js
function getMaxOfArray(numArray) {
  return Math.max.apply(null, numArray);
}
```

The spread syntax is a shorter way of writing the `apply` solution to get the maximum of an array:

```js
const arr = [1, 2, 3];
const max = Math.max(...arr);
```

However, both spread (`...`) and `apply` will either fail or return the wrong result if the array has too many elements, because they try to pass the array elements as function parameters. See Using apply and built-in functions for more details. The `reduce` solution does not have this problem.

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.max` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `max` | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Math.max`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Math.min()`

JavaScript › Reference › Standard built-in objects › Math

# Math.min()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/min · MDN page last modified 2025-07-20

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math.min()`** static method returns the smallest of the numbers given as input parameters, or `Infinity` if there are no parameters.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.min()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(Math.min(2, 3, 1));
// Expected output: 1

console.log(Math.min(-2, -3, -1));
// Expected output: -3

const array = [2, 3, 1];

console.log(Math.min(...array));
// Expected output: 1
```

## Syntax

js

```
Math.min()
Math.min(value1)
Math.min(value1, value2)
Math.min(value1, value2, /* …, */ valueN)
```

### Parameters

**`value1`, …, `valueN`**

Zero or more numbers among which the lowest value will be selected and returned.

### Return value

The smallest of the given numbers. Returns `NaN` if any of the parameters is or is converted into `NaN`. Returns `Infinity` if no parameters are provided.

## Description

Because `min()` is a static method of `Math`, you always use it as `Math.min()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

`Math.min.length` is 2, which weakly signals that it's designed to handle at least two parameters.

## Examples

### Using Math.min()

This finds the min of x and y and assigns it to z:

```js
const x = 10;
const y = -20;
const z = Math.min(x, y); // -20
```

### Clipping a value with Math.min()

`Math.min()` is often used to clip a value so that it is always less than or equal to a boundary. For instance, this

```js
let x = f(foo);

if (x > boundary) {
  x = boundary;
}
```

may be written as this

```js
const x = Math.min(f(foo), boundary);
```

`Math.max()` can be used in a similar way to clip a value at the other end.

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.min |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `min` | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Math.min`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Math.max()`

JavaScript › Reference › Standard built-in objects › Math

# Math.pow()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/pow · MDN page last modified 2025-07-10

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math.pow()`** static method returns the value of a base raised to a power. That is

$$\texttt{Math.pow}(x, y) = x^y$$

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.pow()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(Math.pow(7, 3));
// Expected output: 343

console.log(Math.pow(4, 0.5));
// Expected output: 2

console.log(Math.pow(7, -2));
// Expected output: 0.02040816326530612
//                  (1/49)

console.log(Math.pow(-7, 0.5));
// Expected output: NaN
```

## Syntax

js

```
Math.pow(base, exponent)
```

### Parameters

**`base`**

The base number.

**`exponent`**

The exponent number.

### Return value

A number representing `base` taken to the power of `exponent`. Returns `NaN` in one of the following cases:

- `exponent` is `NaN`.
- `base` is `NaN` and `exponent` is not `0`.
- `base` is ±1 and `exponent` is `±Infinity`.
- `base < 0` and `exponent` is not an integer.

## Description

`Math.pow()` is equivalent to the `**` operator, except `Math.pow()` only accepts numbers.

`Math.pow(NaN, 0)` (and the equivalent `NaN ** 0`) is the only case where NaN doesn't propagate through mathematical operations — it returns `1` despite the operand being `NaN`. In addition, the behavior where `base` is 1 and `exponent` is non-finite (±Infinity or `NaN`) is different from IEEE 754, which specifies that the result should be 1, whereas JavaScript returns `NaN` to preserve backward compatibility with its original behavior.

Because `pow()` is a static method of `Math`, use it as `Math.pow()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

## Examples

### Using Math.pow()

js

```
// Basic cases
Math.pow(7, 2); // 49
Math.pow(7, 3); // 343
Math.pow(2, 10); // 1024

// Fractional exponents
Math.pow(4, 0.5); // 2 (square root of 4)
Math.pow(8, 1 / 3); // 2 (cube root of 8)
Math.pow(2, 0.5); // 1.4142135623730951 (square root of 2)
Math.pow(2, 1 / 3); // 1.2599210498948732 (cube root of 2)

// Signed exponents
Math.pow(7, -2); // 0.02040816326530612 (1/49)
Math.pow(8, -1 / 3); // 0.5

// Signed bases
Math.pow(-7, 2); // 49 (squares are positive)
Math.pow(-7, 3); // -343 (cubes can be negative)
Math.pow(-7, 0.5); // NaN (negative numbers don't have a real square root)
// Due to "even" and "odd" roots laying close to each other,
// and limits in the floating number precision,
// negative bases with fractional exponents always return NaN,
// even when the mathematical result is real
Math.pow(-7, 1 / 3); // NaN

// Zero and infinity
Math.pow(0, 0); // 1 (anything ** ±0 is 1)
Math.pow(Infinity, 0.1); // Infinity (positive exponent)
Math.pow(Infinity, -1); // 0 (negative exponent)
Math.pow(-Infinity, 1); // -Infinity (positive odd integer exponent)
Math.pow(-Infinity, 1.5); // Infinity (positive exponent)
Math.pow(-Infinity, -1); // -0 (negative odd integer exponent)
Math.pow(-Infinity, -1.5); // 0 (negative exponent)
Math.pow(0, 1); // 0 (positive exponent)
Math.pow(0, -1); // Infinity (negative exponent)
Math.pow(-0, 1); // -0 (positive odd integer exponent)
Math.pow(-0, 1.5); // 0 (positive exponent)
Math.pow(-0, -1); // -Infinity (negative odd integer exponent)
Math.pow(-0, -1.5); // Infinity (negative exponent)
Math.pow(0.9, Infinity); // 0
Math.pow(1, Infinity); // NaN
Math.pow(1.1, Infinity); // Infinity
Math.pow(0.9, -Infinity); // Infinity
```

(code continues on the next page)

*(code continued from the previous page)*

```
Math.pow(1, -Infinity); // NaN
Math.pow(1.1, -Infinity); // 0

// NaN: only Math.pow(NaN, 0) does not result in NaN
Math.pow(NaN, 0); // 1
Math.pow(NaN, 1); // NaN
Math.pow(1, NaN); // NaN
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.pow` |

## Browser compatibility

| | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| Feature | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| pow | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Math.pow`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Math.cbrt()`
- `Math.exp()`
- `Math.log()`
- `Math.sqrt()`
- Exponentiation (**)

---

JavaScript › Reference › Standard built-in objects › Math

# Math.random()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/random · MDN page last modified 2025-07-10

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math.random()`** static method returns a floating-point, pseudo-random number that's greater than or equal to 0 and less than 1, with approximately uniform distribution over that range — which you can then scale to your desired range. The implementation selects the initial seed to the random number generation algorithm; it cannot be chosen or reset by the user.

> **Note:** `Math.random()` *does not* provide cryptographically secure random numbers. Do not use them for anything related to security. Use the Web Crypto API instead, and more precisely the `Crypto.getRandomValues()` method.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.random()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function getRandomInt(max) {
  return Math.floor(Math.random() * max);
}

console.log(getRandomInt(3));
// Expected output: 0, 1 or 2

console.log(getRandomInt(1));
// Expected output: 0

console.log(Math.random());
// Expected output: a number from 0 to <1
```

## Syntax

js

```
Math.random()
```

### Parameters

None.

### Return value

A floating-point, pseudo-random number between 0 (inclusive) and 1 (exclusive).

## Examples

Note that as numbers in JavaScript are IEEE 754 floating point numbers with round-to-nearest-even behavior, the ranges claimed for the functions below (excluding the one for `Math.random()` itself) aren't exact. Usually, the claimed upper bound is not attainable, but if `Math.random()` returns a number very close to 1, the tiny difference may not be representable at the requested maximum, therefore causing the upper bound to be attained.

### Getting a random number between 0 (inclusive) and 1 (exclusive)

js

```
function getRandom() {
  return Math.random();
}
```

### Getting a random number between two values

This example returns a random number between the specified values. The returned value is no lower than (and may possibly equal) `min`, and is less than (and not equal) `max`.

js

```
function getRandomArbitrary(min, max) {
  return Math.random() * (max - min) + min;
}
```

### Getting a random integer between two values

This example returns a random *integer* between the specified values. The value is no lower than `min` (or the next integer greater than `min` if `min` isn't an integer), and is less than (but not equal to) `max`.

```js
function getRandomInt(min, max) {
  const minCeiled = Math.ceil(min);
  const maxFloored = Math.floor(max);
  return Math.floor(Math.random() * (maxFloored - minCeiled) + minCeiled); // The maximum is
exclusive and the minimum is inclusive
}
```

> **Note:** It might be tempting to use `Math.round()` to accomplish that, but doing so would cause your random numbers to follow a non-uniform distribution, which may not be acceptable for your needs.

### Getting a random integer between two values, inclusive

While the `getRandomInt()` function above is inclusive at the minimum, it's exclusive at the maximum. What if you need the results to be inclusive at both the minimum and the maximum? The `getRandomIntInclusive()` function below accomplishes that.

```js
function getRandomIntInclusive(min, max) {
  const minCeiled = Math.ceil(min);
  const maxFloored = Math.floor(max);
  return Math.floor(Math.random() * (maxFloored - minCeiled + 1) + minCeiled); // The
maximum is inclusive and the minimum is inclusive
}
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.random` |

## Browser compatibility

| | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| Feature | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `random` | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Math.random`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Crypto.getRandomValues()`

JavaScript › Reference › Standard built-in objects › Math

# Math.round()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/round · MDN page last modified 2025-07-10

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math.round()`** static method returns the value of a number rounded to the nearest integer.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.round()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(Math.round(0.9));
// Expected output: 1

console.log(Math.round(5.95), Math.round(5.5), Math.round(5.05));
// Expected output: 6 6 5

console.log(Math.round(-5.05), Math.round(-5.5), Math.round(-5.95));
// Expected output: -5 -5 -6
```

## Syntax

js

```
Math.round(x)
```

### Parameters

**x**

A number.

### Return value

The value of x rounded to the nearest integer.

## Description

If the fractional portion of the argument is greater than 0.5, the argument is rounded to the integer with the next higher absolute value. If it is less than 0.5, the argument is rounded to the integer with the lower absolute value. If the fractional portion is exactly 0.5, the argument is rounded to the next integer in the direction of +∞.

> **Note:** This differs from many languages' `round()` functions, which often round half-increments *away from zero*, giving a different result in the case of negative numbers with a fractional part of exactly 0.5.

`Math.round(x)` is not exactly the same as `Math.floor(x + 0.5)`. When x is -0, or -0.5 ≤ x < 0, `Math.round(x)` returns -0, while `Math.floor(x + 0.5)` returns 0. However, neglecting that difference and potential precision errors, `Math.round(x)` and `Math.floor(x + 0.5)` are generally equivalent.

Because `round()` is a static method of `Math`, you always use it as `Math.round()`, rather than as a method of a `Math` object you created (`Math` has no constructor).

## Examples

### Using round

```js
Math.round(-Infinity); // -Infinity
Math.round(-20.51); // -21
Math.round(-20.5); // -20
Math.round(-0.1); // -0
Math.round(0); // 0
Math.round(20.49); // 20
Math.round(20.5); // 21
Math.round(42); // 42
Math.round(Infinity); // Infinity
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.round` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `round` | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Math.round`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Number.prototype.toPrecision()`
- `Number.prototype.toFixed()`
- `Math.abs()`
- `Math.ceil()`
- `Math.floor()`
- `Math.sign()`
- `Math.trunc()`

JavaScript › Reference › Standard built-in objects › Math

# Math.sign()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/sign · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since September 2015.

The **`Math.sign()`** static method returns 1 or -1, indicating the sign of the number passed as argument. If the input is 0 or -0, it will be returned as-is.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.sign()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(Math.sign(3));
// Expected output: 1

console.log(Math.sign(-3));
// Expected output: -1

console.log(Math.sign(0));
// Expected output: 0

console.log(Math.sign("-3"));
// Expected output: -1
```

## Syntax

js

```
Math.sign(x)
```

### Parameters

**x**

A number.

### Return value

A number representing the sign of `x`:

- If `x` is positive, returns `1`.
- If `x` is negative, returns `-1`.
- If `x` is positive zero, returns `0`.
- If `x` is negative zero, returns `-0`.
- Otherwise, returns `NaN`.

## Description

Because `sign()` is a static method of `Math`, you always use it as `Math.sign()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

## Examples

### Using Math.sign()

```js
Math.sign(3); // 1
Math.sign(-3); // -1
Math.sign("-3"); // -1
Math.sign(0); // 0
Math.sign(-0); // -0
Math.sign(NaN); // NaN
Math.sign("foo"); // NaN
Math.sign(); // NaN
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.sign |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Nodejs |
| sign | 38 | 12 | 25 | 25 | 9 | 38 | 25 | 25 | 9 | 3.0 | 38 | 9 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Math.sign`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Math.sign` in `core-js`
- es-shims polyfill of `Math.sign`
- `Math.abs()`
- `Math.ceil()`
- `Math.floor()`
- `Math.round()`
- `Math.trunc()`

JavaScript › Reference › Standard built-in objects › Math

# Math.sin()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/sin · MDN page last modified 2025-12-29

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math.sin()`** static method returns the sine of a number in radians.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.sin()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function getCircleY(radians, radius) {
  return Math.sin(radians) * radius;
}

console.log(getCircleY(1, 10));
// Expected output: 8.414709848078965

console.log(getCircleY(2, 10));
// Expected output: 9.092974268256818

console.log(getCircleY(Math.PI, 10));
// Expected output: 1.2246467991473533e-15
```

## Syntax

js

```
Math.sin(x)
```

### Parameters

**x**

A number representing an angle in radians.

### Return value

The sine of x, between -1 and 1, inclusive. If x is `Infinity`, `-Infinity`, or `NaN`, returns `NaN`.

## Description

Because `sin()` is a static method of `Math`, you always use it as `Math.sin()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

## Examples

### Using Math.sin()

```js
Math.sin(-Infinity); // NaN
Math.sin(-0); // -0
Math.sin(0); // 0
Math.sin(1); // 0.8414709848078965
Math.sin(Math.PI / 2); // 1
Math.sin(Infinity); // NaN
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.sin |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| sin | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Math.sin`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Math.acos()`
- `Math.asin()`
- `Math.atan()`
- `Math.atan2()`
- `Math.cos()`
- `Math.tan()`
- CSS `sin()` function

JavaScript › Reference › Standard built-in objects › Math

# Math.sinh()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/sinh · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math.sinh()`** static method returns the hyperbolic sine of a number. That is,

$$\texttt{Math.sinh}\,(\texttt{x}) = \sinh(x) = \frac{\mathrm{e}^{x} - \mathrm{e}^{-x}}{2}$$

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.sinh()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(Math.sinh(0));
// Expected output: 0

console.log(Math.sinh(1));
// Expected output: 1.1752011936438014

console.log(Math.sinh(-1));
// Expected output: -1.1752011936438014

console.log(Math.sinh(2));
// Expected output: 3.626860407847019
```

## Syntax

js

```
Math.sinh(x)
```

### Parameters

**x**

A number.

### Return value

The hyperbolic sine of x.

## Description

Because `sinh()` is a static method of `Math`, you always use it as `Math.sinh()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

## Examples

### Using Math.sinh()

js

```
Math.sinh(-Infinity); // -Infinity
Math.sinh(-0); // -0
Math.sinh(0); // 0
Math.sinh(1); // 1.1752011936438014
Math.sinh(Infinity); // Infinity
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.sinh |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `sinh` | 38 | 12 | 25 | 25 | 8 | 38 | 25 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Math.sinh`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Math.sinh` in `core-js`
- `Math.acosh()`
- `Math.asinh()`
- `Math.atanh()`
- `Math.cosh()`
- `Math.tanh()`

---

JavaScript › Reference › Standard built-in objects › Math

# Math.sqrt()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/sqrt · MDN page last modified 2025-07-10

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math.sqrt()`** static method returns the square root of a number. That is

$$\forall x \geq 0,\ \texttt{Math.sqrt}\,(\texttt{x}) = \sqrt{x} = \text{the unique } y \geq 0 \text{ such that } y^2 = x$$

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.sqrt()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function calcHypotenuse(a, b) {
  return Math.sqrt(a * a + b * b);
}

console.log(calcHypotenuse(3, 4));
// Expected output: 5

console.log(calcHypotenuse(5, 12));
// Expected output: 13

console.log(calcHypotenuse(0, 0));
// Expected output: 0
```

## Syntax

```js
Math.sqrt(x)
```

### Parameters

**x**

A number greater than or equal to 0.

### Return value

The square root of x, a nonnegative number. If `x < 0`, returns `NaN`.

## Description

Because `sqrt()` is a static method of `Math`, you always use it as `Math.sqrt()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

## Examples

### Using Math.sqrt()

```js
Math.sqrt(-1); // NaN
Math.sqrt(-0); // -0
Math.sqrt(0); // 0
Math.sqrt(1); // 1
Math.sqrt(2); // 1.414213562373095
Math.sqrt(9); // 3
Math.sqrt(Infinity); // Infinity
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.sqrt |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| sqrt | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Math.sqrt`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Math.cbrt()`
- `Math.exp()`
- `Math.log()`
- `Math.pow()`

---

JavaScript › Reference › Standard built-in objects › Math

# Math.sumPrecise()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/sumPrecise · MDN page last modified 2026-10-26

**Baseline 2026: Newly available** — works across the latest browser versions since April 2026; may not work in older ones.

The **`Math.sumPrecise()`** static method takes an iterable of numbers and returns the sum of them. It is more precise than summing them up in a loop, because it avoids floating point precision loss in intermediate results.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.sumPrecise()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(Math.sumPrecise([1, 2]));
// Expected output: 3

console.log(Math.sumPrecise([1e20, 0.1, -1e20]));
// Expected output: 0.1
```

## Syntax

js

```
Math.sumPrecise(numbers)
```

### Parameters

**`numbers`**

An iterable (such as an `Array`) of numbers.

### Return value

A number that is the sum of the numbers in the `numbers` iterable. If the iterable is empty, the return value is `-0` (*not* `0`).

### Exceptions

**`TypeError`**

If `numbers` is not an iterable, or if any of the numbers in the iterable is not of the number type.

## Description

Because `sumPrecise()` is a static method of `Math`, you always use it as `Math.sumPrecise()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

The method is called `Math.sumPrecise()` because it is more precise than naïvely summing up numbers in a loop. Consider the following example:

```js
let sum = 0;
const numbers = [1e20, 0.1, -1e20];
for (const number of numbers) {
  sum += number;
}
console.log(sum); // 0
```

The output is 0. This is because `1e20 + 0.1` cannot be represented precisely in 64-bit floats, so the intermediate result is rounded to `1e20`. Then, the sum of `1e20` and `-1e20` is `0`, so the final result is `0`.

`Math.sumPrecise()` avoids this issue by using some specialized summing algorithm. It works as if the floating point numbers are summed up using their precise mathematical values, and the final result is then converted to the nearest representable 64-bit float. This still cannot avoid the `0.1 + 0.2` precision problem:

```js
console.log(Math.sumPrecise([0.1, 0.2])); // 0.30000000000000004
```

Because the floating point literals `0.1` and `0.2` already represent mathematical values greater than `0.1` and `0.2`, and their sum's closest 64-bit float representation is actually `0.30000000000000004`.

## Examples

### Using Math.sumPrecise()

```js
console.log(Math.sumPrecise([1, 2, 3])); // 6
console.log(Math.sumPrecise([1e20, 0.1, -1e20])); // 0.1
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.sumprecise |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| sumPrecise | 147 | 147 | 137 | 131 | 26.2 | 147 | 137 | 98 | 26.2 | **No** | 147 | 26.2 | 1.2.18 | 2.7.8 | **No** |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Math.sumPrecise`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Math.sumPrecise` in `core-js`
- es-shims polyfill of `Math.sumPrecise`
- `Array.prototype.reduce()`

---

JavaScript › Reference › Standard built-in objects › Math

# Math.tan()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/tan · MDN page last modified 2025-12-29

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math.tan()`** static method returns the tangent of a number in radians.

## Try it

*▷ Interactive example on MDN: "JavaScript Demo: Math.tan()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function getTanFromDegrees(degrees) {
  return Math.tan((degrees * Math.PI) / 180);
}

console.log(getTanFromDegrees(0));
// Expected output: 0

console.log(getTanFromDegrees(45));
// Expected output: 0.9999999999999999

console.log(getTanFromDegrees(90));
// Expected output: 16331239353195370
```

## Syntax

js

```
Math.tan(x)
```

### Parameters

**x**

A number representing an angle in radians.

### Return value

The tangent of `x`. If `x` is `Infinity`, `-Infinity`, or `NaN`, returns `NaN`.

> **Note:** Due to floating point precision, it's not possible to obtain the exact value π/2, so the result is always finite if not `NaN`.

## Description

Because `tan()` is a static method of `Math`, you always use it as `Math.tan()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

## Examples

### Using Math.tan()

```js
Math.tan(-Infinity); // NaN
Math.tan(-0); // -0
Math.tan(0); // 0
Math.tan(1); // 1.5574077246549023
Math.tan(Math.PI / 4); // 0.9999999999999999 (Floating point error)
Math.tan(Infinity); // NaN
```

### Math.tan() and π/2

It's not possible to calculate `tan(π/2)` exactly.

```js
Math.tan(Math.PI / 2); // 16331239353195370
Math.tan(Math.PI / 2 + Number.EPSILON); // -6218431163823738
```

### Using Math.tan() with a degree value

Because the `Math.tan()` function accepts radians, but it is often easier to work with degrees, the following function accepts a value in degrees, converts it to radians and returns the tangent.

```js
function getTanDeg(deg) {
  const rad = (deg * Math.PI) / 180;
  return Math.tan(rad);
}
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.tan` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `tan` | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Math.tan`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Math.acos()`
- `Math.asin()`
- `Math.atan()`
- `Math.atan2()`
- `Math.cos()`
- `Math.sin()`
- CSS `tan()` function

---

JavaScript › Reference › Standard built-in objects › Math

# Math.tanh()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/tanh · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math.tanh()`** static method returns the hyperbolic tangent of a number. That is,

$$\texttt{Math.tanh}\,(x) = \tanh(x) = \frac{\sinh(x)}{\cosh(x)} = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{e^{2x} - 1}{e^{2x} + 1}$$

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.tanh()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(Math.tanh(-1));
// Expected output: -0.7615941559557649

console.log(Math.tanh(0));
// Expected output: 0

console.log(Math.tanh(Infinity));
// Expected output: 1

console.log(Math.tanh(1));
// Expected output: 0.7615941559557649
```

## Syntax

js

```
Math.tanh(x)
```

### Parameters

**x**

A number.

### Return value

The hyperbolic tangent of `x`.

## Description

Because `tanh()` is a static method of `Math`, you always use it as `Math.tanh()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

## Examples

### Using Math.tanh()

```js
Math.tanh(-Infinity); // -1
Math.tanh(-0); // -0
Math.tanh(0); // 0
Math.tanh(1); // 0.7615941559557649
Math.tanh(Infinity); // 1
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.tanh` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `tanh` | 38 | 12 | 25 | 25 | 8 | 38 | 25 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Math.tanh`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Math.tanh` in `core-js`
- `Math.acosh()`
- `Math.asinh()`
- `Math.atanh()`
- `Math.cosh()`
- `Math.sinh()`

JavaScript › Reference › Standard built-in objects › Math

# Math.trunc()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/trunc · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math.trunc()`** static method returns the integer part of a number by removing any fractional digits.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.trunc()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(Math.trunc(13.37));
// Expected output: 13

console.log(Math.trunc(42.84));
// Expected output: 42

console.log(Math.trunc(0.123));
// Expected output: 0

console.log(Math.trunc(-0.123));
// Expected output: -0
```

## Syntax

js

```
Math.trunc(x)
```

### Parameters

**x**

A number.

### Return value

The integer part of `x`.

## Description

The way `Math.trunc()` works is more straightforward than the other three `Math` methods: `Math.floor()`, `Math.ceil()` and `Math.round()`; it *truncates* (cuts off) the dot and the digits to the right of it, no matter whether the argument is a positive or negative number.

Because `trunc()` is a static method of `Math`, you always use it as `Math.trunc()`, rather than as a method of a `Math` object you created (`Math` is not a constructor).

## Examples

### Using Math.trunc()

```js
Math.trunc(-Infinity); // -Infinity
Math.trunc("-1.123"); // -1
Math.trunc(-0.123); // -0
Math.trunc(-0); // -0
Math.trunc(0); // 0
Math.trunc(0.123); // 0
Math.trunc(13.37); // 13
Math.trunc(42.84); // 42
Math.trunc(Infinity); // Infinity
```

### Using bitwise no-ops to truncate numbers

> **Warning:** This is not a polyfill for `Math.trunc()` because of non-negligible edge cases.

Bitwise operations convert their operands to 32-bit integers, which people have historically taken advantage of to truncate float-point numbers. Common techniques include:

```js
const original = 3.14;
const truncated1 = ~~original; // Double negation
const truncated2 = original & -1; // Bitwise AND with -1
const truncated3 = original | 0; // Bitwise OR with 0
const truncated4 = original ^ 0; // Bitwise XOR with 0
const truncated5 = original >> 0; // Bitwise shifting by 0
```

Beware that this is essentially `toInt32`, which is not the same as `Math.trunc`. When the value does not satisfy $-2^{31} - 1 <$ `value` $< 2^{31}$ ($-2147483649 <$ `value` $< 2147483648$), the conversion would overflow.

```js
const a = ~~2147483648; // -2147483648
const b = ~~-2147483649; // 2147483647
const c = ~~4294967296; // 0
```

Only use `~~` as a substitution for `Math.trunc()` when you are confident that the range of input falls within the range of 32-bit integers.

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.trunc |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `trunc` | 38 | 12 | 25 | 25 | 8 | 38 | 25 | 25 | 8 | 3.0 | 38 | 8 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Math.trunc`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Math.trunc` in `core-js`
- `Math.abs()`
- `Math.ceil()`
- `Math.floor()`
- `Math.round()`
- `Math.sign()`

---

JavaScript › Reference › Standard built-in objects › Math

# Math.E

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/E · MDN page last modified 2025-07-10

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math.E`** static data property represents Euler's number, the base of natural logarithms, e, which is approximately 2.718.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.E". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function compoundOneYear(interestRate, currentVal) {
  return currentVal * Math.E ** interestRate;
}

console.log(Math.E);
// Expected output: 2.718281828459045

console.log((1 + 1 / 1000000) ** 1000000);
// Expected output: 2.718280469 (approximately)

console.log(compoundOneYear(0.05, 100));
// Expected output: 105.12710963760242
```

## Value

$$\texttt{Math.E} = e \approx 2.718$$

| Property attributes of `Math.E` | |
|---|---|
| Writable | no |
| Enumerable | no |
| Configurable | no |

## Description

Because `E` is a static property of `Math`, you always use it as `Math.E`, rather than as a property of a `Math` object you created (`Math` is not a constructor).

## Examples

### Using Math.E

The following function returns e:

```js
function getNapier() {
  return Math.E;
}

getNapier(); // 2.718281828459045
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.e |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| E | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Math.E`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Math.exp()`
- `Math.log()`
- `Math.log1p()`

---

JavaScript › Reference › Standard built-in objects › Math

# Math.LN10

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/LN10 · MDN page last modified 2025-07-10

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math.LN10`** static data property represents the natural logarithm of 10, approximately 2.303.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.LN10". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function getNatLog10() {
  return Math.LN10;
}

console.log(getNatLog10());
// Expected output: 2.302585092994046
```

## Value

$$\texttt{Math.LN10} = \ln(10) \approx 2.303$$

| Property attributes of `Math.LN10` | |
|---|---|
| Writable | no |
| Enumerable | no |
| Configurable | no |

## Description

Because `LN10` is a static property of `Math`, you always use it as `Math.LN10`, rather than as a property of a `Math` object you created (`Math` is not a constructor).

## Examples

### Using Math.LN10

The following function returns the natural log of 10:

js

```
function getNatLog10() {
  return Math.LN10;
}

getNatLog10(); // 2.302585092994046
```

## Specifications

| Specification |
|---|
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.ln10` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `LN10` | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Math.LN10`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Math.exp()`
- `Math.log()`
- `Math.log10()`

---

JavaScript › Reference › Standard built-in objects › Math

# Math.LN2

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/LN2 · MDN page last modified 2025-07-10

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math.LN2`** static data property represents the natural logarithm of 2, approximately 0.693:

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.LN2". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function getNatLog2() {
  return Math.LN2;
}

console.log(getNatLog2());
// Expected output: 0.6931471805599453
```

## Value

$$\texttt{Math.LN2} = \ln(2) \approx 0.693$$

| Property attributes of `Math.LN2` | |
|---|---|
| Writable | no |
| Enumerable | no |
| Configurable | no |

## Description

Because `LN2` is a static property of `Math`, you always use it as `Math.LN2`, rather than as a property of a `Math` object you created (`Math` is not a constructor).

## Examples

### Using Math.LN2

The following function returns the natural log of 2:

```js
function getNatLog2() {
  return Math.LN2;
}

getNatLog2(); // 0.6931471805599453
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.ln2 |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Nodejs |
| LN2 | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Math.LN2`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Math.exp()`
- `Math.log()`
- `Math.log2()`

---

JavaScript › Reference › Standard built-in objects › Math

# Math.LOG10E

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/LOG10E · MDN page last modified 2025-07-10

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math.LOG10E`** static data property represents the base 10 logarithm of e, approximately 0.434.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.LOG10E". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function getLog10e() {
  return Math.LOG10E;
}

console.log(getLog10e());
// Expected output: 0.4342944819032518
```

## Value

$$\texttt{Math.LOG10E} = \log_{10}(e) \approx 0.434$$

| Property attributes of `Math.LOG10E` | |
|---|---|
| Writable | no |
| Enumerable | no |
| Configurable | no |

## Description

Because `LOG10E` is a static property of `Math`, you always use it as `Math.LOG10E`, rather than as a property of a `Math` object you created (`Math` is not a constructor).

## Examples

### Using Math.LOG10E

The following function returns the base 10 logarithm of e:

js

```
function getLog10e() {
  return Math.LOG10E;
}

getLog10e(); // 0.4342944819032518
```

## Specifications

| Specification |
|---|
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.log10e` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `LOG10E` | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Math.LOG10E`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Math.exp()`
- `Math.log()`
- `Math.log10()`

---

JavaScript › Reference › Standard built-in objects › Math

# Math.LOG2E

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/LOG2E · MDN page last modified 2025-07-10

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math.LOG2E`** static data property represents the base 2 logarithm of e, approximately 1.443.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.LOG2E". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function getLog2e() {
  return Math.LOG2E;
}

console.log(getLog2e());
// Expected output: 1.4426950408889634
```

## Value

$$\texttt{Math.LOG2E} = \log_2(e) \approx 1.443$$

| Property attributes of `Math.LOG2E` | |
|---|---|
| Writable | no |
| Enumerable | no |
| Configurable | no |

## Description

Because `LOG2E` is a static property of `Math`, you always use it as `Math.LOG2E`, rather than as a property of a `Math` object you created (`Math` is not a constructor).

## Examples

### Using Math.LOG2E

The following function returns the base 2 logarithm of e:

```js
function getLog2e() {
  return Math.LOG2E;
}

getLog2e(); // 1.4426950408889634
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.log2e |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| LOG2E | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Math.LOG2E`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Math.exp()`
- `Math.log()`
- `Math.log2()`

---

JavaScript › Reference › Standard built-in objects › Math

# Math.PI

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/PI · MDN page last modified 2025-07-10

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math.PI`** static data property represents the ratio of the circumference of a circle to its diameter, approximately 3.14159.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.PI". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function calculateCircumference(radius) {
  return 2 * Math.PI * radius;
}

console.log(Math.PI);
// Expected output: 3.141592653589793

console.log(calculateCircumference(10));
// Expected output: 62.83185307179586
```

## Value

$$\texttt{Math.PI} = \pi \approx 3.14159$$

| Property attributes of `Math.PI` | |
|---|---|
| Writable | no |
| Enumerable | no |
| Configurable | no |

## Description

Because `PI` is a static property of `Math`, you always use it as `Math.PI`, rather than as a property of a `Math` object you created (`Math` is not a constructor).

## Examples

### Using Math.PI

The following function uses `Math.PI` to calculate the circumference of a circle with a passed radius.

js

```
function calculateCircumference(radius) {
  return Math.PI * (radius + radius);
}

calculateCircumference(1); // 6.283185307179586
```

## Specifications

| Specification |
|---|
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.pi` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `PI` | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Math.PI`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Math`

---

JavaScript › Reference › Standard built-in objects › Math

# Math.SQRT1_2

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/SQRT1_2 · MDN page last modified 2025-11-03

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math.SQRT1_2`** static data property represents the square root of 1/2, which is approximately 0.707.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.SQRT1_2". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function getRoot1Over2() {
  return Math.SQRT1_2;
}

console.log(getRoot1Over2());
// Expected output: 0.7071067811865476
```

## Value

$$\texttt{Math.SQRT1_2} = \sqrt{\frac{1}{2}} \approx 0.707$$

| Property attributes of `Math.SQRT1_2` | |
|---|---|
| Writable | no |
| Enumerable | no |
| Configurable | no |

## Description

`Math.SQRT1_2` is a constant and a more performant equivalent to `Math.sqrt(0.5)`.

Because `SQRT1_2` is a static property of `Math`, you always use it as `Math.SQRT1_2`, rather than as a property of a `Math` object you created (`Math` is not a constructor).

## Examples

### Using Math.SQRT1_2

The following function calculates the side length of a square given its diagonal length:

```js
function getSquareSideLength(diagonalLength) {
  return diagonalLength * Math.SQRT1_2;
}
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.sqrt1_2` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `SQRT1_2` | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Math.SQRT1_2`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Math.pow()`
- `Math.sqrt()`

---

JavaScript › Reference › Standard built-in objects › Math

# Math.SQRT2

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/SQRT2 · MDN page last modified 2025-07-10

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Math.SQRT2`** static data property represents the square root of 2, approximately 1.414.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Math.SQRT2". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function getRoot2() {
  return Math.SQRT2;
}

console.log(getRoot2());
// Expected output: 1.4142135623730951
```

## Value

$$\texttt{Math.SQRT2} = \sqrt{2} \approx 1.414$$

| Property attributes of `Math.SQRT2` | |
|---|---|
| Writable | no |
| Enumerable | no |
| Configurable | no |

## Description

`Math.SQRT2` is a constant and a more performant equivalent to `Math.sqrt(2)`.

Because `SQRT2` is a static property of `Math`, you always use it as `Math.SQRT2`, rather than as a property of a `Math` object you created (`Math` is not a constructor).

## Examples

### Using Math.SQRT2

The following function returns the square root of 2:

js

```
function getRoot2() {
  return Math.SQRT2;
}

getRoot2(); // 1.4142135623730951
```

## Specifications

| Specification |
|---|
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-math.sqrt2` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| SQRT2 | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Math.SQRT2`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Math.pow()`
- `Math.sqrt()`

JavaScript › Reference › Standard built-in objects

# NaN

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/NaN · MDN page last modified 2025-07-08

**Baseline: Widely available** — available across major browsers since July 2015.

The **NaN** global property is a value representing Not-A-Number.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: NaN". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function sanitize(x) {
  if (isNaN(x)) {
    return NaN;
  }
  return x;
}

console.log(sanitize("1"));
// Expected output: "1"

console.log(sanitize("NotANumber"));
// Expected output: NaN
```

## Value

The same number value as `Number.NaN`.

| Property attributes of NaN | |
|---|---|
| Writable | no |
| Enumerable | no |
| Configurable | no |

## Description

`NaN` is a property of the *global object*. In other words, it is a variable in global scope.

In modern browsers, `NaN` is a non-configurable, non-writable property. Even when this is not the case, avoid overriding it.

There are five different types of operations that return `NaN`:

- Failed number conversion (e.g., explicit ones like `parseInt("blabla")`, `Number(undefined)`, or implicit ones like `Math.abs(undefined)`)
- Math operation where the result is not a real number (e.g., `Math.sqrt(-1)`)
- Indeterminate form (e.g., `0 * Infinity`, `1 ** Infinity`, `Infinity / Infinity`, `Infinity - Infinity`)
- A method or expression whose operand is or gets coerced to `NaN` (e.g., `7 ** NaN`, `7 * "blabla"`) — this means `NaN` is contagious
- Other cases where an invalid value is to be represented as a number (e.g., an invalid Date `new Date("blabla").getTime()`, `"".charCodeAt(1)`)

NaN and its behaviors are not invented by JavaScript. Its semantics in floating point arithmetic (including that NaN !== NaN) are specified by IEEE 754. NaN's behaviors include:

- If NaN is involved in a mathematical operation (but not bitwise operations), the result is usually also NaN. (See counter-example below.)
- When NaN is one of the operands of any relational comparison (>, <, >=, <=), the result is always false.
- NaN compares unequal (via ==, !=, ===, and !==) to any other value — including to another NaN value.

NaN is also one of the falsy values in JavaScript.

## Examples

### Testing against NaN

To tell if a value is NaN, use Number.isNaN() or isNaN() to most clearly determine whether a value is NaN — or, since NaN is the only value that compares unequal to itself, you can perform a self-comparison like x !== x.

js

```
NaN === NaN; // false
Number.NaN === NaN; // false
isNaN(NaN); // true
isNaN(Number.NaN); // true
Number.isNaN(NaN); // true

function valueIsNaN(v) {
  return v !== v;
}
valueIsNaN(1); // false
valueIsNaN(NaN); // true
valueIsNaN(Number.NaN); // true
```

However, do note the difference between isNaN() and Number.isNaN(): the former will return true if the value is currently NaN, or if it is going to be NaN after it is coerced to a number, while the latter will return true only if the value is currently NaN:

js

```
isNaN("hello world"); // true
Number.isNaN("hello world"); // false
```

For the same reason, using a BigInt value will throw an error with isNaN() and not with Number.isNaN():

js

```
isNaN(1n); // TypeError: Conversion from 'BigInt' to 'number' is not allowed.
Number.isNaN(1n); // false
```

Additionally, some array methods cannot find NaN, while others can. Namely, the index-finding ones (indexOf(), lastIndexOf()) cannot find NaN, while the value-finding ones (includes()) can:

```js
const arr = [2, 4, NaN, 12];
arr.indexOf(NaN); // -1
arr.includes(NaN); // true

// Methods accepting a properly defined predicate can always find NaN
arr.findIndex((n) => Number.isNaN(n)); // 2
```

For more information about `NaN` and its comparison, see Equality comparison and sameness.

### Observably distinct NaN values

It's possible to produce two floating point numbers with different binary representations but are both `NaN`, because in IEEE 754 encoding, any floating point number with exponent `0x7ff` and a non-zero mantissa is `NaN`. In JavaScript, you can do bit-level manipulation using typed arrays.

```js
const f2b = (x) => new Uint8Array(new Float64Array([x]).buffer);
const b2f = (x) => new Float64Array(x.buffer)[0];
// Get a byte representation of NaN
const n = f2b(NaN);
const m = f2b(NaN);
// Change the sign bit, which doesn't matter for NaN
n[7] += 2 ** 7;
// n[0] += 2**7; for big endian processors
const nan2 = b2f(n);
console.log(nan2); // NaN
console.log(Object.is(nan2, NaN)); // true
console.log(f2b(NaN)); // Uint8Array(8) [0, 0, 0, 0, 0, 0, 248, 127]
console.log(f2b(nan2)); // Uint8Array(8) [0, 0, 0, 0, 0, 0, 248, 255]
// Change the first bit, which is the least significant bit of the mantissa and doesn't
matter for NaN
m[0] = 1;
// m[7] = 1; for big endian processors
const nan3 = b2f(m);
console.log(nan3); // NaN
console.log(Object.is(nan3, NaN)); // true
console.log(f2b(NaN)); // Uint8Array(8) [0, 0, 0, 0, 0, 0, 248, 127]
console.log(f2b(nan3)); // Uint8Array(8) [1, 0, 0, 0, 0, 0, 248, 127]
```

### Silently escaping NaN

`NaN` propagates through mathematical operations, so it's usually sufficient to test for `NaN` once at the end of calculation to detect error conditions. The only case where `NaN` gets silently escaped is when using exponentiation with an exponent of `0`, which immediately returns `1` without testing the base's value.

```js
NaN ** 0 === 1; // true
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/global-object.html#sec-value-properties-of-the-global-object-nan |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| NaN | 1 | 12 | 1 | 4 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.NaN`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Number.NaN`
- `Number.isNaN()`
- `isNaN()`

JavaScript › Reference › Standard built-in objects

# Number

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Number · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since July 2015.

**`Number`** values represent floating-point numbers like `37` or `-9.25`.

The `Number` constructor contains constants and methods for working with numbers. Values of other types can be converted to numbers using the `Number()` function.

## Description

Numbers are most commonly expressed in literal forms like `255` or `3.14159`. The lexical grammar contains a more detailed reference.

```js
255; // two-hundred and fifty-five
255.0; // same number
255 === 255.0; // true
255 === 0xff; // true (hexadecimal notation)
255 === 0b11111111; // true (binary notation)
255 === 0.255e3; // true (decimal exponential notation)
```

A number literal like `37` in JavaScript code is a floating-point value, not an integer. There is no separate integer type in common everyday use. (JavaScript also has a `BigInt` type, but it's not designed to replace Number for everyday uses. `37` is still a number, not a BigInt.)

When used as a function, `Number(value)` converts a string or other value to the Number type. If the value can't be converted, it returns `NaN`.

```js
Number("123"); // returns the number 123
Number("123") === 123; // true

Number("unicorn"); // NaN
Number(undefined); // NaN
```

### Number encoding

The JavaScript `Number` type is a double-precision 64-bit binary format IEEE 754 value, like `double` in Java or C#. This means it can represent fractional values, but there are some limits to the stored number's magnitude and precision. Very briefly, an IEEE 754 double-precision number uses 64 bits to represent 3 parts:

- 1 bit for the *sign* (positive or negative)
- 11 bits for the *exponent* (-1022 to 1023)
- 52 bits for the *mantissa* (representing a number between 0 and 1)

The mantissa (also called *significand*) is the part of the number representing the actual value (significant digits). The exponent is the power of 2 that the mantissa should be multiplied by. Thinking about it as scientific notation:

$$\text{Number} = (-1)^{\text{sign}} \cdot (1 + \text{mantissa}) \cdot 2^{\text{exponent}}$$

The mantissa is stored with 52 bits, interpreted as digits after `1.…` in a binary fractional number. Therefore, the mantissa's precision is $2^{-52}$ (obtainable via `Number.EPSILON`), or about 15 to 17

significant decimal digits; arithmetic above that level of precision is subject to rounding.

The largest value a number can hold is $2^{1023} \times (2 - 2^{-52})$ (with the exponent being 1023 and the mantissa being 0.1111… in base 2), which is obtainable via `Number.MAX_VALUE`. Values higher than that are replaced with the special number constant `Infinity`.

Integers can only be represented without loss of precision in the range $-2^{53} + 1$ to $2^{53} - 1$, inclusive (obtainable via `Number.MIN_SAFE_INTEGER` and `Number.MAX_SAFE_INTEGER`), because the mantissa can only hold 53 bits (including the leading 1).

More details on this are described in the ECMAScript standard.

### Number coercion

Many built-in operations that expect numbers first coerce their arguments to numbers (which is largely why `Number` objects behave similarly to number primitives). The operation can be summarized as follows:

- Numbers are returned as-is.
- `undefined` turns into `NaN`.
- `null` turns into `0`.
- `true` turns into `1`; `false` turns into `0`.
- Strings are converted by parsing them as if they contain a number literal. Parsing failure results in `NaN`. There are some minor differences compared to an actual number literal:
  - Leading and trailing whitespace/line terminators are ignored.
  - A leading `0` digit does not cause the number to become an octal literal (or get rejected in strict mode).
  - `+` and `-` are allowed at the start of the string to indicate its sign. (In actual code, they "look like" part of the literal, but are actually separate unary operators.) However, the sign can only appear once, and must not be followed by whitespace.
  - `Infinity` and `-Infinity` are recognized as literals. In actual code, they are global variables.
  - Empty or whitespace-only strings are converted to `0`.
  - Numeric separators are not allowed.
- BigInts throw a `TypeError` to prevent unintended implicit coercion causing loss of precision.
- Symbols throw a `TypeError`.
- Objects are first converted to a primitive by calling their `[Symbol.toPrimitive]()` (with `"number"` as hint), `valueOf()`, and `toString()` methods, in that order. The resulting primitive is then converted to a number.

There are two ways to achieve nearly the same effect in JavaScript.

- Unary plus: `+x` does exactly the number coercion steps explained above to convert `x`.
- The `Number()` function: `Number(x)` uses the same algorithm to convert `x`, except that BigInts don't throw a `TypeError`, but return their number value, with possible loss of precision.

`Number.parseFloat()` and `Number.parseInt()` are similar to `Number()` but only convert strings, and have slightly different parsing rules. For example, `parseInt()` doesn't recognize the decimal point, and `parseFloat()` doesn't recognize the `0x` prefix.

### Integer conversion

Some operations expect integers, most notably those that work with array/string indices, date/time components, and number radixes. After performing the number coercion steps above, the result is truncated to an integer (by discarding the fractional part). If the number is ±Infinity, it's returned as-is. If the number is `NaN` or `-0`, it's returned as `0`. The result is therefore always an integer (which is not `-0`) or ±Infinity.

Notably, when converted to integers, both `undefined` and `null` become `0`, because `undefined` is converted to `NaN`, which also becomes `0`.

### Fixed-width number conversion

JavaScript has some lower-level functions that deal with the binary encoding of integer numbers, most notably bitwise operators and `TypedArray` objects. Bitwise operators always convert the operands to 32-bit integers. In these cases, after converting the value to a number, the number is then normalized to the given width by first truncating the fractional part and then taking the lowest bits in the integer's two's complement encoding.

```js
new Int32Array([1.1, 1.9, -1.1, -1.9]); // Int32Array(4) [ 1, 1, -1, -1 ]

new Int8Array([257, -257]); // Int8Array(2) [ 1, -1 ]
// 257 = 0001 0000 0001
//     =      0000 0001 (mod 2^8)
//     = 1
// -257 = 1110 1111 1111
//      =      1111 1111 (mod 2^8)
//      = -1 (as signed integer)

new Uint8Array([257, -257]); // Uint8Array(2) [ 1, 255 ]
// -257 = 1110 1111 1111
//      =      1111 1111 (mod 2^8)
//      = 255 (as unsigned integer)
```

## Constructor

**`Number()`**

Creates `Number` objects. When called as a function, it returns primitive values of type Number.

## Static properties

**`Number.EPSILON`**

The smallest interval between two representable numbers.

**`Number.MAX_SAFE_INTEGER`**

The maximum safe integer in JavaScript ($2^{53}$ - 1).

**`Number.MAX_VALUE`**

The largest positive representable number.

**`Number.MIN_SAFE_INTEGER`**

The minimum safe integer in JavaScript (-($2^{53}$ - 1)).

**`Number.MIN_VALUE`**

The smallest positive representable number—that is, the positive number closest to zero (without actually being zero).

**`Number.NaN`**

Special "**N**ot **a** **N**umber" value.

**`Number.NEGATIVE_INFINITY`**

Special value representing negative infinity. Returned on overflow.

**`Number.POSITIVE_INFINITY`**

Special value representing infinity. Returned on overflow.

## Static methods

**`Number.isFinite()`**

Determine whether the passed value is a finite number.

**`Number.isInteger()`**

Determine whether the passed value is an integer.

**`Number.isNaN()`**

Determine whether the passed value is `NaN`.

**`Number.isSafeInteger()`**

Determine whether the passed value is a safe integer (number between $-(2^{53} - 1)$ and $2^{53} - 1$).

**`Number.parseFloat()`**

This is the same as the global `parseFloat()` function.

**`Number.parseInt()`**

This is the same as the global `parseInt()` function.

## Instance properties

These properties are defined on `Number.prototype` and shared by all `Number` instances.

**`Number.prototype.constructor`**

The constructor function that created the instance object. For `Number` instances, the initial value is the `Number` constructor.

## Instance methods

**`Number.prototype.toExponential()`**

Returns a string representing the number in exponential notation.

**`Number.prototype.toFixed()`**

Returns a string representing the number in fixed-point notation.

**`Number.prototype.toLocaleString()`**

Returns a string with a language sensitive representation of this number. Overrides the `Object.prototype.toLocaleString()` method.

**`Number.prototype.toPrecision()`**

Returns a string representing the number to a specified precision in fixed-point or exponential notation.

**`Number.prototype.toString()`**

Returns a string representing the specified object in the specified *radix* ("base"). Overrides the `Object.prototype.toString()` method.

**`Number.prototype.valueOf()`**

Returns the primitive value of the specified object. Overrides the `Object.prototype.valueOf()` method.

## Examples

### Using the Number object to assign values to numeric variables

The following example uses the `Number` object's properties to assign values to several numeric variables:

```js
const biggestNum = Number.MAX_VALUE;
const smallestNum = Number.MIN_VALUE;
const infiniteNum = Number.POSITIVE_INFINITY;
const negInfiniteNum = Number.NEGATIVE_INFINITY;
const notANum = Number.NaN;
```

### Integer range for Number

The following example shows the minimum and maximum integer values that can be represented as `Number` object.

```js
const biggestInt = Number.MAX_SAFE_INTEGER; // (2**53 - 1) => 9007199254740991
const smallestInt = Number.MIN_SAFE_INTEGER; // -(2**53 - 1) => -9007199254740991
```

When parsing data that has been serialized to JSON, integer values falling outside of this range can be expected to become corrupted when JSON parser coerces them to `Number` type.

A possible workaround is to use `String` instead.

Larger numbers can be represented using the `BigInt` type.

### Using Number() to convert a Date object

The following example converts the `Date` object to a numerical value using `Number` as a function:

```js
const d = new Date("1995-12-17T03:24:00");
console.log(Number(d));
```

This logs `819199440000`.

### Convert numeric strings and null to numbers

```js
Number("123"); // 123
Number("123") === 123; // true
Number("12.3"); // 12.3
Number("12.00"); // 12
Number("123e-1"); // 12.3
Number(""); // 0
Number(null); // 0
Number("0x11"); // 17
Number("0b11"); // 3
Number("0o11"); // 9
Number("foo"); // NaN
Number("100a"); // NaN
Number("-Infinity"); // -Infinity
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-number-objects |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| Number | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| EPSILON | 34 | 12 | 25 | 21 | 9 | 34 | 25 | 21 | 9 | 2.0 | 37 | 9 | 1.0.0 | 1.0 | 0.12.0 |
| MAX_SAFE_INTEGER | 34 | 12 | 31 | 21 | 9 | 34 | 31 | 21 | 9 | 2.0 | 37 | 9 | 1.0.0 | 1.0 | 0.12.0 |
| MAX_VALUE | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| MIN_SAFE_INTEGER | 34 | 12 | 31 | 21 | 9 | 34 | 31 | 21 | 9 | 2.0 | 37 | 9 | 1.0.0 | 1.0 | 0.12.0 |
| MIN_VALUE | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| NEGATIVE_INFINITY | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| NaN | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| Number() constructor | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| POSITIVE_INFINITY | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| isFinite | 19 | 12 | 16 | 15 | 9 | 25 | 16 | 14 | 9 | 1.5 | 4.4 | 9 | 1.0.0 | 1.0 | 0.10.0 |
| isInteger | 34 | 12 | 16 | 21 | 9 | 34 | 16 | 21 | 9 | 2.0 | 37 | 9 | 1.0.0 | 1.0 | 0.12.0 |
| isNaN | 25 | 12 | 15 | 15 | 9 | 25 | 15 | 14 | 9 | 1.5 | 4.4 | 9 | 1.0.0 | 1.0 | 0.10.0 |
| isSafeInteger | 34 | 12 | 32 | 21 | 9 | 34 | 32 | 21 | 9 | 2.0 | 37 | 9 | 1.0.0 | 1.0 | 0.12.0 |
| parseFloat | 34 | 12 | 25 | 21 | 9 | 34 | 25 | 21 | 9 | 2.0 | 37 | 9 | 1.0.0 | 1.0 | 0.12.0 |
| parseInt | 34 | 12 | 25 | 21 | 9 | 34 | 25 | 21 | 9 | 2.0 | 37 | 9 | 1.0.0 | 1.0 | 0.12.0 |
| toExponential | 1 | 12 | 1 | 7 | 2 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| toFixed | 1 | 12 | 1 | 7 | 2 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| toLocaleString | 1 | 12[1] | 1 | 4 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| locales parameter | 24 | 12 | 29 | 15 | 10 | 26 | 56 | 14 | 10 | 1.5 | 4.4 | 10 | 1.0.0 | 1.8[2,3] | 13.0.0[4,5] |

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `options` parameter | 24 | 12 | 29 | 15 | 10 | 26 | 56 | 14 | 10 | 1.5 | 4.4 | 10 | 1.0.0 | 1.0 | 0.12.0 |
| `toPrecision` | 1 | 12 | 1 | 7 | 2 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| `toString` | 1 | 12 | 1 | 4 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| `valueOf` | 1 | 12 | 1 | 4 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

1. Before Edge 18, numbers are rounded to 15 decimal digits. For example, `(1000000000000005).toLocaleString('en-US')` returns `"1,000,000,000,000,010"`.
2. Also from version 1.0–1.8: partial support.
3. Also from version 1.0–1.8: Only the locale data for `en-US` is available.
4. Also from version 0.12.0–13.0.0: partial support.
5. Also from version 0.12.0–13.0.0: Before version 13.0.0, only the locale data for `en-US` is available by default. When other locales are specified, the function silently falls back to `en-US`. To make full ICU (locale) data available before version 13, see Node.js documentation on the `--with-intl` option and how to provide the data.

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Number`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of modern `Number` behavior (with support binary and octal literals) in `core-js`
- `NaN`
- Arithmetic operators
- `Math`
- `BigInt`

---

JavaScript › Reference › Standard built-in objects › Number

# Number() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Number/Number · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Number()`** constructor creates `Number` objects. When called as a function, it returns primitive values of type Number.

## Syntax

```
new Number(value)
Number(value)
```

> **Note:** `Number()` can be called with or without `new`, but with different effects. See Return value.

### Parameters

**`value`**

The numeric value of the object being created.

### Return value

When `Number()` is called as a function (without `new`), it returns `value` coerced to a number primitive. Specially, BigInts values are converted to numbers instead of throwing. If `value` is absent, it becomes `0`.

When `Number()` is called as a constructor (with `new`), it uses the coercion process above and returns a wrapping `Number` object, which is **not** a primitive.

> **Warning:** You should rarely find yourself using `Number` as a constructor.

## Examples

### Creating Number objects

js

```
const a = new Number("123"); // a === 123 is false
const b = Number("123"); // b === 123 is true
a instanceof Number; // is true
b instanceof Number; // is false
typeof a; // "object"
typeof b; // "number"
```

### Using Number() to convert a BigInt to a number

`Number()` is the only case where a BigInt can be converted to a number without throwing, because it's very explicit.

js · ✗ bad example

```
+1n; // TypeError: Cannot convert a BigInt value to a number
0 + 1n; // TypeError: Cannot mix BigInt and other types, use explicit conversions
```

js · ✓ good example

```
Number(1n); // 1
```

Note that this may result in loss of precision, if the BigInt is too large to be safely represented.

js

```
BigInt(Number(2n ** 54n + 1n)) === 2n ** 54n + 1n; // false
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-number-constructor |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `Number()` constructor | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Number.Number`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of modern `Number` behavior (with support binary and octal literals) in `core-js`
- `NaN`
- `Math`
- `BigInt`

---



# Number.isFinite()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Number/isFinite · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since September 2015.

The **`Number.isFinite()`** static method determines whether the passed value is a finite number — that is, it checks that a given value is a number, and the number is neither positive `Infinity`, negative `Infinity`, nor `NaN`.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Number.isFinite()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(Number.isFinite(1 / 0));
// Expected output: false

console.log(Number.isFinite(10 / 5));
// Expected output: true

console.log(Number.isFinite(0 / 0));
// Expected output: false
```

## Syntax

js

```
Number.isFinite(value)
```

### Parameters

**`value`**

The value to be tested for finiteness.

### Return value

The boolean value `true` if the given value is a finite number. Otherwise `false`.

## Examples

### Using isFinite()

```js
Number.isFinite(Infinity); // false
Number.isFinite(NaN); // false
Number.isFinite(-Infinity); // false

Number.isFinite(0); // true
Number.isFinite(2e64); // true
```

### Difference between Number.isFinite() and global isFinite()

In comparison to the global `isFinite()` function, this method doesn't first convert the parameter to a number. This means only values of the type number *and* are finite return `true`, and non-numbers always return `false`.

```js
isFinite("0"); // true; coerced to number 0
Number.isFinite("0"); // false
isFinite(null); // true; coerced to number 0
Number.isFinite(null); // false
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-number.isfinite` |

## Browser compatibility

| | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| Feature | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `isFinite` | 19 | 12 | 16 | 15 | 9 | 25 | 16 | 14 | 9 | 1.5 | 4.4 | 9 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Number.isFinite`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Number.isFinite` in `core-js`
- es-shims polyfill of `Number.isFinite`
- `Number`
- `isFinite()`

JavaScript › Reference › Standard built-in objects › Number

# Number.isInteger()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Number/isInteger · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since September 2015.

The **`Number.isInteger()`** static method determines whether the passed value is an integer.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Number.isInteger()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function fits(x, y) {
  if (Number.isInteger(y / x)) {
    return "Fits!";
  }
  return "Does NOT fit!";
}

console.log(fits(5, 10));
// Expected output: "Fits!"

console.log(fits(5, 11));
// Expected output: "Does NOT fit!"
```

## Syntax

js

```
Number.isInteger(value)
```

### Parameters

**`value`**

The value to be tested for being an integer.

### Return value

The boolean value `true` if the given value is an integer. Otherwise `false`.

## Description

If the target value is an integer, return `true`, otherwise return `false`. If the value is `NaN` or `Infinity`, return `false`. The method will also return `true` for floating point numbers that can be represented as integer. It will always return `false` if the value is not a number.

Note that some number literals, while looking like non-integers, actually represent integers — due to the precision limit of ECMAScript floating-point number encoding (IEEE-754). For example, `5.0000000000000001` only differs from `5` by `1e-16`, which is too small to be represented. (For reference, `Number.EPSILON` stores the distance between 1 and the next representable floating-point number greater than 1, and that is about `2.22e-16`.) Therefore, `5.000000000000000` will be represented with the same encoding as `5`, thus making `Number.isInteger(5.0000000000000001)` return `true`.

In a similar sense, numbers around the magnitude of `Number.MAX_SAFE_INTEGER` will suffer from loss of precision and make `Number.isInteger` return `true` even when it's not an integer. (The actual threshold varies based on how many bits are needed to represent the decimal — for example, `Number.isInteger(4500000000000000.1)` is `true`, but `Number.isInteger(4500000000000000.5)` is `false`.)

## Examples

### Using isInteger

```js
Number.isInteger(0); // true
Number.isInteger(1); // true
Number.isInteger(-100000); // true
Number.isInteger(99999999999999999999); // true

Number.isInteger(0.1); // false
Number.isInteger(Math.PI); // false

Number.isInteger(NaN); // false
Number.isInteger(Infinity); // false
Number.isInteger(-Infinity); // false
Number.isInteger("10"); // false
Number.isInteger(true); // false
Number.isInteger(false); // false
Number.isInteger([1]); // false

Number.isInteger(5.0); // true
Number.isInteger(5.000000000000001); // false
Number.isInteger(5.0000000000000001); // true, because of loss of precision
Number.isInteger(4500000000000000.1); // true, because of loss of precision
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-number.isinteger` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `isInteger` | 34 | 12 | 16 | 21 | 9 | 34 | 16 | 21 | 9 | 2.0 | 37 | 9 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Number.isInteger`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Number.isInteger` in `core-js`
- es-shims polyfill of `Number.isInteger`
- `Number`

---

JavaScript › Reference › Standard built-in objects › Number

# Number.isNaN()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Number/isNaN · MDN page last modified 2026-10-26
**Baseline: Widely available** — available across major browsers since September 2015.

The **`Number.isNaN()`** static method determines whether the passed value is the number value `NaN`, and returns `false` if the input is not of the Number type. It is a more robust version of the original, global `isNaN()` function.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Number.isNaN()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function typeOfNaN(x) {
  if (Number.isNaN(x)) {
    return "Number NaN";
  }
  if (isNaN(x)) {
    return "NaN";
  }
}

console.log(typeOfNaN("100F"));
// Expected output: "NaN"

console.log(typeOfNaN(NaN));
// Expected output: "Number NaN"
```

## Syntax

js

```
Number.isNaN(value)
```

### Parameters

**`value`**

The value to be tested for `NaN`.

### Return value

The boolean value `true` if the given value is a number with value `NaN`. Otherwise, `false`.

## Description

The function `Number.isNaN()` provides a convenient way to check for equality with `NaN`. Note that you cannot test for equality with `NaN` using either the `==` or `===` operators, because unlike all other value comparisons in JavaScript, these evaluate to `false` whenever one operand is `NaN`, even if the other operand is also `NaN`.

Since `x !== x` is only true for `NaN` among all possible JavaScript values, `Number.isNaN(x)` can also be replaced with a test for `x !== x`, despite the latter being less readable.

As opposed to the global `isNaN()` function, the `Number.isNaN()` method doesn't force-convert the parameter to a number. This makes it safe to pass values that would normally convert to `NaN` but aren't actually the same value as `NaN`. This also means that only values of the Number type that are also `NaN` return `true`.

## Examples

### Using isNaN()

```js
Number.isNaN(NaN); // true
Number.isNaN(Number.NaN); // true
Number.isNaN(0 / 0); // true
Number.isNaN(37); // false
```

### Difference between Number.isNaN() and global isNaN()

`Number.isNaN()` doesn't attempt to convert the parameter to a number, so non-numbers always return `false`. The following are all `false`:

```js
Number.isNaN("NaN");
Number.isNaN(undefined);
Number.isNaN({});
Number.isNaN("blabla");
Number.isNaN(true);
Number.isNaN(null);
Number.isNaN("37");
Number.isNaN("37.37");
Number.isNaN("");
Number.isNaN(" ");
```

The global `isNaN()` coerces its parameter to a number:

```js
isNaN("NaN"); // true
isNaN(undefined); // true
isNaN({}); // true
isNaN("blabla"); // true
isNaN(true); // false, this is coerced to 1
isNaN(null); // false, this is coerced to 0
isNaN("37"); // false, this is coerced to 37
isNaN("37.37"); // false, this is coerced to 37.37
isNaN(""); // false, this is coerced to 0
isNaN(" "); // false, this is coerced to 0
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-number.isnan` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `isNaN` | 25 | 12 | 15 | 15 | 9 | 25 | 15 | 14 | 9 | 1.5 | 4.4 | 9 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Number.isNaN`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Number.isNaN` in `core-js`
- es-shims polyfill of `Number.isNaN`
- `Number`
- `isNaN()`

---

JavaScript › Reference › Standard built-in objects › Number

# Number.isSafeInteger()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Number/isSafeInteger · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since September 2015.

The **`Number.isSafeInteger()`** static method determines whether the provided value is a number that is a *safe integer*.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Number.isSafeInteger()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function warn(x) {
  if (Number.isSafeInteger(x)) {
    return "Precision safe.";
  }
  return "Precision may be lost!";
}

console.log(warn(2 ** 53));
// Expected output: "Precision may be lost!"

console.log(warn(2 ** 53 - 1));
// Expected output: "Precision safe."
```

## Syntax

```js
Number.isSafeInteger(testValue)
```

### Parameters

**`testValue`**

The value to be tested for being a safe integer.

### Return value

The boolean value `true` if the given value is a number that is a safe integer. Otherwise `false`.

## Description

The safe integers consist of all integers from $-(2^{53} - 1)$ to $2^{53} - 1$, inclusive (±9,007,199,254,740,991). A safe integer is an integer that:

- can be exactly represented as an IEEE-754 double precision number, and
- whose IEEE-754 representation cannot be the result of rounding any other integer to fit the IEEE-754 representation.

For example, $2^{53} - 1$ is a safe integer: it can be exactly represented, and no other integer rounds to it under any IEEE-754 rounding mode. In contrast, $2^{53}$ is *not* a safe integer: it can be exactly represented in IEEE-754, but the integer $2^{53} + 1$ can't be directly represented in IEEE-754 but instead rounds to $2^{53}$ under round-to-nearest and round-to-zero rounding.

Handling values larger or smaller than ~9 quadrillion with full precision requires using an arbitrary precision arithmetic library. See What Every Programmer Needs to Know about Floating Point Arithmetic for more information on floating point representations of numbers.

For larger integers, consider using the `BigInt` type.

## Examples

### Using isSafeInteger()

```js
Number.isSafeInteger(3); // true
Number.isSafeInteger(2 ** 53); // false
Number.isSafeInteger(2 ** 53 - 1); // true
Number.isSafeInteger(NaN); // false
Number.isSafeInteger(Infinity); // false
Number.isSafeInteger("3"); // false
Number.isSafeInteger(3.1); // false
Number.isSafeInteger(3.0); // true
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-number.issafeinteger |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `isSafeInteger` | 34 | 12 | 32 | 21 | 9 | 34 | 32 | 21 | 9 | 2.0 | 37 | 9 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Number.isSafeInteger`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Number.isSafeInteger` in `core-js`
- es-shims polyfill of `Number.isSafeInteger`
- `Number`
- `Number.MIN_SAFE_INTEGER`
- `Number.MAX_SAFE_INTEGER`
- `BigInt`

---

JavaScript › Reference › Standard built-in objects › Number

# Number.parseFloat()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Number/parseFloat · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since September 2015.

The **`Number.parseFloat()`** static method parses an argument and returns a floating point number. If a number cannot be parsed from the argument, it returns `NaN`.

## Try it

*▷ Interactive example on MDN: "JavaScript Demo: Number.parseFloat()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function circumference(r) {
  if (Number.isNaN(Number.parseFloat(r))) {
    return 0;
  }
  return parseFloat(r) * 2.0 * Math.PI;
}

console.log(circumference("4.567abcdefgh"));
// Expected output: 28.695307297889173

console.log(circumference("abcdefgh"));
// Expected output: 0
```

## Syntax

```js
Number.parseFloat(string)
```

### Parameters

**`string`**

The value to parse, coerced to a string. Leading whitespace in this argument is ignored.

### Return value

A floating point number parsed from the given `string`.

Or `NaN` when the first non-whitespace character cannot be converted to a number.

## Examples

### Number.parseFloat vs. parseFloat

This method has the same functionality as the global `parseFloat()` function:

```js
Number.parseFloat === parseFloat; // true
```

Its purpose is modularization of globals.

See `parseFloat()` for more detail and examples.

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-number.parsefloat` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `parseFloat` | 34 | 12 | 25 | 21 | 9 | 34 | 25 | 21 | 9 | 2.0 | 37 | 9 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Number.parseFloat`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Number.parseFloat` in `core-js`
- es-shims polyfill of `Number.parseFloat`
- `Number`
- `parseFloat()`

JavaScript › Reference › Standard built-in objects › Number

# Number.parseInt()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Number/parseInt · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since September 2015.

The **`Number.parseInt()`** static method parses a string argument and returns an integer of the specified radix or base.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Number.parseInt()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function roughScale(x, base) {
  const parsed = Number.parseInt(x, base);
  if (Number.isNaN(parsed)) {
    return 0;
  }
  return parsed * 100;
}

console.log(roughScale(" 0xF", 16));
// Expected output: 1500

console.log(roughScale("321", 2));
// Expected output: 0
```

## Syntax

js

```
Number.parseInt(string)
Number.parseInt(string, radix)
```

### Parameters

**`string`**

The value to parse, coerced to a string. Leading whitespace in this argument is ignored.

**`radix`** Optional

An integer between `2` and `36` that represents the *radix* (the base in mathematical numeral systems) of the `string`.

If `radix` is undefined or `0`, it is assumed to be `10` except when the number begins with the code unit pairs `0x` or `0X`, in which case a radix of `16` is assumed.

### Return value

An integer parsed from the given `string`.

If the `radix` is smaller than `2` or bigger than `36`, or the first non-whitespace character cannot be converted to a number, `NaN` is returned.

## Examples

### Number.parseInt vs. parseInt

This method has the same functionality as the global `parseInt()` function:

```js
Number.parseInt === parseInt; // true
```

Its purpose is modularization of globals. Please see `parseInt()` for more detail and examples.

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-number.parseint |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Nodejs |
| `parseInt` | 34 | 12 | 25 | 21 | 9 | 34 | 25 | 21 | 9 | 2.0 | 37 | 9 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Number.parseInt`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Number.parseInt` in `core-js`
- es-shims polyfill of `Number.parseInt`
- `Number`
- `parseInt()`

---

JavaScript › Reference › Standard built-in objects › Number

# Number.EPSILON

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Number/EPSILON · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since September 2015.

The **`Number.EPSILON`** static data property represents the difference between 1 and the smallest floating point number greater than 1.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Number.EPSILON". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const result = Math.abs(0.2 - 0.3 + 0.1);

console.log(result);
// Expected output: 2.7755575615628914e-17

console.log(result < Number.EPSILON);
// Expected output: true
```

## Value

$2^{-52}$, or approximately `2.2204460492503130808472633361816E-16`.

| Property attributes of `Number.EPSILON` | |
|---|---|
| Writable | no |
| Enumerable | no |
| Configurable | no |

## Description

`Number.EPSILON` is the difference between 1 and the next greater number representable in the Number format, because double precision floating point format only has 52 bits to represent the mantissa, and the lowest bit has a significance of $2^{-52}$.

Note that the absolute accuracy of floating numbers decreases as the number gets larger, because the exponent grows while the mantissa's accuracy stays the same. `Number.MIN_VALUE` is the smallest representable positive number, which is much smaller than `Number.EPSILON`.

Because `EPSILON` is a static property of `Number`, you always use it as `Number.EPSILON`, rather than as a property of a number value.

## Examples

### Testing equality

Any number encoding system occupying a finite number of bits, of whatever base you choose (e.g., decimal or binary), will *necessarily* be unable to represent all numbers exactly, because you are trying to represent an infinite number of points on the number line using a finite amount of memory. For example, a base-10 (decimal) system cannot represent 1/3 exactly, and a base-2 (binary) system cannot represent `0.1` exactly. Thus, for example, `0.1 + 0.2` is not exactly equal to `0.3`:

js

```
console.log(0.1 + 0.2); // 0.30000000000000004
console.log(0.1 + 0.2 === 0.3); // false
```

For this reason, it is often advised that **floating point numbers should never be compared with `===`**. Instead, we can deem two numbers as equal if they are *close enough* to each other. The `Number.EPSILON` constant is usually a reasonable threshold for errors if the arithmetic is around the magnitude of `1`, because `EPSILON`, in essence, specifies how accurate the number "1" is.

```js
function equal(x, y) {
  return Math.abs(x - y) < Number.EPSILON;
}

const x = 0.2;
const y = 0.3;
const z = 0.1;
console.log(equal(x + z, y)); // true
```

However, `Number.EPSILON` is inappropriate for any arithmetic operating on a larger magnitude. If your data is on the $10^3$ order of magnitude, the decimal part will have a much smaller accuracy than `Number.EPSILON`:

```js
function equal(x, y) {
  return Math.abs(x - y) < Number.EPSILON;
}

const x = 1000.1;
const y = 1000.2;
const z = 2000.3;
console.log(x + y); // 2000.3000000000002; error of 10^-13 instead of 10^-16
console.log(equal(x + y, z)); // false
```

In this case, a larger tolerance is required. As the numbers compared have a magnitude of approximately `2000`, a multiplier such as `2000 * Number.EPSILON` creates enough tolerance for this instance.

```js
function equal(x, y, tolerance = Number.EPSILON) {
  return Math.abs(x - y) < tolerance;
}

const x = 1000.1;
const y = 1000.2;
const z = 2000.3;
console.log(equal(x + y, z, 2000 * Number.EPSILON)); // true
```

In addition to magnitude, it is important to consider the *accuracy* of your input. For example, if the numbers are collected from a form input and the input value can only be adjusted by steps of `0.1` (i.e., `<input type="number" step="0.1">`), it usually makes sense to allow a much larger tolerance, such as `0.01`, since the data only has a precision of `0.1`.

> **Note:** Important takeaway: do not simply use `Number.EPSILON` as a threshold for equality testing. Use a threshold that is appropriate for the magnitude and accuracy of the numbers you are comparing.

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-number.epsilon |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| EPSILON | 34 | 12 | 25 | 21 | 9 | 34 | 25 | 21 | 9 | 2.0 | 37 | 9 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Number.EPSILON`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Number.EPSILON` in `core-js`
- es-shims polyfill of `Number.EPSILON`
- `Number`

---

JavaScript › Reference › Standard built-in objects › Number

# Number.MAX_SAFE_INTEGER

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Number/MAX_SAFE_INTEGER · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since September 2015.

The **`Number.MAX_SAFE_INTEGER`** static data property represents the maximum safe integer in JavaScript ($2^{53} - 1$).

For larger integers, consider using `BigInt`.

## Try it

*▷ Interactive example on MDN: "JavaScript Demo: Number.MAX_SAFE_INTEGER". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const x = Number.MAX_SAFE_INTEGER + 1;
const y = Number.MAX_SAFE_INTEGER + 2;

console.log(Number.MAX_SAFE_INTEGER);
// Expected output: 9007199254740991

console.log(x);
// Expected output: 9007199254740992

console.log(x === y);
// Expected output: true
```

## Value

`9007199254740991` (9,007,199,254,740,991, or ~9 quadrillion).

| Property attributes of `Number.MAX_SAFE_INTEGER` | |
|---|---|
| Writable | no |
| Enumerable | no |
| Configurable | no |

## Description

Double precision floating point format only has 52 bits to represent the mantissa, so it can only safely represent integers between $-(2^{53} - 1)$ and $2^{53} - 1$. "Safe" in this context refers to the ability to represent integers exactly and to compare them correctly. For example, `Number.MAX_SAFE_INTEGER + 1 === Number.MAX_SAFE_INTEGER + 2` will evaluate to true, which is mathematically incorrect. See `Number.isSafeInteger()` for more information.

As mentioned in `Number.EPSILON`, the precision of numbers depends on their magnitude. `Number.MAX_SAFE_INTEGER` represents the largest value at which integer-level operations can be performed precisely, but you can still perform meaningful arithmetic on numbers larger than that, just without integer-level precision. The largest representable number in JavaScript is actually `Number.MAX_VALUE`, which is approximately $1.7976931348623157 \times 10^{308}$.

Because `MAX_SAFE_INTEGER` is a static property of `Number`, you always use it as `Number.MAX_SAFE_INTEGER`, rather than as a property of a number value.

## Examples

### Return value of MAX_SAFE_INTEGER

```js
Number.MAX_SAFE_INTEGER; // 9007199254740991
```

### Relationship between MAX_SAFE_INTEGER and EPSILON

`Number.EPSILON` is $2^{-52}$, while `MAX_SAFE_INTEGER` is $2^{53} - 1$ — both of them are derived from the width of the mantissa, which is 53 bits (with the highest bit always being 1). Multiplying them will give a value very close — but not equal — to 2.

```js
Number.MAX_SAFE_INTEGER * Number.EPSILON; // 1.9999999999999998
```

## Specifications

| Specification |
|---|
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-number.max_safe_integer |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| MAX_SAFE_INTEGER | 34 | 12 | 31 | 21 | 9 | 34 | 31 | 21 | 9 | 2.0 | 37 | 9 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Number.MAX_SAFE_INTEGER`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Number.MAX_SAFE_INTEGER` in `core-js`
- es-shims polyfill of `Number.MAX_SAFE_INTEGER`
- `Number.MIN_SAFE_INTEGER`
- `Number.MAX_VALUE`
- `Number.isSafeInteger()`
- `BigInt`

---

JavaScript › Reference › Standard built-in objects › Number

# Number.MAX_VALUE

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Number/MAX_VALUE · MDN page last modified 2025-10-22

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Number.MAX_VALUE`** static data property represents the maximum numeric value representable in JavaScript.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Number.MAX_VALUE". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function multiply(x, y) {
  if (x * y > Number.MAX_VALUE) {
    return "Process as Infinity";
  }
  return x * y;
}

console.log(multiply(1.7976931348623157e308, 1));
// Expected output: 1.7976931348623157e+308

console.log(multiply(1.7976931348623157e308, 2));
// Expected output: "Process as Infinity"
```

## Value

$2^{1024}$ - $2^{971}$, or approximately `1.7976931348623157E+308`.

| Property attributes of `Number.MAX_VALUE` | |
|---|---|
| Writable | no |
| Enumerable | no |
| Configurable | no |

## Description

Values larger than `MAX_VALUE` are represented as `Infinity` and will lose their actual value. As mentioned in `Number.EPSILON`, the precision of numbers depends on their magnitude. Integers can only be represented precisely up to `Number.MAX_SAFE_INTEGER`, which is $2^{53}$ - 1.

Because `MAX_VALUE` is a static property of `Number`, you always use it as `Number.MAX_VALUE`, rather than as a property of a number value.

## Examples

### Using MAX_VALUE

The following code multiplies two numeric values. If the result is less than or equal to `MAX_VALUE`, the `func1` function is called; otherwise, the `func2` function is called.

```js
if (num1 * num2 <= Number.MAX_VALUE) {
  func1();
} else {
  func2();
}
```

## Specifications

| Specification |
|---|
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-number.max_value` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `MAX_VALUE` | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Number.MAX_VALUE`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Number.MIN_VALUE`
- `Number.MAX_SAFE_INTEGER`
- `Number`

---



# Number.MIN_SAFE_INTEGER

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Number/MIN_SAFE_INTEGER · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since September 2015.

The **`Number.MIN_SAFE_INTEGER`** static data property represents the minimum safe integer in JavaScript, or -($2^{53}$ - 1).

To represent integers smaller than this, consider using `BigInt`.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Number.MIN_SAFE_INTEGER". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const x = Number.MIN_SAFE_INTEGER - 1;
const y = Number.MIN_SAFE_INTEGER - 2;

console.log(Number.MIN_SAFE_INTEGER);
// Expected output: -9007199254740991

console.log(x);
// Expected output: -9007199254740992

console.log(x === y);
// Expected output: true
```

## Value

`-9007199254740991` (-9,007,199,254,740,991, or about -9 quadrillion).

| Property attributes of `Number.MIN_SAFE_INTEGER` | |
|---|---|
| Writable | no |
| Enumerable | no |
| Configurable | no |

## Description

Double precision floating point format only has 52 bits to represent the mantissa, so it can only safely represent integers between -($2^{53}$ – 1) and $2^{53}$ – 1. Safe in this context refers to the ability to represent integers exactly and to correctly compare them. For example, `Number.MIN_SAFE_INTEGER - 1 === Number.MIN_SAFE_INTEGER - 2` will evaluate to true, which is mathematically incorrect. See `Number.isSafeInteger()` for more information.

As mentioned in `Number.EPSILON`, the precision of numbers depends on their magnitude. `Number.MIN_SAFE_INTEGER` represents the smallest value at which integer-level operations can be performed precisely, but you can still perform meaningful arithmetic on numbers more negative than that, just without integer-level precision. The largest representable number in JavaScript is actually `Number.MAX_VALUE`, which is approximately $1.7976931348623157 \times 10^{308}$.

Because `MIN_SAFE_INTEGER` is a static property of `Number`, you always use it as `Number.MIN_SAFE_INTEGER`, rather than as a property of a number value.

## Examples

### Using MIN_SAFE_INTEGER

```js
Number.MIN_SAFE_INTEGER; // -9007199254740991
-(2 ** 53 - 1); // -9007199254740991
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-number.min_safe_integer |

## Browser compatibility

| | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| Feature | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Nodejs |
| MIN_SAFE_INTEGER | 34 | 12 | 31 | 21 | 9 | 34 | 31 | 21 | 9 | 2.0 | 37 | 9 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Number.MIN_SAFE_INTEGER`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Number.MIN_SAFE_INTEGER` in `core-js`
- es-shims polyfill of `Number.MIN_SAFE_INTEGER`
- `Number.MAX_SAFE_INTEGER`
- `Number.isSafeInteger()`
- `BigInt`

JavaScript › Reference › Standard built-in objects › Number

# Number.MIN_VALUE

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Number/MIN_VALUE · MDN page last modified 2025-07-10

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Number.MIN_VALUE`** static data property represents the smallest positive numeric value representable in JavaScript.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Number.MIN_VALUE". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function divide(x, y) {
  if (x / y < Number.MIN_VALUE) {
    return "Process as 0";
  }
  return x / y;
}

console.log(divide(5e-324, 1));
// Expected output: 5e-324

console.log(divide(5e-324, 2));
// Expected output: "Process as 0"
```

## Value

$2^{-1074}$, or `5E-324`.

| Property attributes of `Number.MIN_VALUE` | |
|---|---|
| Writable | no |
| Enumerable | no |
| Configurable | no |

## Description

`Number.MIN_VALUE` is the smallest positive number (not the most negative number) that can be represented within float precision — in other words, the number closest to 0. The ECMAScript spec doesn't define a precise value that implementations are required to support — instead the spec says, *"must be the smallest non-zero positive value that can actually be represented by the implementation"*. This is because small IEEE-754 floating point numbers are denormalized, but implementations are not required to support this representation, in which case `Number.MIN_VALUE` may be larger.

In practice, its precise value in mainstream engines like V8 (used by Chrome, Edge, Node.js), SpiderMonkey (used by Firefox), and JavaScriptCore (used by Safari) is $2^{-1074}$, or `5E-324`.

Because `MIN_VALUE` is a static property of `Number`, you always use it as `Number.MIN_VALUE`, rather than as a property of a number value.

## Examples

### Using MIN_VALUE

The following code divides two numeric values. If the result is greater than or equal to `MIN_VALUE`, the `func1` function is called; otherwise, the `func2` function is called.

```js
if (num1 / num2 >= Number.MIN_VALUE) {
  func1();
} else {
  func2();
}
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-number.min_value |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| MIN_VALUE | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Number.MIN_VALUE`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Number.MAX_VALUE`

---

JavaScript › Reference › Standard built-in objects › Number

# Number.NaN

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Number/NaN · MDN page last modified 2025-07-10

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Number.NaN`** static data property represents Not-A-Number, which is equivalent to `NaN`. For more information about the behaviors of `NaN`, see the description for the global property.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Number.NaN". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function clean(x) {
  if (x === Number.NaN) {
    // Can never be true
    return null;
  }
  if (isNaN(x)) {
    return 0;
  }
}

console.log(clean(Number.NaN));
// Expected output: 0
```

## Value

The number value `NaN`.

| Property attributes of `Number.NaN` | |
|---|---|
| Writable | no |
| Enumerable | no |
| Configurable | no |

## Description

Because `NaN` is a static property of `Number`, you always use it as `Number.NaN`, rather than as a property of a number value.

## Examples

### Checking whether values are numeric

js

```
function sanitize(x) {
  if (isNaN(x)) {
    return Number.NaN;
  }
  return x;
}
```

## Specifications

| Specification |
|---|
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-number.nan` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| NaN | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Number.NaN`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `NaN`
- `Number.isNaN()`

---

JavaScript › Reference › Standard built-in objects › Number

# Number.NEGATIVE_INFINITY

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Number/NEGATIVE_INFINITY · MDN page last modified 2025-07-10

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Number.NEGATIVE_INFINITY`** static data property represents the negative Infinity value.

## Try it

*▷ Interactive example on MDN: "JavaScript Demo: Number.NEGATIVE_INFINITY". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function checkNumber(smallNumber) {
  if (smallNumber === Number.NEGATIVE_INFINITY) {
    return "Process number as -Infinity";
  }
  return smallNumber;
}

console.log(checkNumber(-Number.MAX_VALUE));
// Expected output: -1.7976931348623157e+308

console.log(checkNumber(-Number.MAX_VALUE * 2));
// Expected output: "Process number as -Infinity"
```

## Value

The same as the negative value of the global `Infinity` property.

| Property attributes of `Number.NEGATIVE_INFINITY` | |
|---|---|
| Writable | no |
| Enumerable | no |
| Configurable | no |

## Description

The `Number.NEGATIVE_INFINITY` value behaves slightly differently than mathematical infinity:

- Any positive value, including `POSITIVE_INFINITY`, multiplied by `NEGATIVE_INFINITY` is `NEGATIVE_INFINITY`.
- Any negative value, including `NEGATIVE_INFINITY`, multiplied by `NEGATIVE_INFINITY` is `POSITIVE_INFINITY`.
- Any positive value divided by `NEGATIVE_INFINITY` is negative zero (as defined in IEEE 754).
- Any negative value divided by `NEGATIVE_INFINITY` is positive zero (as defined in IEEE 754).
- Zero multiplied by `NEGATIVE_INFINITY` is `NaN`.
- `NaN` multiplied by `NEGATIVE_INFINITY` is `NaN`.
- `NEGATIVE_INFINITY`, divided by any negative value except `NEGATIVE_INFINITY`, is `POSITIVE_INFINITY`.
- `NEGATIVE_INFINITY`, divided by any positive value except `POSITIVE_INFINITY`, is `NEGATIVE_INFINITY`.
- `NEGATIVE_INFINITY`, divided by either `NEGATIVE_INFINITY` or `POSITIVE_INFINITY`, is `NaN`.
- `x > Number.NEGATIVE_INFINITY` is true for any number *x* that isn't `NEGATIVE_INFINITY`.

You might use the `Number.NEGATIVE_INFINITY` property to indicate an error condition that returns a finite number in case of success. Note, however, that `NaN` would be more appropriate in such a case.

Because `NEGATIVE_INFINITY` is a static property of `Number`, you always use it as `Number.NEGATIVE_INFINITY`, rather than as a property of a number value.

## Examples

### Using NEGATIVE_INFINITY

In the following example, the variable `smallNumber` is assigned a value that is smaller than the minimum value. When the `if` statement executes, `smallNumber` has the value `-Infinity`, so `smallNumber` is set to a more manageable value before continuing.

```js
let smallNumber = -Number.MAX_VALUE * 2;

if (smallNumber === Number.NEGATIVE_INFINITY) {
  smallNumber = returnFinite();
}
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-number.negative_infinity |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `NEGATIVE_INFINITY` | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Number.NEGATIVE_INFINITY`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Number.POSITIVE_INFINITY`
- `Number.isFinite()`
- `Infinity`
- `isFinite()`

---

JavaScript › Reference › Standard built-in objects › Number

# Number.POSITIVE_INFINITY

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Number/POSITIVE_INFINITY · MDN page last modified 2025-07-10

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Number.POSITIVE_INFINITY`** static data property represents the positive Infinity value.

## Try it

*▷ Interactive example on MDN: "JavaScript Demo: Number.POSITIVE_INFINITY". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function checkNumber(bigNumber) {
  if (bigNumber === Number.POSITIVE_INFINITY) {
    return "Process number as Infinity";
  }
  return bigNumber;
}

console.log(checkNumber(Number.MAX_VALUE));
// Expected output: 1.7976931348623157e+308

console.log(checkNumber(Number.MAX_VALUE * 2));
// Expected output: "Process number as Infinity"
```

## Value

The same as the value of the global `Infinity` property.

| Property attributes of `Number.POSITIVE_INFINITY` | |
|---|---|
| Writable | no |

| Property attributes of `Number.POSITIVE_INFINITY` | |
|---|---|
| Enumerable | no |
| Configurable | no |

## Description

The `Number.POSITIVE_INFINITY` value behaves slightly differently than mathematical infinity:

- Any positive value, including `POSITIVE_INFINITY`, multiplied by `POSITIVE_INFINITY` is `POSITIVE_INFINITY`.
- Any negative value, including `NEGATIVE_INFINITY`, multiplied by `POSITIVE_INFINITY` is `NEGATIVE_INFINITY`.
- Any positive number divided by `POSITIVE_INFINITY` is positive zero (as defined in IEEE 754).
- Any negative number divided by `POSITIVE_INFINITY` is negative zero (as defined in IEEE 754.
- Zero multiplied by `POSITIVE_INFINITY` is `NaN`.
- `NaN` multiplied by `POSITIVE_INFINITY` is `NaN`.
- `POSITIVE_INFINITY`, divided by any negative value except `NEGATIVE_INFINITY`, is `NEGATIVE_INFINITY`.
- `POSITIVE_INFINITY`, divided by any positive value except `POSITIVE_INFINITY`, is `POSITIVE_INFINITY`.
- `POSITIVE_INFINITY`, divided by either `NEGATIVE_INFINITY` or `POSITIVE_INFINITY`, is `NaN`.
- `Number.POSITIVE_INFINITY > x` is true for any number *x* that isn't `POSITIVE_INFINITY`.

You might use the `Number.POSITIVE_INFINITY` property to indicate an error condition that returns a finite number in case of success. Note, however, that `NaN` would be more appropriate in such a case.

Because `POSITIVE_INFINITY` is a static property of `Number`, you always use it as `Number.POSITIVE_INFINITY`, rather than as a property of a number value.

## Examples

### Using POSITIVE_INFINITY

In the following example, the variable `bigNumber` is assigned a value that is larger than the maximum value. When the `if` statement executes, `bigNumber` has the value `Infinity`, so `bigNumber` is set to a more manageable value before continuing.

```js
let bigNumber = Number.MAX_VALUE * 2;

if (bigNumber === Number.POSITIVE_INFINITY) {
  bigNumber = returnFinite();
}
```

## Specifications

| Specification |
|---|
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-number.positive_infinity |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| POSITIVE_INFINITY | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Number.POSITIVE_INFINITY`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Number.NEGATIVE_INFINITY`
- `Number.isFinite()`
- `Infinity`
- `isFinite()`

---

JavaScript › Reference › Standard built-in objects › Number

# Number.prototype.toExponential()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Number/toExponential · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since July 2015.

The **`toExponential()`** method of `Number` values returns a string representing this number in exponential notation.

## Try it

*▷ Interactive example on MDN: "JavaScript Demo: Number.prototype.toExponential()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function expo(x, f) {
  return Number.parseFloat(x).toExponential(f);
}

console.log(expo(123456, 2));
// Expected output: "1.23e+5"

console.log(expo("123456"));
// Expected output: "1.23456e+5"

console.log(expo("oink"));
// Expected output: "NaN"
```

## Syntax

js

```
toExponential()
toExponential(fractionDigits)
```

### Parameters

**`fractionDigits`** Optional

Optional. An integer specifying the number of digits after the decimal point. Defaults to as many digits as necessary to specify the number.

### Return value

A string representing the given `Number` object in exponential notation with one digit before the decimal point, rounded to `fractionDigits` digits after the decimal point.

### Exceptions

**`RangeError`**

Thrown if `fractionDigits` is not between `0` and `100` (inclusive).

**`TypeError`**

Thrown if this method is invoked on an object that is not a `Number`.

## Description

If the `fractionDigits` argument is omitted, the number of digits after the decimal point defaults to the number of digits necessary to represent the value uniquely.

If you use the `toExponential()` method for a numeric literal and the numeric literal has no exponent and no decimal point, leave whitespace(s) before the dot that precedes the method call to prevent the dot from being interpreted as a decimal point.

If a number has more digits than requested by the `fractionDigits` parameter, the number is rounded to the nearest number represented by `fractionDigits` digits. See the discussion of rounding in the description of the `toFixed()` method, which also applies to `toExponential()`.

## Examples

### Using toExponential

js

```
const numObj = 77.1234;

console.log(numObj.toExponential()); // 7.71234e+1
console.log(numObj.toExponential(4)); // 7.7123e+1
console.log(numObj.toExponential(2)); // 7.71e+1
console.log((77.1234).toExponential()); // 7.71234e+1
console.log((77).toExponential()); // 7.7e+1
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-number.prototype.toexponential |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `toExponential` | 1 | 12 | 1 | 7 | 2 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Number.toExponential`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Number.prototype.toExponential` with many bug fixes in `core-js`
- es-shims polyfill of `Number.prototype.toExponential`
- `Number.prototype.toFixed()`
- `Number.prototype.toPrecision()`
- `Number.prototype.toString()`

---

JavaScript › Reference › Standard built-in objects › Number

# Number.prototype.toFixed()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Number/toFixed · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since July 2015.

The **`toFixed()`** method of `Number` values returns a string representing this number using fixed-point notation with the specified number of decimal places.

## Try it

*▷ Interactive example on MDN: "JavaScript Demo: Number.toFixed()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function financial(x) {
  return Number.parseFloat(x).toFixed(2);
}

console.log(financial(123.456));
// Expected output: "123.46"

console.log(financial(0.004));
// Expected output: "0.00"

console.log(financial("1.23e+5"));
// Expected output: "123000.00"
```

## Syntax

js

```
toFixed()
toFixed(digits)
```

### Parameters

**`digits`** Optional

The number of digits to appear after the decimal point; should be a value between `0` and `100`, inclusive. If this argument is omitted, it is treated as `0`.

### Return value

A string representing the given number using fixed-point notation. Scientific notation is used if the number's magnitude (ignoring sign) is greater than or equal to $10^{21}$ (same return value as `Number.prototype.toString()`).

### Exceptions

**`RangeError`**

Thrown if `digits` is not between `0` and `100` (inclusive).

**`TypeError`**

Thrown if this method is invoked on an object that is not a `Number`.

## Description

The `toFixed()` method returns a string representation of a number without using exponential notation and with exactly `digits` digits after the decimal point. The number is rounded if necessary, and the fractional part is padded with zeros if necessary so that it has the specified length.

If the absolute value of the number is greater or equal to $10^{21}$, this method uses the same algorithm as `Number.prototype.toString()` and returns a string in exponential notation. `toFixed()` returns `"Infinity"`, `"NaN"`, or `"-Infinity"` if the value of the number is non-finite.

The output of `toFixed()` may be more precise than `toString()` for some values, because `toString()` only prints enough significant digits to distinguish the number from adjacent number values. For example:

```js
(1000000000000000128).toString(); // '1000000000000000100'
(1000000000000000128).toFixed(0); // '1000000000000000128'
```

However, choosing a `digits` precision that's too high can return unexpected results, because decimal fractional numbers cannot be represented precisely in floating point. For example:

```js
(0.3).toFixed(50); // '0.29999999999999998889776975374843459576368331909180'
```

## Examples

### Using toFixed()

```js
const numObj = 12345.6789;

numObj.toFixed(); // '12346'; rounding, no fractional part
numObj.toFixed(1); // '12345.7'; it rounds up
numObj.toFixed(6); // '12345.678900'; additional zeros
(1.23e20).toFixed(2); // '123000000000000000000.00'
(1.23e-10).toFixed(2); // '0.00'
```

(code continues on the next page)

*(code continued from the previous page)*

```
(2.34).toFixed(1); // '2.3'
(2.35).toFixed(1); // '2.4'; it rounds up
(2.55).toFixed(1); // '2.5'
// it rounds down as it can't be represented exactly by a float and the
// closest representable float is lower
(2.449999999999999).toFixed(1); // '2.5'
// it rounds up as it's less than Number.EPSILON away from 2.45.
// This literal actually encodes the same number value as 2.45

(6.02 * 10 ** 23).toFixed(50); // '6.019999999999999e+23'; large numbers still use
exponential notation
```

### Using toFixed() with negative numbers

Because member access has higher precedence than unary minus, you need to group the negative number expression to get a string.

js

```
-2.34.toFixed(1); // -2.3; a number
(-2.34).toFixed(1); // '-2.3'
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-number.prototype.tofixed` |

## Browser compatibility

| | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| Feature | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `toFixed` | 1 | 12 | 1 | 7 | 2 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Number.toFixed`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of modern `Number.prototype.toFixed` behavior in `core-js`
- `Number.prototype.toExponential()`
- `Number.prototype.toPrecision()`
- `Number.prototype.toString()`
- `Number.EPSILON`

JavaScript › Reference › Standard built-in objects › Number

# Number.prototype.toLocaleString()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Number/toLocaleString · MDN page last modified 2025-10-30

**Baseline: Widely available** — available across major browsers since July 2015.

The **`toLocaleString()`** method of `Number` values returns a string with a language-sensitive representation of this number. In implementations with `Intl.NumberFormat` API support, this method delegates to `Intl.NumberFormat`.

Every time `toLocaleString` is called, it has to perform a search in a big database of localization strings, which is potentially inefficient. When the method is called many times with the same arguments, it is better to create an `Intl.NumberFormat` object and use its `format()` method, because a `NumberFormat` object remembers the arguments passed to it and may decide to cache a slice of the database, so future `format` calls can search for localization strings within a more constrained context.

## Try it

*▷ Interactive example on MDN: "JavaScript Demo: Number.prototype.toLocaleString()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function eArabic(x) {
  return x.toLocaleString("ar-EG");
}

console.log(eArabic(123456.789));
// Expected output: "١٢٣,٤٥٦,٧٨٩"

console.log(eArabic("123456.789"));
// Expected output: "123456.789"

console.log(eArabic(NaN));
// Expected output: "ليس رقم"
```

## Syntax

js

```
toLocaleString()
toLocaleString(locales)
toLocaleString(locales, options)
```

### Parameters

The `locales` and `options` parameters customize the behavior of the function and let applications specify the language whose formatting conventions should be used.

In implementations that support the `Intl.NumberFormat` API, these parameters correspond exactly to the `Intl.NumberFormat()` constructor's parameters. Implementations without `Intl.NumberFormat` support are asked to ignore both parameters, making the locale used and the form of the string returned entirely implementation-dependent.

**`locales`** Optional

A string with a BCP 47 language tag, or an array of such strings. Corresponds to the `locales` parameter of the `Intl.NumberFormat()` constructor.

In implementations without `Intl.NumberFormat` support, this parameter is ignored and the host's locale is usually used.

**`options`** Optional

An object adjusting the output format. Corresponds to the `options` parameter of the `Intl.NumberFormat()` constructor.

In implementations without `Intl.NumberFormat` support, this parameter is ignored.

See the `Intl.NumberFormat()` constructor for details on these parameters and how to use them.

### Return value

A string representing the given number according to language-specific conventions.

In implementations with `Intl.NumberFormat`, this is equivalent to `new Intl.NumberFormat(locales, options).format(number)`.

> **Note:** Most of the time, the formatting returned by `toLocaleString()` is consistent. However, the output may vary between implementations, even within the same locale — output variations are by design and allowed by the specification. It may also not be what you expect. For example, the string may use non-breaking spaces or be surrounded by bidirectional control characters. You should not compare the results of `toLocaleString()` to hardcoded constants.

## Examples

### Using toLocaleString()

Basic use of this method without specifying a `locale` returns a formatted string in the default locale and with default options.

```js
const number = 3500;

console.log(number.toLocaleString()); // "3,500" if in U.S. English locale
```

### Checking for support for locales and options parameters

The `locales` and `options` parameters may not be supported in all implementations, because support for the internationalization API is optional, and some systems may not have the necessary data. For implementations without internationalization support, `toLocaleString()` always uses the system's locale, which may not be what you want. Because any implementation that supports the `locales` and `options` parameters must support the `Intl` API, you can check the existence of the latter for support:

```js
function toLocaleStringSupportsLocales() {
  return (
    typeof Intl === "object" &&
    !!Intl &&
    typeof Intl.NumberFormat === "function"
  );
}
```

### Using locales

This example shows some of the variations in localized number formats. In order to get the format of the language used in the user interface of your application, make sure to specify that language (and possibly some fallback languages) using the `locales` argument:

```js
const number = 123456.789;

// German uses comma as decimal separator and period for thousands
console.log(number.toLocaleString("de-DE"));
// 123.456,789

// Arabic in most Arabic speaking countries uses Eastern Arabic digits
console.log(number.toLocaleString("ar-EG"));
// ١٢٣٤٥٦٫٧٨٩

// India uses thousands/lakh/crore separators
console.log(number.toLocaleString("en-IN"));
// 1,23,456.789

// the nu extension key requests a numbering system, e.g. Chinese decimal
console.log(number.toLocaleString("zh-Hans-CN-u-nu-hanidec"));
// 一二三,四五六.七八九

// when requesting a language that may not be supported, such as
// Balinese, include a fallback language, in this case Indonesian
console.log(number.toLocaleString(["ban", "id"]));
// 123.456,789
```

### Using options

The results provided by `toLocaleString()` can be customized using the `options` parameter:

```js
const number = 123456.789;

// request a currency format
console.log(
  number.toLocaleString("de-DE", { style: "currency", currency: "EUR" }),
);
// 123.456,79 €

// the Japanese yen doesn't use a minor unit
console.log(
  number.toLocaleString("ja-JP", { style: "currency", currency: "JPY" }),
);
// ￥123,457

// limit to three significant digits
console.log(number.toLocaleString("en-IN", { maximumSignificantDigits: 3 }));
// 1,23,000

// Use the host default language with options for number formatting
const num = 30000.65;
console.log(
  num.toLocaleString(undefined, {
    minimumFractionDigits: 2,
    maximumFractionDigits: 2,
```

(code continues on the next page)

*(code continued from the previous page)*

```
  }),
);
// "30,000.65" where English is the default language, or
// "30.000,65" where German is the default language, or
// "30 000,65" where French is the default language
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-number.prototype.tolocalestring |
| ECMAScript® 2027 Internationalization API Specification<br>https://tc39.es/ecma402/#sup-number.prototype.tolocalestring |

## Browser compatibility

| | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| Feature | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `toLocaleString` | 1 | 12[1] | 1 | 4 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| `locales` parameter | 24 | 12 | 29 | 15 | 10 | 26 | 56 | 14 | 10 | 1.5 | 4.4 | 10 | 1.0.0 | 1.8[2,3] | 13.0.0[4,5] |
| `options` parameter | 24 | 12 | 29 | 15 | 10 | 26 | 56 | 14 | 10 | 1.5 | 4.4 | 10 | 1.0.0 | 1.0 | 0.12.0 |

1. Before Edge 18, numbers are rounded to 15 decimal digits. For example, `(1000000000000005).toLocaleString('en-US')` returns `"1,000,000,000,000,010"`.
2. Also from version 1.0–1.8: partial support.
3. Also from version 1.0–1.8: Only the locale data for `en-US` is available.
4. Also from version 0.12.0–13.0.0: partial support.
5. Also from version 0.12.0–13.0.0: Before version 13.0.0, only the locale data for `en-US` is available by default. When other locales are specified, the function silently falls back to `en-US`. To make full ICU (locale) data available before version 13, see Node.js documentation on the `--with-intl` option and how to provide the data.

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Number.toLocaleString`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Intl.NumberFormat`
- `Number.prototype.toString()`

---

JavaScript › Reference › Standard built-in objects › Number

# Number.prototype.toPrecision()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Number/toPrecision · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since July 2015.

The **`toPrecision()`** method of `Number` values returns a string representing this number to the specified number of significant digits.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Number.prototype.toPrecision()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function precise(x) {
  return x.toPrecision(4);
}

console.log(precise(123.456));
// Expected output: "123.5"

console.log(precise(0.004));
// Expected output: "0.004000"

console.log(precise(1.23e5));
// Expected output: "1.230e+5"
```

## Syntax

js

```
toPrecision()
toPrecision(precision)
```

### Parameters

**`precision`** Optional

An integer specifying the number of significant digits.

### Return value

A string representing the given number, using the given number of significant digits. Scientific notation is used if the exponent is greater than or equal to `precision` or less than -6. Has the same behavior as `Number.prototype.toString()` if the `precision` argument is omitted.

### Exceptions

**`RangeError`**

Thrown if `precision` is not between `1` and `100` (inclusive).

**`TypeError`**

Thrown if this method is invoked on an object that is not a `Number`.

## Examples

### Using `toPrecision`

js

```
// This number has exponent 0, so it will never use exponential notation
let num = 5.123456;

console.log(num.toPrecision()); // '5.123456'
console.log(num.toPrecision(5)); // '5.1235'
console.log(num.toPrecision(2)); // '5.1'
console.log(num.toPrecision(1)); // '5'
```

*(code continues on the next page)*

*(code continued from the previous page)*

```
// This number has exponent -4, so it will never use exponential notation
num = 0.000123;

console.log(num.toPrecision()); // '0.000123'
console.log(num.toPrecision(5)); // '0.00012300'
console.log(num.toPrecision(2)); // '0.00012'
console.log(num.toPrecision(1)); // '0.0001'

// This number has exponent 3, so it will use exponential notation if precision is less than
4
num = 1234.5;
console.log(num.toPrecision(1)); // '1e+3'
console.log(num.toPrecision(2)); // '1.2e+3'
console.log(num.toPrecision(6)); // '1234.50'

// This number has exponent -7, so it will always use exponential notation
num = 0.00000012345;
console.log(num.toPrecision(1)); // '1e-7'
console.log(num.toPrecision(10)); // '1.234500000e-7'
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-number.prototype.toprecision` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `toPrecision` | 1 | 12 | 1 | 7 | 2 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Number.toPrecision`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of modern `Number.prototype.toPrecision` behavior in `core-js`
- `Number.prototype.toFixed()`
- `Number.prototype.toExponential()`
- `Number.prototype.toString()`

JavaScript › Reference › Standard built-in objects › Number

# Number.prototype.toString()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Number/toString · MDN page last modified 2025-07-10

**Baseline: Widely available** — available across major browsers since July 2015.

The **`toString()`** method of `Number` values returns a string representing this number value.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Number.prototype.toString()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function hexColor(c) {
  if (c < 256) {
    return Math.abs(c).toString(16);
  }
  return 0;
}

console.log(hexColor(233));
// Expected output: "e9"

console.log(hexColor("11"));
// Expected output: "b"
```

## Syntax

js

```
toString()
toString(radix)
```

### Parameters

**`radix`** Optional

An integer in the range 2 through 36 specifying the base to use for representing the number value. Defaults to 10.

### Return value

A string representing the specified number value. Scientific notation is used if radix is 10 and the number's magnitude (ignoring sign) is greater than or equal to $10^{21}$ or less than $10^{-6}$.

### Exceptions

**`RangeError`**

Thrown if `radix` is less than 2 or greater than 36.

**`TypeError`**

Thrown if this method is invoked on an object that is not a `Number`.

## Description

The `Number` object overrides the `toString` method of `Object`; it does not inherit `Object.prototype.toString()`. For `Number` values, the `toString` method returns a string representation of the value in the specified radix.

For radixes above 10, the letters of the alphabet indicate digits greater than 9. For example, for hexadecimal numbers (base 16) `a` through `f` are used.

If the specified number value is negative, the sign is preserved. This is the case even if the radix is 2; the string returned is the positive binary representation of the number value preceded by a `-` sign, **not** the two's complement of the number value.

Both `0` and `-0` have `"0"` as their string representation. `Infinity` returns `"Infinity"` and `NaN` returns `"NaN"`.

If the number is not a whole number, the decimal point `.` is used to separate the decimal places. Scientific notation is used if the radix is 10 and the number's magnitude (ignoring sign) is greater than or equal to $10^{21}$ or less than $10^{-6}$. In this case, the returned string always explicitly specifies the sign of the exponent.

```js
console.log((10 ** 21.5).toString()); // "3.1622776601683794e+21"
console.log((10 ** 21.5).toString(8)); // "526665530627250154000000"
```

The underlying representation for floating point numbers is base-2 scientific notation (see number encoding). However, the `toString()` method doesn't directly use this most precise representation of the number value. Rather, the algorithm uses the least number of significant figures necessary to distinguish the output from adjacent number values. For example, if the number is large, there will be many equivalent string representations of the same floating point number, and `toString()` will choose the one with the most 0s to the right (for any given radix).

```js
console.log((100000000000000128).toString()); // "100000000000000100"
console.log(100000000000000100 === 100000000000000128); // true
```

On the other hand, `Number.prototype.toFixed()` and `Number.prototype.toPrecision()` allow you to specify the precision and can be more precise than `toString()`.

The `toString()` method requires its `this` value to be a `Number` primitive or wrapper object. It throws a `TypeError` for other `this` values without attempting to coerce them to number values.

Because `Number` doesn't have a `[Symbol.toPrimitive]()` method, JavaScript calls the `toString()` method automatically when a `Number` *object* is used in a context expecting a string, such as in a template literal. However, Number *primitive* values do not consult the `toString()` method to be coerced to strings — rather, they are directly converted using the same algorithm as the initial `toString()` implementation.

```js
Number.prototype.toString = () => "Overridden";
console.log(`${1}`); // "1"
console.log(`${new Number(1)}`); // "Overridden"
```

## Examples

### Using toString()

```js
const count = 10;
console.log(count.toString()); // "10"

console.log((17).toString()); // "17"
console.log((17.2).toString()); // "17.2"

const x = 6;
console.log(x.toString(2)); // "110"
console.log((254).toString(16)); // "fe"
console.log((-10).toString(2)); // "-1010"
console.log((-0xff).toString(2)); // "-11111111"
```

### Converting radix of number strings

If you have a string representing a number in a non-decimal radix, you can use `parseInt()` and `toString()` to convert it to a different radix.

```js
const hex = "CAFEBABE";
const bin = parseInt(hex, 16).toString(2); // "11001010111111101011101010111110"
```

Beware of loss of precision: if the original number string is too large (larger than `Number.MAX_SAFE_INTEGER`, for example), you should use a `BigInt` instead. However, the `BigInt` constructor only has support for strings representing number literals (i.e., strings starting with `0b`, `0o`, `0x`). In case your original radix is not one of binary, octal, decimal, or hexadecimal, you may need to hand-write your radix converter, or use a library.

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-number.prototype.tostring |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| toString | 1 | 12 | 1 | 4 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Number.toString`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Number.prototype.toFixed()`
- `Number.prototype.toExponential()`
- `Number.prototype.toPrecision()`

---

JavaScript › Reference › Standard built-in objects › Number

# Number.prototype.valueOf()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Number/valueOf · MDN page last modified 2025-07-10

**Baseline: Widely available** — available across major browsers since July 2015.

The **`valueOf()`** method of `Number` values returns the value of this number.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Number.prototype.valueOf()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const numObj = new Number(42);
console.log(typeof numObj);
// Expected output: "object"

const num = numObj.valueOf();
console.log(num);
// Expected output: 42

console.log(typeof num);
// Expected output: "number"
```

## Syntax

js

```
valueOf()
```

### Parameters

None.

### Return value

A number representing the primitive value of the specified `Number` object.

## Description

This method is usually called internally by JavaScript and not explicitly in web code.

## Examples

### Using valueOf

```js
const numObj = new Number(10);
console.log(typeof numObj); // object

const num = numObj.valueOf();
console.log(num); // 10
console.log(typeof num); // number
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-number.prototype.valueof |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| valueOf | 1 | 12 | 1 | 4 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Number.valueOf`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Object.prototype.valueOf()`

JavaScript › Reference › Standard built-in objects

# Object

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object · MDN page last modified 2026-05-22

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Object`** type represents one of JavaScript's data types. It is used to store various keyed collections and more complex entities. Objects can be created using the `Object()` constructor or the object initializer / literal syntax.

## Description

Nearly all objects in JavaScript are instances of `Object`; a typical object inherits properties (including methods) from `Object.prototype`, although these properties may be shadowed (a.k.a. overridden). The only objects that don't inherit from `Object.prototype` are those with `null` prototype, or descended from other `null` prototype objects.

Changes to the `Object.prototype` object are seen by **all** objects through prototype chaining, unless the properties and methods subject to those changes are overridden further along the prototype chain. This provides a very powerful although potentially dangerous mechanism to override or extend object behavior. To make it more secure, `Object.prototype` is the only object in the core JavaScript language that has immutable prototype — the prototype of `Object.prototype` is always `null` and not changeable.

### Object prototype properties

You should avoid calling any of the `Object.prototype` methods directly from the instance, especially those that are not intended to be polymorphic (i.e., only its initial behavior makes sense and no descending object could override it in a meaningful way). All objects descending from `Object.prototype` may define a custom own property that has the same name, but with entirely different semantics from what you expect. Furthermore, these properties are not inherited by `null`-prototype objects. All modern JavaScript utilities for working with objects are static. More specifically:

- `valueOf()`, `toString()`, and `toLocaleString()` exist to be polymorphic and you should expect the object to define its own implementation with sensible behaviors, so you can call them as instance methods. However, `valueOf()` and `toString()` are usually implicitly called through type conversion and you don't need to call them yourself in your code.
- `__defineGetter__()`, `__defineSetter__()`, `__lookupGetter__()`, and `__lookupSetter__()` are deprecated and should not be used. Use the static alternatives `Object.defineProperty()` and `Object.getOwnPropertyDescriptor()` instead.
- The `__proto__` property is deprecated and should not be used. The `Object.getPrototypeOf()` and `Object.setPrototypeOf()` alternatives are static methods.
- The `propertyIsEnumerable()` and `hasOwnProperty()` methods can be replaced with the `Object.getOwnPropertyDescriptor()` and `Object.hasOwn()` static methods, respectively.
- The `isPrototypeOf()` method can usually be replaced with `instanceof`, if you are checking the `prototype` property of a constructor.

In case where a semantically equivalent static method doesn't exist, or if you really want to use the `Object.prototype` method, you should directly `call()` the `Object.prototype` method on your target object instead, to prevent the object from having an overriding property that produces unexpected results.

```js
const obj = {
  foo: 1,
  // You should not define such a method on your own object,
  // but you may not be able to prevent it from happening if
  // you are receiving the object from external input
  propertyIsEnumerable() {
    return false;
  },
};

obj.propertyIsEnumerable("foo"); // false; unexpected result
Object.prototype.propertyIsEnumerable.call(obj, "foo"); // true; expected result
```

### Deleting a property from an object

There isn't any method in an Object itself to delete its own properties (such as `Map.prototype.delete()`). To do so, one must use the `delete` operator.

### null-prototype objects

Almost all objects in JavaScript ultimately inherit from `Object.prototype` (see inheritance and the prototype chain). However, you may create `null`-prototype objects using `Object.create(null)` or the object initializer syntax with `__proto__: null` (note: the `__proto__` key in object literals is different from the deprecated `Object.prototype.__proto__` property). You can also change the prototype of an existing object to `null` by calling `Object.setPrototypeOf(obj, null)`.

```js
const obj = Object.create(null);
const obj2 = { __proto__: null };
```

An object with a `null` prototype can behave in unexpected ways, because it doesn't inherit any object methods from `Object.prototype`. This is especially true when debugging, since common object-property converting/detecting utility functions may generate errors, or lose information (especially if using silent error-traps that ignore errors).

For example, the lack of `Object.prototype.toString()` often makes debugging intractable:

```js
const normalObj = {}; // create a normal object
const nullProtoObj = Object.create(null); // create an object with "null" prototype

console.log(`normalObj is: ${normalObj}`); // shows "normalObj is: [object Object]"
console.log(`nullProtoObj is: ${nullProtoObj}`); // throws error: Cannot convert object to
primitive value

alert(normalObj); // shows [object Object]
alert(nullProtoObj); // throws error: Cannot convert object to primitive value
```

Other methods will fail as well.

```js
normalObj.valueOf(); // shows {}
nullProtoObj.valueOf(); // throws error: nullProtoObj.valueOf is not a function

normalObj.hasOwnProperty("p"); // shows "true"
nullProtoObj.hasOwnProperty("p"); // throws error: nullProtoObj.hasOwnProperty is not a
function

normalObj.constructor; // shows "Object() { [native code] }"
nullProtoObj.constructor; // shows "undefined"
```

We can add the `toString` method back to the null-prototype object by assigning it one:

```js
nullProtoObj.toString = Object.prototype.toString; // since new object lacks toString, add
the original generic one back

console.log(nullProtoObj.toString()); // shows "[object Object]"
console.log(`nullProtoObj is: ${nullProtoObj}`); // shows "nullProtoObj is: [object Object]"
```

Unlike normal objects, in which `toString()` is on the object's prototype, the `toString()` method here is an own property of `nullProtoObj`. This is because `nullProtoObj` has no (`null`) prototype.

You can also revert a null-prototype object back to an ordinary object using `Object.setPrototypeOf(nullProtoObj, Object.prototype)`.

In practice, objects with `null` prototype are usually used as a cheap substitute for maps. The presence of `Object.prototype` properties will cause some bugs:

```js
const ages = { alice: 18, bob: 27 };

function hasPerson(name) {
  return name in ages;
}

function getAge(name) {
  return ages[name];
}

hasPerson("hasOwnProperty"); // true
getAge("toString"); // [Function: toString]
```

Using a null-prototype object removes this hazard without introducing too much complexity to the `hasPerson` and `getAge` functions:

```js
const ages = Object.create(null, {
  alice: { value: 18, enumerable: true },
  bob: { value: 27, enumerable: true },
});

hasPerson("hasOwnProperty"); // false
getAge("toString"); // undefined
```

In such case, the addition of any method should be done cautiously, as they can be confused with the other key-value pairs stored as data.

Making your object not inherit from `Object.prototype` also prevents prototype pollution attacks. If a malicious script adds a property to `Object.prototype`, it will be accessible on every object in your program, except objects that have null prototype.

```js
const user = {};

// A malicious script:
Object.prototype.authenticated = true;

// Unexpectedly allowing unauthenticated user to pass through
if (user.authenticated) {
  // access confidential data
}
```

JavaScript also has built-in APIs that produce `null`-prototype objects, especially those that use objects as ad hoc key-value collections. For example:

- The return value of `Object.groupBy()`
- The `groups` and `indices.groups` properties of the result of `RegExp.prototype.exec()`
- `Array.prototype[Symbol.unscopables]` (all `[Symbol.unscopables]` objects should have `null`-prototype)
- `import.meta`
- Module namespace objects, obtained through `import * as ns from "module";` or `import()`

The term "`null`-prototype object" often also includes any object without `Object.prototype` in its prototype chain. Such objects can be created with `extends null` when using classes.

### Object coercion

Many built-in operations that expect objects first coerce their arguments to objects. The operation can be summarized as follows:

- Objects are returned as-is.
- `undefined` and `null` throw a `TypeError`.
- `Number`, `String`, `Boolean`, `Symbol`, `BigInt` primitives are wrapped into their corresponding object wrappers.

There are two ways to achieve nearly the same effect in JavaScript.

- `Object.prototype.valueOf()`: `Object.prototype.valueOf.call(x)` does exactly the object coercion steps explained above to convert `x`.
- The `Object()` function: `Object(x)` uses the same algorithm to convert `x`, except that `undefined` and `null` don't throw a `TypeError`, but return a plain object.

Places that use object coercion include:

- The `object` parameter of `for...in` loops.
- The `this` value of `Array` methods.
- Parameters of `Object` methods such as `Object.keys()`.
- Auto-boxing when a property is accessed on a primitive value, since primitives do not have properties.
- The `this` value when calling a non-strict function. Primitives are boxed while `null` and `undefined` are replaced with the global object.

Unlike conversion to primitives, the object coercion process itself is not observable in any way, since it doesn't invoke custom code like `toString` or `valueOf` methods.

## Constructor

**`Object()`**
Turns the input into an object.

## Static methods

**`Object.assign()`**
Copies the values of all enumerable own properties from one or more source objects to a target object.

**`Object.create()`**
Creates a new object with the specified prototype object and properties.

**`Object.defineProperties()`**
Adds the named properties described by the given descriptors to an object.

**`Object.defineProperty()`**
Adds the named property described by a given descriptor to an object.

**`Object.entries()`**
Returns an array containing all of the `[key, value]` pairs of a given object's **own** enumerable string properties.

**`Object.freeze()`**
Freezes an object. Other code cannot delete or change its properties.

**`Object.fromEntries()`**
Returns a new object from an iterable of `[key, value]` pairs. (This is the reverse of `Object.entries`).

**`Object.getOwnPropertyDescriptor()`**
Returns a property descriptor for a named property on an object.

**`Object.getOwnPropertyDescriptors()`**
Returns an object containing all own property descriptors for an object.

**`Object.getOwnPropertyNames()`**
Returns an array containing the names of all of the given object's **own** enumerable and non-enumerable properties.

**`Object.getOwnPropertySymbols()`**
Returns an array of all symbol properties found directly upon a given object.

**`Object.getPrototypeOf()`**
Returns the prototype (internal `[[Prototype]]` property) of the specified object.

**`Object.groupBy()`**
Groups the elements of a given iterable according to the string values returned by a provided callback function. The returned object has separate properties for each group, containing arrays with the elements in the group.

**`Object.hasOwn()`**
Returns `true` if the specified object has the indicated property as its *own* property, or `false` if the property is inherited or does not exist.

**`Object.is()`**
Compares if two values are the same value. Equates all `NaN` values (which differs from both `IsLooselyEqual` used by `==` and `IsStrictlyEqual` used by `===`).

**`Object.isExtensible()`**
Determines if extending of an object is allowed.

**`Object.isFrozen()`**
Determines if an object was frozen.

**`Object.isSealed()`**
Determines if an object is sealed.

**`Object.keys()`**
Returns an array containing the names of all of the given object's **own** enumerable string properties.

**`Object.preventExtensions()`**
Prevents any extensions of an object.

**`Object.seal()`**
Prevents other code from deleting properties of an object.

**`Object.setPrototypeOf()`**
Sets the object's prototype (its internal `[[Prototype]]` property).

**`Object.values()`**
Returns an array containing the values that correspond to all of a given object's **own** enumerable string properties.

## Instance properties

These properties are defined on `Object.prototype` and shared by all `Object` instances.

**`Object.prototype.__proto__`** Deprecated
Points to the object which was used as prototype when the object was instantiated.

**`Object.prototype.constructor`**
The constructor function that created the instance object. For plain `Object` instances, the initial value is the `Object` constructor. Instances of other constructors each inherit the `constructor` property from their respective `Constructor.prototype` object.

## Instance methods

**`Object.prototype.__defineGetter__()`** Deprecated
Associates a function with a property that, when accessed, executes that function and returns its return value.

**`Object.prototype.__defineSetter__()`** Deprecated
Associates a function with a property that, when set, executes that function which modifies the property.

**`Object.prototype.__lookupGetter__()`** Deprecated
Returns the function bound as a getter to the specified property.

**`Object.prototype.__lookupSetter__()`** Deprecated

Returns the function bound as a setter to the specified property.

**`Object.prototype.hasOwnProperty()`**

Returns a boolean indicating whether an object contains the specified property as a direct property of that object and not inherited through the prototype chain.

**`Object.prototype.isPrototypeOf()`**

Returns a boolean indicating whether the object this method is called upon is in the prototype chain of the specified object.

**`Object.prototype.propertyIsEnumerable()`**

Returns a boolean indicating whether the specified property is the object's enumerable own property.

**`Object.prototype.toLocaleString()`**

Calls `toString()`.

**`Object.prototype.toString()`**

Returns a string representation of the object.

**`Object.prototype.valueOf()`**

Returns the primitive value of the specified object.

## Examples

### Constructing empty objects

The following example creates empty objects using the `new` keyword with different arguments:

```js
const o1 = new Object();
const o2 = new Object(undefined);
const o3 = new Object(null);
```

### Using Object() constructor to turn primitives into an Object of their respective type

You can use the `Object()` constructor to create an object wrapper of a primitive value.

The following examples create variables `o1` and `o2` which are objects storing `Boolean` and `BigInt` values:

```js
// Equivalent to const o1 = new Boolean(true)
const o1 = new Object(true);

// No equivalent because BigInt() can't be called as a constructor,
// and calling it as a regular function won't create an object
const o2 = new Object(1n);
```

### Object prototypes

When altering the behavior of existing `Object.prototype` methods, consider injecting code by wrapping your extension before or after the existing logic. For example, this (untested) code will pre-conditionally execute custom logic before the built-in logic or someone else's extension is executed.

When modifying prototypes with hooks, pass `this` and the arguments (the call state) to the current behavior by calling `apply()` on the function. This pattern can be used for any prototype, such as

`Node.prototype`, `Function.prototype`, etc.

```js
const current = Object.prototype.valueOf;

// Since my property "-prop-value" is cross-cutting and isn't always
// on the same prototype chain, I want to modify Object.prototype:
Object.prototype.valueOf = function (...args) {
  if (Object.hasOwn(this, "-prop-value")) {
    return this["-prop-value"];
  }
  // It doesn't look like one of my objects, so let's fall back on
  // the default behavior by reproducing the current behavior as best we can.
  // The apply behaves like "super" in some other languages.
  // Even though valueOf() doesn't take arguments, some other hook may.
  return current.apply(this, args);
};
```

> **Warning:** Modifying the `prototype` property of any built-in constructor is considered a bad practice and risks forward compatibility.

You can read more about prototypes in Inheritance and the prototype chain.

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-object-objects` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `Object` | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 1 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| `Object()` constructor | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 1 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| `assign` | 45 | 12 | 34 | 32 | 9 | 45 | 34 | 32 | 9 | 5.0 | 45 | 9 | 1.0.0 | 1.0 | 4.0.0 |
| `constructor` | 1 | 12 | 1 | 4 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| `create` | 5 | 12 | 4 | 11.6 | 5 | 18 | 4 | 12 | 5 | 1.0 | 4.4 | 5 | 1.0.0 | 1.0 | 0.10.0 |
| `__defineGetter__` Deprecated | 1 | 12 | 1[1] | 9.5 | 3 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| `defineProperties` | 5 | 12 | 4 | 11.6 | 5 | 18 | 4 | 12 | 5 | 1.0 | 4.4 | 5 | 1.0.0 | 1.0 | 0.10.0 |
| `defineProperty` | 5 | 12 | 4 | 11.6 | 5.1[2] | 18 | 4 | 12 | 6[3] | 1.0 | 4.4 | 6[3] | 1.0.0 | 1.0 | 0.10.0 |
| `__defineSetter__` Deprecated | 1 | 12 | 1[1] | 9.5 | 3 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| `entries` | 54 | 14 | 47 | 41 | 10.1 | 54 | 47 | 41 | 10.3 | 6.0 | 54 | 10.3 | 1.0.0 | 1.0 | 7.0.0 |
| `freeze` | 6 | 12 | 4 | 12 | 5.1 | 18 | 4 | 12 | 5 | 1.0 | 4.4 | 5 | 1.0.0 | 1.0 | 0.10.0 |
| `fromEntries` | 73 | 79 | 63 | 60 | 12.1 | 73 | 63 | 52 | 12.2 | 11.0 | 73 | 12.2 | 1.0.0 | 1.0 | 12.0.0 |
| `getOwnPropertyDescriptor` | 5 | 12 | 4 | 12 | 5 | 18 | 4 | 12 | 5 | 1.0 | 4.4 | 5 | 1.0.0 | 1.0 | 0.10.0 |

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Nodejs |
| `getOwnPropertyDescriptors` | 54 | 15 | 50 | 41 | 10 | 54 | 50 | 41 | 10 | 6.0 | 54 | 10 | 1.0.0 | 1.0 | 7.0.0 |
| `getOwnPropertyNames` | 5 | 12 | 4 | 12 | 5 | 18 | 4 | 12 | 5 | 1.0 | 4.4 | 5 | 1.0.0 | 1.0 | 0.10.0 |
| `getOwnPropertySymbols` | 38 | 12 | 36 | 25 | 9 | 38 | 36 | 25 | 9 | 3.0 | 38 | 9 | 1.0.0 | 1.0 | 0.12.0 |
| `getPrototypeOf` | 5 | 12 | 3.5 | 12.1 | 5 | 18 | 4 | 12.1 | 5 | 1.0 | 4.4 | 5 | 1.0.0 | 1.0 | 0.10.0 |
| `groupBy` | 117 | 117 | 119 | 103 | 17.4[4] | 117 | 119 | 78 | 17.4[4] | 24.0 | 117 | 17.4[4] | 1.0.19 | 1.37 | 21.0.0 |
| `hasOwn` | 93 | 93 | 92 | 79 | 15.4 | 93 | 92 | 66 | 15.4 | 17.0 | 93 | 15.4 | 1.0.0 | 1.13 | 16.9.0 |
| `hasOwnProperty` | 1 | 12 | 1 | 5 | 3 | 18 | 4 | 10.1 | 1 | 1.0 | 1 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| `is` | 19 | 12 | 22 | 15 | 9 | 25 | 22 | 14 | 9 | 1.5 | 4.4 | 9 | 1.0.0 | 1.0 | 0.10.0 |
| `isExtensible` | 6 | 12 | 4 | 12 | 5.1 | 18 | 4 | 12 | 5 | 1.0 | 4.4 | 5 | 1.0.0 | 1.0 | 0.10.0 |
| `isFrozen` | 6 | 12 | 4 | 12 | 5.1 | 18 | 4 | 12 | 5 | 1.0 | 4.4 | 5 | 1.0.0 | 1.0 | 0.10.0 |
| `isPrototypeOf` | 1 | 12 | 1 | 4 | 3 | 18 | 4 | 10.1 | 1 | 1.0 | 1 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| `isSealed` | 6 | 12 | 4 | 12 | 5.1 | 18 | 4 | 12 | 5 | 1.0 | 4.4 | 5 | 1.0.0 | 1.0 | 0.10.0 |
| `keys` | 5 | 12 | 4 | 12 | 5 | 18 | 4 | 12 | 5 | 1.0 | 4.4 | 5 | 1.0.0 | 1.0 | 0.10.0 |
| `__lookupGetter__` Deprecated | 1 | 12 | 1 | 9.5 | 3 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| `__lookupSetter__` Deprecated | 1 | 12 | 1 | 9.5 | 3 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| `preventExtensions` | 6 | 12 | 4 | 12 | 5.1 | 18 | 4 | 12 | 5 | 1.0 | 4.4 | 5 | 1.0.0 | 1.0 | 0.10.0 |
| ES2015 behavior for non-object argument | 44 | 12 | 35 | 31 | 9 | 44 | 35 | 32 | 9 | 4.0 | 44 | 9 | 1.0.0 | 1.0 | 4.0.0 |
| `propertyIsEnumerable` | 1 | 12 | 1 | 4 | 3 | 18 | 4 | 10.1 | 1 | 1.0 | 1 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| `__proto__` Deprecated | 1 | 12 | 1 | 10.5 | 3 | 18 | 4 | 11 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 2.9[5,6] | 0.10.0 |
| `seal` | 6 | 12 | 4 | 12 | 5.1 | 18 | 4 | 12 | 5 | 1.0 | 4.4 | 5 | 1.0.0 | 1.0 | 0.10.0 |
| `setPrototypeOf` | 34 | 12 | 31 | 21 | 9 | 34 | 31 | 21 | 9 | 2.0 | 37 | 9 | 1.0.0 | 1.0 | 0.12.0 |
| `toLocaleString` | 1 | 12 | 1 | 4 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 1 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| `toString()` | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 1 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| `valueOf` | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 1 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| `values` | 54 | 14 | 47 | 41 | 10.1 | 54 | 47 | 41 | 10.3 | 6.0 | 54 | 10.3 | 1.0.0 | 1.0 | 7.0.0 |

1. Starting with Firefox 48, this method can no longer be called at the global scope without any object. A `TypeError` will be thrown otherwise. Previously, the global object was used in these cases automatically, but this is no longer the case.
2. Also supported in Safari 5, but not on DOM objects.
3. Also supported in Safari for iOS 4.2, but not on DOM objects.
4. Also from version 16.4–17.4: uses the non-standard name `Array.prototype.groupToMap`.
5. Disabled by default; the user must enable the runtime flag `--unsafe-proto`.
6. Also from version 1.39: disabled by default; the user must enable the runtime flag `--unstable-unsafe-proto`.

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Object`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Object initializer
- Prototype pollution attack

JavaScript › Reference › Standard built-in objects › Object

# Object() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/Object · MDN page last modified 2025-07-10

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Object()`** constructor turns the input into an object. Its behavior depends on the input's type.

## Syntax

```js
new Object()
new Object(value)

Object()
Object(value)
```

> **Note:** `Object()` can be called with or without `new`, but sometimes with different effects. See Return value.

### Parameters

**`value`** Optional

Any value.

### Return value

When the `Object()` constructor itself is called or constructed, its return value is an object:

- If the value is `null` or `undefined`, it creates and returns an empty object.
- If the value is an object already, it returns the value.
- Otherwise, it returns an object of a type that corresponds to the given value. For example, passing a `BigInt` primitive returns a `BigInt` wrapper object.

When `Object()` is constructed but `new.target` is not the `Object` constructor itself, the behavior is slightly different — it initializes a new object with `new.target.prototype` as its prototype. Any argument value is ignored. This may happen, for example, when `Object()` is implicitly called via `super()` in the constructor of a class that `extends Object`. In this case, even if you pass a number to `super()`, the `this` value inside the constructor does not become a `Number` instance.

## Examples

### Creating a new Object

```js
const o = new Object();
o.foo = 42;

console.log(o);
// { foo: 42 }
```

### Using Object given undefined and null types

The following examples store an empty `Object` object in `o`:

```js
const o = new Object();
```

```js
const o = new Object(undefined);
```

```js
const o = new Object(null);
```

### Obtaining wrapper objects for BigInt and Symbol

The `BigInt()` and `Symbol()` constructors throw an error when called with `new`, to prevent the common mistake of creating a wrapper object instead of the primitive value. The only way to create a wrapper object for these types is to call `Object()` with them:

```js
const numberObj = new Number(1);
console.log(typeof numberObj); // "object"

const bigintObj = Object(1n);
console.log(typeof bigintObj); // "object"

const symbolObj = Object(Symbol("foo"));
console.log(typeof symbolObj); // "object"
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-object-constructor` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `Object()` constructor | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 1 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Object.Object`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Object initializer

JavaScript › Reference › Standard built-in objects › Object

# Object.assign()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/assign · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since September 2015.

The **`Object.assign()`** static method copies all enumerable own properties from one or more *source objects* to a *target object*. It returns the modified target object.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Object.assign()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const target = { a: 1, b: 2 };
const source = { b: 4, c: 5 };

const returnedTarget = Object.assign(target, source);

console.log(target);
// Expected output: Object { a: 1, b: 4, c: 5 }

console.log(returnedTarget === target);
// Expected output: true
```

## Syntax

js

```
Object.assign(target)
Object.assign(target, source1)
Object.assign(target, source1, source2)
Object.assign(target, source1, source2, /* …, */ sourceN)
```

### Parameters

**`target`**

The target object — what to apply the sources' properties to, which is returned after it is modified. If a primitive value is provided as the target, it will be converted to an object.

**`source1`, ..., `sourceN`**

The source object(s) — objects containing the properties you want to apply.

### Return value

The target object.

### Exceptions

**`TypeError`**

Thrown in one of the following cases:

- The `target` parameter is `null` or `undefined`.
- Assignment of a property on the target object fails; for example, because the property is non-writable on the target object, or because its setter throws an error.

## Description

Properties in the target object are overwritten by properties in the sources if they have the same key. Later sources' properties overwrite earlier ones.

The `Object.assign()` method only copies *enumerable* and *own* properties from a source object to a target object. It uses `[[Get]]` on the source and `[[Set]]` on the target, so it will invoke getters and setters. Therefore it *assigns* properties, versus copying or defining new properties. This may make it unsuitable for merging new properties into a prototype if the merge sources contain getters.

For copying property definitions (including their enumerability) into prototypes, use `Object.getOwnPropertyDescriptor()` and `Object.defineProperty()` instead.

Both `String` and `Symbol` properties are copied.

In case of an error, for example if a property is non-writable, a `TypeError` is raised, and the `target` object is changed if any properties are added before the error is raised.

> **Note:** `Object.assign()` does not throw on `null` or `undefined` sources.

## Examples

### Cloning an object

```js
const obj = { a: 1 };
const copy = Object.assign({}, obj);
console.log(copy); // { a: 1 }
```

### Warning for deep clone

For deep cloning, we need to use alternatives like `structuredClone()`, because `Object.assign()` copies property values.

If the source value is a reference to an object, it only copies the reference value.

```js
const obj1 = { a: 0, b: { c: 0 } };
const obj2 = Object.assign({}, obj1);
console.log(obj2); // { a: 0, b: { c: 0 } }

obj1.a = 1;
console.log(obj1); // { a: 1, b: { c: 0 } }
console.log(obj2); // { a: 0, b: { c: 0 } }

obj2.a = 2;
console.log(obj1); // { a: 1, b: { c: 0 } }
console.log(obj2); // { a: 2, b: { c: 0 } }

obj2.b.c = 3;
console.log(obj1); // { a: 1, b: { c: 3 } }
console.log(obj2); // { a: 2, b: { c: 3 } }
```

*(code continues on the next page)*

(code continued from the previous page)

```
// Deep Clone
const obj3 = { a: 0, b: { c: 0 } };
const obj4 = structuredClone(obj3);
obj3.a = 4;
obj3.b.c = 4;
console.log(obj4); // { a: 0, b: { c: 0 } }
```

### Merging objects

js

```
const o1 = { a: 1 };
const o2 = { b: 2 };
const o3 = { c: 3 };

const obj = Object.assign(o1, o2, o3);
console.log(obj); // { a: 1, b: 2, c: 3 }
console.log(o1); // { a: 1, b: 2, c: 3 }, target object itself is changed.
```

### Merging objects with same properties

js

```
const o1 = { a: 1, b: 1, c: 1 };
const o2 = { b: 2, c: 2 };
const o3 = { c: 3 };

const obj = Object.assign({}, o1, o2, o3);
console.log(obj); // { a: 1, b: 2, c: 3 }
```

The properties are overwritten by other objects that have the same properties later in the parameters order.

### Copying symbol-typed properties

js

```
const o1 = { a: 1 };
const o2 = { [Symbol("foo")]: 2 };

const obj = Object.assign({}, o1, o2);
console.log(obj); // { a : 1, [Symbol("foo")]: 2 } (cf. bug 1207182 on Firefox)
Object.getOwnPropertySymbols(obj); // [Symbol(foo)]
```

### Properties on the prototype chain and non-enumerable properties cannot be copied

js

```
const obj = Object.create(
  // foo is on obj's prototype chain.
  { foo: 1 },
  {
    bar: {
      value: 2, // bar is a non-enumerable property.
    },
    baz: {
      value: 3,
      enumerable: true, // baz is an own enumerable property.
    },
  },
);
```

(code continues on the next page)

*(code continued from the previous page)*

```
const copy = Object.assign({}, obj);
console.log(copy); // { baz: 3 }
```

### Primitives will be wrapped to objects

js

```
const v1 = "abc";
const v2 = true;
const v3 = 10;
const v4 = Symbol("foo");

const obj = Object.assign({}, v1, null, v2, undefined, v3, v4);
// Primitives will be wrapped, null and undefined will be ignored.
// Note, only string wrappers can have own enumerable properties.
console.log(obj); // { "0": "a", "1": "b", "2": "c" }

// Primitives as the target are also wrapped to objects
const number = Object.assign(3, { a: 1 });
console.log(number); // Number {3, a: 1}
console.log(typeof number); // object
console.log(number.a); // 1

// null and undefined as targets throw TypeError
try {
  Object.assign(null, { a: 1 });
} catch (e) {
  console.log(e.message); // "Cannot convert undefined or null to object"
}
```

### Exceptions will interrupt the ongoing copying task

js

```
const target = Object.defineProperty({}, "foo", {
  value: 1,
  writable: false,
}); // target.foo is a read-only property

Object.assign(target, { bar: 2 }, { foo2: 3, foo: 3, foo3: 3 }, { baz: 4 });
// TypeError: "foo" is read-only
// The Exception is thrown when assigning target.foo

console.log(target.bar); // 2, the first source was copied successfully.
console.log(target.foo2); // 3, the first property of the second source was copied
successfully.
console.log(target.foo); // 1, exception is thrown here.
console.log(target.foo3); // undefined, assign method has finished, foo3 will not be copied.
console.log(target.baz); // undefined, the third source will not be copied either.
```

### Copying accessors

js

```
const obj = {
  foo: 1,
  get bar() {
    return 2;
  },
};
```

*(code continues on the next page)*

(code continued from the previous page)

```
let copy = Object.assign({}, obj);
console.log(copy);
// { foo: 1, bar: 2 }
// The value of copy.bar is obj.bar's getter's return value.

// This is an assign function that copies full descriptors
function completeAssign(target, ...sources) {
  sources.forEach((source) => {
    const descriptors = Object.keys(source).reduce((descriptors, key) => {
      descriptors[key] = Object.getOwnPropertyDescriptor(source, key);
      return descriptors;
    }, {});

    // By default, Object.assign copies enumerable Symbols, too
    Object.getOwnPropertySymbols(source).forEach((sym) => {
      const descriptor = Object.getOwnPropertyDescriptor(source, sym);
      if (descriptor.enumerable) {
        descriptors[sym] = descriptor;
      }
    });
    Object.defineProperties(target, descriptors);
  });
  return target;
}

copy = completeAssign({}, obj);
console.log(copy);
// { foo:1, get bar() { return 2 } }
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-object.assign` |

## Browser compatibility

| | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| Feature | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `assign` | 45 | 12 | 34 | 32 | 9 | 45 | 34 | 32 | 9 | 5.0 | 45 | 9 | 1.0.0 | 1.0 | 4.0.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Object.assign`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Object.assign` in `core-js`
- es-shims polyfill of `Object.assign`
- `Object.defineProperties()`
- Enumerability and ownership of properties
- Spread in object literals

JavaScript › Reference › Standard built-in objects › Object

# Object.create()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/create · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Object.create()`** static method creates a new object, using an existing object as the prototype of the newly created object.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Object.create()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const person = {
  isHuman: false,
  printIntroduction() {
    console.log(`My name is ${this.name}. Am I human? ${this.isHuman}`);
  },
};

const me = Object.create(person);

me.name = "Matthew"; // "name" is a property set on "me", but not on "person"
me.isHuman = true; // Inherited properties can be overwritten

me.printIntroduction();
// Expected output: "My name is Matthew. Am I human? true"
```

## Syntax

js

```
Object.create(proto)
Object.create(proto, propertiesObject)
```

### Parameters

**`proto`**

The object which should be the prototype of the newly-created object.

**`propertiesObject`** Optional

If specified and not `undefined`, an object whose enumerable own properties specify property descriptors to be added to the newly-created object, with the corresponding property names. These properties correspond to the second argument of `Object.defineProperties()`.

### Return value

A new object with the specified prototype object and properties.

### Exceptions

**`TypeError`**

Thrown if `proto` is neither `null` nor an `Object`.

## Examples

### Classical inheritance with Object.create()

Below is an example of how to use `Object.create()` to achieve classical inheritance. This is for a single inheritance, which is all that JavaScript supports.

js

```
// Shape - superclass
function Shape() {
  this.x = 0;
  this.y = 0;
}

// superclass method
Shape.prototype.move = function (x, y) {
  this.x += x;
  this.y += y;
  console.info("Shape moved.");
};

// Rectangle - subclass
function Rectangle() {
  Shape.call(this); // call super constructor.
}

// subclass extends superclass
Rectangle.prototype = Object.create(Shape.prototype, {
  // If you don't set Rectangle.prototype.constructor to Rectangle,
  // it will take the prototype.constructor of Shape (parent).
  // To avoid that, we set the prototype.constructor to Rectangle (child).
  constructor: {
    value: Rectangle,
    enumerable: false,
    writable: true,
    configurable: true,
  },
});

const rect = new Rectangle();

console.log("Is rect an instance of Rectangle?", rect instanceof Rectangle); // true
console.log("Is rect an instance of Shape?", rect instanceof Shape); // true
rect.move(1, 1); // Logs 'Shape moved.'
```

Note that there are caveats to watch out for using `create()`, such as re-adding the `constructor` property to ensure proper semantics. Although `Object.create()` is believed to have better performance than mutating the prototype with `Object.setPrototypeOf()`, the difference is in fact negligible if no instances have been created and property accesses haven't been optimized yet. In modern code, the class syntax should be preferred in any case.

### Using propertiesObject argument with Object.create()

`Object.create()` allows fine-tuned control over the object creation process. The object initializer syntax is, in fact, a syntax sugar of `Object.create()`. With `Object.create()`, we can create objects with a designated prototype and also some properties. Note that the second parameter maps keys to

*property descriptors* — this means you can control each property's enumerability, configurability, etc. as well, which you can't do in object initializers.

```js
o = {};
// Is equivalent to:
o = Object.create(Object.prototype);

o = Object.create(Object.prototype, {
  // foo is a regular data property
  foo: {
    writable: true,
    configurable: true,
    value: "hello",
  },
  // bar is an accessor property
  bar: {
    configurable: false,
    get() {
      return 10;
    },
    set(value) {
      console.log("Setting `o.bar` to", value);
    },
  },
});

// Create a new object whose prototype is a new, empty
// object and add a single property 'p', with value 42.
o = Object.create({}, { p: { value: 42 } });
```

With `Object.create()`, we can create an object with `null` as prototype. The equivalent syntax in object initializers would be the `__proto__` key.

```js
o = Object.create(null);
// Is equivalent to:
o = { __proto__: null };
```

By default properties are *not* writable, enumerable or configurable.

```js
o.p = 24; // throws in strict mode
o.p; // 42

o.q = 12;
for (const prop in o) {
  console.log(prop);
}
// 'q'

delete o.p;
// false; throws in strict mode
```

To specify a property with the same attributes as in an initializer, explicitly specify `writable`, `enumerable` and `configurable`.

```js
o2 = Object.create(
  {},
  {
    p: {
      value: 42,
      writable: true,
      enumerable: true,
      configurable: true,
    },
  },
);
// This is not equivalent to:
// o2 = Object.create({ p: 42 })
// which will create an object with prototype { p: 42 }
```

You can use `Object.create()` to mimic the behavior of the `new` operator.

```js
function Constructor() {}
o = new Constructor();
// Is equivalent to:
o = Object.create(Constructor.prototype);
```

Of course, if there is actual initialization code in the `Constructor` function, the `Object.create()` method cannot reflect it.

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-object.create` |

## Browser compatibility

| | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| Feature | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `create` | 5 | 12 | 4 | 11.6 | 5 | 18 | 4 | 12 | 5 | 1.0 | 4.4 | 5 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Object.create`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Object.create` in `core-js`
- `Object.defineProperty()`
- `Object.defineProperties()`
- `Object.prototype.isPrototypeOf()`
- `Reflect.construct()`
- Object.getPrototypeOf by John Resig (2008)

JavaScript › Reference › Standard built-in objects › Object

# Object.defineProperties()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/defineProperties · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Object.defineProperties()`** static method defines new or modifies existing properties directly on an object, returning the object.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Object.defineProperties()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const object = {};

Object.defineProperties(object, {
  property1: {
    value: 42,
    writable: true,
  },
  property2: {},
});

console.log(object.property1);
// Expected output: 42
```

## Syntax

js

```
Object.defineProperties(obj, props)
```

### Parameters

**`obj`**

The object on which to define or modify properties.

**`props`**

An object whose keys represent the names of properties to be defined or modified and whose values are objects describing those properties. Each value in `props` must be either a data descriptor or an accessor descriptor; it cannot be both (see `Object.defineProperty()` for more details).

Data descriptors and accessor descriptors may optionally contain the following keys:

**`configurable`**

`true` if and only if the type of this property descriptor may be changed and if the property may be deleted from the corresponding object. **Defaults to `false`.**

**`enumerable`**

`true` if and only if this property shows up during enumeration of the properties on the corresponding object. **Defaults to `false`.**

A data descriptor also has the following optional keys:

**`value`**

The value associated with the property. Can be any valid JavaScript value (number, object, function, etc.). **Defaults to `undefined`.**

**`writable`**

`true` if and only if the value associated with the property may be changed with an assignment operator. **Defaults to `false`.**

An accessor descriptor also has the following optional keys:

**`get`**

A function which serves as a getter for the property, or `undefined` if there is no getter. The function's return value will be used as the value of the property. **Defaults to `undefined`.**

**`set`**

A function which serves as a setter for the property, or `undefined` if there is no setter. The function will receive as its only argument the new value being assigned to the property. **Defaults to `undefined`.**

If a descriptor has neither of `value`, `writable`, `get` and `set` keys, it is treated as a data descriptor. If a descriptor has both `value` or `writable` and `get` or `set` keys, an exception is thrown.

### Return value

The object that was passed to the function.

## Examples

### Using Object.defineProperties

```js
const obj = {};
Object.defineProperties(obj, {
  property1: {
    value: true,
    writable: true,
  },
  property2: {
    value: "Hello",
    writable: false,
  },
  // etc. etc.
});
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-object.defineproperties |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `defineProperties` | 5 | 12 | 4 | 11.6 | 5 | 18 | 4 | 12 | 5 | 1.0 | 4.4 | 5 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Object.defineProperties`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Object.defineProperties` in `core-js`
- es-shims polyfill of `Object.defineProperties`
- `Object.defineProperty()`
- `Object.keys()`
- Enumerability and ownership of properties

---

JavaScript › Reference › Standard built-in objects › Object

# Object.defineProperty()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/defineProperty · MDN page last modified 2025-07-20

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Object.defineProperty()`** static method defines a new property directly on an object, or modifies an existing property on an object, and returns the object.

## Try it

*▷ Interactive example on MDN: "JavaScript Demo: Object.defineProperty()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const object = {};

Object.defineProperty(object, "foo", {
  value: 42,
  writable: false,
});

object.foo = 77;
// Throws an error in strict mode

console.log(object.foo);
// Expected output: 42
```

## Syntax

js

```
Object.defineProperty(obj, prop, descriptor)
```

### Parameters

**`obj`**

The object on which to define the property.

**`prop`**

A string or `Symbol` specifying the key of the property to be defined or modified.

**`descriptor`**

The descriptor for the property being defined or modified.

### Return value

The object that was passed to the function, with the specified property added or modified.

## Description

`Object.defineProperty()` allows a precise addition to or modification of a property on an object. Normal property addition through assignment creates properties which show up during property enumeration (`for...in`, `Object.keys()`, etc.), whose values may be changed and which may be deleted. This method allows these extra details to be changed from their defaults. By default, properties added using `Object.defineProperty()` are not writable, not enumerable, and not configurable. In addition, `Object.defineProperty()` uses the `[[DefineOwnProperty]]` internal method, instead of `[[Set]]`, so it does not invoke setters, even when the property is already present.

Property descriptors present in objects come in two main flavors: data descriptors and accessor descriptors. A **data descriptor** is a property with a value that may or may not be writable. An **accessor descriptor** is a property described by a getter-setter pair of functions. A descriptor must be one of these two flavors; it cannot be both.

Both data and accessor descriptors are objects. They share the following optional keys (please note: the **defaults** mentioned here are in the case of defining properties using `Object.defineProperty()`):

**`configurable`**

when this is set to `false`,

- the type of this property cannot be changed between data property and accessor property, and
- the property may not be deleted, and
- other attributes of its descriptor cannot be changed (however, if it's a data descriptor with `writable: true`, the `value` can be changed, and `writable` can be changed to `false`).

**Defaults to `false`.**

**`enumerable`**

`true` if and only if this property shows up during enumeration of the properties on the corresponding object. **Defaults to `false`.**

A **data descriptor** also has the following optional keys:

**`value`**

The value associated with the property. Can be any valid JavaScript value (number, object, function, etc.). **Defaults to `undefined`.**

**`writable`**

`true` if the value associated with the property may be changed with an assignment operator. **Defaults to `false`.**

An **accessor descriptor** also has the following optional keys:

**`get`**
: A function which serves as a getter for the property, or `undefined` if there is no getter. When the property is accessed, this function is called without arguments and with `this` set to the object through which the property is accessed (this may not be the object on which the property is defined due to inheritance). The return value will be used as the value of the property. **Defaults to `undefined`.**

**`set`**
: A function which serves as a setter for the property, or `undefined` if there is no setter. When the property is assigned, this function is called with one argument (the value being assigned to the property) and with `this` set to the object through which the property is assigned. **Defaults to `undefined`.**

If a descriptor doesn't have any of the `value`, `writable`, `get`, and `set` keys, it is treated as a data descriptor. If a descriptor is both a data descriptor (because it has `value` or `writable`) and an accessor descriptor (because it has `get` or `set`), an exception is thrown.

These attributes are not necessarily the descriptor's own properties. Inherited properties will be considered as well. In order to ensure these defaults are preserved, you might freeze existing objects in the descriptor object's prototype chain upfront, specify all options explicitly, or create a `null`-prototype object.

```js
const obj = {};
// 1. Using a null prototype: no inherited properties
const descriptor = Object.create(null);
descriptor.value = "static";

// not enumerable, not configurable, not writable as defaults
Object.defineProperty(obj, "key", descriptor);

// 2. Being explicit by using a throw-away object literal with all attributes present
Object.defineProperty(obj, "key2", {
  enumerable: false,
  configurable: false,
  writable: false,
  value: "static",
});

// 3. Prevents adding or removing the object prototype properties
// (value, get, set, enumerable, writable, configurable)
Object.freeze(Object.prototype);
```

When the property already exists, `Object.defineProperty()` attempts to modify the property according to the values in the descriptor and the property's current configuration.

If the old descriptor had its `configurable` attribute set to `false`, the property is said to be *non-configurable*. It is not possible to change any attribute of a non-configurable accessor property, and it is not possible to switch between data and accessor property types. For data properties with `writable: true`, it is possible to modify the value and change the `writable` attribute from `true` to `false`. A `TypeError` is thrown when attempts are made to change non-configurable property attributes (except `value` and `writable`, if permitted), except when defining a value same as the original value on a data property.

When the current property is configurable, defining an attribute to `undefined` effectively deletes it. For example, if `o.k` is an accessor property, `Object.defineProperty(o, "k", { set: undefined })` will remove the setter, making `k` only have a getter and become readonly. If an attribute is absent from the new descriptor, the old descriptor attribute's value is kept (it won't be implicitly re-defined to `undefined`). It is possible to toggle between data and accessor property by giving a descriptor of a different "flavor". For example, if the new descriptor is a data descriptor (with `value` or `writable`), the original descriptor's `get` and `set` attributes will both be dropped.

## Examples

### Creating a property

When the property specified doesn't exist in the object, `Object.defineProperty()` creates a new property as described. Fields may be omitted from the descriptor and default values for those fields are inputted.

```js
const o = {}; // Creates a new object

// Example of an object property added
// with defineProperty with a data property descriptor
Object.defineProperty(o, "a", {
  value: 37,
  writable: true,
  enumerable: true,
  configurable: true,
});
// 'a' property exists in the o object and its value is 37

// Example of an object property added
// with defineProperty with an accessor property descriptor
let bValue = 38;
Object.defineProperty(o, "b", {
  get() {
    return bValue;
  },
  set(newValue) {
    bValue = newValue;
  },
  enumerable: true,
  configurable: true,
});
o.b; // 38
// 'b' property exists in the o object and its value is 38
// The value of o.b is now always identical to bValue,
// unless o.b is redefined

// You cannot try to mix both:
Object.defineProperty(o, "conflict", {
  value: 0x9f91102,
  get() {
    return 0xdeadbeef;
  },
});
// throws a TypeError: value appears
// only in data descriptors,
// get appears only in accessor descriptors
```

## Modifying a property

When modifying an existing property, the current property configuration determines if the operator succeeds, does nothing, or throws a `TypeError`.

### Writable attribute

When the `writable` property attribute is `false`, the property is said to be "non-writable". It cannot be reassigned. Trying to write to a non-writable property doesn't change it and results in an error in strict mode.

```js
const o = {}; // Creates a new object

Object.defineProperty(o, "a", {
  value: 37,
  writable: false,
});

console.log(o.a); // 37
o.a = 25; // No error thrown
// (it would throw in strict mode,
// even if the value had been the same)
console.log(o.a); // 37; the assignment didn't work

// strict mode
(() => {
  "use strict";
  const o = {};
  Object.defineProperty(o, "b", {
    value: 2,
    writable: false,
  });
  o.b = 3; // throws TypeError: "b" is read-only
  return o.b; // returns 2 without the line above
})();
```

### Enumerable attribute

The `enumerable` property attribute defines whether the property is considered by `Object.assign()` or the spread operator. For non-`Symbol` properties, it also defines whether it shows up in a `for...in` loop and `Object.keys()` or not. For more information, see Enumerability and ownership of properties.

```js
const o = {};
Object.defineProperty(o, "a", {
  value: 1,
  enumerable: true,
});
Object.defineProperty(o, "b", {
  value: 2,
  enumerable: false,
});
Object.defineProperty(o, "c", {
  value: 3,
}); // enumerable defaults to false
o.d = 4; // enumerable defaults to true when creating a property by setting it
Object.defineProperty(o, Symbol.for("e"), {
  value: 5,
  enumerable: true,
```

*(code continues on the next page)*

(code continued from the previous page)

```
});
Object.defineProperty(o, Symbol.for("f"), {
  value: 6,
  enumerable: false,
});

for (const i in o) {
  console.log(i);
}
// Logs 'a' and 'd' (always in that order)

Object.keys(o); // ['a', 'd']

o.propertyIsEnumerable("a"); // true
o.propertyIsEnumerable("b"); // false
o.propertyIsEnumerable("c"); // false
o.propertyIsEnumerable("d"); // true
o.propertyIsEnumerable(Symbol.for("e")); // true
o.propertyIsEnumerable(Symbol.for("f")); // false

const p = { ...o };
p.a; // 1
p.b; // undefined
p.c; // undefined
p.d; // 4
p[Symbol.for("e")]; // 5
p[Symbol.for("f")]; // undefined
```

### Configurable attribute

The `configurable` attribute controls whether the property can be deleted from the object and whether its attributes (other than `value` and `writable`) can be changed.

This example illustrates a non-configurable accessor property.

js

```
const o = {};
Object.defineProperty(o, "a", {
  get() {
    return 1;
  },
  configurable: false,
});

Object.defineProperty(o, "a", {
  configurable: true,
}); // throws a TypeError
Object.defineProperty(o, "a", {
  enumerable: true,
}); // throws a TypeError
Object.defineProperty(o, "a", {
  set() {},
}); // throws a TypeError (set was undefined previously)
Object.defineProperty(o, "a", {
  get() {
    return 1;
  },
}); // throws a TypeError
// (even though the new get does exactly the same thing)
Object.defineProperty(o, "a", {
  value: 12,
```

(code continues on the next page)

*(code continued from the previous page)*

```
}); // throws a TypeError
// ('value' can be changed when 'configurable' is false, but only when the property is a
writable data property)

console.log(o.a); // 1
delete o.a; // Nothing happens; throws an error in strict mode
console.log(o.a); // 1
```

If the `configurable` attribute of `o.a` had been `true`, none of the errors would be thrown and the property would be deleted at the end.

This example illustrates a non-configurable but writable data property. The property's `value` can still be changed, and `writable` can still be toggled from `true` to `false`.

js

```
const o = {};
Object.defineProperty(o, "b", {
  writable: true,
  configurable: false,
});
console.log(o.b); // undefined
Object.defineProperty(o, "b", {
  value: 1,
}); // Even when configurable is false, because the object is writable, we may still replace
the value
console.log(o.b); // 1
o.b = 2; // We can change the value with assignment operators as well
console.log(o.b); // 2
// Toggle the property's writability
Object.defineProperty(o, "b", {
  writable: false,
});
Object.defineProperty(o, "b", {
  value: 1,
}); // TypeError: because the property is neither writable nor configurable, it cannot be
modified
// At this point, there's no way to further modify 'b'
// or restore its writability
```

This example illustrates a configurable but non-writable data property. The property's `value` may still be replaced with `defineProperty` (but not with assignment operators), and `writable` may be toggled.

js

```
const o = {};
Object.defineProperty(o, "b", {
  writable: false,
  configurable: true,
});
Object.defineProperty(o, "b", {
  value: 1,
}); // We can replace the value with defineProperty
console.log(o.b); // 1
o.b = 2; // throws TypeError in strict mode: cannot change a non-writable property's value
with assignment
```

This example illustrates a non-configurable and non-writable data property. There's no way to update any attribute of the property, including its `value`.

```js
const o = {};
Object.defineProperty(o, "b", {
  writable: false,
  configurable: false,
});
Object.defineProperty(o, "b", {
  value: 1,
}); // TypeError: the property cannot be modified because it is neither writable nor
configurable.
```

### Adding properties and default values

It is important to consider the way default values of attributes are applied. There is often a difference between using property accessors to assign a value and using `Object.defineProperty()`, as shown in the example below.

```js
const o = {};

o.a = 1;
// is equivalent to:
Object.defineProperty(o, "a", {
  value: 1,
  writable: true,
  configurable: true,
  enumerable: true,
});

// On the other hand,
Object.defineProperty(o, "a", { value: 1 });
// is equivalent to:
Object.defineProperty(o, "a", {
  value: 1,
  writable: false,
  configurable: false,
  enumerable: false,
});
```

### Custom setters and getters

The example below shows how to implement a self-archiving object. When `temperature` property is set, the `archive` array gets a log entry.

```js
function Archiver() {
  let temperature = null;
  const archive = [];

  Object.defineProperty(this, "temperature", {
    get() {
      console.log("get!");
      return temperature;
    },
    set(value) {
      temperature = value;
      archive.push({ val: temperature });
```

(code continues on the next page)

*(code continued from the previous page)*

```
    },
  });

  this.getArchive = () => archive;
}

const arc = new Archiver();
arc.temperature; // 'get!'
arc.temperature = 11;
arc.temperature = 13;
arc.getArchive(); // [{ val: 11 }, { val: 13 }]
```

In this example, a getter always returns the same value.

js

```
const pattern = {
  get() {
    return "I always return this string, whatever you have assigned";
  },
  set() {
    this.myName = "this is my name string";
  },
};

function TestDefineSetAndGet() {
  Object.defineProperty(this, "myProperty", pattern);
}

const instance = new TestDefineSetAndGet();
instance.myProperty = "test";
console.log(instance.myProperty);
// I always return this string, whatever you have assigned

console.log(instance.myName); // this is my name string
```

### Inheritance of properties

If an accessor property is inherited, its `get` and `set` methods will be called when the property is accessed and modified on descendant objects. If these methods use a variable to store the value, this value will be shared by all objects.

js

```
function MyClass() {}

let value;
Object.defineProperty(MyClass.prototype, "x", {
  get() {
    return value;
  },
  set(x) {
    value = x;
  },
});

const a = new MyClass();
const b = new MyClass();
a.x = 1;
console.log(b.x); // 1
```

This can be fixed by storing the value in another property. In `get` and `set` methods, `this` points to the object which is used to access or modify the property.

```js
function MyClass() {}

Object.defineProperty(MyClass.prototype, "x", {
  get() {
    return this.storedX;
  },
  set(x) {
    this.storedX = x;
  },
});

const a = new MyClass();
const b = new MyClass();
a.x = 1;
console.log(b.x); // undefined
```

Unlike accessor properties, data properties are always set on the object itself, not on a prototype. However, if a non-writable data property is inherited, it is still prevented from being modified on the object.

```js
function MyClass() {}

MyClass.prototype.x = 1;
Object.defineProperty(MyClass.prototype, "y", {
  writable: false,
  value: 1,
});

const a = new MyClass();
a.x = 2;
console.log(a.x); // 2
console.log(MyClass.prototype.x); // 1
a.y = 2; // Ignored, throws in strict mode
console.log(a.y); // 1
console.log(MyClass.prototype.y); // 1
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-object.defineproperty |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `defineProperty` | 5 | 12 | 4 | 11.6 | 5.1[1] | 18 | 4 | 12 | 6[2] | 1.0 | 4.4 | 6[2] | 1.0.0 | 1.0 | 0.10.0 |

1. Also supported in Safari 5, but not on DOM objects.
2. Also supported in Safari for iOS 4.2, but not on DOM objects.

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Object.defineProperty`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Enumerability and ownership of properties
- `Object.defineProperties()`
- `Object.prototype.propertyIsEnumerable()`
- `Object.getOwnPropertyDescriptor()`
- `get`
- `set`
- `Object.create()`
- `Reflect.defineProperty()`

---

JavaScript › Reference › Standard built-in objects › Object

# Object.entries()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/entries · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since March 2017.

The **`Object.entries()`** static method returns an array of a given object's own enumerable string-keyed property key-value pairs.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Object.entries()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const object = {
  a: "some string",
  b: 42,
};

for (const [key, value] of Object.entries(object)) {
  console.log(`${key}: ${value}`);
}

// Expected output:
// "a: some string"
// "b: 42"
```

## Syntax

```js
Object.entries(obj)
```

### Parameters

**`obj`**

An object.

### Return value

An array of the given object's own enumerable string-keyed property key-value pairs. Each key-value pair is an array with two elements: the first element is the property key (which is always a string), and the second element is the property value.

## Description

`Object.entries()` returns an array whose elements are arrays corresponding to the enumerable string-keyed property key-value pairs found directly upon `object`. This is the same as iterating with a `for...in` loop, except that a `for...in` loop enumerates properties in the prototype chain as well. The order of the array returned by `Object.entries()` is the same as that provided by a `for...in` loop.

If you only need the property keys, use `Object.keys()` instead. If you only need the property values, use `Object.values()` instead.

## Examples

### Using Object.entries()

```js
const obj = { foo: "bar", baz: 42 };
console.log(Object.entries(obj)); // [ ['foo', 'bar'], ['baz', 42] ]

const arrayLike = { 0: "a", 1: "b", 2: "c" };
console.log(Object.entries(arrayLike)); // [ ['0', 'a'], ['1', 'b'], ['2', 'c'] ]

const randomKeyOrder = { 100: "a", 2: "b", 7: "c" };
console.log(Object.entries(randomKeyOrder)); // [ ['2', 'b'], ['7', 'c'], ['100', 'a'] ]

// getFoo is a non-enumerable property
const myObj = Object.create(
  {},
  {
    getFoo: {
      value() {
        return this.foo;
      },
    },
  },
);
myObj.foo = "bar";
console.log(Object.entries(myObj)); // [ ['foo', 'bar'] ]
```

### Using Object.entries() on primitives

Non-object arguments are coerced to objects. `undefined` and `null` cannot be coerced to objects and throw a `TypeError` upfront. Only strings may have own enumerable properties, while all other primitives return an empty array.

```js
// Strings have indices as enumerable own properties
console.log(Object.entries("foo")); // [ ['0', 'f'], ['1', 'o'], ['2', 'o'] ]

// Other primitives except undefined and null have no own properties
console.log(Object.entries(100)); // []
```

### Converting an Object to a Map

The `Map()` constructor accepts an iterable of `entries`. With `Object.entries`, you can easily convert from `Object` to `Map`:

```js
const obj = { foo: "bar", baz: 42 };
const map = new Map(Object.entries(obj));
console.log(map); // Map(2) {"foo" => "bar", "baz" => 42}
```

### Iterating through an Object

Using array destructuring, you can iterate through objects easily.

```js
// Using for...of loop
const obj = { a: 5, b: 7, c: 9 };
for (const [key, value] of Object.entries(obj)) {
  console.log(`${key} ${value}`); // "a 5", "b 7", "c 9"
}

// Using array methods
Object.entries(obj).forEach(([key, value]) => {
  console.log(`${key} ${value}`); // "a 5", "b 7", "c 9"
});
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-object.entries |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `entries` | 54 | 14 | 47 | 41 | 10.1 | 54 | 47 | 41 | 10.3 | 6.0 | 54 | 10.3 | 1.0.0 | 1.0 | 7.0.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Object.entries`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Object.entries` in `core-js`
- es-shims polyfill of `Object.entries`
- Enumerability and ownership of properties
- `Object.keys()`
- `Object.values()`
- `Object.prototype.propertyIsEnumerable()`
- `Object.create()`
- `Object.fromEntries()`
- `Object.getOwnPropertyNames()`
- `Map.prototype.entries()`

---

JavaScript › Reference › Standard built-in objects › Object

# Object.freeze()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/freeze · MDN page last modified 2026-05-02

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Object.freeze()`** static method *freezes* an object. Freezing an object prevents extensions and makes existing properties non-writable and non-configurable. A frozen object can no longer be changed: new properties cannot be added, existing properties cannot be removed, their enumerability, configurability, writability, or value cannot be changed, and the object's prototype cannot be re-assigned. `freeze()` returns the same object that was passed in.

Freezing an object is the highest integrity level that JavaScript provides.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Object.freeze()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const obj = {
  prop: 42,
};

Object.freeze(obj);

obj.prop = 33;
// Throws an error in strict mode

console.log(obj.prop);
// Expected output: 42
```

## Syntax

js

```
Object.freeze(obj)
```

### Parameters

**obj**

The object to freeze.

### Return value

The object that was passed to the function.

## Description

Freezing an object is equivalent to preventing extensions and then changing all existing properties' descriptors' `configurable` to `false` — and for data properties, `writable` to `false` as well. Nothing can be added to or removed from the properties set of a frozen object. Any attempt to do so will fail, either silently or by throwing a `TypeError` exception (most commonly, but not exclusively, when in strict mode).

For data properties of a frozen object, their values cannot be changed since the `writable` and `configurable` attributes are set to `false`. Accessor properties (getters and setters) work the same — the property value returned by the getter may still change, and the setter can still be called without throwing errors when setting the property. Note that values that are objects can still be modified, unless they are also frozen. As an object, an array can be frozen; after doing so, its elements cannot be altered and no elements can be added to or removed from the array.

Private elements are not properties and do not have the concept of property descriptors. Freezing an object with private elements does not prevent the values of these private elements from being changed. (Freezing objects is usually meant as a security measure against external code, but external code cannot access private elements anyway.) Private elements cannot be added or removed from the object, whether the object is frozen or not.

`freeze()` returns the same object that was passed into the function. It *does not* create a frozen copy.

A `TypedArray` or a `DataView` with elements will cause a `TypeError`, as they are views over memory and will definitely cause other possible issues:

```js
Object.freeze(new Uint8Array(0)); // No elements
// Uint8Array []

Object.freeze(new Uint8Array(1)); // Has elements
// TypeError: Cannot freeze array buffer views with elements

Object.freeze(new DataView(new ArrayBuffer(32))); // No elements
// DataView {}

Object.freeze(new Float64Array(new ArrayBuffer(64), 32, 0)); // No elements
// Float64Array []

Object.freeze(new Float64Array(new ArrayBuffer(64), 32, 2)); // Has elements
// TypeError: Cannot freeze array buffer views with elements
```

Note that as the standard three properties (`buf.byteLength`, `buf.byteOffset` and `buf.buffer`) are read-only (as are those of an `ArrayBuffer` or `SharedArrayBuffer`), there is no reason for attempting to freeze these properties.

Unlike `Object.seal()`, existing properties in objects frozen with `Object.freeze()` are made immutable and data properties cannot be re-assigned.

## Examples

### Freezing objects

```js
const obj = {
  prop() {},
  foo: "bar",
};

// Before freezing: new properties may be added,
// and existing properties may be changed or removed
obj.foo = "baz";
obj.lumpy = "woof";
delete obj.prop;

// Freeze.
const o = Object.freeze(obj);

// The return value is just the same object we passed in.
o === obj; // true

// The object has become frozen.
Object.isFrozen(obj); // === true

// Now any changes will fail
obj.foo = "quux"; // silently does nothing
// silently doesn't add the property
obj.quaxxor = "the friendly duck";

// In strict mode such attempts will throw TypeErrors
function fail() {
  "use strict";
  obj.foo = "sparky"; // throws a TypeError
  delete obj.foo; // throws a TypeError
  delete obj.quaxxor; // returns true since attribute 'quaxxor' was never added
  obj.sparky = "arf"; // throws a TypeError
}

fail();

// Attempted changes through Object.defineProperty;
// both statements below throw a TypeError.
Object.defineProperty(obj, "ohai", { value: 17 });
Object.defineProperty(obj, "foo", { value: "eit" });

// It's also impossible to change the prototype
// both statements below will throw a TypeError.
Object.setPrototypeOf(obj, { x: 20 });
obj.__proto__ = { x: 20 };
```

### Freezing arrays

```js
const a = [0];
Object.freeze(a); // The array cannot be modified now.

a[0] = 1; // fails silently

// In strict mode such attempt will throw a TypeError
function fail() {
  "use strict";
  a[0] = 1;
}

fail();

// Attempted to push
a.push(2); // throws a TypeError
```

The object being frozen is *immutable*. However, it is not necessarily *constant*. The following example shows that a frozen object is not constant (freeze is shallow).

```js
const obj1 = {
  internal: {},
};

Object.freeze(obj1);
obj1.internal.a = "value";

obj1.internal.a; // 'value'
```

To be a constant object, the entire reference graph (direct and indirect references to other objects) must reference only immutable frozen objects. The object being frozen is said to be immutable because the entire object *state* (values and references to other objects) within the whole object is fixed. Note that strings, numbers, and booleans are always immutable and that Functions and Arrays are objects.

### Deep freezing

The result of calling `Object.freeze(object)` only applies to the immediate properties of `object` itself and will prevent future property addition, removal or value re-assignment operations *only* on `object`. If the value of those properties are objects themselves, those objects are not frozen and may be the target of property addition, removal or value re-assignment operations.

```js
const employee = {
  name: "Mayank",
  designation: "Developer",
  address: {
    street: "Rohini",
    city: "Delhi",
  },
};
```

*(code continues on the next page)*

(code continued from the previous page)

```
Object.freeze(employee);

employee.name = "Dummy"; // fails silently in non-strict mode
employee.address.city = "Noida"; // attributes of child object can be modified

console.log(employee.address.city); // "Noida"
```

To make an object immutable, recursively freeze each non-primitive property (deep freeze). Use the pattern on a case-by-case basis based on your design when you know the object contains no cycles in the reference graph, otherwise an endless loop will be triggered. For example, functions created with the `function` syntax have a `prototype` property with a `constructor` property that points to the function itself, so they have cycles by default. Other functions, such as arrow functions, can still be frozen.

An enhancement to `deepFreeze()` would be to store the objects it has already visited, so you can suppress calling `deepFreeze()` recursively when an object is in the process of being made immutable. For one example, see using `WeakSet` to detect circular references. You still run a risk of freezing an object that shouldn't be frozen, such as `window`.

js

```
function deepFreeze(object) {
  // Retrieve the property names defined on object
  const propNames = Reflect.ownKeys(object);

  // Freeze properties before freezing self
  for (const name of propNames) {
    const value = object[name];

    if ((value && typeof value === "object") || typeof value === "function") {
      deepFreeze(value);
    }
  }

  return Object.freeze(object);
}

const obj2 = {
  internal: {
    a: null,
  },
};

deepFreeze(obj2);

obj2.internal.a = "anotherValue"; // fails silently in non-strict mode
obj2.internal.a; // null
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-object.freeze |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `freeze` | 6 | 12 | 4 | 12 | 5.1 | 18 | 4 | 12 | 5 | 1.0 | 4.4 | 5 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Object.freeze`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Object.isFrozen()`
- `Object.preventExtensions()`
- `Object.isExtensible()`
- `Object.seal()`
- `Object.isSealed()`

---

JavaScript › Reference › Standard built-in objects › Object

# Object.fromEntries()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/fromEntries · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since January 2020.

The **`Object.fromEntries()`** static method transforms a list of key-value pairs into an object.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Object.fromEntries()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const entries = new Map([
  ["foo", "bar"],
  ["baz", 42],
]);

const obj = Object.fromEntries(entries);

console.log(obj);
// Expected output: Object { foo: "bar", baz: 42 }
```

## Syntax

js

```
Object.fromEntries(iterable)
```

### Parameters

**`iterable`**

An iterable, such as an `Array` or `Map`, containing a list of objects. Each object should have two properties:

**`0`**

A string or symbol representing the property key.

**`1`**

The property value.

Typically, this object is implemented as a two-element array, with the first element being the property key and the second element being the property value.

### Return value

A new object whose properties are given by the entries of the iterable.

## Description

The `Object.fromEntries()` method takes a list of key-value pairs and returns a new object whose properties are given by those entries. The `iterable` argument is expected to be an object that implements a `[Symbol.iterator]()` method. The method returns an iterator object that produces two-element array-like objects. The first element is a value that will be used as a property key, and the second element is the value to associate with that property key.

`Object.fromEntries()` performs the reverse of `Object.entries()`, except that `Object.entries()` only returns string-keyed properties, while `Object.fromEntries()` can also create symbol-keyed properties.

> **Note:** Unlike `Array.from()`, `Object.fromEntries()` does not use the value of `this`, so calling it on another constructor does not create objects of that type.

## Examples

### Converting a Map to an Object

With `Object.fromEntries`, you can convert from `Map` to `Object`:

```js
const map = new Map([
  ["foo", "bar"],
  ["baz", 42],
]);
const obj = Object.fromEntries(map);
console.log(obj); // { foo: "bar", baz: 42 }
```

### Converting an Array to an Object

With `Object.fromEntries`, you can convert from `Array` to `Object`:

```js
const arr = [
  ["0", "a"],
  ["1", "b"],
  ["2", "c"],
];
const obj = Object.fromEntries(arr);
console.log(obj); // { 0: "a", 1: "b", 2: "c" }
```

### Object transformations

With `Object.fromEntries`, its reverse method `Object.entries()`, and array manipulation methods, you are able to transform objects like this:

```js
const object1 = { a: 1, b: 2, c: 3 };

const object2 = Object.fromEntries(
  Object.entries(object1).map(([key, val]) => [key, val * 2]),
);

console.log(object2);
// { a: 2, b: 4, c: 6 }
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-object.fromentries` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `fromEntries` | 73 | 79 | 63 | 60 | 12.1 | 73 | 63 | 52 | 12.2 | 11.0 | 73 | 12.2 | 1.0.0 | 1.0 | 12.0.0 |

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Object.fromEntries`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Object.fromEntries` in `core-js`
- es-shims polyfill of `Object.fromEntries`
- `Object.entries()`
- `Object.keys()`
- `Object.values()`
- `Object.prototype.propertyIsEnumerable()`
- `Object.create()`
- `Map.prototype.entries()`

- `Map.prototype.keys()`
- `Map.prototype.values()`

---

JavaScript › Reference › Standard built-in objects › Object

# Object.getOwnPropertyDescriptor()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/getOwnPropertyDescriptor · MDN page last modified 2025-07-20

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Object.getOwnPropertyDescriptor()`** static method returns an object describing the configuration of a specific property on a given object (that is, one directly present on an object and not in the object's prototype chain). The object returned is mutable but mutating it has no effect on the original property's configuration.

## Try it

*▷ Interactive example on MDN: "JavaScript Demo: Object.getOwnPropertyDescriptor()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const object = {
  foo: 42,
};

const descriptor = Object.getOwnPropertyDescriptor(object, "foo");

console.log(descriptor.configurable);
// Expected output: true

console.log(descriptor.value);
// Expected output: 42
```

## Syntax

js

```
Object.getOwnPropertyDescriptor(obj, prop)
```

### Parameters

**`obj`**

The object in which to look for the property.

**`prop`**

The name or `Symbol` of the property whose description is to be retrieved.

### Return value

A property descriptor of the given property if it exists on the object, `undefined` otherwise.

## Description

This method permits examination of the precise description of a property. A *property* in JavaScript consists of either a string-valued name or a `Symbol` and a property descriptor. Further information about property descriptor types and their attributes can be found in `Object.defineProperty()`.

A *property descriptor* is a record with some of the following attributes:

**`value`**
The value associated with the property (data descriptors only).

**`writable`**
`true` if and only if the value associated with the property may be changed (data descriptors only).

**`get`**
A function which serves as a getter for the property, or `undefined` if there is no getter (accessor descriptors only).

**`set`**
A function which serves as a setter for the property, or `undefined` if there is no setter (accessor descriptors only).

**`configurable`**
`true` if and only if the type of this property descriptor may be changed and if the property may be deleted from the corresponding object.

**`enumerable`**
`true` if and only if this property shows up during enumeration of the properties on the corresponding object.

## Examples

### Using Object.getOwnPropertyDescriptor()

js

```
let o, d;

o = {
  get foo() {
    return 17;
  },
};
d = Object.getOwnPropertyDescriptor(o, "foo");
console.log(d);
// {
//   configurable: true,
//   enumerable: true,
//   get: [Function: get foo],
//   set: undefined
// }

o = { bar: 42 };
d = Object.getOwnPropertyDescriptor(o, "bar");
console.log(d);
// {
//   configurable: true,
//   enumerable: true,
//   value: 42,
//   writable: true
// }
```

*(code continues on the next page)*

(code continued from the previous page)

```
o = { [Symbol.for("baz")]: 73 };
d = Object.getOwnPropertyDescriptor(o, Symbol.for("baz"));
console.log(d);
// {
//   configurable: true,
//   enumerable: true,
//   value: 73,
//   writable: true
// }

o = {};
Object.defineProperty(o, "qux", {
  value: 8675309,
  writable: false,
  enumerable: false,
});
d = Object.getOwnPropertyDescriptor(o, "qux");
console.log(d);
// {
//   value: 8675309,
//   writable: false,
//   enumerable: false,
//   configurable: false
// }
```

### Non-object coercion

In ES5, if the first argument to this method is not an object (a primitive), then it will cause a `TypeError`. In ES2015, a non-object first argument will be coerced to an object at first.

js

```
Object.getOwnPropertyDescriptor("foo", 0);
// TypeError: "foo" is not an object  // ES5 code

Object.getOwnPropertyDescriptor("foo", 0);
// Object returned by ES2015 code: {
//   configurable: false,
//   enumerable: true,
//   value: "f",
//   writable: false
// }
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-object.getownpropertydescriptor |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `getOwnPropertyDescriptor` | 5 | 12 | 4 | 12 | 5 | 18 | 4 | 12 | 5 | 1.0 | 4.4 | 5 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Object.getOwnPropertyDescriptor`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Object.defineProperty()`
- `Reflect.getOwnPropertyDescriptor()`

---

JavaScript › Reference › Standard built-in objects › Object

# Object.getOwnPropertyDescriptors()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/getOwnPropertyDescriptors · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since April 2017.

The **`Object.getOwnPropertyDescriptors()`** static method returns all own property descriptors of a given object.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Object.getOwnPropertyDescriptors()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const object = {
  foo: 42,
};

const descriptors = Object.getOwnPropertyDescriptors(object);

console.log(descriptors.foo.writable);
// Expected output: true

console.log(descriptors.foo.value);
// Expected output: 42
```

## Syntax

js

```
Object.getOwnPropertyDescriptors(obj)
```

### Parameters

**`obj`**

The object for which to get all own property descriptors.

### Return value

An object containing all own property descriptors of an object. Might be an empty object, if there are no properties.

## Description

This method permits examination of the precise description of all own properties of an object. A *property* in JavaScript consists of either a string-valued name or a `Symbol` and a property descriptor. Further information about property descriptor types and their attributes can be found in `Object.defineProperty()`.

A *property descriptor* is a record with some of the following attributes:

**`value`**
: The value associated with the property (data descriptors only).

**`writable`**
: `true` if and only if the value associated with the property may be changed (data descriptors only).

**`get`**
: A function which serves as a getter for the property, or `undefined` if there is no getter (accessor descriptors only).

**`set`**
: A function which serves as a setter for the property, or `undefined` if there is no setter (accessor descriptors only).

**`configurable`**
: `true` if and only if the type of this property descriptor may be changed and if the property may be deleted from the corresponding object.

**`enumerable`**
: `true` if and only if this property shows up during enumeration of the properties on the corresponding object.

## Examples

### Creating a shallow copy

Whereas the `Object.assign()` method will only copy enumerable and own properties from a source object to a target object, you are able to use this method and `Object.create()` for a shallow copy between two unknown objects:

```js
Object.create(
  Object.getPrototypeOf(obj),
  Object.getOwnPropertyDescriptors(obj),
);
```

### Creating a subclass

A typical way of creating a subclass is to define the subclass, set its prototype to an instance of the superclass, and then define properties on that instance. This can get awkward especially for getters and setters. Instead, you can use this code to set the prototype:

```js
function superclass() {}
superclass.prototype = {
  // Define the superclass constructor, methods, and properties here
};
function subclass() {}
subclass.prototype = Object.create(superclass.prototype, {
  // Define the subclass constructor, methods, and properties here
});
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-object.getownpropertydescriptors |

## Browser compatibility

| | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| Feature | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| getOwnPropertyDescriptors | 54 | 15 | 50 | 41 | 10 | 54 | 50 | 41 | 10 | 6.0 | 54 | 10 | 1.0.0 | 1.0 | 7.0.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Object.getOwnPropertyDescriptors`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Object.getOwnPropertyDescriptors` in `core-js`
- es-shims polyfill of `Object.getOwnPropertyDescriptors`
- `Object.getOwnPropertyDescriptor()`
- `Object.defineProperty()`

---

JavaScript › Reference › Standard built-in objects › Object

# Object.getOwnPropertyNames()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/getOwnPropertyNames · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Object.getOwnPropertyNames()`** static method returns an array of all properties (including non-enumerable properties except for those which use Symbol) found directly in a given object.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Object.getOwnPropertyNames()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const object = {
  a: 1,
  b: 2,
  c: 3,
};

console.log(Object.getOwnPropertyNames(object));
// Expected output: Array ["a", "b", "c"]
```

## Syntax

js

```
Object.getOwnPropertyNames(obj)
```

### Parameters

**`obj`**

The object whose enumerable and non-enumerable properties are to be returned.

### Return value

An array of strings that corresponds to the properties found directly in the given object.

## Description

`Object.getOwnPropertyNames()` returns an array whose elements are strings corresponding to the enumerable and non-enumerable properties found directly in a given object `obj`. The ordering of the enumerable properties in the array is consistent with the ordering exposed by a `for...in` loop (or by `Object.keys()`) over the properties of the object. The non-negative integer keys of the object (both enumerable and non-enumerable) are added in ascending order to the array first, followed by the string keys in the order of insertion.

In ES5, if the argument to this method is not an object (a primitive), then it will cause a `TypeError`. In ES2015, a non-object argument will be coerced to an object.

js

```
Object.getOwnPropertyNames("foo");
// TypeError: "foo" is not an object (ES5 code)

Object.getOwnPropertyNames("foo");
// ["0", "1", "2", "length"]  (ES2015 code)
```

## Examples

### Using Object.getOwnPropertyNames()

```js
const arr = ["a", "b", "c"];
console.log(Object.getOwnPropertyNames(arr).sort());
// ["0", "1", "2", "length"]

// Array-like object
const obj = { 0: "a", 1: "b", 2: "c" };
console.log(Object.getOwnPropertyNames(obj).sort());
// ["0", "1", "2"]

Object.getOwnPropertyNames(obj).forEach((val, idx, array) => {
  console.log(`${val} -> ${obj[val]}`);
});
// 0 -> a
// 1 -> b
// 2 -> c

// non-enumerable property
const myObj = Object.create(
  {},
  {
    getFoo: {
      value() {
        return this.foo;
      },
      enumerable: false,
    },
  },
);
myObj.foo = 1;

console.log(Object.getOwnPropertyNames(myObj).sort()); // ["foo", "getFoo"]
```

If you want only the enumerable properties, see `Object.keys()` or use a `for...in` loop (note that this will also return enumerable properties found along the prototype chain for the object unless the latter is filtered with `Object.hasOwn()`).

Items on the prototype chain are not listed:

```js
function ParentClass() {}
ParentClass.prototype.inheritedMethod = function () {};

function ChildClass() {
  this.prop = 5;
  this.method = function () {};
}
ChildClass.prototype = new ParentClass();
ChildClass.prototype.prototypeMethod = function () {};

console.log(Object.getOwnPropertyNames(new ChildClass()));
// ["prop", "method"]
```

### Get non-enumerable properties only

This uses the `Array.prototype.filter()` function to remove the enumerable keys (obtained with `Object.keys()`) from a list of all keys (obtained with `Object.getOwnPropertyNames()`) thus giving only

the non-enumerable keys as output.

```js
const target = myObject;
const enumAndNonEnum = Object.getOwnPropertyNames(target);
const enumOnly = new Set(Object.keys(target));
const nonEnumOnly = enumAndNonEnum.filter((key) => !enumOnly.has(key));

console.log(nonEnumOnly);
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-object.getownpropertynames |

## Browser compatibility

| | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| Feature | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| getOwnPropertyNames | 5 | 12 | 4 | 12 | 5 | 18 | 4 | 12 | 5 | 1.0 | 4.4 | 5 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Object.getOwnPropertyNames`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Object.getOwnPropertyNames` in `core-js`
- Enumerability and ownership of properties
- `Object.hasOwn()`
- `Object.prototype.propertyIsEnumerable()`
- `Object.create()`
- `Object.keys()`
- `Array.prototype.forEach()`

JavaScript › Reference › Standard built-in objects › Object

# Object.getOwnPropertySymbols()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/getOwnPropertySymbols · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since September 2015.

The **`Object.getOwnPropertySymbols()`** static method returns an array of all symbol properties found directly upon a given object.

## Try it

*▷ Interactive example on MDN: "JavaScript Demo: Object.getOwnPropertySymbols()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const object = {};
const a = Symbol("a");
const b = Symbol.for("b");

object[a] = "localSymbol";
object[b] = "globalSymbol";

const objectSymbols = Object.getOwnPropertySymbols(object);

console.log(objectSymbols.length);
// Expected output: 2
```

## Syntax

js

```
Object.getOwnPropertySymbols(obj)
```

### Parameters

**`obj`**

The object whose symbol properties are to be returned.

### Return value

An array of all symbol properties found directly upon the given object.

## Description

Similar to `Object.getOwnPropertyNames()`, you can get all symbol properties of a given object as an array of symbols. Note that `Object.getOwnPropertyNames()` itself does not contain the symbol properties of an object and only the string properties.

As all objects have no own symbol properties initially, `Object.getOwnPropertySymbols()` returns an empty array unless you have set symbol properties on your object.

## Examples

### Using Object.getOwnPropertySymbols()

```js
const obj = {};
const a = Symbol("a");
const b = Symbol.for("b");

obj[a] = "localSymbol";
obj[b] = "globalSymbol";

const objectSymbols = Object.getOwnPropertySymbols(obj);

console.log(objectSymbols.length); // 2
console.log(objectSymbols); // [Symbol(a), Symbol(b)]
console.log(objectSymbols[0]); // Symbol(a)
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-object.getownpropertysymbols |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| getOwnPropertySymbols | 38 | 12 | 36 | 25 | 9 | 38 | 36 | 25 | 9 | 3.0 | 38 | 9 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Object.getOwnPropertySymbols`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Object.getOwnPropertySymbols` in `core-js`
- `Object.getOwnPropertyNames()`
- `Symbol`

---

JavaScript › Reference › Standard built-in objects › Object

# Object.getPrototypeOf()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/getPrototypeOf · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Object.getPrototypeOf()`** static method returns the prototype (i.e., the value of the internal `[[Prototype]]` property) of the specified object.

## Try it

*▷ Interactive example on MDN: "JavaScript Demo: Object.getPrototypeOf()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const prototype = {};
const object = Object.create(prototype);

console.log(Object.getPrototypeOf(object) === prototype);
// Expected output: true
```

## Syntax

js

```
Object.getPrototypeOf(obj)
```

### Parameters

**`obj`**

The object whose prototype is to be returned.

### Return value

The prototype of the given object, which may be `null`.

## Examples

### Using getPrototypeOf

js

```
const proto = {};
const obj = Object.create(proto);
Object.getPrototypeOf(obj) === proto; // true
```

### Non-object coercion

In ES5, it will throw a `TypeError` exception if the `obj` parameter isn't an object. In ES2015, the parameter will be coerced to an `Object`.

js

```
Object.getPrototypeOf("foo");
// TypeError: "foo" is not an object (ES5 code)
Object.getPrototypeOf("foo");
// String.prototype                 (ES2015 code)
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-object.getprototypeof |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `getPrototypeOf` | 5 | 12 | 3.5 | 12.1 | 5 | 18 | 4 | 12.1 | 5 | 1.0 | 4.4 | 5 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Object.getPrototypeOf`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Object.getPrototypeOf` in `core-js`
- es-shims polyfill of `Object.getPrototypeOf`
- `Object.prototype.isPrototypeOf()`
- `Object.setPrototypeOf()`
- `Object.prototype.__proto__`
- `Reflect.getPrototypeOf()`
- Object.getPrototypeOf by John Resig (2008)

---

JavaScript › Reference › Standard built-in objects › Object

# Object.groupBy()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/groupBy · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since March 2024.

> **Note:** In some versions of some browsers, this method was implemented as the method `Array.prototype.group()`. Due to web compatibility issues, it is now implemented as a static method. Check the browser compatibility table for details.

The **`Object.groupBy()`** static method groups the elements of a given iterable according to the string values returned by a provided callback function. The returned object has separate properties for each group, containing arrays with the elements in the group.

This method should be used when group names can be represented by strings. If you need to group elements using a key that is some arbitrary value, use `Map.groupBy()` instead.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Object.groupBy()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const inventory = [
  { name: "asparagus", type: "vegetables", quantity: 9 },
  { name: "bananas", type: "fruit", quantity: 5 },
  { name: "goat", type: "meat", quantity: 23 },
  { name: "cherries", type: "fruit", quantity: 12 },
  { name: "fish", type: "meat", quantity: 22 },
];

const result = Object.groupBy(inventory, ({ quantity }) =>
  quantity < 6 ? "restock" : "sufficient",
);
console.log(result.restock);
// [{ name: "bananas", type: "fruit", quantity: 5 }]
```

## Syntax

js

```
Object.groupBy(items, callbackFn)
```

### Parameters

**`items`**

An iterable (such as an `Array`) whose elements will be grouped.

**`callbackFn`**

A function to execute for each element in the iterable. It should return a value that can get coerced into a property key (string or symbol) indicating the group of the current element. The function is called with the following arguments:

**`element`**

The current element being processed.

**`index`**

The index of the current element being processed.

### Return value

A null-prototype object with properties for all groups, each assigned to an array containing the elements of the associated group.

## Description

`Object.groupBy()` calls a provided `callbackFn` function once for each element in an iterable. The callback function should return a string or symbol (values that are neither type are coerced to strings) indicating the group of the associated element. The values returned by `callbackFn` are used as keys for the object returned by `Object.groupBy()`. Each key has an associated array containing all the elements for which the callback returned the same value.

The elements in the returned object and the original iterable are the same (not deep copies). Changing the internal structure of the elements will be reflected in both the original iterable and the returned object.

## Examples

### Using Object.groupBy()

First we define an array containing objects representing an inventory of different foodstuffs. Each food has a `type` and a `quantity`.

```js
const inventory = [
  { name: "asparagus", type: "vegetables", quantity: 5 },
  { name: "bananas", type: "fruit", quantity: 0 },
  { name: "goat", type: "meat", quantity: 23 },
  { name: "cherries", type: "fruit", quantity: 5 },
  { name: "fish", type: "meat", quantity: 22 },
];
```

The code below groups the elements by the value of their `type` property.

```js
const result = Object.groupBy(inventory, ({ type }) => type);

/* Result is:
{
  vegetables: [
    { name: 'asparagus', type: 'vegetables', quantity: 5 },
  ],
  fruit: [
    { name: "bananas", type: "fruit", quantity: 0 },
    { name: "cherries", type: "fruit", quantity: 5 }
  ],
  meat: [
    { name: "goat", type: "meat", quantity: 23 },
    { name: "fish", type: "meat", quantity: 22 }
  ]
}
*/
```

The arrow function just returns the `type` of each array element each time it is called. Note that the function argument `{ type }` is a basic example of object destructuring syntax for function arguments. This unpacks the `type` property of an object passed as a parameter, and assigns it to a variable named `type` in the body of the function. This is a very succinct way to access the relevant values of elements within a function.

We can also create groups inferred from values in one or more properties of the elements. Below is a very similar example that puts the items into `ok` or `restock` groups based on the value of the `quantity` field.

```js
function myCallback({ quantity }) {
  return quantity > 5 ? "ok" : "restock";
}

const result2 = Object.groupBy(inventory, myCallback);

/* Result is:
{
  restock: [
    { name: "asparagus", type: "vegetables", quantity: 5 },
```

(code continues on the next page)

*(code continued from the previous page)*

```
    { name: "bananas", type: "fruit", quantity: 0 },
    { name: "cherries", type: "fruit", quantity: 5 }
  ],
  ok: [
    { name: "goat", type: "meat", quantity: 23 },
    { name: "fish", type: "meat", quantity: 22 }
  ]
}
*/
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-object.groupby |

## Browser compatibility

| | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| Feature | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| groupBy | 117 | 117 | 119 | 103 | 17.4[1] | 117 | 119 | 78 | 17.4[1] | 24.0 | 117 | 17.4[1] | 1.0.19 | 1.37 | 21.0.0 |

1. Also from version 16.4–17.4: uses the non-standard name `Array.prototype.groupToMap`.

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Object.groupBy`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Object.groupBy` in `core-js`
- es-shims polyfill of `Object.groupBy`
- Indexed collections guide
- `Array.prototype.reduce()`
- `Object.fromEntries()`
- `Map.groupBy()`

---

JavaScript › Reference › Standard built-in objects › Object

# Object.hasOwn()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/hasOwn · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since March 2022.

The **`Object.hasOwn()`** static method returns `true` if the specified object has the indicated property as its *own* property. If the property is inherited, or does not exist, the method returns `false`.

> **Note:** `Object.hasOwn()` is intended as a replacement for `Object.prototype.hasOwnProperty()`.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Object.hasOwn()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const object = {
  prop: "exists",
};

console.log(Object.hasOwn(object, "prop"));
// Expected output: true

console.log(Object.hasOwn(object, "toString"));
// Expected output: false

console.log(Object.hasOwn(object, "undeclaredPropertyValue"));
// Expected output: false
```

## Syntax

js

```
Object.hasOwn(obj, prop)
```

### Parameters

**`obj`**

The JavaScript object instance to test.

**`prop`**

The `String` name or Symbol of the property to test.

### Return value

`true` if the specified object has directly defined the specified property. Otherwise `false`

## Description

The `Object.hasOwn()` method returns `true` if the specified property is a direct property of the object — even if the property value is `null` or `undefined`. The method returns `false` if the property is inherited, or has not been declared at all. Unlike the `in` operator, this method does not check for the specified property in the object's prototype chain.

It is recommended over `Object.prototype.hasOwnProperty()` because it works for `null`-prototype objects and with objects that have overridden the inherited `hasOwnProperty()` method. While it is possible to work around these problems by accessing `Object.prototype.hasOwnProperty()` on another object (like `Object.prototype.hasOwnProperty.call(obj, prop)`), `Object.hasOwn()` is more intuitive and concise.

## Examples

### Using Object.hasOwn() to test for a property's existence

The following code shows how to determine whether the `example` object contains a property named `prop`.

```js
const example = {};
Object.hasOwn(example, "prop"); // false - 'prop' has not been defined

example.prop = "exists";
Object.hasOwn(example, "prop"); // true - 'prop' has been defined

example.prop = null;
Object.hasOwn(example, "prop"); // true - own property exists with value of null

example.prop = undefined;
Object.hasOwn(example, "prop"); // true - own property exists with value of undefined
```

### Direct vs. inherited properties

The following example differentiates between direct properties and properties inherited through the prototype chain:

```js
const example = {};
example.prop = "exists";

// `hasOwn` will only return true for direct properties:
Object.hasOwn(example, "prop"); // true
Object.hasOwn(example, "toString"); // false
Object.hasOwn(example, "hasOwnProperty"); // false

// The `in` operator will return true for direct or inherited properties:
"prop" in example; // true
"toString" in example; // true
"hasOwnProperty" in example; // true
```

### Iterating over the properties of an object

To iterate over the enumerable properties of an object, you *should* use:

```js
const example = { foo: true, bar: true };
for (const name of Object.keys(example)) {
  // …
}
```

But if you need to use `for...in`, you can use `Object.hasOwn()` to skip the inherited properties:

```js
const example = { foo: true, bar: true };
for (const name in example) {
  if (Object.hasOwn(example, name)) {
    // …
  }
}
```

### Checking if an Array index exists

The elements of an `Array` are defined as direct properties, so you can use `hasOwn()` method to check whether a particular index exists:

```js
const fruits = ["Apple", "Banana", "Watermelon", "Orange"];
Object.hasOwn(fruits, 3); // true ('Orange')
Object.hasOwn(fruits, 4); // false - not defined
```

### Problematic cases for hasOwnProperty()

This section demonstrates that `Object.hasOwn()` is immune to the problems that affect `hasOwnProperty()`. Firstly, it can be used with objects that have re-implemented `hasOwnProperty()`. In the example below, the re-implemented `hasOwnProperty()` method reports false for *every* property, but the behavior of `Object.hasOwn()` remains unaffected:

```js
const foo = {
  hasOwnProperty() {
    return false;
  },
  bar: "The dragons be out of office",
};

console.log(foo.hasOwnProperty("bar")); // false

console.log(Object.hasOwn(foo, "bar")); // true
```

It can also be used with `null`-prototype objects. These do not inherit from `Object.prototype`, and so `hasOwnProperty()` is inaccessible.

```js
const foo = Object.create(null);
foo.prop = "exists";

console.log(foo.hasOwnProperty("prop"));
// Uncaught TypeError: foo.hasOwnProperty is not a function

console.log(Object.hasOwn(foo, "prop")); // true
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-object.hasown |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| hasOwn | 93 | 93 | 92 | 79 | 15.4 | 93 | 92 | 66 | 15.4 | 17.0 | 93 | 15.4 | 1.0.0 | 1.13 | 16.9.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Object.hasOwn`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Object.hasOwn` in `core-js`
- es-shims polyfill of `Object.hasOwn`
- `Object.prototype.hasOwnProperty()`
- Enumerability and ownership of properties
- `Object.getOwnPropertyNames()`
- `for...in`
- `in`
- Inheritance and the prototype chain

---

JavaScript › Reference › Standard built-in objects › Object

# Object.is()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/is · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since September 2015.

The **`Object.is()`** static method determines whether two values are the same value.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Object.is()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(Object.is("1", 1));
// Expected output: false

console.log(Object.is(NaN, NaN));
// Expected output: true

console.log(Object.is(-0, 0));
// Expected output: false

const obj = {};
console.log(Object.is(obj, {}));
// Expected output: false
```

## Syntax

js

```
Object.is(value1, value2)
```

### Parameters

**`value1`**

The first value to compare.

**`value2`**

The second value to compare.

### Return value

A boolean indicating whether or not the two arguments are the same value.

## Description

`Object.is()` determines whether two values are the same value. Two values are the same if one of the following holds:

- both `undefined`
- both `null`
- both `true` or both `false`
- both strings of the same length with the same characters in the same order
- both the same object (meaning both values reference the same object in memory)
- both BigInts with the same numeric value
- both symbols that reference the same symbol value
- both numbers and
  - both `+0`
  - both `-0`
  - both `NaN`
  - or both non-zero, not `NaN`, and have the same value

`Object.is()` is not equivalent to the `==` operator. The `==` operator applies various coercions to both sides (if they are not the same type) before testing for equality (resulting in such behavior as `"" == false` being `true`), but `Object.is()` doesn't coerce either value.

`Object.is()` is also *not* equivalent to the `===` operator. The only difference between `Object.is()` and `===` is in their treatment of signed zeros and `NaN` values. The `===` operator (and the `==` operator) treats the number values `-0` and `+0` as equal, but treats `NaN` as not equal to each other.

## Examples

### Using Object.is()

js

```
// Case 1: Evaluation result is the same as using ===
Object.is(25, 25); // true
Object.is("foo", "foo"); // true
Object.is("foo", "bar"); // false
Object.is(null, null); // true
Object.is(undefined, undefined); // true
Object.is(window, window); // true
Object.is([], []); // false
const foo = { a: 1 };
const bar = { a: 1 };
const sameFoo = foo;
Object.is(foo, foo); // true
Object.is(foo, bar); // false
Object.is(foo, sameFoo); // true
```

(code continues on the next page)

*(code continued from the previous page)*

```
// Case 2: Signed zero
Object.is(0, -0); // false
Object.is(+0, -0); // false
Object.is(-0, -0); // true

// Case 3: NaN
Object.is(NaN, 0 / 0); // true
Object.is(NaN, Number.NaN); // true
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-object.is` |

## Browser compatibility

| | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| Feature | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `is` | 19 | 12 | 22 | 15 | 9 | 25 | 22 | 14 | 9 | 1.5 | 4.4 | 9 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Object.is`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Object.is` in `core-js`
- es-shims polyfill of `Object.is`
- Equality comparisons and sameness

---

JavaScript › Reference › Standard built-in objects › Object

# Object.isExtensible()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/isExtensible · MDN page last modified 2025-07-20

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Object.isExtensible()`** static method determines if an object is extensible (whether it can have new properties added to it).

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Object.isExtensible()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const object = {};

console.log(Object.isExtensible(object));
// Expected output: true

Object.preventExtensions(object);

console.log(Object.isExtensible(object));
// Expected output: false
```

## Syntax

js

```
Object.isExtensible(obj)
```

### Parameters

**`obj`**

The object which should be checked.

### Return value

A `Boolean` indicating whether or not the given object is extensible.

## Description

Objects are extensible by default: they can have new properties added to them, and their `[[Prototype]]` can be re-assigned. An object can be marked as non-extensible using one of `Object.preventExtensions()`, `Object.seal()`, `Object.freeze()`, or `Reflect.preventExtensions()`.

## Examples

### Using Object.isExtensible

js

```
// New objects are extensible.
const empty = {};
Object.isExtensible(empty); // true

// They can be made un-extensible
Object.preventExtensions(empty);
Object.isExtensible(empty); // false

// Sealed objects are by definition non-extensible.
const sealed = Object.seal({});
Object.isExtensible(sealed); // false

// Frozen objects are also by definition non-extensible.
const frozen = Object.freeze({});
Object.isExtensible(frozen); // false
```

### Non-object argument

In ES5, if the argument to this method is not an object (a primitive), then it will cause a `TypeError`. In ES2015, it will return `false` without any errors if a non-object argument is passed, since primitives are, by definition, immutable.

```js
Object.isExtensible(1);
// TypeError: 1 is not an object (ES5 code)

Object.isExtensible(1);
// false                          (ES2015 code)
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-object.isextensible |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `isExtensible` | 6 | 12 | 4 | 12 | 5.1 | 18 | 4 | 12 | 5 | 1.0 | 4.4 | 5 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Object.isExtensible`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Object.preventExtensions()`
- `Object.seal()`
- `Object.isSealed()`
- `Object.freeze()`
- `Object.isFrozen()`
- `Reflect.isExtensible()`

---

JavaScript › Reference › Standard built-in objects › Object

# Object.isFrozen()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/isFrozen · MDN page last modified 2025-07-20

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Object.isFrozen()`** static method determines if an object is frozen.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Object.isFrozen()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const object = {
  foo: 42,
};

console.log(Object.isFrozen(object));
// Expected output: false

Object.freeze(object);

console.log(Object.isFrozen(object));
// Expected output: true
```

## Syntax

js

```
Object.isFrozen(obj)
```

### Parameters

**`obj`**

The object which should be checked.

### Return value

A `Boolean` indicating whether or not the given object is frozen.

## Description

An object is frozen if and only if it is not extensible, all its properties are non-configurable, and all its data properties (that is, properties which are not accessor properties with getter or setter components) are non-writable.

## Examples

### Using Object.isFrozen

js

```
// A new object is extensible, so it is not frozen.
Object.isFrozen({}); // false

// An empty object which is not extensible
// is vacuously frozen.
const vacuouslyFrozen = Object.preventExtensions({});
Object.isFrozen(vacuouslyFrozen); // true

// A new object with one property is also extensible,
// ergo not frozen.
const oneProp = { p: 42 };
Object.isFrozen(oneProp); // false
```

*(code continues on the next page)*

*(code continued from the previous page)*

```
// Preventing extensions to the object still doesn't
// make it frozen, because the property is still
// configurable (and writable).
Object.preventExtensions(oneProp);
Object.isFrozen(oneProp); // false

// Deleting that property makes the object vacuously frozen.
delete oneProp.p;
Object.isFrozen(oneProp); // true

// A non-extensible object with a non-writable
// but still configurable property is not frozen.
const nonWritable = { e: "plep" };
Object.preventExtensions(nonWritable);
Object.defineProperty(nonWritable, "e", {
  writable: false,
}); // make non-writable
Object.isFrozen(nonWritable); // false

// Changing that property to non-configurable
// then makes the object frozen.
Object.defineProperty(nonWritable, "e", {
  configurable: false,
}); // make non-configurable
Object.isFrozen(nonWritable); // true

// A non-extensible object with a non-configurable
// but still writable property also isn't frozen.
const nonConfigurable = { release: "the kraken!" };
Object.preventExtensions(nonConfigurable);
Object.defineProperty(nonConfigurable, "release", {
  configurable: false,
});
Object.isFrozen(nonConfigurable); // false

// Changing that property to non-writable
// then makes the object frozen.
Object.defineProperty(nonConfigurable, "release", {
  writable: false,
});
Object.isFrozen(nonConfigurable); // true

// A non-extensible object with a configurable
// accessor property isn't frozen.
const accessor = {
  get food() {
    return "yum";
  },
};
Object.preventExtensions(accessor);
Object.isFrozen(accessor); // false

// When we make that property non-configurable it becomes frozen.
Object.defineProperty(accessor, "food", {
  configurable: false,
});
Object.isFrozen(accessor); // true
```

*(code continues on the next page)*

(code continued from the previous page)

```
// But the easiest way for an object to be frozen
// is if Object.freeze has been called on it.
const frozen = { 1: 81 };
Object.isFrozen(frozen); // false
Object.freeze(frozen);
Object.isFrozen(frozen); // true

// By definition, a frozen object is non-extensible.
Object.isExtensible(frozen); // false

// Also by definition, a frozen object is sealed.
Object.isSealed(frozen); // true
```

### Non-object argument

In ES5, if the argument to this method is not an object (a primitive), then it will cause a `TypeError`. In ES2015, it will return `true` without any errors if a non-object argument is passed, since primitives are, by definition, immutable.

js

```
Object.isFrozen(1);
// TypeError: 1 is not an object (ES5 code)

Object.isFrozen(1);
// true                          (ES2015 code)
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-object.isfrozen` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `isFrozen` | 6 | 12 | 4 | 12 | 5.1 | 18 | 4 | 12 | 5 | 1.0 | 4.4 | 5 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Object.isFrozen`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Object.freeze()`
- `Object.preventExtensions()`
- `Object.isExtensible()`
- `Object.seal()`
- `Object.isSealed()`

JavaScript › Reference › Standard built-in objects › Object

# Object.isSealed()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/isSealed · MDN page last modified 2025-07-20

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Object.isSealed()`** static method determines if an object is sealed.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Object.isSealed()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const object = {
  foo: 42,
};

console.log(Object.isSealed(object));
// Expected output: false

Object.seal(object);

console.log(Object.isSealed(object));
// Expected output: true
```

## Syntax

js

```
Object.isSealed(obj)
```

### Parameters

**`obj`**

The object which should be checked.

### Return value

A `Boolean` indicating whether or not the given object is sealed.

## Description

Returns `true` if the object is sealed, otherwise `false`. An object is sealed if it is not extensible and if all its properties are non-configurable and therefore not removable (but not necessarily non-writable).

## Examples

### Using Object.isSealed

js

```
// Objects aren't sealed by default.
const empty = {};
Object.isSealed(empty); // false
```

*(code continues on the next page)*

(code continued from the previous page)

```
// If you make an empty object non-extensible,
// it is vacuously sealed.
Object.preventExtensions(empty);
Object.isSealed(empty); // true

// The same is not true of a non-empty object,
// unless its properties are all non-configurable.
const hasProp = { fee: "fie foe fum" };
Object.preventExtensions(hasProp);
Object.isSealed(hasProp); // false

// But make them all non-configurable
// and the object becomes sealed.
Object.defineProperty(hasProp, "fee", {
  configurable: false,
});
Object.isSealed(hasProp); // true

// The easiest way to seal an object, of course,
// is Object.seal.
const sealed = {};
Object.seal(sealed);
Object.isSealed(sealed); // true

// A sealed object is, by definition, non-extensible.
Object.isExtensible(sealed); // false

// A sealed object might be frozen,
// but it doesn't have to be.
Object.isFrozen(sealed); // true
// (all properties also non-writable)

const s2 = Object.seal({ p: 3 });
Object.isFrozen(s2); // false
// ('p' is still writable)

const s3 = Object.seal({
  get p() {
    return 0;
  },
});
Object.isFrozen(s3); // true
// (only configurability matters for accessor properties)
```

### Non-object argument

In ES5, if the argument to this method is not an object (a primitive), then it will cause a `TypeError`. In ES2015, it will return `true` without any errors if a non-object argument is passed, since primitives are, by definition, immutable.

js

```
Object.isSealed(1);
// TypeError: 1 is not an object (ES5 code)

Object.isSealed(1);
// true                         (ES2015 code)
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-object.issealed` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `isSealed` | 6 | 12 | 4 | 12 | 5.1 | 18 | 4 | 12 | 5 | 1.0 | 4.4 | 5 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Object.isSealed`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Object.seal()`
- `Object.preventExtensions()`
- `Object.isExtensible()`
- `Object.freeze()`
- `Object.isFrozen()`

---

JavaScript › Reference › Standard built-in objects › Object

# Object.keys()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/keys · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Object.keys()`** static method returns an array of a given object's own enumerable string-keyed property names.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Object.keys()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const object = {
  a: "some string",
  b: 42,
  c: false,
};

console.log(Object.keys(object));
// Expected output: Array ["a", "b", "c"]
```

## Syntax

```js
Object.keys(obj)
```

### Parameters

**`obj`**

An object.

### Return value

An array of strings representing the given object's own enumerable string-keyed property keys.

## Description

`Object.keys()` returns an array whose elements are strings corresponding to the enumerable string-keyed property names found directly upon `object`. This is the same as iterating with a `for...in` loop, except that a `for...in` loop enumerates properties in the prototype chain as well. The order of the array returned by `Object.keys()` is the same as that provided by a `for...in` loop.

If you need the property values, use `Object.values()` instead. If you need both the property keys and values, use `Object.entries()` instead.

## Examples

### Using Object.keys()

```js
// Basic array
const arr = ["a", "b", "c"];
console.log(Object.keys(arr)); // ['0', '1', '2']

// Array-like object
const obj = { 0: "a", 1: "b", 2: "c" };
console.log(Object.keys(obj)); // ['0', '1', '2']

// Array-like object with random key ordering
const anObj = { 100: "a", 2: "b", 7: "c" };
console.log(Object.keys(anObj)); // ['2', '7', '100']

// getFoo is a non-enumerable property
const myObj = Object.create(
  {},
  {
    getFoo: {
      value() {
        return this.foo;
      },
    },
  },
);
myObj.foo = 1;
console.log(Object.keys(myObj)); // ['foo']
```

If you want *all* string-keyed own properties, including non-enumerable ones, see `Object.getOwnPropertyNames()`.

### Using Object.keys() on primitives

Non-object arguments are coerced to objects. `undefined` and `null` cannot be coerced to objects and throw a `TypeError` upfront. Only strings may have own enumerable properties, while all other primitives return an empty array.

```js
// Strings have indices as enumerable own properties
console.log(Object.keys("foo")); // ['0', '1', '2']

// Other primitives except undefined and null have no own properties
console.log(Object.keys(100)); // []
```

> **Note:** In ES5, passing a non-object to `Object.keys()` threw a `TypeError`.

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-object.keys` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| keys | 5 | 12 | 4 | 12 | 5 | 18 | 4 | 12 | 5 | 1.0 | 4.4 | 5 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Object.keys`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Object.keys` in `core-js`
- es-shims polyfill of `Object.keys`
- Enumerability and ownership of properties
- `Object.entries()`
- `Object.values()`
- `Object.prototype.propertyIsEnumerable()`
- `Object.create()`
- `Object.getOwnPropertyNames()`
- `Map.prototype.keys()`

JavaScript › Reference › Standard built-in objects › Object

# Object.preventExtensions()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/preventExtensions · MDN page last modified 2025-07-20

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Object.preventExtensions()`** static method prevents new properties from ever being added to an object (i.e., prevents future extensions to the object). It also prevents the object's prototype from being re-assigned.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Object.preventExtensions()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const object = {};

Object.preventExtensions(object);

try {
  Object.defineProperty(object, "foo", {
    value: 42,
  });
} catch (e) {
  console.log(e);
  // Expected output: TypeError: Cannot define property foo, object is not extensible
}
```

## Syntax

js

```
Object.preventExtensions(obj)
```

### Parameters

**`obj`**

The object which should be made non-extensible.

### Return value

The object being made non-extensible.

## Description

An object is extensible if new properties can be added to it. `Object.preventExtensions()` marks an object as no longer extensible, so that it will never have properties beyond the ones it had at the time it was marked as non-extensible. Note that the properties of a non-extensible object, in general, may still be *deleted*. Attempting to add new properties to a non-extensible object will fail, either silently or, in strict mode, throwing a `TypeError`.

Unlike `Object.seal()` and `Object.freeze()`, `Object.preventExtensions()` invokes an intrinsic JavaScript behavior and cannot be replaced with a composition of several other operations. It also has

its `Reflect` counterpart (which only exists for intrinsic operations), `Reflect.preventExtensions()`.

`Object.preventExtensions()` only prevents addition of own properties. Properties can still be added to the object prototype.

This method makes the `[[Prototype]]` of the target immutable; any `[[Prototype]]` re-assignment will throw a `TypeError`. This behavior is specific to the internal `[[Prototype]]` property; other properties of the target object will remain mutable.

There is no way to make an object extensible again once it has been made non-extensible.

## Examples

### Using Object.preventExtensions

js

```
// Object.preventExtensions returns the object
// being made non-extensible.
const obj = {};
const obj2 = Object.preventExtensions(obj);
obj === obj2; // true

// Objects are extensible by default.
const empty = {};
Object.isExtensible(empty); // true

// They can be made un-extensible
Object.preventExtensions(empty);
Object.isExtensible(empty); // false

// Object.defineProperty throws when adding
// a new property to a non-extensible object.
const nonExtensible = { removable: true };
Object.preventExtensions(nonExtensible);
Object.defineProperty(nonExtensible, "new", {
  value: 8675309,
}); // throws a TypeError

// In strict mode, attempting to add new properties
// to a non-extensible object throws a TypeError.
function fail() {
  "use strict";
  // throws a TypeError
  nonExtensible.newProperty = "FAIL";
}
fail();
```

A non-extensible object's prototype is immutable:

js

```
const fixed = Object.preventExtensions({});
// throws a 'TypeError'.
fixed.__proto__ = { oh: "hai" };
```

### Non-object argument

In ES5, if the argument to this method is not an object (a primitive), then it will cause a `TypeError`. In ES2015, a non-object argument will be returned as-is without any errors, since primitives are already,

by definition, immutable.

```js
Object.preventExtensions(1);
// TypeError: 1 is not an object (ES5 code)

Object.preventExtensions(1);
// 1                            (ES2015 code)
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-object.preventextensions |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| preventExtensions | 6 | 12 | 4 | 12 | 5.1 | 18 | 4 | 12 | 5 | 1.0 | 4.4 | 5 | 1.0.0 | 1.0 | 0.10.0 |
| ES2015 behavior for non-object argument | 44 | 12 | 35 | 31 | 9 | 44 | 35 | 32 | 9 | 4.0 | 44 | 9 | 1.0.0 | 1.0 | 4.0.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Object.preventExtensions`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Object.isExtensible()`
- `Object.seal()`
- `Object.isSealed()`
- `Object.freeze()`
- `Object.isFrozen()`
- `Reflect.preventExtensions()`

---

JavaScript › Reference › Standard built-in objects › Object

# Object.seal()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/seal · MDN page last modified 2025-07-20

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Object.seal()`** static method *seals* an object. Sealing an object prevents extensions and makes existing properties non-configurable. A sealed object has a fixed set of properties: new properties cannot be added, existing properties cannot be removed, their enumerability and configurability cannot be changed, and its prototype cannot be re-assigned. Values of existing properties can still be changed as long as they are writable. `seal()` returns the same object that was passed in.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Object.seal()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const object = {
  foo: 42,
};

Object.seal(object);
object.foo = 33;
console.log(object.foo);
// Expected output: 33

delete object.foo; // Cannot delete when sealed
console.log(object.foo);
// Expected output: 33
```

## Syntax

js

```
Object.seal(obj)
```

### Parameters

**`obj`**

The object which should be sealed.

### Return value

The object being sealed.

## Description

Sealing an object is equivalent to preventing extensions and then changing all existing properties' descriptors to `configurable: false`. This has the effect of making the set of properties on the object fixed. Making all properties non-configurable also prevents them from being converted from data properties to accessor properties and vice versa, but it does not prevent the values of data properties from being changed. Attempting to delete or add properties to a sealed object, or to convert a data property to accessor or vice versa, will fail, either silently or by throwing a `TypeError` (most commonly, although not exclusively, when in strict mode code).

Private elements are not properties and do not have the concept of property descriptors. Private elements cannot be added or removed from the object, whether the object is sealed or not.

The prototype chain remains untouched. However, due to the effect of preventing extensions, the `[[Prototype]]` cannot be reassigned.

Unlike `Object.freeze()`, objects sealed with `Object.seal()` may have their existing properties changed, as long as they are writable.

## Examples

### Using Object.seal

```js
const obj = {
  prop() {},
  foo: "bar",
};

// New properties may be added, existing properties
// may be changed or removed.
obj.foo = "baz";
obj.lumpy = "woof";
delete obj.prop;

const o = Object.seal(obj);

o === obj; // true
Object.isSealed(obj); // true

// Changing property values on a sealed object
// still works.
obj.foo = "quux";

// But you can't convert data properties to accessors,
// or vice versa.
Object.defineProperty(obj, "foo", {
  get() {
    return "g";
  },
}); // throws a TypeError

// Now any changes, other than to property values,
// will fail.
obj.quaxxor = "the friendly duck";
// silently doesn't add the property
delete obj.foo;
// silently doesn't delete the property

// … and in strict mode such attempts
// will throw TypeErrors.
function fail() {
  "use strict";
  delete obj.foo; // throws a TypeError
  obj.sparky = "arf"; // throws a TypeError
}
fail();

// Attempted additions through
// Object.defineProperty will also throw.
Object.defineProperty(obj, "ohai", {
  value: 17,
}); // throws a TypeError
Object.defineProperty(obj, "foo", {
  value: "eit",
}); // changes existing property value
```

### Non-object argument

In ES5, if the argument to this method is not an object (a primitive), then it will cause a `TypeError`. In ES2015, a non-object argument will be returned as-is without any errors, since primitives are already, by definition, immutable.

```js
Object.seal(1);
// TypeError: 1 is not an object (ES5 code)

Object.seal(1);
// 1                              (ES2015 code)
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-object.seal` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `seal` | 6 | 12 | 4 | 12 | 5.1 | 18 | 4 | 12 | 5 | 1.0 | 4.4 | 5 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Object.seal`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Object.isSealed()`
- `Object.preventExtensions()`
- `Object.isExtensible()`
- `Object.freeze()`
- `Object.isFrozen()`

---

JavaScript › Reference › Standard built-in objects › Object

# Object.setPrototypeOf()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/setPrototypeOf · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since September 2015.

The **`Object.setPrototypeOf()`** static method sets the prototype (i.e., the internal `[[Prototype]]` property) of a specified object to another object or `null`.

> **Warning:** Changing the `[[Prototype]]` of an object is, by the nature of how modern JavaScript engines optimize property accesses, currently a very slow operation in every browser and JavaScript engine. In addition, the effects of altering inheritance are subtle and far-flung, and are not limited to the time spent in the `Object.setPrototypeOf(...)` statement, but may extend to ***any*** code that has access to any object whose `[[Prototype]]` has been altered. You can read more in JavaScript engine fundamentals: optimizing prototypes.
>
> Because this feature is a part of the language, it is still the burden on engine developers to implement that feature performantly (ideally). Until engine developers address this issue, if you are concerned about performance, you should avoid setting the `[[Prototype]]` of an object. Instead, create a new object with the desired `[[Prototype]]` using `Object.create()`.

## Try it

*▷ Interactive example on MDN: "JavaScript Demo: Object.setPrototypeOf()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const obj = {};
const parent = { foo: "bar" };

console.log(obj.foo);
// Expected output: undefined

Object.setPrototypeOf(obj, parent);

console.log(obj.foo);
// Expected output: "bar"
```

## Syntax

js

```
Object.setPrototypeOf(obj, prototype)
```

### Parameters

**`obj`**

The object which is to have its prototype set.

**`prototype`**

The object's new prototype (an object or `null`).

### Return value

The specified object.

### Exceptions

**`TypeError`**

Thrown in one of the following cases:

- The `obj` parameter is `undefined` or `null`.
- The `obj` parameter is non-extensible, or it's an immutable prototype exotic object, such as `Object.prototype` or `window`. However, the error is not thrown if the new prototype is the same value as the original prototype of `obj`.

- The `prototype` parameter is not an object or `null`.

## Description

`Object.setPrototypeOf()` is generally considered the proper way to set the prototype of an object. You should always use it in favor of the deprecated `Object.prototype.__proto__` accessor.

If the `obj` parameter is not an object (e.g., number, string, etc.), this method does nothing — without coercing it to an object or attempting to set its prototype — and directly returns `obj` as a primitive value. If `prototype` is the same value as the prototype of `obj`, then `obj` is directly returned, without causing a `TypeError` even when `obj` has immutable prototype.

For security concerns, there are certain built-in objects that are designed to have an *immutable prototype*. This prevents prototype pollution attacks, especially proxy-related ones. The core language only specifies `Object.prototype` as an immutable prototype exotic object, whose prototype is always `null`. In browsers, `window` and `location` are two other very common examples.

```js
Object.isExtensible(Object.prototype); // true; you can add more properties
Object.setPrototypeOf(Object.prototype, {}); // TypeError: Immutable prototype object '#
<Object>' cannot have their prototype set
Object.setPrototypeOf(Object.prototype, null); // No error; the prototype of
`Object.prototype` is already `null`
```

## Examples

### Pseudoclassical inheritance using Object.setPrototypeOf()

Inheritance in JS using classes.

```js
class Human {}
class SuperHero extends Human {}

const superMan = new SuperHero();
```

However, if we want to implement subclasses without using `class`, we can do the following:

```js
function Human(name, level) {
  this.name = name;
  this.level = level;
}

function SuperHero(name, level) {
  Human.call(this, name, level);
}

Object.setPrototypeOf(SuperHero.prototype, Human.prototype);
```

*(code continues on the next page)*

(code continued from the previous page)

```
// Set the `[[Prototype]]` of `SuperHero.prototype`
// to `Human.prototype`
// To set the prototypal inheritance chain

Human.prototype.speak = function () {
  return `${this.name} says hello.`;
};

SuperHero.prototype.fly = function () {
  return `${this.name} is flying.`;
};

const superMan = new SuperHero("Clark Kent", 1);

console.log(superMan.fly());
console.log(superMan.speak());
```

The similarity between classical inheritance (with classes) and pseudoclassical inheritance (with constructors' `prototype` property) as done above is mentioned in Inheritance chains.

Since function constructors' `prototype` property is writable, you can reassign it to a new object created with `Object.create()` to achieve the same inheritance chain as well. There are caveats to watch out when using `create()`, such as remembering to re-add the `constructor` property.

In the example below, which also uses classes, `SuperHero` is made to inherit from `Human` without using `extends` by using `setPrototypeOf()` instead.

> **Warning:** It is not advisable to use `setPrototypeOf()` instead of `extends` due to performance and readability reasons.

js

```
class Human {}
class SuperHero {}

// Set the instance properties
Object.setPrototypeOf(SuperHero.prototype, Human.prototype);

// Hook up the static properties
Object.setPrototypeOf(SuperHero, Human);

const superMan = new SuperHero();
```

Subclassing without `extends` is mentioned in ES-6 subclassing.

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-object.setprototypeof |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `setPrototypeOf` | 34 | 12 | 31 | 21 | 9 | 34 | 31 | 21 | 9 | 2.0 | 37 | 9 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Object.setPrototypeOf`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Object.setPrototypeOf` in `core-js`
- `Reflect.setPrototypeOf()`
- `Object.prototype.isPrototypeOf()`
- `Object.getPrototypeOf()`
- `Object.prototype.__proto__`
- Inheritance chain
- ES6 In Depth: Subclassing on hacks.mozilla.org (2015)

---

JavaScript › Reference › Standard built-in objects › Object

# Object.values()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/values · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since March 2017.

The **`Object.values()`** static method returns an array of a given object's own enumerable string-keyed property values.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Object.values()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const object = {
  a: "some string",
  b: 42,
  c: false,
};

console.log(Object.values(object));
// Expected output: Array ["some string", 42, false]
```

## Syntax

js

```
Object.values(obj)
```

### Parameters

**`obj`**

An object.

### Return value

An array containing the given object's own enumerable string-keyed property values.

## Description

`Object.values()` returns an array whose elements are values of enumerable string-keyed properties found directly upon `object`. This is similar to accessing the values inside a `for...in` loop, except that a `for...in` loop enumerates properties in the prototype chain as well. The order of the array returned by `Object.values()` is the same as that provided by a `for...in` loop.

If you need the property keys, use `Object.keys()` instead. If you need both the property keys and values, use `Object.entries()` instead.

## Examples

### Using Object.values()

```js
const obj = { foo: "bar", baz: 42 };
console.log(Object.values(obj)); // ['bar', 42]

// Array-like object
const arrayLikeObj1 = { 0: "a", 1: "b", 2: "c" };
console.log(Object.values(arrayLikeObj1)); // ['a', 'b', 'c']

// Array-like object with random key ordering
// When using numeric keys, the values are returned in the keys' numerical order
const arrayLikeObj2 = { 100: "a", 2: "b", 7: "c" };
console.log(Object.values(arrayLikeObj2)); // ['b', 'c', 'a']

// getFoo is a non-enumerable property
const myObj = Object.create(
  {},
  {
    getFoo: {
      value() {
        return this.foo;
      },
    },
  },
);
myObj.foo = "bar";
console.log(Object.values(myObj)); // ['bar']
```

### Using Object.values() on primitives

Non-object arguments are coerced to objects. `undefined` and `null` cannot be coerced to objects and throw a `TypeError` upfront. Only strings may have own enumerable properties, while all other primitives return an empty array.

```js
// Strings have indices as enumerable own properties
console.log(Object.values("foo")); // ['f', 'o', 'o']

// Other primitives except undefined and null have no own properties
console.log(Object.values(100)); // []
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-object.values` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `values` | 54 | 14 | 47 | 41 | 10.1 | 54 | 47 | 41 | 10.3 | 6.0 | 54 | 10.3 | 1.0.0 | 1.0 | 7.0.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Object.values`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Object.values` in `core-js`
- es-shims polyfill of `Object.values`
- Enumerability and ownership of properties
- `Object.keys()`
- `Object.entries()`
- `Object.prototype.propertyIsEnumerable()`
- `Object.create()`
- `Object.getOwnPropertyNames()`
- `Map.prototype.values()`

---

JavaScript › Reference › Standard built-in objects › Object

# Object.prototype.__defineGetter__()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/__defineGetter__ · MDN page last modified 2026-10-26

**Limited availability** — not Baseline: does not work in some of the most widely-used browsers.

> **Note:** This feature is deprecated in favor of defining getters using the object initializer syntax or the `Object.defineProperty()` API. This method's behavior is only specified for web compatibility, and is not required to be implemented in any platform. It may not work everywhere.

The **`__defineGetter__()`** method of `Object` instances binds an object's property to a function to be called when that property is looked up.

## Syntax

```js
__defineGetter__(prop, func)
```

### Parameters

**`prop`**

A string containing the name of the property that the getter `func` is bound to.

**`func`**

A function to be bound to a lookup of the specified property.

### Return value

None (`undefined`).

### Exceptions

**`TypeError`**

Thrown if `func` is not a function.

## Description

All objects that inherit from `Object.prototype` (that is, all except `null`-prototype objects) inherit the `__defineGetter__()` method. This method allows a getter to be defined on a pre-existing object. This is equivalent to `Object.defineProperty(obj, prop, { get: func, configurable: true, enumerable: true })`, which means the property is enumerable and configurable, and any existing setter, if present, is preserved.

`__defineGetter__()` is defined in the spec as "normative optional", which means no implementation is required to implement this. However, all major browsers implement it, and due to its continued usage, it's unlikely to be removed. If a browser implements `__defineGetter__()`, it also needs to implement the `__lookupGetter__()`, `__lookupSetter__()`, and `__defineSetter__()` methods.

## Examples

### Using __defineGetter__()

```js
const o = {};
o.__defineGetter__("gimmeFive", function () {
  return 5;
});
console.log(o.gimmeFive); // 5
```

### Defining a getter property in standard ways

You can use the `get` syntax to define a getter when the object is first initialized.

```js
const o = {
  get gimmeFive() {
    return 5;
  },
};
console.log(o.gimmeFive); // 5
```

You may also use `Object.defineProperty()` to define a getter on an object after it's been created. Compared to `__defineGetter__()`, this method allows you to control the getter's enumerability and configurability, as well as defining symbol properties. The `Object.defineProperty()` method also works with `null`-prototype objects, which don't inherit from `Object.prototype` and therefore don't have the `__defineGetter__()` method.

```js
const o = {};
Object.defineProperty(o, "gimmeFive", {
  get() {
    return 5;
  },
  configurable: true,
  enumerable: true,
});
console.log(o.gimmeFive); // 5
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-object.prototype.__defineGetter__ |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `__defineGetter__` Deprecated | 1 | 12 | 1[1] | 9.5 | 3 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

1. Starting with Firefox 48, this method can no longer be called at the global scope without any object. A `TypeError` will be thrown otherwise. Previously, the global object was used in these cases automatically, but this is no longer the case.

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Object.defineGetter`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Object.prototype.__defineGetter__` in `core-js`
- `Object.prototype.__defineSetter__()`
- `get`
- `Object.defineProperty()`
- `Object.prototype.__lookupGetter__()`

- `Object.prototype.__lookupSetter__()`
- JS Guide: Defining Getters and Setters
- Firefox bug 647423

---

JavaScript › Reference › Standard built-in objects › Object

# Object.prototype.__defineSetter__()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/__defineSetter__ · MDN page last modified 2026-10-26

**Limited availability** — not Baseline: does not work in some of the most widely-used browsers.

> **Note:** This feature is deprecated in favor of defining setters using the object initializer syntax or the `Object.defineProperty()` API. This method's behavior is only specified for web compatibility, and is not required to be implemented in any platform. It may not work everywhere.

The **`__defineSetter__()`** method of `Object` instances binds an object's property to a function to be called when an attempt is made to set that property.

## Syntax

```js
__defineSetter__(prop, func)
```

### Parameters

**`prop`**

A string containing the name of the property that the setter `func` is bound to.

**`func`**

A function to be called when there is an attempt to set the specified property. This function receives the following parameter:

**`val`**

The value attempted to be assigned to `prop`.

### Return value

None (`undefined`).

### Exceptions

**`TypeError`**

Thrown if `func` is not a function.

## Description

All objects that inherit from `Object.prototype` (that is, all except `null`-prototype objects) inherit the `__defineSetter__()` method. This method allows a setter to be defined on a pre-existing object. This is equivalent to `Object.defineProperty(obj, prop, { set: func, configurable: true, enumerable: true })`, which means the property is enumerable and configurable, and any existing getter, if present, is preserved.

`__defineSetter__()` is defined in the spec as "normative optional", which means no implementation is required to implement this. However, all major browsers implement it, and due to its continued usage, it's unlikely to be removed. If a browser implements `__defineSetter__()`, it also needs to implement the `__lookupGetter__()`, `__lookupSetter__()`, and `__defineGetter__()` methods.

## Examples

### Using __defineSetter__()

```js
const o = {};
o.__defineSetter__("value", function (val) {
  this.anotherValue = val;
});
o.value = 5;
console.log(o.value); // undefined
console.log(o.anotherValue); // 5
```

### Defining a setter property in standard ways

You can use the `set` syntax to define a setter when the object is first initialized.

```js
const o = {
  set value(val) {
    this.anotherValue = val;
  },
};
o.value = 5;
console.log(o.value); // undefined
console.log(o.anotherValue); // 5
```

You may also use `Object.defineProperty()` to define a setter on an object after it's been created. Compared to `__defineSetter__()`, this method allows you to control the setter's enumerability and configurability, as well as defining symbol properties. The `Object.defineProperty()` method also works with `null`-prototype objects, which don't inherit from `Object.prototype` and therefore don't have the `__defineSetter__()` method.

```js
const o = {};
Object.defineProperty(o, "value", {
  set(val) {
    this.anotherValue = val;
  },
  configurable: true,
  enumerable: true,
});
o.value = 5;
console.log(o.value); // undefined
console.log(o.anotherValue); // 5
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-object.prototype.__defineSetter__ |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `__defineSetter__` Deprecated | 1 | 12 | 1[1] | 9.5 | 3 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

1. Starting with Firefox 48, this method can no longer be called at the global scope without any object. A `TypeError` will be thrown otherwise. Previously, the global object was used in these cases automatically, but this is no longer the case.

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Object.defineSetter`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Object.prototype.__defineSetter__` in `core-js`
- `Object.prototype.__defineGetter__()`
- `set`
- `Object.defineProperty()`
- `Object.prototype.__lookupGetter__()`
- `Object.prototype.__lookupSetter__()`
- JS Guide: Defining Getters and Setters
- Firefox bug 647423

---

JavaScript › Reference › Standard built-in objects › Object

# Object.prototype.__lookupGetter__()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/__lookupGetter__ · MDN page last modified 2026-10-26

**Limited availability** — not Baseline: does not work in some of the most widely-used browsers.

> **Note:** This feature is deprecated in favor of the `Object.getOwnPropertyDescriptor()` API. This method's behavior is only specified for web compatibility, and is not required to be implemented in any platform. It may not work everywhere.

The **`__lookupGetter__()`** method of `Object` instances returns the function bound as a getter to the specified property.

## Syntax

```js
__lookupGetter__(prop)
```

### Parameters

**prop**

A string containing the name of the property whose getter should be returned.

### Return value

The function bound as a getter to the specified property. Returns `undefined` if no such property is found, or the property is a data property.

## Description

All objects that inherit from `Object.prototype` (that is, all except `null`-prototype objects) inherit the `__lookupGetter__()` method. If a getter has been defined for an object's property, it's not possible to reference the getter function through that property, because that property refers to the return value of that function. `__lookupGetter__()` can be used to obtain a reference to the getter function.

`__lookupGetter__()` walks up the prototype chain to find the specified property. If any object along the prototype chain has the specified own property, the `get` attribute of the property descriptor for that property is returned. If that property is a data property, `undefined` is returned. If the property is not found along the entire prototype chain, `undefined` is also returned.

`__lookupGetter__()` is defined in the spec as "normative optional", which means no implementation is required to implement this. However, all major browsers implement it, and due to its continued usage, it's unlikely to be removed. If a browser implements `__lookupGetter__()`, it also needs to implement the `__lookupSetter__()`, `__defineGetter__()`, and `__defineSetter__()` methods.

## Examples

### Using __lookupGetter__()

```js
const obj = {
  get foo() {
    return Math.random() > 0.5 ? "foo" : "bar";
  },
};

obj.__lookupGetter__("foo");
// [Function: get foo]
```

### Looking up a property's getter in the standard way

You should use the `Object.getOwnPropertyDescriptor()` API to look up a property's getter. Compared to `__lookupGetter__()`, this method allows looking up symbol properties. The `Object.getOwnPropertyDescriptor()` method also works with `null`-prototype objects, which don't inherit from `Object.prototype` and therefore don't have the `__lookupGetter__()` method. If `__lookupGetter__()`'s behavior of walking up the prototype chain is important, you may implement it yourself with `Object.getPrototypeOf()`.

js

```
const obj = {
  get foo() {
    return Math.random() > 0.5 ? "foo" : "bar";
  },
};

Object.getOwnPropertyDescriptor(obj, "foo").get;
// [Function: get foo]
```

js

```
const obj2 = {
  __proto__: {
    get foo() {
      return Math.random() > 0.5 ? "foo" : "bar";
    },
  },
};

function findGetter(obj, prop) {
  while (obj) {
    const desc = Object.getOwnPropertyDescriptor(obj, prop);
    if (desc) {
      return desc.get;
    }
    obj = Object.getPrototypeOf(obj);
  }
}

console.log(findGetter(obj2, "foo")); // [Function: get foo]
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-object.prototype.__lookupGetter__ |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| __lookupGetter__ Deprecated | 1 | 12 | 1 | 9.5 | 3 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Object.lookupGetter`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Object.prototype.__lookupGetter__` in `core-js`
- `Object.prototype.__lookupSetter__()`
- `get`

- `Object.getOwnPropertyDescriptor()`
- `Object.prototype.__defineGetter__()`
- `Object.prototype.__defineSetter__()`
- JS Guide: Defining Getters and Setters

---

JavaScript › Reference › Standard built-in objects › Object

# Object.prototype.__lookupSetter__()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/__lookupSetter__ · MDN page last modified 2026-10-26

**Limited availability** — not Baseline: does not work in some of the most widely-used browsers.

> **Note:** This feature is deprecated in favor of the `Object.getOwnPropertyDescriptor()` API. This method's behavior is only specified for web compatibility, and is not required to be implemented in any platform. It may not work everywhere.

The **`__lookupSetter__()`** method of `Object` instances returns the function bound as a setter to the specified property.

## Syntax

```js
__lookupSetter__(prop)
```

### Parameters

**`prop`**

A string containing the name of the property whose setter should be returned.

### Return value

The function bound as a setter to the specified property. Returns `undefined` if no such property is found, or the property is a data property.

## Description

All objects that inherit from `Object.prototype` (that is, all except `null`-prototype objects) inherit the `__lookupSetter__()` method. If a setter has been defined for an object's property, it's not possible to reference the setter function through that property, because that property only calls the function when it's being set. `__lookupSetter__()` can be used to obtain a reference to the setter function.

`__lookupSetter__()` walks up the prototype chain to find the specified property. If any object along the prototype chain has the specified own property, the `set` attribute of the property descriptor for that property is returned. If that property is a data property, `undefined` is returned. If the property is not found along the entire prototype chain, `undefined` is also returned.

`__lookupSetter__()` is defined in the spec as "normative optional", which means no implementation is required to implement this. However, all major browsers implement it, and due to its continued usage, it's unlikely to be removed. If a browser implements `__lookupSetter__()`, it also needs to implement the `__lookupGetter__()`, `__defineGetter__()`, and `__defineSetter__()` methods.

## Examples

### Using __lookupSetter__()

```js
const obj = {
  set foo(value) {
    this.bar = value;
  },
};

obj.__lookupSetter__("foo");
// [Function: set foo]
```

### Looking up a property's setter in the standard way

You should use the `Object.getOwnPropertyDescriptor()` API to look up a property's setter. Compared to `__lookupSetter__()`, this method allows looking up symbol properties. The `Object.getOwnPropertyDescriptor()` method also works with `null`-prototype objects, which don't inherit from `Object.prototype` and therefore don't have the `__lookupSetter__()` method. If `__lookupSetter__()`'s behavior of walking up the prototype chain is important, you may implement it yourself with `Object.getPrototypeOf()`.

```js
const obj = {
  set foo(value) {
    this.bar = value;
  },
};

Object.getOwnPropertyDescriptor(obj, "foo").set;
// [Function: set foo]
```

```js
const obj2 = {
  __proto__: {
    set foo(value) {
      this.bar = value;
    },
  },
};

function findSetter(obj, prop) {
  while (obj) {
    const desc = Object.getOwnPropertyDescriptor(obj, prop);
    if (desc) {
      return desc.set;
    }
    obj = Object.getPrototypeOf(obj);
  }
}

console.log(findSetter(obj2, "foo")); // [Function: set foo]
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-object.prototype.__lookupSetter__` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Nodejs |
| `__lookupSetter__` Deprecated | 1 | 12 | 1 | 9.5 | 3 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Object.lookupSetter`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Object.prototype.__lookupSetter__` in `core-js`
- `Object.prototype.__lookupGetter__()`
- `set`
- `Object.getOwnPropertyDescriptor()`
- `Object.prototype.__defineGetter__()`
- `Object.prototype.__defineSetter__()`
- JS Guide: Defining Getters and Setters

---

JavaScript › Reference › Standard built-in objects › Object

# Object.prototype.hasOwnProperty()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/hasOwnProperty · MDN page last modified 2025-07-20

**Baseline: Widely available** — available across major browsers since July 2015.

The **`hasOwnProperty()`** method of `Object` instances returns a boolean indicating whether this object has the specified property as its own property (as opposed to inheriting it).

> **Note:** `Object.hasOwn()` is recommended over `hasOwnProperty()`, in browsers where it is supported.

## Try it

*▷ Interactive example on MDN: "JavaScript Demo: Object.prototype.hasOwnProperty()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const object = {};
object.foo = 42;

console.log(object.hasOwnProperty("foo"));
// Expected output: true

console.log(object.hasOwnProperty("toString"));
// Expected output: false

console.log(object.hasOwnProperty("hasOwnProperty"));
// Expected output: false
```

## Syntax

js

```
hasOwnProperty(prop)
```

### Parameters

**`prop`**

The `String` name or Symbol of the property to test.

### Return value

Returns `true` if the object has the specified property as own property; `false` otherwise.

## Description

The **`hasOwnProperty()`** method returns `true` if the specified property is a direct property of the object — even if the value is `null` or `undefined`. The method returns `false` if the property is inherited, or has not been declared at all. Unlike the `in` operator, this method does not check for the specified property in the object's prototype chain.

The method can be called on *most* JavaScript objects, because most objects descend from `Object`, and hence inherit its methods. For example `Array` is an `Object`, so you can use `hasOwnProperty()` method to check whether an index exists:

js

```
const fruits = ["Apple", "Banana", "Watermelon", "Orange"];
fruits.hasOwnProperty(3); // true ('Orange')
fruits.hasOwnProperty(4); // false - not defined
```

The method will not be available in objects where it is reimplemented, or on `null`-prototype objects (as these don't inherit from `Object.prototype`). Examples for these cases are given below.

## Examples

### Using hasOwnProperty to test for an own property's existence

The following code shows how to determine whether the `example` object contains a property named `prop`.

```js
const example = {};
example.hasOwnProperty("prop"); // false

example.prop = "exists";
example.hasOwnProperty("prop"); // true - 'prop' has been defined

example.prop = null;
example.hasOwnProperty("prop"); // true - own property exists with value of null

example.prop = undefined;
example.hasOwnProperty("prop"); // true - own property exists with value of undefined
```

### Direct vs. inherited properties

The following example differentiates between direct properties and properties inherited through the prototype chain:

```js
const example = {};
example.prop = "exists";

// `hasOwnProperty` will only return true for direct properties:
example.hasOwnProperty("prop"); // true
example.hasOwnProperty("toString"); // false
example.hasOwnProperty("hasOwnProperty"); // false

// The `in` operator will return true for direct or inherited properties:
"prop" in example; // true
"toString" in example; // true
"hasOwnProperty" in example; // true
```

### Iterating over the properties of an object

The following example shows how to iterate over the enumerable properties of an object without executing on inherited properties.

```js
const buz = {
  fog: "stack",
};

for (const name in buz) {
  if (buz.hasOwnProperty(name)) {
    console.log(`this is fog (${name}) for sure. Value: ${buz[name]}`);
  } else {
    console.log(name); // toString or something else
  }
}
```

Note that the `for...in` loop only iterates enumerable items: the absence of non-enumerable properties emitted from the loop does not imply that `hasOwnProperty` itself is confined strictly to enumerable items. You can iterate over non-enumerable properties with `Object.getOwnPropertyNames()`.

### Using hasOwnProperty as a property name

JavaScript does not protect the property name `hasOwnProperty`; an object that has a property with this name may return incorrect results:

```js
const foo = {
  hasOwnProperty() {
    return false;
  },
  bar: "Here be dragons",
};

foo.hasOwnProperty("bar"); // re-implementation always returns false
```

The recommended way to overcome this problem is to instead use `Object.hasOwn()` (in browsers that support it). Other alternatives include using an *external* `hasOwnProperty`:

```js
const foo = { bar: "Here be dragons" };

// Use Object.hasOwn() method - recommended
Object.hasOwn(foo, "bar"); // true

// Use the hasOwnProperty property from the Object prototype
Object.prototype.hasOwnProperty.call(foo, "bar"); // true

// Use another Object's hasOwnProperty
// and call it with 'this' set to foo
({}).hasOwnProperty.call(foo, "bar"); // true
```

Note that in the first two cases there are no newly created objects.

### Objects created with Object.create(null)

`null`-prototype objects do not inherit from `Object.prototype`, making `hasOwnProperty()` inaccessible.

```js
const foo = Object.create(null);
foo.prop = "exists";
foo.hasOwnProperty("prop"); // Uncaught TypeError: foo.hasOwnProperty is not a function
```

The solutions in this case are the same as for the previous section: use `Object.hasOwn()` by preference, otherwise use an external object's `hasOwnProperty()`.

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-object.prototype.hasownproperty |

## Browser compatibility

| | Desktop | | | | | Mobile | | | | | | | Server | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Feature | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `hasOwnProperty` | 1 | 12 | 1 | 5 | 3 | 18 | 4 | 10.1 | 1 | 1.0 | 1 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Object.hasOwnProperty`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Object.hasOwn()`
- Enumerability and ownership of properties
- `Object.getOwnPropertyNames()`
- `for...in`
- `in`
- Inheritance and the prototype chain

---

JavaScript › Reference › Standard built-in objects › Object

# Object.prototype.isPrototypeOf()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/isPrototypeOf · MDN page last modified 2026-05-22

**Baseline: Widely available** — available across major browsers since July 2015.

The **`isPrototypeOf()`** method of `Object` instances checks if this object exists in another object's prototype chain.

> **Note:** `isPrototypeOf()` differs from the `instanceof` operator. In the expression `object instanceof AFunction`, `object`'s prototype chain is checked against `AFunction.prototype`, not against `AFunction` itself.

## Try it

*▷ Interactive example on MDN: "JavaScript Demo: Object.prototype.isPrototypeOf()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function Foo() {}
function Bar() {}

Bar.prototype = Object.create(Foo.prototype);

const bar = new Bar();

console.log(Foo.prototype.isPrototypeOf(bar));
// Expected output: true
console.log(Bar.prototype.isPrototypeOf(bar));
// Expected output: true
```

## Syntax

```
isPrototypeOf(object)
```

### Parameters

**`object`**

The object whose prototype chain will be searched.

### Return value

A boolean indicating whether the calling object (`this`) lies in the prototype chain of `object`. Directly returns `false` when `object` is not an object (i.e., a primitive).

### Exceptions

**`TypeError`**

Thrown if `this` is `null` or `undefined` (because it can't be converted to an object).

## Description

All objects that inherit from `Object.prototype` (that is, all except `null`-prototype objects) inherit the `isPrototypeOf()` method. This method allows you to check whether or not the object exists within another object's prototype chain. If the `object` passed as the parameter is not an object (i.e., a primitive), the method directly returns `false`. Otherwise, the `this` value is converted to an object, and the prototype chain of `object` is searched for the `this` value, until the end of the chain is reached or the `this` value is found.

## Examples

### Using isPrototypeOf()

This example demonstrates that `Baz.prototype`, `Bar.prototype`, `Foo.prototype` and `Object.prototype` exist in the prototype chain for object `baz`:

```
class Foo {}
class Bar extends Foo {}
class Baz extends Bar {}

const foo = new Foo();
const bar = new Bar();
const baz = new Baz();

// prototype chains:
// foo: Foo --> Object
// bar: Bar --> Foo --> Object
// baz: Baz --> Bar --> Foo --> Object
console.log(Baz.prototype.isPrototypeOf(baz)); // true
console.log(Baz.prototype.isPrototypeOf(bar)); // false
console.log(Baz.prototype.isPrototypeOf(foo)); // false
console.log(Bar.prototype.isPrototypeOf(baz)); // true
console.log(Bar.prototype.isPrototypeOf(foo)); // false
```

(code continues on the next page)

(code continued from the previous page)

```
console.log(Foo.prototype.isPrototypeOf(baz)); // true
console.log(Foo.prototype.isPrototypeOf(bar)); // true
console.log(Object.prototype.isPrototypeOf(baz)); // true
```

The `isPrototypeOf()` method — along with the `instanceof` operator — comes in particularly handy if you have code that can only function when dealing with objects descended from a specific prototype chain; e.g., to guarantee that certain methods or properties will be present on that object.

For example, to execute some code that's only safe to run if a `baz` object has `Foo.prototype` in its prototype chain, you can do this:

js

```
if (Foo.prototype.isPrototypeOf(baz)) {
  // do something safe
}
```

However, `Foo.prototype` existing in `baz`'s prototype chain doesn't imply `baz` was created using `Foo` as its constructor. For example, `baz` could be directly assigned with `Foo.prototype` as its prototype. In this case, if your code reads private fields of `Foo` from `baz`, it would still fail:

js

```
class Foo {
  #value = "foo";
  static getValue(x) {
    return x.#value;
  }
}

const baz = { __proto__: Foo.prototype };

if (Foo.prototype.isPrototypeOf(baz)) {
  console.log(Foo.getValue(baz)); // TypeError: Cannot read private member #value from an
object whose class did not declare it
}
```

The same applies to `instanceof`. If you need to read private fields in a secure way, offer a branded check method using `in` instead.

js

```
class Foo {
  #value = "foo";
  static getValue(x) {
    return x.#value;
  }
  static isFoo(x) {
    return #value in x;
  }
}

const baz = { __proto__: Foo.prototype };

if (Foo.isFoo(baz)) {
  // Doesn't run, because baz is not a Foo
  console.log(Foo.getValue(baz));
}
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-object.prototype.isprototypeof` |

## Browser compatibility

| | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| Feature | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `isPrototypeOf` | 1 | 12 | 1 | 4 | 3 | 18 | 4 | 10.1 | 1 | 1.0 | 1 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Object.isPrototypeOf`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `instanceof`
- `Object.getPrototypeOf()`
- `Object.setPrototypeOf()`
- Inheritance and the prototype chain

---

JavaScript › Reference › Standard built-in objects › Object

# Object.prototype.propertyIsEnumerable()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/propertyIsEnumerable · MDN page last modified 2025-07-20

**Baseline: Widely available** — available across major browsers since July 2015.

The **`propertyIsEnumerable()`** method of `Object` instances returns a boolean indicating whether the specified property is this object's enumerable own property.

## Try it

*▷ Interactive example on MDN: "JavaScript Demo: Object.prototype.propertyIsEnumerable()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const object = {};
const array = [];
object.foo = 42;
array[0] = 42;

console.log(object.propertyIsEnumerable("foo"));
// Expected output: true

console.log(array.propertyIsEnumerable(0));
// Expected output: true

console.log(array.propertyIsEnumerable("length"));
// Expected output: false
```

## Syntax

js

```
propertyIsEnumerable(prop)
```

### Parameters

**prop**

The name of the property to test. Can be a string or a `Symbol`.

### Return value

A boolean value indicating whether the specified property is enumerable and is the object's own property.

## Description

All objects that inherit from `Object.prototype` (that is, all except `null`-prototype objects) inherit the `propertyIsEnumerable()` method. This method determines if the specified property, string or symbol, is an enumerable own property of the object. If the object does not have the specified property, this method returns `false`.

This method is equivalent to `Object.getOwnPropertyDescriptor(obj, prop)?.enumerable ?? false`.

## Examples

### Using propertyIsEnumerable()

The following example shows the use of `propertyIsEnumerable()` on objects and arrays.

```js
const o = {};
const a = [];
o.prop = "is enumerable";
a[0] = "is enumerable";

o.propertyIsEnumerable("prop"); // true
a.propertyIsEnumerable(0); // true
```

### User-defined vs. built-in objects

Most built-in properties are non-enumerable by default, while user-created object properties are often enumerable, unless explicitly designated otherwise.

```js
const a = ["is enumerable"];

a.propertyIsEnumerable(0); // true
a.propertyIsEnumerable("length"); // false

Math.propertyIsEnumerable("random"); // false
globalThis.propertyIsEnumerable("Math"); // false
```

### Direct vs. inherited properties

Only enumerable own properties cause `propertyIsEnumerable()` to return `true`, although all enumerable properties, including inherited ones, are visited by the `for...in` loop.

```js
const o1 = {
  enumerableInherited: "is enumerable",
};
Object.defineProperty(o1, "nonEnumerableInherited", {
  value: "is non-enumerable",
  enumerable: false,
});
const o2 = {
  // o1 is the prototype of o2
  __proto__: o1,
  enumerableOwn: "is enumerable",
};
Object.defineProperty(o2, "nonEnumerableOwn", {
  value: "is non-enumerable",
  enumerable: false,
});

o2.propertyIsEnumerable("enumerableInherited"); // false
o2.propertyIsEnumerable("nonEnumerableInherited"); // false
o2.propertyIsEnumerable("enumerableOwn"); // true
o2.propertyIsEnumerable("nonEnumerableOwn"); // false
```

### Testing symbol properties

`Symbol` properties are also supported by `propertyIsEnumerable()`. Note that most enumeration methods only visit string properties; enumerability of symbol properties is only useful when using `Object.assign()` or spread syntax. For more information, see Enumerability and ownership of properties.

```js
const sym = Symbol("enumerable");
const sym2 = Symbol("non-enumerable");
const o = {
  [sym]: "is enumerable",
};
Object.defineProperty(o, sym2, {
  value: "is non-enumerable",
  enumerable: false,
});

o.propertyIsEnumerable(sym); // true
o.propertyIsEnumerable(sym2); // false
```

### Usage with null-prototype objects

Because `null`-prototype objects do not inherit from `Object.prototype`, they do not inherit the `propertyIsEnumerable()` method. You must call `Object.prototype.propertyIsEnumerable` with the object as `this` instead.

```js
const o = {
  __proto__: null,
  enumerableOwn: "is enumerable",
};

o.propertyIsEnumerable("enumerableOwn"); // TypeError: o.propertyIsEnumerable is not a
function
Object.prototype.propertyIsEnumerable.call(o, "enumerableOwn"); // true
```

Alternatively, you may use `Object.getOwnPropertyDescriptor()` instead, which also helps to distinguish between non-existent properties and actually non-enumerable properties.

```js
const o = {
  __proto__: null,
  enumerableOwn: "is enumerable",
};

Object.getOwnPropertyDescriptor(o, "enumerableOwn")?.enumerable; // true
Object.getOwnPropertyDescriptor(o, "nonExistent")?.enumerable; // undefined
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-object.prototype.propertyisenumerable |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `propertyIsEnumerable` | 1 | 12 | 1 | 4 | 3 | 18 | 4 | 10.1 | 1 | 1.0 | 1 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Object.propertyIsEnumerable`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Enumerability and ownership of properties
- `for...in`
- `Object.keys()`
- `Object.defineProperty()`

---

JavaScript › Reference › Standard built-in objects › Object

# Object.prototype.toLocaleString()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/toLocaleString · MDN page last modified 2025-07-20

**Baseline: Widely available** — available across major browsers since July 2015.

The **`toLocaleString()`** method of `Object` instances returns a string representing this object. This method is meant to be overridden by derived objects for locale-specific purposes.

## Try it

*▷ Interactive example on MDN: "JavaScript Demo: Object.prototype.toLocaleString()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const date = new Date(Date.UTC(2012, 11, 20, 3, 0, 0));

console.log(date.toLocaleString("ar-EG"));
// Expected output: "٢٠/١٢/٢٠١٢ ٤:٠٠:٠٠ ص"

const number = 123456.789;

console.log(number.toLocaleString("de-DE"));
// Expected output: "123.456,789"
```

## Syntax

js

```
toLocaleString()
```

### Parameters

None. However, all objects that override this method are expected to accept at most two parameters, corresponding to `locales` and `options`, such as `Number.prototype.toLocaleString`. The parameter positions should not be used for any other purpose.

### Return value

The return value of calling `this.toString()`.

## Description

All objects that inherit from `Object.prototype` (that is, all except `null`-prototype objects) inherit the `toLocaleString()` method. `Object`'s `toLocaleString` returns the result of calling `this.toString()`.

This function is provided to give objects a generic `toLocaleString` method, even though not all may use it. In the core language, these built-in objects override `toLocaleString` to provide locale-specific formatting:

- `Array`: `Array.prototype.toLocaleString()`
- `Number`: `Number.prototype.toLocaleString()`
- `Date`: `Date.prototype.toLocaleString()`
- `TypedArray`: `TypedArray.prototype.toLocaleString()`
- `BigInt`: `BigInt.prototype.toLocaleString()`

## Examples

### Using the base toLocaleString() method

The base `toLocaleString()` method simply calls `toString()`.

```js
const obj = {
  toString() {
    return "My Object";
  },
};
console.log(obj.toLocaleString()); // "My Object"
```

### Array toLocaleString() override

`Array.prototype.toLocaleString()` is used to print array values as a string by invoking each element's `toLocaleString()` method and joining the results with a locale-specific separator. For example:

```js
const testArray = [4, 7, 10];

const euroPrices = testArray.toLocaleString("fr", {
  style: "currency",
  currency: "EUR",
});
// "4,00 €,7,00 €,10,00 €"
```

### Date toLocaleString() override

`Date.prototype.toLocaleString()` is used to print out date displays more suitable for specific locales. For example:

```js
const testDate = new Date();
// "Fri May 29 2020 18:04:24 GMT+0100 (British Summer Time)"

const deDate = testDate.toLocaleString("de");
// "29.5.2020, 18:04:24"

const frDate = testDate.toLocaleString("fr");
// "29/05/2020, 18:04:24"
```

### Number toLocaleString() override

`Number.prototype.toLocaleString()` is used to print out number displays more suitable for specific locales, e.g., with the correct separators. For example:

```js
const testNumber = 2901234564;
// "2901234564"

const deNumber = testNumber.toLocaleString("de");
// "2.901.234.564"

const frNumber = testNumber.toLocaleString("fr");
// "2 901 234 564"
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-object.prototype.tolocalestring` |

## Browser compatibility

| | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| Feature | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `toLocaleString` | 1 | 12 | 1 | 4 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 1 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Object.toLocaleString`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Object.prototype.toString()`

JavaScript › Reference › Standard built-in objects › Object

# Object.prototype.toString()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/toString · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since July 2015.

The **`toString()`** method of `Object` instances returns a string representing this object. This method is meant to be overridden by derived objects for custom type coercion logic.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Object.prototype.toString()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const map = new Map();

console.log(map.toString());
// Expected output: "[object Map]"
```

## Syntax

js

```
toString()
```

### Parameters

By default `toString()` takes no parameters. However, objects that inherit from `Object` may override it with their own implementations that do take parameters. For example, the `Number.prototype.toString()` and `BigInt.prototype.toString()` methods take an optional `radix` parameter.

### Return value

A string representing the object.

## Description

JavaScript calls the `toString` method to convert an object to a primitive value. You rarely need to invoke the `toString` method yourself; JavaScript automatically invokes it when encountering an object where a primitive value is expected.

This method is called in priority by string conversion, but numeric conversion and primitive conversion call `valueOf()` in priority. However, because the base `valueOf()` method returns an object, the `toString()` method is usually called in the end, unless the object overrides `valueOf()`. For example, `+[1]` returns `1`, because its `toString()` method returns `"1"`, which is then converted to a number.

All objects that inherit from `Object.prototype` (that is, all except `null`-prototype objects) inherit the `toString()` method. When you create a custom object, you can override `toString()` to call a custom method, so that your custom object can be converted to a string value. Alternatively, you can add a `[Symbol.toPrimitive]()` method, which allows even more control over the conversion process, and will always be preferred over `valueOf` or `toString` for any type conversion.

To use the base `Object.prototype.toString()` with an object that has it overridden (or to invoke it on `null` or `undefined`), you need to call `Function.prototype.call()` or `Function.prototype.apply()` on it, passing the object you want to inspect as the first parameter (called `thisArg`).

```js
const arr = [1, 2, 3];

arr.toString(); // "1,2,3"
Object.prototype.toString.call(arr); // "[object Array]"
```

`Object.prototype.toString()` returns `"[object Type]"`, where `Type` is the object type. If the object has a `Symbol.toStringTag` property whose value is a string, that value will be used as the `Type`. Many built-in objects, including `Map` and `Symbol`, have a `Symbol.toStringTag`. Some objects predating ES6 do not have `Symbol.toStringTag`, but have a special tag nonetheless. They include (the tag is the same as the type name given below):

- `Array`
- `Function` (anything whose `typeof` returns `"function"`)
- `Error`
- `Boolean`
- `Number`
- `String`
- `Date`
- `RegExp`

The `arguments` object returns `"[object Arguments]"`. Everything else, including user-defined classes, unless with a custom `Symbol.toStringTag`, will return `"[object Object]"`.

`Object.prototype.toString()` invoked on `null` and `undefined` returns `[object Null]` and `[object Undefined]`, respectively.

## Examples

### Overriding toString for custom objects

You can create a function to be called in place of the default `toString()` method. The `toString()` function you create should return a string value. If it returns an object and the method is called implicitly during type conversion, then its result is ignored and the value of a related method, `valueOf()`, is used instead, or a `TypeError` is thrown if none of these methods return a primitive.

The following code defines a `Dog` class.

```js
class Dog {
  constructor(name, breed, color, sex) {
    this.name = name;
    this.breed = breed;
    this.color = color;
    this.sex = sex;
  }
}
```

If you call the `toString()` method, either explicitly or implicitly, on an instance of `Dog`, it returns the default value inherited from `Object`:

```js
const theDog = new Dog("Gabby", "Lab", "chocolate", "female");

theDog.toString(); // "[object Object]"
`${theDog}`; // "[object Object]"
```

The following code overrides the default `toString()` method. This method generates a string containing the `name`, `breed`, `color`, and `sex` of the object.

```js
class Dog {
  constructor(name, breed, color, sex) {
    this.name = name;
    this.breed = breed;
    this.color = color;
    this.sex = sex;
  }
  toString() {
    return `Dog ${this.name} is a ${this.sex} ${this.color} ${this.breed}`;
  }
}
```

With the preceding code in place, any time an instance of `Dog` is used in a string context, JavaScript automatically calls the `toString()` method.

```js
const theDog = new Dog("Gabby", "Lab", "chocolate", "female");

`${theDog}`; // "Dog Gabby is a female chocolate Lab"
```

### Using toString() to detect object class

`toString()` can be used with every object and (by default) allows you to get its class.

```js
const toString = Object.prototype.toString;

toString.call(new Date()); // [object Date]
toString.call(new String()); // [object String]
// Math has its Symbol.toStringTag
toString.call(Math); // [object Math]

toString.call(undefined); // [object Undefined]
toString.call(null); // [object Null]
```

Using `toString()` in this way is unreliable; objects can change the behavior of `Object.prototype.toString()` by defining a `Symbol.toStringTag` property, leading to unexpected results. For example:

```js
const myDate = new Date();
Object.prototype.toString.call(myDate); // [object Date]

myDate[Symbol.toStringTag] = "myDate";
Object.prototype.toString.call(myDate); // [object myDate]

Date.prototype[Symbol.toStringTag] = "prototype polluted";
Object.prototype.toString.call(new Date()); // [object prototype polluted]
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-object.prototype.tostring` |

## Browser compatibility

| | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| Feature | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `toString()` | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 1 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Object.toString`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Object.prototype.toString` with `Symbol.toStringTag` support in `core-js`
- `Object.prototype.valueOf()`
- `Number.prototype.toString()`
- `Symbol.toPrimitive`
- `Symbol.toStringTag`

---

JavaScript › Reference › Standard built-in objects › Object

# Object.prototype.valueOf()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/valueOf · MDN page last modified 2025-08-19

**Baseline: Widely available** — available across major browsers since July 2015.

The **`valueOf()`** method of `Object` instances converts the `this` value to an object. This method is meant to be overridden by derived objects for custom type conversion logic.

## Try it

*▷ Interactive example on MDN: "JavaScript Demo: Object.prototype.valueOf()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function MyNumberType(n) {
  this.number = n;
}

MyNumberType.prototype.valueOf = function () {
  return this.number;
};

const object = new MyNumberType(4);

console.log(object + 3);
// Expected output: 7
```

## Syntax

js

```
valueOf()
```

### Parameters

None.

### Return value

The `this` value, converted to an object.

> **Note:** In order for `valueOf` to be useful during type conversion, it must return a primitive. Because all primitive types have their own `valueOf()` methods, calling `primitiveValue.valueOf()` generally does not invoke `Object.prototype.valueOf()`.

## Description

JavaScript calls the `valueOf` method to convert an object to a primitive value. You rarely need to invoke the `valueOf` method yourself; JavaScript automatically invokes it when encountering an object where a primitive value is expected.

This method is called in priority by numeric conversion and primitive conversion, but string conversion calls `toString()` in priority, and `toString()` is very likely to return a string value (even for the `Object.prototype.toString()` base implementation), so `valueOf()` is usually not called in this case.

All objects that inherit from `Object.prototype` (that is, all except `null`-prototype objects) inherit the `toString()` method. The `Object.prototype.valueOf()` base implementation is deliberately useless: by returning an object, its return value will never be used by any primitive conversion algorithm. Many built-in objects override this method to return an appropriate primitive value. When you create a custom object, you can override `valueOf()` to call a custom method, so that your custom object can be converted to a primitive value. Generally, `valueOf()` is used to return a value that is most meaningful for the object — unlike `toString()`, it does not need to be a string. Alternatively, you can add a

[Symbol.toPrimitive]() method, which allows even more control over the conversion process, and will always be preferred over valueOf or toString for any type conversion.

## Examples

### Using valueOf()

The base valueOf() method returns the this value itself, converted to an object if it isn't already. Therefore its return value will never be used by any primitive conversion algorithm.

```js
const obj = { foo: 1 };
console.log(obj.valueOf() === obj); // true

console.log(Object.prototype.valueOf.call("primitive"));
// [String: 'primitive'] (a wrapper object)
```

### Overriding valueOf for custom objects

You can create a function to be called in place of the default valueOf method. Your function should take no arguments, since it won't be passed any when called during type conversion.

For example, you can add a valueOf method to your custom class Box.

```js
class Box {
  #value;
  constructor(value) {
    this.#value = value;
  }
  valueOf() {
    return this.#value;
  }
}
```

With the preceding code in place, any time an object of type Box is used in a context where it is to be represented as a primitive value (but not specifically a string), JavaScript automatically calls the function defined in the preceding code.

```js
const box = new Box(123);
console.log(box + 456); // 579
console.log(box == 123); // true
```

An object's valueOf method is usually invoked by JavaScript, but you can invoke it yourself as follows:

```js
box.valueOf();
```

### Using unary plus on objects

Unary plus performs number coercion on its operand, which, for most objects without [Symbol.toPrimitive](), means calling its valueOf(). However, if the object doesn't have a custom valueOf() method, the base implementation will cause valueOf() to be ignored and the return value of toString() to be used instead.

```js
+new Date(); // the current timestamp; same as new Date().getTime()
+{}; // NaN (toString() returns "[object Object]")
+[]; // 0 (toString() returns an empty string list)
+[1]; // 1 (toString() returns "1")
+[1, 2]; // NaN (toString() returns "1,2")
+new Set([1]); // NaN (toString() returns "[object Set]")
+{ valueOf: () => 42 }; // 42
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-object.prototype.valueof |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `valueOf` | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 1 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Object.valueOf`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Object.prototype.toString()`
- `parseInt()`
- `Symbol.toPrimitive`

---

JavaScript › Reference › Standard built-in objects › Object

# Object.prototype.__proto__

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/proto · MDN page last modified 2026-08-21

**Limited availability** — not Baseline: does not work in some of the most widely-used browsers.

> **Warning:** Changing the `[[Prototype]]` of an object is, by the nature of how modern JavaScript engines optimize property accesses, currently a very slow operation in every browser and JavaScript engine. In addition, the effects of altering inheritance are subtle and far-flung, and are not limited to the time spent in the `obj.__proto__ = ...` statement, but may extend to ***any*** code that has access to any object whose `[[Prototype]]` has been altered. You can read more in JavaScript engine fundamentals: optimizing prototypes.

> **Note:** The use of `__proto__` is controversial and discouraged. Its existence and exact behavior have only been standardized as a legacy feature to ensure web compatibility, while it presents several security issues and footguns. For better support, prefer `Object.getPrototypeOf()`/`Reflect.getPrototypeOf()` and `Object.setPrototypeOf()`/`Reflect.setPrototypeOf()` instead.

The **__proto__** accessor property of Object instances exposes the [[Prototype]] (either an object or null) of this object.

The __proto__ property can also be used in an object literal definition to set the object [[Prototype]] on creation, as an alternative to Object.create(). See: object initializer / literal syntax. That syntax is standard and optimized for in implementations, and quite different from Object.prototype.__proto__.

## Syntax

```js
obj.__proto__
```

### Return value

If used as a getter, returns the object's [[Prototype]].

### Exceptions

**TypeError**

Thrown if attempting to set the prototype of a non-extensible object or an immutable prototype exotic object, such as Object.prototype or window.

## Description

The __proto__ getter function exposes the value of the internal [[Prototype]] of an object. For objects created using an object literal (unless you use the prototype setter syntax), this value is Object.prototype. For objects created using array literals, this value is Array.prototype. For functions, this value is Function.prototype. You can read more about the prototype chain in Inheritance and the prototype chain.

The __proto__ setter allows the [[Prototype]] of an object to be mutated. The value provided must be an object or null. Providing any other value will do nothing.

Unlike Object.getPrototypeOf() and Object.setPrototypeOf(), which are always available on Object as static properties and always reflect the [[Prototype]] internal property, the __proto__ property doesn't always exist as a property on all objects, and as a result doesn't reflect [[Prototype]] reliably.

The __proto__ property is just an accessor property on Object.prototype consisting of a getter and setter function. A property access for __proto__ that eventually consults Object.prototype will find this property, but an access that does not consult Object.prototype will not. If some other __proto__ property is found before Object.prototype is consulted, that property will hide the one found on Object.prototype.

null-prototype objects don't inherit any property from Object.prototype, including the __proto__ accessor property, so if you try to read __proto__ on such an object, the value is always undefined regardless of the object's actual [[Prototype]], and any assignment to __proto__ would create a new property called __proto__ instead of setting the object's prototype. Furthermore, __proto__ can be redefined as an own property on any object instance through Object.defineProperty() without triggering the setter. In this case, __proto__ will no longer be an accessor for [[Prototype]]. Therefore, always prefer Object.getPrototypeOf() and Object.setPrototypeOf() for setting and getting the [[Prototype]] of an object.

## Examples

### Using __proto__

```js
function Circle() {}
const shape = {};
const circle = new Circle();

// Set the object prototype.
// DEPRECATED. This is for example purposes only. DO NOT DO THIS in real code.
shape.__proto__ = circle;

// Get the object prototype
console.log(shape.__proto__ === Circle); // false
```

```js
function ShapeA() {}
const ShapeB = {
  a() {
    console.log("aaa");
  },
};

ShapeA.prototype.__proto__ = ShapeB;
console.log(ShapeA.prototype.__proto__); // { a: [Function: a] }

const shapeA = new ShapeA();
shapeA.a(); // aaa
console.log(ShapeA.prototype === shapeA.__proto__); // true
```

```js
function ShapeC() {}
const ShapeD = {
  a() {
    console.log("a");
  },
};

const shapeC = new ShapeC();
shapeC.__proto__ = ShapeD;
shapeC.a(); // a
console.log(ShapeC.prototype === shapeC.__proto__); // false
```

```js
function Test() {}
Test.prototype.myName = function () {
  console.log("myName");
};

const test = new Test();
console.log(test.__proto__ === Test.prototype); // true
test.myName(); // myName

const obj = {};
obj.__proto__ = Test.prototype;
obj.myName(); // myName
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-object.prototype.__proto__` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `__proto__` Deprecated | 1 | 12 | 1 | 10.5 | 3 | 18 | 4 | 11 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 2.9[1,2] | 0.10.0 |

1. Disabled by default; the user must enable the runtime flag `--unsafe-proto`.
2. Also from version 1.39: disabled by default; the user must enable the runtime flag `--unstable-unsafe-proto`.

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Object.proto`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Object.prototype.isPrototypeOf()`
- `Object.getPrototypeOf()`
- `Object.setPrototypeOf()`
- Prototype pollution attack

---

JavaScript › Reference › Standard built-in objects › Object

# Object.prototype.constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/constructor · MDN page last modified 2025-07-10

**Baseline: Widely available** — available across major browsers since July 2015.

The **`constructor`** data property of an `Object` instance returns a reference to the constructor function that created the instance object. Note that the value of this property is a reference to *the function itself*, not a string containing the function's name.

> **Note:** This is a property of JavaScript objects. For the `constructor` method in classes, see its own reference page.

## Value

A reference to the constructor function that created the instance object.

| Property attributes of `Object.prototype.constructor` | |
| --- | --- |
| Writable | yes |
| Enumerable | no |
| Configurable | yes |

> **Note:** This property is created by default on the `prototype` property of every constructor function and is inherited by all objects created by that constructor.

## Description

Any object (with the exception of `null` prototype objects) will have a `constructor` property on its `[[Prototype]]`. Objects created with literals will also have a `constructor` property that points to the constructor type for that object — for example, array literals create `Array` objects, and object literals create plain objects.

```js
const o1 = {};
o1.constructor === Object; // true

const o2 = new Object();
o2.constructor === Object; // true

const a1 = [];
a1.constructor === Array; // true

const a2 = new Array();
a2.constructor === Array; // true

const n = 3;
n.constructor === Number; // true
```

Note that `constructor` usually comes from the constructor's `prototype` property. If you have a longer prototype chain, you can usually expect every object in the chain to have a `constructor` property.

```js
const o = new TypeError(); // Inheritance: TypeError -> Error -> Object
const proto = Object.getPrototypeOf;

Object.hasOwn(o, "constructor"); // false
proto(o).constructor === TypeError; // true
proto(proto(o)).constructor === Error; // true
proto(proto(proto(o))).constructor === Object; // true
```

## Examples

### Displaying the constructor of an object

The following example creates a constructor (`Tree`) and an object of that type (`theTree`). The example then displays the `constructor` property for the object `theTree`.

```js
function Tree(name) {
  this.name = name;
}

const theTree = new Tree("Redwood");
console.log(`theTree.constructor is ${theTree.constructor}`);
```

This example displays the following output:

```
theTree.constructor is function Tree(name) {
  this.name = name;
}
```

### Assigning the constructor property to an object

One can assign the `constructor` property of non-primitives.

```js
const arr = [];
arr.constructor = String;
arr.constructor === String; // true
arr instanceof String; // false
arr instanceof Array; // true

const foo = new Foo();
foo.constructor = "bar";
foo.constructor === "bar"; // true

// etc.
```

This does not overwrite the old `constructor` property — it was originally present on the instance's `[[Prototype]]`, not as its own property.

```js
const arr = [];
Object.hasOwn(arr, "constructor"); // false
Object.hasOwn(Object.getPrototypeOf(arr), "constructor"); // true

arr.constructor = String;
Object.hasOwn(arr, "constructor"); // true — the instance property shadows the one on its
prototype
```

But even when `Object.getPrototypeOf(a).constructor` is re-assigned, it won't change other behaviors of the object. For example, the behavior of `instanceof` is controlled by `Symbol.hasInstance`, not `constructor`:

```js
const arr = [];
arr.constructor = String;
arr instanceof String; // false
arr instanceof Array; // true
```

There is nothing protecting the `constructor` property from being re-assigned or shadowed, so using it to detect the type of a variable should usually be avoided in favor of less fragile ways like `instanceof` and `Symbol.toStringTag` for objects, or `typeof` for primitives.

### Changing the constructor of a constructor function's prototype

Every constructor has a `prototype` property, which will become the instance's `[[Prototype]]` when called via the `new` operator. `ConstructorFunction.prototype.constructor` will therefore become a property on the instance's `[[Prototype]]`, as previously demonstrated.

However, if `ConstructorFunction.prototype` is re-assigned, the `constructor` property will be lost. For example, the following is a common way to create an inheritance pattern:

```js
function Parent() {
  // …
}
Parent.prototype.parentMethod = function () {};

function Child() {
  Parent.call(this); // Make sure everything is initialized properly
}
// Pointing the [[Prototype]] of Child.prototype to Parent.prototype
Child.prototype = Object.create(Parent.prototype);
```

The `constructor` of instances of `Child` will be `Parent` due to `Child.prototype` being re-assigned.

This is usually not a big deal — the language almost never reads the `constructor` property of an object. The only exception is when using `[Symbol.species]` to create new instances of a class, but such cases are rare, and you should be using the `extends` syntax to subclass builtins anyway.

However, ensuring that `Child.prototype.constructor` always points to `Child` itself is crucial when some caller is using `constructor` to access the original class from an instance. Take the following case: the object has the `create()` method to create itself.

```js
function Parent() {
  // …
}
function CreatedConstructor() {
  Parent.call(this);
}

CreatedConstructor.prototype = Object.create(Parent.prototype);

CreatedConstructor.prototype.create = function () {
  return new this.constructor();
};

new CreatedConstructor().create().create(); // TypeError: new
CreatedConstructor().create().create is undefined, since constructor === Parent
```

In the example above, an exception is thrown, since the `constructor` links to `Parent`. To avoid this, just assign the necessary constructor you are going to use.

```js
function Parent() {
  // …
}
function CreatedConstructor() {
  // …
}
```

*(code continues on the next page)*

(code continued from the previous page)

```
CreatedConstructor.prototype = Object.create(Parent.prototype, {
  // Return original constructor to Child
  constructor: {
    value: CreatedConstructor,
    enumerable: false, // Make it non-enumerable, so it won't appear in `for...in` loop
    writable: true,
    configurable: true,
  },
});

CreatedConstructor.prototype.create = function () {
  return new this.constructor();
};

new CreatedConstructor().create().create(); // it's pretty fine
```

Note that when manually adding the `constructor` property, it's crucial to make the property non-enumerable, so `constructor` won't be visited in `for...in` loops — as it normally isn't.

If the code above looks like too much boilerplate, you may also consider using `Object.setPrototypeOf()` to manipulate the prototype chain.

js

```
function Parent() {
  // …
}
function CreatedConstructor() {
  // …
}

Object.setPrototypeOf(CreatedConstructor.prototype, Parent.prototype);

CreatedConstructor.prototype.create = function () {
  return new this.constructor();
};

new CreatedConstructor().create().create(); // still works without re-creating constructor
property
```

`Object.setPrototypeOf()` comes with its potential performance downsides because all previously created objects involved in the prototype chain have to be re-compiled; but if the above initialization code happens before `Parent` or `CreatedConstructor` are constructed, the effect should be minimal.

Let's consider one more involved case.

js

```
function ParentWithStatic() {}

ParentWithStatic.startPosition = { x: 0, y: 0 }; // Static member property
ParentWithStatic.getStartPosition = function () {
  return this.startPosition;
};

function Child(x, y) {
  this.position = { x, y };
}
```

(code continues on the next page)

(code continued from the previous page)

```
Child.prototype = Object.create(ParentWithStatic.prototype, {
  // Return original constructor to Child
  constructor: {
    value: Child,
    enumerable: false,
    writable: true,
    configurable: true,
  },
});

Child.prototype.getOffsetByInitialPosition = function () {
  const position = this.position;
  // Using this.constructor, in hope that getStartPosition exists as a static method
  const startPosition = this.constructor.getStartPosition();

  return {
    offsetX: startPosition.x - position.x,
    offsetY: startPosition.y - position.y,
  };
};

new Child(1, 1).getOffsetByInitialPosition();
// Error: this.constructor.getStartPosition is undefined, since the
// constructor is Child, which doesn't have the getStartPosition static method
```

For this example to work properly, we can reassign the `Parent`'s static properties to `Child`:

js

```
// …
Object.assign(Child, ParentWithStatic); // Notice that we assign it before we create() a
prototype below
Child.prototype = Object.create(ParentWithStatic.prototype, {
  // Return original constructor to Child
  constructor: {
    value: Child,
    enumerable: false,
    writable: true,
    configurable: true,
  },
});
// …
```

But even better, we can make the constructor functions themselves extend each other, as classes' `extends` do.

js

```
function ParentWithStatic() {}

ParentWithStatic.startPosition = { x: 0, y: 0 }; // Static member property
ParentWithStatic.getStartPosition = function () {
  return this.startPosition;
};
```

(code continues on the next page)

(code continued from the previous page)

```
function Child(x, y) {
  this.position = { x, y };
}

// Properly create inheritance!
Object.setPrototypeOf(Child.prototype, ParentWithStatic.prototype);
Object.setPrototypeOf(Child, ParentWithStatic);

Child.prototype.getOffsetByInitialPosition = function () {
  const position = this.position;
  const startPosition = this.constructor.getStartPosition();

  return {
    offsetX: startPosition.x - position.x,
    offsetY: startPosition.y - position.y,
  };
};

console.log(new Child(1, 1).getOffsetByInitialPosition()); // { offsetX: -1, offsetY: -1 }
```

Again, using `Object.setPrototypeOf()` may have adverse performance effects, so make sure it happens immediately after the constructor declaration and before any instances are created — to avoid objects being "tainted".

> **Note:** Manually updating or setting the constructor can lead to different and sometimes confusing consequences. To prevent this, just define the role of `constructor` in each specific case. In most cases, `constructor` is not used and reassigning it is not necessary.

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-object.prototype.constructor` |

## Browser compatibility

| | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| Feature | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `constructor` | 1 | 12 | 1 | 4 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Object.constructor`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `class`
- `constructor`
- Constructor

JavaScript › Reference › Standard built-in objects

# parseFloat()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/parseFloat · MDN page last modified 2025-07-08

**Baseline: Widely available** — available across major browsers since July 2015.

The **`parseFloat()`** function parses a string argument and returns a floating point number.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: parseFloat()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function circumference(r) {
  return parseFloat(r) * 2.0 * Math.PI;
}

console.log(circumference(4.567));
// Expected output: 28.695307297889173

console.log(circumference("4.567abcdefgh"));
// Expected output: 28.695307297889173

console.log(circumference("abcdefgh"));
// Expected output: NaN
```

## Syntax

js

```
parseFloat(string)
```

### Parameters

**`string`**

The value to parse, coerced to a string. Leading whitespace in this argument is ignored.

### Return value

A floating point number parsed from the given `string`, or `NaN` when the first non-whitespace character cannot be converted to a number.

> **Note:** JavaScript does not have the distinction of "floating point numbers" and "integers" on the language level. `parseInt()` and `parseFloat()` only differ in their parsing behavior, but not necessarily their return values. For example, `parseInt("42")` and `parseFloat("42")` would return the same value: a `Number` 42.

## Description

The `parseFloat` function converts its first argument to a string, parses that string as a decimal number literal, then returns a number or `NaN`. The number syntax it accepts can be summarized as:

- The characters accepted by `parseFloat()` are plus sign (`+`), minus sign (`-` U+002D HYPHEN-MINUS), decimal digits (`0` – `9`), decimal point (`.`), exponent indicator (`e` or `E`), and the `"Infinity"` literal.
- The `+`/`-` signs can only appear strictly at the beginning of the string, or immediately following the `e`/`E` character. The decimal point can only appear once, and only before the `e`/`E` character. The `e`/`E`

character can only appear once, and only if there is at least one digit before it.
- Leading spaces in the argument are trimmed and ignored.
- `parseFloat()` can also parse and return `Infinity` or `-Infinity` if the string starts with `"Infinity"` or `"-Infinity"` preceded by none or more white spaces.
- `parseFloat()` picks the longest substring starting from the beginning that generates a valid number literal. If it encounters an invalid character, it returns the number represented up to that point, ignoring the invalid character and all characters following it.
- If the argument's first character can't start a legal number literal per the syntax above, `parseFloat` returns `NaN`.

Syntax-wise, `parseFloat()` parses a subset of the syntax that the `Number()` function accepts. Namely, `parseFloat()` does not support non-decimal literals with `0x`, `0b`, or `0o` prefixes but supports everything else. However, `parseFloat()` is more lenient than `Number()` because it ignores trailing invalid characters, which would cause `Number()` to return `NaN`.

Similar to number literals and `Number()`, the number returned from `parseFloat()` may not be exactly equal to the number represented by the string, due to floating point range and inaccuracy. For numbers outside the `-1.7976931348623158e+308` – `1.7976931348623158e+308` range (see `Number.MAX_VALUE`), `-Infinity` or `Infinity` is returned.

## Examples

### Using parseFloat()

The following examples all return `3.14`:

```js
parseFloat(3.14);
parseFloat("3.14");
parseFloat("  3.14  ");
parseFloat("314e-2");
parseFloat("0.0314E+2");
parseFloat("3.14some non-digit characters");
parseFloat({
  toString() {
    return "3.14";
  },
});
```

### parseFloat() returning NaN

The following example returns `NaN`:

```js
parseFloat("FF2");
```

Anecdotally, because the string `NaN` itself is invalid syntax as accepted by `parseFloat()`, passing `"NaN"` returns `NaN` as well.

```js
parseFloat("NaN"); // NaN
```

### Returning Infinity

Infinity values are returned when the number is outside the double-precision 64-bit IEEE 754-2019 format range:

```js
parseFloat("1.7976931348623159e+308"); // Infinity
parseFloat("-1.7976931348623159e+308"); // -Infinity
```

Infinity is also returned when the string starts with `"Infinity"` or `"-Infinity"`:

```js
parseFloat("Infinity"); // Infinity
parseFloat("-Infinity"); // -Infinity
```

### Interaction with BigInt values

`parseFloat()` does not handle `BigInt` values. It stops at the `n` character, and treats the preceding string as a normal integer, with possible loss of precision. If a BigInt value is passed to `parseFloat()`, it will be converted to a string, and the string will be parsed as a floating-point number, which may result in loss of precision as well.

**js · ✗ bad example**

```
parseFloat(900719925474099267n); // 900719925474099300
parseFloat("900719925474099267n"); // 900719925474099300
```

You should pass the string to the `BigInt()` function instead, without the trailing `n` character.

**js · ✓ good example**

```
BigInt("900719925474099267");
// 900719925474099267n
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/global-object.html#sec-parsefloat-string |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `parseFloat` | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.parseFloat`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- parseInt()
- Number.parseFloat()
- Number.parseInt()
- Number.prototype.toFixed()

JavaScript › Reference › Standard built-in objects

# parseInt()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/parseInt · MDN page last modified 2025-07-08

**Baseline: Widely available** — available across major browsers since July 2015.

The **`parseInt()`** function parses a string argument and returns an integer of the specified radix (the base in mathematical numeral systems).

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: parseInt()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(parseInt("123"));
// 123 (default base-10)
console.log(parseInt("123", 10));
// 123 (explicitly specify base-10)
console.log(parseInt("   123 "));
// 123 (whitespace is ignored)
console.log(parseInt("077"));
// 77 (leading zeros are ignored)
console.log(parseInt("1.9"));
// 1 (decimal part is truncated)
console.log(parseInt("ff", 16));
// 255 (lower-case hexadecimal)
console.log(parseInt("0xFF", 16));
// 255 (upper-case hexadecimal with "0x" prefix)
console.log(parseInt("xyz"));
// NaN (input can't be converted to an integer)
```

## Syntax

js

```
parseInt(string)
parseInt(string, radix)
```

### Parameters

**`string`**

A string starting with an integer. Leading whitespace in this argument is ignored.

**`radix`** Optional

An integer between `2` and `36` that represents the *radix* (the base in mathematical numeral systems) of the `string`. It is converted to a 32-bit integer; if it's nonzero and outside the range of [2, 36] after conversion, the function will always return `NaN`. If `0` or not provided, the radix will be inferred based on `string`'s value. Be careful — this does *not* always default to `10`! The description below explains in more detail what happens when `radix` is not provided.

### Return value

An integer parsed from the given `string`, or `NaN` when

- the `radix` as a 32-bit integer is smaller than `2` or bigger than `36`, or
- the first non-whitespace character cannot be converted to a number.

> **Note:** JavaScript does not have the distinction of "floating point numbers" and "integers" on the language level. `parseInt()` and `parseFloat()` only differ in their parsing behavior, but not necessarily their return values. For example, `parseInt("42")` and `parseFloat("42")` would return the same value: a `Number` 42.

## Description

The `parseInt` function converts its first argument to a string, parses that string, then returns an integer or `NaN`.

If not `NaN`, the return value will be the integer that is the first argument taken as a number in the specified `radix`. (For example, a `radix` of `10` converts from a decimal number, `8` converts from octal, `16` from hexadecimal, and so on.)

The `radix` argument is converted to a number. If it's unprovided, or if the value becomes 0, `NaN` or `Infinity` (`undefined` is coerced to `NaN`), JavaScript assumes the following:

1. If the input `string`, with leading whitespace and possible `+/-` signs removed, begins with `0x` or `0X` (a zero, followed by lowercase or uppercase X), `radix` is assumed to be `16` and the rest of the string is parsed as a hexadecimal number.
2. If the input `string` begins with any other value, the radix is `10` (decimal).

> **Note:** Other prefixes like `0b`, which are valid in number literals, are treated as normal digits by `parseInt()`. `parseInt()` does *not* treat strings beginning with a `0` character as octal values either. The only prefix that `parseInt()` recognizes is `0x` or `0X` for hexadecimal values — everything else is parsed as a decimal value if `radix` is missing. `Number()` or `BigInt()` can be used instead to parse these prefixes.

If the radix is `16`, `parseInt()` allows the string to be optionally prefixed by `0x` or `0X` after the optional sign character (`+/-`).

If the radix value (coerced if necessary) is not in range [2, 36] (inclusive) `parseInt` returns `NaN`.

For radices above `10`, letters of the English alphabet indicate numerals greater than `9`. For example, for hexadecimal numbers (base `16`), `A` through `F` are used. The letters are case-insensitive.

`parseInt` understands exactly two signs: `+` for positive, and `-` for negative. It is done as an initial step in the parsing after whitespace is removed. If no signs are found, the algorithm moves to the following step; otherwise, it removes the sign and runs the number-parsing on the rest of the string.

If `parseInt` encounters a character in the input string that is not a valid numeral in the specified `radix`, it ignores it and all succeeding characters and returns the integer value parsed up to that point. For example, `parseInt("2", 2)` returns `NaN` because `2` is not a valid numeral in the binary number system. Likewise, although `1e3` technically encodes an integer (and will be correctly parsed to the integer `1000` by `parseFloat()`), `parseInt("1e3", 10)` returns `1`, because `e` is not a valid numeral in base 10. Because `.` is not a numeral either, the return value will always be an integer.

If the first character cannot be converted to a number with the radix in use, `parseInt` returns `NaN`. Leading whitespace is allowed.

For arithmetic purposes, the `NaN` value is not a number in any radix. You can call the `Number.isNaN` function to determine if the result of `parseInt` is `NaN`. If `NaN` is passed on to arithmetic operations, the operation result will also be `NaN`.

Because large numbers use the `e` character in their string representation (e.g., `6.022e23` for $6.022 \times 10^{23}$), using `parseInt` to truncate numbers will produce unexpected results when used on very large or very small numbers. `parseInt` should *not* be used as a substitute for `Math.trunc()`.

To convert a number to its string literal in a particular radix, use `thatNumber.toString(radix)`.

Because `parseInt()` returns a number, it may suffer from loss of precision if the integer represented by the string is outside the safe range. The `BigInt()` function supports parsing integers of arbitrary length accurately, by returning a `BigInt`.

## Examples

### Using parseInt()

The following examples all return `15`:

```js
parseInt("0xF", 16);
parseInt("F", 16);
parseInt("17", 8);
parseInt("015", 10);
parseInt("15,123", 10);
parseInt("FXX123", 16);
parseInt("1111", 2);
parseInt("15 * 3", 10);
parseInt("15e2", 10);
parseInt("15px", 10);
parseInt("12", 13);
```

The following examples all return `NaN`:

```js
parseInt("Hello", 8); // Not a number at all
parseInt("546", 2); // Digits other than 0 or 1 are invalid for binary radix
```

The following examples all return `-15`:

```js
parseInt("-F", 16);
parseInt("-0F", 16);
parseInt("-0XF", 16);
parseInt("-17", 8);
parseInt("-15", 10);
parseInt("-1111", 2);
parseInt("-15e1", 10);
parseInt("-12", 13);
```

The following example returns `224`:

```js
parseInt("0e0", 16);
```

`parseInt()` does not handle `BigInt` values. It stops at the `n` character, and treats the preceding string as a normal integer, with possible loss of precision.

js · ✗ bad example

```
parseInt("900719925474099267n");
// 900719925474099300
```

You should pass the string to the `BigInt()` function instead, without the trailing `n` character.

js · ✓ good example

```
BigInt("900719925474099267");
// 900719925474099267n
```

`parseInt` doesn't work with numeric separators:

js · ✗ bad example

```
parseInt("123_456"); // 123
```

### Using parseInt() on non-strings

`parseInt()` can have interesting results when working on non-strings combined with a high radix; for example, `36` (which makes all alphanumeric characters valid numeric digits).

js

```
parseInt(null, 36); // 1112745: The string "null" is 1112745 in base 36
parseInt(undefined, 36); // 86464843759093: The string "undefined" is 86464843759093 in base
36
```

In general, it's a bad idea to use `parseInt()` on non-strings, especially to use it as a substitution for `Math.trunc()`. It may work on small numbers:

js

```
parseInt(15.99, 10); // 15
parseInt(-15.1, 10); // -15
```

However, it only happens to work because the string representation of these numbers uses basic fractional notation (`"15.99"`, `"-15.1"`), where `parseInt()` stops at the decimal point. Numbers greater than or equal to 1e+21 or less than or equal to 1e-7 use exponential notation (`"1.5e+22"`, `"1.51e-8"`) in their string representation, and `parseInt()` will stop at the `e` character or decimal point, which always comes after the first digit. This means for large and small numbers, `parseInt()` will return a one-digit integer:

js · ✗ bad example

```
parseInt(4.7 * 1e22, 10); // Very large number becomes 4
parseInt(0.00000000000434, 10); // Very small number becomes 4

parseInt(0.0000001, 10); // 1
parseInt(0.000000123, 10); // 1
parseInt(1e-7, 10); // 1
parseInt(100000000000000000000, 10); // 1
parseInt(123000000000000000000000, 10); // 1
parseInt(1e21, 10); // 1
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/global-object.html#sec-parseint-string-radix |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `parseInt` | 1 | 12 | 1 | 3 | 1 | 18 | 4 | 10.1 | 1 | 1.0 | 4.4 | 1 | 1.0.0 | 1.0 | 0.10.0 |
| Parses leading-zero strings as decimal, not octal | 23 | 12 | 21 | 15 | 6 | 25 | 21 | 14 | 6 | 1.5 | 4.4 | 6 | 1.0.0 | 1.0 | 0.10.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.parseInt`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `parseFloat()`
- `Number()` constructor
- `Number.parseFloat()`
- `Number.parseInt()`
- `isNaN()`
- `Number.prototype.toString()`
- `Object.prototype.valueOf()`
- `BigInt()` constructor

JavaScript › Reference › Standard built-in objects

# Promise

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Promise`** object represents the eventual completion (or failure) of an asynchronous operation and its resulting value.

To learn about the way promises work and how you can use them, we advise you to read Using promises first.

## Description

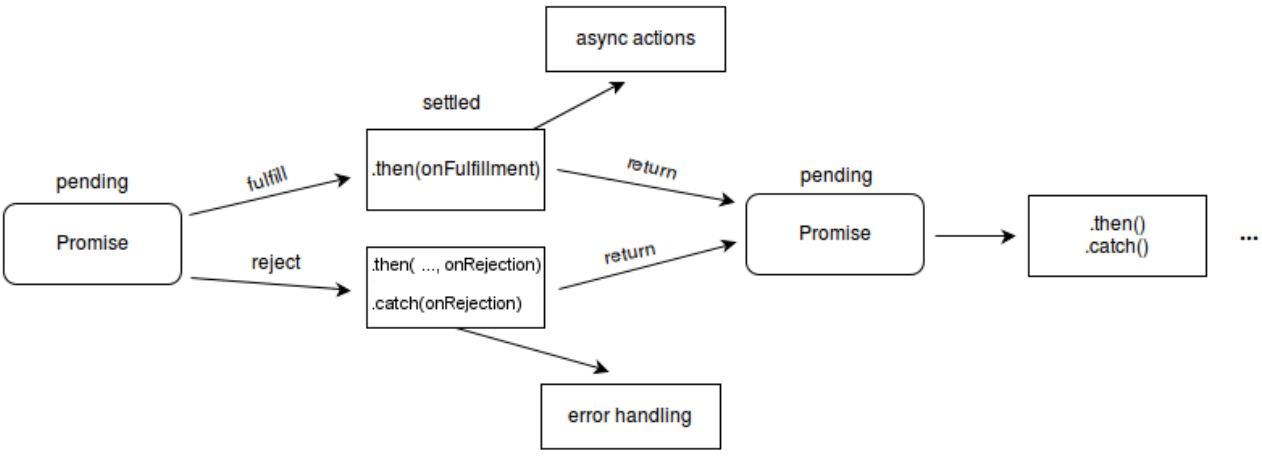
A `Promise` is a proxy for a value not necessarily known when the promise is created. It allows you to associate handlers with an asynchronous action's eventual success value or failure reason. This lets asynchronous methods return values like synchronous methods: instead of immediately returning the final value, the asynchronous method returns a *promise* to supply the value at some point in the future.

A `Promise` is in one of these states:

- *pending*: initial state, neither fulfilled nor rejected.
- *fulfilled*: meaning that the operation was completed successfully.
- *rejected*: meaning that the operation failed.

The *eventual state* of a pending promise can either be *fulfilled* with a value or *rejected* with a reason (error). When either of these options occurs, the associated handlers queued up by a promise's `then` method are called. If the promise has already been fulfilled or rejected when a corresponding handler is attached, the handler will be called, so there is no race condition between an asynchronous operation completing and its handlers being attached.

A promise is said to be *settled* if it is either fulfilled or rejected, but not pending.




You will also hear the term *resolved* used with promises — this means that the promise is settled or "locked-in" to match the eventual state of another promise, and further resolving or rejecting it has no effect. The States and fates document from the original Promise proposal contains more details about promise terminology. Colloquially, "resolved" promises are often equivalent to "fulfilled" promises, but as illustrated in "States and fates", resolved promises can be pending or rejected as well. For example:

```js
new Promise((resolveOuter) => {
  resolveOuter(
    new Promise((resolveInner) => {
      setTimeout(resolveInner, 1000);
    }),
  );
});
```

This promise is already *resolved* at the time when it's created (because the `resolveOuter` is called synchronously), but it is resolved with another promise, and therefore won't be *fulfilled* until 1 second later, when the inner promise fulfills. In practice, the "resolution" is often done behind the scenes and not observable, and only its fulfillment or rejection are.

> **Note:** Several other languages have mechanisms for lazy evaluation and deferring a computation, which they also call "promises", e.g., Scheme. Promises in JavaScript represent processes that are already happening, which can be chained with callback functions. If you are looking to lazily evaluate an expression, consider using a function with no arguments e.g., `f = () => expression` to create the lazily-evaluated expression, and `f()` to evaluate the expression immediately.

`Promise` itself has no first-class protocol for cancellation, but you may be able to directly cancel the underlying asynchronous operation, typically using `AbortController`.

### Chained Promises

The promise methods `then()`, `catch()`, and `finally()` are used to associate further action with a promise that becomes settled. The `then()` method takes up to two arguments; the first argument is a callback function for the fulfilled case of the promise, and the second argument is a callback function for the rejected case. The `catch()` and `finally()` methods call `then()` internally and make error handling less verbose. For example, a `catch()` is really just a `then()` without passing the fulfillment handler. As these methods return promises, they can be chained. For example:

```js
const myPromise = new Promise((resolve, reject) => {
  setTimeout(() => {
    resolve("foo");
  }, 300);
});

myPromise
  .then(handleFulfilledA, handleRejectedA)
  .then(handleFulfilledB, handleRejectedB)
  .then(handleFulfilledC, handleRejectedC);
```

We will use the following terminology: *initial promise* is the promise on which `then` is called; *new promise* is the promise returned by `then`. The two callbacks passed to `then` are called *fulfillment handler* and *rejection handler*, respectively.

The settled state of the initial promise determines which handler to execute.

- If the initial promise is fulfilled, the fulfillment handler is called with the fulfillment value.
- If the initial promise is rejected, the rejection handler is called with the rejection reason.

The completion of the handler determines the settled state of the new promise.

- If the handler returns a thenable value, the new promise settles in the same state as the returned value.
- If the handler returns a non-thenable value, the new promise is fulfilled with the returned value.
- If the handler throws an error, the new promise is rejected with the thrown error.
- If the initial promise has no corresponding handler attached, the new promise will settle to the same state as the initial promise — that is, without a rejection handler, a rejected promise stays rejected with the same reason.

For example, in the code above, if `myPromise` rejects, `handleRejectedA` will be called, and if `handleRejectedA` completes normally (without throwing or returning a rejected promise), the promise returned by the first `then` will be fulfilled instead of staying rejected. Therefore, if an error must be handled immediately, but we want to maintain the error state down the chain, we must throw an error of some type in the rejection handler. On the other hand, in the absence of an immediate need, we can leave out error handling until the final `catch()` handler.

```js
myPromise
  .then(handleFulfilledA)
  .then(handleFulfilledB)
  .then(handleFulfilledC)
  .catch(handleRejectedAny);
```

Using arrow functions for the callback functions, implementation of the promise chain might look something like this:

```js
myPromise
  .then((value) => `${value} and bar`)
  .then((value) => `${value} and bar again`)
  .then((value) => `${value} and again`)
  .then((value) => `${value} and again`)
  .then((value) => {
    console.log(value);
  })
  .catch((err) => {
    console.error(err);
  });
```

> **Note:** For faster execution, all synchronous actions should preferably be done within one handler, otherwise it would take several ticks to execute all handlers in sequence.

JavaScript maintains a job queue. Each time, JavaScript picks a job from the queue and executes it to completion. The jobs are defined by the executor of the `Promise()` constructor, the handlers passed to `then`, or any platform API that returns a promise. The promises in a chain represent the dependency relationship between these jobs. When a promise settles, the respective handlers associated with it are added to the back of the job queue.

A promise can participate in more than one chain. For the following code, the fulfillment of `promiseA` will cause both `handleFulfilled1` and `handleFulfilled2` to be added to the job queue. Because `handleFulfilled1` is registered first, it will be invoked first.

```js
const promiseA = new Promise(myExecutorFunc);
const promiseB = promiseA.then(handleFulfilled1, handleRejected1);
const promiseC = promiseA.then(handleFulfilled2, handleRejected2);
```

An action can be assigned to an already settled promise. In this case, the action is added immediately to the back of the job queue and will be performed when all existing jobs are completed. Therefore, an action for an already "settled" promise will occur only after the current synchronous code completes and at least one loop-tick has passed. This guarantees that promise actions are asynchronous.

```js
const promiseA = new Promise((resolve, reject) => {
  resolve(777);
});
// At this point, "promiseA" is already settled.
promiseA.then((val) => console.log("asynchronous logging has val:", val));
console.log("immediate logging");

// produces output in this order:
// immediate logging
// asynchronous logging has val: 777
```

### Thenables

The JavaScript ecosystem had made multiple Promise implementations long before it became part of the language. Despite being represented differently internally, at the minimum, all Promise-like objects implement the *Thenable* interface. A thenable implements the `.then()` method, which is called with two callbacks: one for when the promise is fulfilled, one for when it's rejected. Promises are thenables as well.

To interoperate with the existing Promise implementations, the language allows using thenables in place of promises. For example, `Promise.resolve` will not only resolve promises, but also trace thenables.

```js
// This is not a Promises/A+ compliant thenable! It calls onFulfilled
// synchronously. For demonstration only.
const thenable = {
  then(onFulfilled, onRejected) {
    onFulfilled({
      // The thenable is fulfilled with another thenable
      then(onFulfilled, onRejected) {
        onFulfilled(42);
      },
    });
  },
};

Promise.resolve(thenable); // A promise fulfilled with 42
```

The `then()` method is responsible for scheduling the execution of the provided `onFulfilled` and `onRejected` callbacks. Its semantics, including error handling and asynchronicity, are precisely defined in the Promises/A+ specification, and we shall not repeat them here. It's very rare that you need to implement a thenable yourself; even if you are not using native promises, you would probably be using a Promise library such as Bluebird.

### Promise concurrency

The `Promise` class offers four main static methods to facilitate async task concurrency:

**`Promise.all()`**
Fulfills when **all** of the promises fulfill; rejects when **any** of the promises rejects.

**`Promise.allSettled()`**
Fulfills when **all** promises settle.

**`Promise.any()`**
Fulfills when **any** of the promises fulfills; rejects when **all** of the promises reject.

**`Promise.race()`**
Settles when **any** of the promises settles. In other words, fulfills when any of the promises fulfills; rejects when any of the promises rejects.

All these methods take an iterable of promises (thenables, to be exact) and return a new promise. They all support subclassing, which means they can be called on subclasses of `Promise`, and the result will be a promise of the subclass type. To do so, the subclass's constructor must implement the same signature as the `Promise()` constructor — accepting a single `executor` function that can be called with the `resolve` and `reject` callbacks as parameters. The subclass must also have a `resolve` static method that can be called like `Promise.resolve()` to resolve values to promises.

There are two other convenience static methods: `Promise.allKeyed()` and `Promise.allSettledKeyed()`, that behave like `Promise.all()` and `Promise.allSettled()`, but take *objects* of promises and return promises that fulfill with *objects* of the same shape. By working with objects instead of arrays, you can associate results with semantically meaningful keys, instead of arbitrary array ordering which can be difficult to maintain.

These methods attach handlers to each input promise using `then()`. Even when the resulting promise has settled early (such as when one input in `Promise.race()` settles), the other handlers are not removed. Repeatedly passing the same pending promise to concurrency methods can accumulate handlers even when those handlers are never used:

```js
const pendingPromise = new Promise(() => {});

for (let i = 0; i < 1000; i++) {
  await Promise.race([Promise.resolve(0), pendingPromise]);
}
// All tasks have completed, but pendingPromise retains the
// handlers attached by all 1000 races.
```

Promises do not provide a way to unsubscribe these handlers; they remain attached while the input promise is pending and reachable. Where possible, cancel the underlying operation by using an `AbortSignal` when the pending promise is no longer useful.

Note that JavaScript is single-threaded by nature, so at a given instant, only one task will be executing, although control can shift between different promises, making execution of the promises appear concurrent. Parallel execution in JavaScript can only be achieved through worker threads.

## Constructor

**`Promise()`**

Creates a new `Promise` object. The constructor is primarily used to wrap functions that do not already support promises.

## Static properties

**`Promise[Symbol.species]`**

Returns the constructor used to construct return values from promise methods.

## Static methods

**`Promise.all()`**

Takes an iterable of promises as input and returns a single `Promise`. This returned promise fulfills when all of the input's promises fulfill (including when an empty iterable is passed), with an array of the fulfillment values. It rejects when any of the input's promises reject, with this first rejection reason.

**`Promise.allKeyed()`** Experimental

Like `Promise.all()`, except that it takes an object of promises and returns a promise that fulfills with an object of the same shape, allowing you to associate results with semantically meaningful keys.

**`Promise.allSettled()`**

Takes an iterable of promises as input and returns a single `Promise`. This returned promise fulfills when all of the input's promises settle (including when an empty iterable is passed), with an array of objects that describe the outcome of each promise.

**`Promise.allSettledKeyed()`** Experimental

Like `Promise.allSettled()`, except that it takes an object of promises and returns a promise that fulfills with an object of the same shape, allowing you to associate results with semantically meaningful keys.

**`Promise.any()`**

Takes an iterable of promises as input and returns a single `Promise`. This returned promise fulfills when any of the input's promises fulfill, with this first fulfillment value. It rejects when all of the input's promises reject (including when an empty iterable is passed), with an `AggregateError` containing an array of rejection reasons.

**`Promise.race()`**

Takes an iterable of promises as input and returns a single `Promise`. This returned promise settles with the eventual state of the first promise that settles.

**`Promise.reject()`**

Returns a new `Promise` object that is rejected with the given reason.

**`Promise.resolve()`**

Returns a `Promise` object that is resolved with the given value. If the value is a thenable (i.e., has a `then` method), the returned promise will "follow" that thenable, adopting its eventual state; otherwise, the returned promise will be fulfilled with the value.

**`Promise.try()`**

Takes a callback of any kind (returns or throws, synchronously or asynchronously) and resolves its result to a `Promise`.

**`Promise.withResolvers()`**

Returns an object containing a new `Promise` object and two functions to resolve or reject it, corresponding to the two parameters passed to the executor of the `Promise()` constructor.

## Instance properties

These properties are defined on `Promise.prototype` and shared by all `Promise` instances.

**`Promise.prototype.constructor`**

The constructor function that created the instance object. For `Promise` instances, the initial value is the `Promise` constructor.

**`Promise.prototype[Symbol.toStringTag]`**

The initial value of the `[Symbol.toStringTag]` property is the string `"Promise"`. This property is used in `Object.prototype.toString()`.

## Instance methods

**`Promise.prototype.catch()`**

Appends a rejection handler callback to the promise, and returns a new promise resolving to the return value of the callback if it is called, or to its original fulfillment value if the promise is instead fulfilled.

**`Promise.prototype.finally()`**

Appends a handler to the promise, and returns a new promise that is resolved when the original promise is resolved. The handler is called when the promise is settled, whether fulfilled or rejected.

**`Promise.prototype.then()`**

Appends fulfillment and rejection handlers to the promise, and returns a new promise resolving to the return value of the called handler, or to its original settled value if the promise was not handled (i.e., if the relevant handler `onFulfilled` or `onRejected` is not a function).

## Examples

### Basic Example

In this example, we use `setTimeout(...)` to simulate async code. In reality, you will probably be using something like XHR or an HTML API.

```js
const myFirstPromise = new Promise((resolve, reject) => {
  // We call resolve(...) when what we were doing asynchronously
  // was successful, and reject(...) when it failed.
  setTimeout(() => {
    resolve("Success!"); // Yay! Everything went well!
  }, 250);
});

myFirstPromise.then((successMessage) => {
  // successMessage is whatever we passed in the resolve(...) function above.
  // It doesn't have to be a string, but if it is only a succeed message, it probably will
be.
  console.log(`Yay! ${successMessage}`);
});
```

### Example with diverse situations

This example shows diverse techniques for using Promise capabilities and diverse situations that can occur. To understand this, start by scrolling to the bottom of the code block, and examine the promise chain. Upon provision of an initial promise, a chain of promises can follow. The chain is composed of `.then()` calls, and typically (but not necessarily) has a single `.catch()` at the end, optionally followed by `.finally()`. In this example, the promise chain is initiated by a custom-written `new Promise()` construct; but in actual practice, promise chains more typically start with an API function (written by someone else) that returns a promise.

The example function `tetheredGetNumber()` shows that a promise generator will utilize `reject()` while setting up an asynchronous call, or within the call-back, or both. The function `promiseGetWord()` illustrates how an API function might generate and return a promise in a self-contained manner.

Note that the function `troubleWithGetNumber()` ends with a `throw`. That is forced because a promise chain goes through all the `.then()` promises, even after an error, and without the `throw`, the error would seem "fixed". This is a hassle, and for this reason, it is common to omit `onRejected` throughout the chain of `.then()` promises, and just have a single `onRejected` in the final `catch()`.

This code can be run under Node.js. Comprehension is enhanced by seeing the errors actually occur. To force more errors, change the `threshold` values.

```js
// To experiment with error handling, "threshold" values cause errors randomly
const THRESHOLD_A = 8; // can use zero 0 to guarantee error

function tetheredGetNumber(resolve, reject) {
  setTimeout(() => {
    const randomInt = Date.now();
    const value = randomInt % 10;
    if (value < THRESHOLD_A) {
      resolve(value);
    } else {
      reject(new RangeError(`Too large: ${value}`));
    }
  }, 500);
}
```

*(code continues on the next page)*

(code continued from the previous page)

```
function determineParity(value) {
  const isOdd = value % 2 === 1;
  return { value, isOdd };
}

function troubleWithGetNumber(reason) {
  const err = new Error("Trouble getting number", { cause: reason });
  console.error(err);
  throw err;
}

function promiseGetWord(parityInfo) {
  return new Promise((resolve, reject) => {
    const { value, isOdd } = parityInfo;
    if (value >= THRESHOLD_A - 1) {
      reject(new RangeError(`Still too large: ${value}`));
    } else {
      parityInfo.wordEvenOdd = isOdd ? "odd" : "even";
      resolve(parityInfo);
    }
  });
}

new Promise(tetheredGetNumber)
  .then(determineParity, troubleWithGetNumber)
  .then(promiseGetWord)
  .then((info) => {
    console.log(`Got: ${info.value}, ${info.wordEvenOdd}`);
    return info;
  })
  .catch((reason) => {
    if (reason.cause) {
      console.error("Had previously handled error");
    } else {
      console.error(`Trouble with promiseGetWord(): ${reason}`);
    }
  })
  .finally((info) => console.log("All done"));
```

## Advanced Example

This small example shows the mechanism of a `Promise`. The `testPromise()` method is called each time the `<button>` is clicked. It creates a promise that will be fulfilled, using `setTimeout()`, to the promise count (number starting from 1) every 1-3 seconds, at random. The `Promise()` constructor is used to create the promise.

The fulfillment of the promise is logged, via a fulfill callback set using `p1.then()`. A few logs show how the synchronous part of the method is decoupled from the asynchronous completion of the promise.

By clicking the button several times in a short amount of time, you'll even see the different promises being fulfilled one after another.

### HTML

html

```
<button id="make-promise">Make a promise!</button>
<div id="log"></div>
```

### JavaScript

js

```
"use strict";

let promiseCount = 0;

function testPromise() {
  const thisPromiseCount = ++promiseCount;
  const log = document.getElementById("log");
  // begin
  log.insertAdjacentHTML("beforeend", `${thisPromiseCount}) Started<br>`);
  // We make a new promise: we promise a numeric count of this promise,
  // starting from 1 (after waiting 3s)
  const p1 = new Promise((resolve, reject) => {
    // The executor function is called with the ability
    // to resolve or reject the promise
    log.insertAdjacentHTML(
      "beforeend",
      `${thisPromiseCount}) Promise constructor<br>`,
    );
    // This is only an example to create asynchronism
    setTimeout(
      () => {
        // We fulfill the promise
        resolve(thisPromiseCount);
      },
      Math.random() * 2000 + 1000,
    );
  });

  // We define what to do when the promise is resolved with the then() call,
  // and what to do when the promise is rejected with the catch() call
  p1.then((val) => {
    // Log the fulfillment value
    log.insertAdjacentHTML("beforeend", `${val}) Promise fulfilled<br>`);
  }).catch((reason) => {
    // Log the rejection reason
    console.log(`Handle rejected promise (${reason}) here.`);
  });
  // end
  log.insertAdjacentHTML("beforeend", `${thisPromiseCount}) Promise made<br>`);
}

const btn = document.getElementById("make-promise");
btn.addEventListener("click", testPromise);
```

### Result

*▷ Live sample output ("Advanced Example sample") is rendered in the browser on MDN and cannot be reproduced on paper. See it running at developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise#advanced_example .*

## Loading an image with XHR

Another example using `Promise` and `XMLHttpRequest` to load an image is shown below. Each step is commented on and allows you to follow the Promise and XHR architecture closely.

html · hidden on the MDN page (supporting code for the live sample)

```
<h1>Promise example</h1>
```

js

```
function imgLoad(url) {
  // Create new promise with the Promise() constructor;
  // This has as its argument a function with two parameters, resolve and reject
  return new Promise((resolve, reject) => {
    // XHR to load an image
    const request = new XMLHttpRequest();
    request.open("GET", url);
    request.responseType = "blob";
    // When the request loads, check whether it was successful
    request.onload = () => {
      if (request.status === 200) {
        // If successful, resolve the promise by passing back the request response
        resolve(request.response);
      } else {
        // If it fails, reject the promise with an error message
        reject(
          Error(
            `Image didn't load successfully; error code: + ${request.statusText}`,
          ),
        );
      }
    };
    // Handle network errors
    request.onerror = () => reject(new Error("There was a network error."));
    // Send the request
    request.send();
  });
}

// Get a reference to the body element, and create a new image object
const body = document.querySelector("body");
const myImage = new Image();
const imgUrl =
  "https://mdn.github.io/shared-assets/images/examples/round-balloon.png";

// Call the function with the URL we want to load, then chain the
// promise then() method with two callbacks
imgLoad(imgUrl).then(
  (response) => {
    // The first runs when the promise resolves, with the request.response
    // specified within the resolve() method.
    const imageURL = URL.createObjectURL(response);
    myImage.src = imageURL;
    body.appendChild(myImage);
  },
  (error) => {
    // The second runs when the promise
    // is rejected, and logs the Error specified with the reject() method.
    console.log(error);
  },
);
```

▷ *Live sample output ("promises sample") is rendered in the browser on MDN and cannot be reproduced on paper. See it running at developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise#promises .*

### Incumbent settings object tracking

A settings object is an environment that provides additional information when JavaScript code is running. This includes the realm and module map, as well as HTML specific information such as the origin. The incumbent settings object is tracked in order to ensure that the browser knows which one to use for a given piece of user code.

To better picture this, we can take a closer look at how the realm might be an issue. A **realm** can be roughly thought of as the global object. What is unique about realms is that they hold all of the necessary information to run JavaScript code. This includes objects like `Array` and `Error`. Each settings object has its own "copy" of these and they are not shared. That can cause some unexpected behavior in relation to promises. In order to get around this, we track something called the **incumbent settings object**. This represents information specific to the context of the user code responsible for a certain function call.

To illustrate this a bit further we can take a look at how an `<iframe>` embedded in a document communicates with its host. Since all web APIs are aware of the incumbent settings object, the following will work in all browsers:

html

```
<!doctype html>
<iframe></iframe>
<!-- we have a realm here -->
<script>
  // we have a realm here as well
  const bound = frames[0].postMessage.bind(frames[0], "some data", "*");
  // bound is a built-in function – there is no user
  // code on the stack, so which realm do we use?
  setTimeout(bound);
  // this still works, because we use the youngest
  // realm (the incumbent) on the stack
</script>
```

The same concept applies to promises. If we modify the above example a little bit, we get this:

html

```
<!doctype html>
<iframe></iframe>
<!-- we have a realm here -->
<script>
  // we have a realm here as well
  const bound = frames[0].postMessage.bind(frames[0], "some data", "*");
  // bound is a built in function – there is no user
  // code on the stack – which realm do we use?
  Promise.resolve(undefined).then(bound);
  // this still works, because we use the youngest
  // realm (the incumbent) on the stack
</script>
```

If we change this so that the `<iframe>` in the document is listening to post messages, we can observe the effect of the incumbent settings object:

html

```
<!-- y.html -->
<!doctype html>
<iframe src="x.html"></iframe>
<script>
  const bound = frames[0].postMessage.bind(frames[0], "some data", "*");
  Promise.resolve(undefined).then(bound);
</script>
```

html

```
<!-- x.html -->
<!doctype html>
<script>
  window.addEventListener("message", (event) => {
    document.querySelector("#text").textContent = "hello";
    // this code will only run in browsers that track the incumbent settings object
    console.log(event);
  });
</script>
```

In the above example, the inner text of the `<iframe>` will be updated only if the incumbent settings object is tracked. This is because without tracking the incumbent, we may end up using the wrong environment to send the message.

> **Note:** Currently, incumbent realm tracking is fully implemented in Firefox, and has partial implementations in Chrome and Safari.

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-promise-objects` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `Promise` | 32 | 12 | 29 | 19 | 8 | 32 | 29 | 19 | 8 | 2.0 | 4.4.3 | 8 | 1.0.0 | 1.0 | 0.12.0 |
| [Symbol.species] | 51 | 79 | 48 | 38 | 10 | 51 | 48 | 41 | 10 | 5.0 | 51 | 10 | 1.0.0 | 1.0 | 6.5.0 |
| `Promise()` constructor | 32 | 12 | 29[1] | 19 | 8[2] | 32 | 29[1] | 19 | 8[2] | 2.0 | 4.4.3 | 8[2] | 1.0.0 | 1.0 | 0.12.0[3] |
| `all()` | 32 | 12 | 29 | 19 | 8 | 32 | 29 | 19 | 8 | 2.0 | 4.4.3 | 8 | 1.0.0 | 1.0 | 0.12.0 |
| `allKeyed` Experimental | **No** | **No** | 155 | **No** | **No** | **No** | 155 | **No** | **No** | **No** | **No** | **No** | ? | ? | ? |
| `allSettled()` | 76 | 79 | 71 | 63 | 13 | 76 | 79 | 54 | 13 | 12.0 | 76 | 13 | 1.0.0 | 1.0 | 12.9.0 |
| `allSettledKeyed` Experimental | **No** | **No** | 155 | **No** | **No** | **No** | 155 | **No** | **No** | **No** | **No** | **No** | ? | ? | ? |
| `any` | 85 | 85 | 79 | 71 | 14 | 85 | 79 | 60 | 14 | 14.0 | 85 | 14 | 1.0.0 | 1.2 | 15.0.0 |
| `catch()` | 32 | 12 | 29 | 19 | 8 | 32 | 29 | 19 | 8 | 2.0 | 4.4.3 | 8 | 1.0.0 | 1.0 | 0.12.0 |
| `finally()` | 63 | 18 | 58 | 50 | 11.1 | 63 | 58 | 46 | 11.3 | 8.0 | 63 | 11.3 | 1.0.0 | 1.0 | 10.0.0 |

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Nodejs |
| Incumbent settings object tracking | **No** | **No** | 50 | **No** | **No** | **No** | 50 | **No** | **No** | **No** | **No** | **No** | ? | **No** | **No** |
| `race()` | 32 | 12 | 29 | 19 | 8 | 32 | 29 | 19 | 8 | 2.0 | 4.4.3 | 8 | 1.0.0 | 1.0 | 0.12.0 |
| `reject()` | 32 | 12 | 29 | 19 | 8 | 32 | 29 | 19 | 8 | 2.0 | 4.4.3 | 8 | 1.0.0 | 1.0 | 0.12.0 |
| `resolve()` | 32 | 12 | 29 | 19 | 8 | 32 | 29 | 19 | 8 | 2.0 | 4.4.3 | 8 | 1.0.0 | 1.0 | 0.12.0 |
| `then()` | 32 | 12 | 29 | 19 | 8 | 32 | 29 | 19 | 8 | 2.0 | 4.4.3 | 8 | 1.0.0 | 1.0 | 0.12.0 |
| `try` | 128 | 128 | 134 | 114 | 18.2 | 128 | 134 | 85 | 18.2 | 28.0 | 128 | 18.2 | 1.1.22 | 1.46 | 23.0.0 |
| `withResolvers` | 119 | 119 | 121 | 105 | 17.4 | 119 | 121 | 79 | 17.4 | 25.0 | 119 | 17.4 | 1.0.0 | 1.38 | 22.0.0 |

1. Constructor requires a new operator since version 37.
2. Constructor requires a new operator since version 10.
3. Constructor requires a new operator since version 4.

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Promise`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Promise` in `core-js`
- Using promises guide
- Promises/A+ specification
- JavaScript Promises: an introduction on web.dev (2013)
- Callbacks, Promises, and Coroutines: Asynchronous Programming Patterns in JavaScript slide show by Domenic Denicola (2011)

---

JavaScript › Reference › Standard built-in objects › Promise

# Promise() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/Promise · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Promise()`** constructor creates `Promise` objects. It is primarily used to wrap callback-based APIs that do not already support promises.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Promise() constructor". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const promise1 = new Promise((resolve, reject) => {
  setTimeout(() => {
    resolve("foo");
  }, 300);
});

promise1.then((value) => {
  console.log(value);
  // Expected output: "foo"
});

console.log(promise1);
// Expected output: [object Promise]
```

## Syntax

js

```
new Promise(executor)
```

> **Note:** `Promise()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

### Parameters

**`executor`**

A `Function` to be executed by the constructor. It receives two functions as parameters: `resolveFunc` and `rejectFunc`. Any errors thrown in the `executor` will cause the promise to be rejected, and the return value will be neglected. The semantics of `executor` are detailed below.

### Return value

When called via `new`, the `Promise` constructor returns a promise object. The promise object will become *resolved* when either of the functions `resolveFunc` or `rejectFunc` are invoked. Note that if you call `resolveFunc` and pass another promise object as an argument, the initial promise can be said to be "resolved", but still not "settled". See the Promise description for more explanation.

## Description

Traditionally (before promises), asynchronous tasks were designed as callbacks.

js

```
readFile("./data.txt", (error, result) => {
  // This callback will be called when the task is done, with the
  // final `error` or `result`. Any operation dependent on the
  // result must be defined within this callback.
});
// Code here is immediately executed after the `readFile` request
// is fired. It does not wait for the callback to be called, hence
// making `readFile` "asynchronous".
```

To take advantage of the readability improvement and language features offered by promises, the `Promise()` constructor allows one to transform the callback-based API to a promise-based one.

> **Note:** If your task is already promise-based, you likely do not need the `Promise()` constructor.

The `executor` is custom code that ties an outcome in a callback to a promise. You, the programmer, write the `executor`. Its signature is expected to be:

```js
function executor(resolveFunc, rejectFunc) {
  // Typically, some asynchronous operation that accepts a callback,
  // like the `readFile` function above
}
```

`resolveFunc` and `rejectFunc` are also functions, and you can give them whatever actual names you want. Their signatures are simple: they accept a single parameter of any type.

```js
resolveFunc(value); // call on resolved
rejectFunc(reason); // call on rejected
```

The `value` parameter passed to `resolveFunc` can be another promise object, in which case the newly constructed promise's state will be "locked in" to the promise passed (as part of the resolution promise). The `rejectFunc` has semantics close to the `throw` statement, so `reason` is typically an `Error` instance. If either `value` or `reason` is omitted, the promise is fulfilled/rejected with `undefined`.

The `executor`'s completion state has limited effect on the promise's state:

- The `executor` return value is ignored. `return` statements within the `executor` merely impact control flow and alter whether a part of the function is executed, but do not have any impact on the promise's fulfillment value. If `executor` exits and it's impossible for `resolveFunc` or `rejectFunc` to be called in the future (for example, there are no async tasks scheduled), then the promise remains pending forever.
- If an error is thrown in the `executor`, the promise is rejected, unless `resolveFunc` or `rejectFunc` has already been called.

> **Note:** The existence of pending promises does not prevent the program from exiting. If the event loop is empty, the program exits despite any pending promises (because those are necessarily forever-pending).

Here's a summary of the typical flow:

1. At the time when the constructor generates the new `Promise` object, it also generates a corresponding pair of functions for `resolveFunc` and `rejectFunc`; these are "tethered" to the `Promise` object.
2. `executor` typically wraps some asynchronous operation which provides a callback-based API. The callback (the one passed to the original callback-based API) is defined within the `executor` code, so it has access to the `resolveFunc` and `rejectFunc`.
3. The `executor` is called synchronously (as soon as the `Promise` is constructed) with the `resolveFunc` and `rejectFunc` functions as arguments.
4. The code within the `executor` has the opportunity to perform some operation. The eventual completion of the asynchronous task is communicated with the promise instance via the side effect

caused by `resolveFunc` or `rejectFunc`. The side effect is that the `Promise` object becomes "resolved".

- If `resolveFunc` is called first, the value passed will be resolved. The promise may stay pending (in case another thenable is passed), become fulfilled (in most cases where a non-thenable value is passed), or become rejected (in case of an invalid resolution value).
- If `rejectFunc` is called first, the promise instantly becomes rejected.
- Once one of the resolving functions (`resolveFunc` or `rejectFunc`) is called, the promise stays resolved. Only the first call to `resolveFunc` or `rejectFunc` affects the promise's eventual state, and subsequent calls to either function can neither change the fulfillment value/rejection reason nor toggle its eventual state from "fulfilled" to "rejected" or opposite.
- If `executor` exits by throwing an error, then the promise is rejected. However, the error is ignored if one of the resolving functions has already been called (so that the promise is already resolved).
- Resolving the promise does not necessarily cause the promise to become fulfilled or rejected (i.e., settled). The promise may still be pending because it's resolved with another thenable, but its eventual state will match that of the resolved thenable.

5. Once the promise settles, it (asynchronously) invokes any further handlers associated through `then()`, `catch()`, or `finally()`. The eventual fulfillment value or rejection reason is passed to the invocation of fulfillment and rejection handlers as an input parameter (see Chained Promises).

For example, the callback-based `readFile` API above can be transformed into a promise-based one.

```js
const readFilePromise = (path) =>
  new Promise((resolve, reject) => {
    readFile(path, (error, result) => {
      if (error) {
        reject(error);
      } else {
        resolve(result);
      }
    });
  });

readFilePromise("./data.txt")
  .then((result) => console.log(result))
  .catch((error) => console.error("Failed to read data"));
```

The `resolve` and `reject` callbacks are only available within the scope of the executor function, which means you can't access them after the promise is constructed. If you want to construct the promise before deciding how to resolve it, you can use the `Promise.withResolvers()` method instead, which exposes the `resolve` and `reject` functions.

### The resolve function

The `resolve` function has the following behaviors:

- If it's called with the same value as the newly created promise (the promise it's "tethered to"), the promise is rejected with a `TypeError`.
- If it's called with a non-thenable value (a primitive, or an object whose `then` property is not callable, including when the property is not present), the promise is immediately fulfilled with that value.
- If it's called with a thenable value (including another `Promise` instance), then the thenable's `then` method is saved and called in the future (it's always called asynchronously). The `then` method will

be called with two callbacks, which are two new functions with the exact same behaviors as the `resolveFunc` and `rejectFunc` passed to the `executor` function. If calling the `then` method throws, then the current promise is rejected with the thrown error.

In the last case, it means code like:

```js
new Promise((resolve, reject) => {
  resolve(thenable);
});
```

Is roughly equivalent to:

```js
new Promise((resolve, reject) => {
  try {
    thenable.then(
      (value) => resolve(value),
      (reason) => reject(reason),
    );
  } catch (e) {
    reject(e);
  }
});
```

Except that in the `resolve(thenable)` case:

1. `resolve` is called synchronously, so that calling `resolve` or `reject` again has no effect, even when the handlers attached through `anotherPromise.then()` are not called yet.
2. The `then` method is called asynchronously, so that the promise will never be instantly resolved if a thenable is passed.

Because `resolve` is called again with whatever `thenable.then()` passes to it as `value`, the resolver function is able to flatten nested thenables, where a thenable calls its `onFulfilled` handler with another thenable. The effect is that the fulfillment handler of a real promise will never receive a thenable as its fulfillment value.

## Examples

### Turning a callback-based API into a promise-based one

To provide a function with promise functionality, have it return a promise by calling the `resolve` and `reject` functions at the correct times.

```js
function myAsyncFunction(url) {
  return new Promise((resolve, reject) => {
    const xhr = new XMLHttpRequest();
    xhr.open("GET", url);
    xhr.onload = () => resolve(xhr.responseText);
    xhr.onerror = () => reject(xhr.statusText);
    xhr.send();
  });
}
```

### Effect of calling resolveFunc

Calling `resolveFunc` causes the promise to become resolved, so that calling `resolveFunc` or `rejectFunc` again has no effect. However, the promise may be in any of the states: pending, fulfilled, or rejected.

This `pendingResolved` promise is resolved the time it's created, because it has already been "locked in" to match the eventual state of the inner promise, and calling `resolveOuter` or `rejectOuter` or throwing an error later in the executor has no effect on its eventual state. However, the inner promise is still pending until 100ms later, so the outer promise is also pending:

```js
const pendingResolved = new Promise((resolveOuter, rejectOuter) => {
  resolveOuter(
    new Promise((resolveInner) => {
      setTimeout(() => {
        resolveInner("inner");
      }, 100);
    }),
  );
});
```

This `fulfilledResolved` promise becomes fulfilled the moment it's resolved, because it's resolved with a non-thenable value. However, when it's created, it's unresolved, because neither `resolve` nor `reject` has been called yet. An unresolved promise is necessarily pending:

```js
const fulfilledResolved = new Promise((resolve, reject) => {
  setTimeout(() => {
    resolve("outer");
  }, 100);
});
```

Calling `rejectFunc` obviously causes the promise to reject. However, there are also two ways to cause the promise to instantly become rejected even when the `resolveFunc` callback is called.

```js
// 1. Resolving with the promise itself
const rejectedResolved1 = new Promise((resolve) => {
  // Note: resolve has to be called asynchronously,
  // so that the rejectedResolved1 variable is initialized
  setTimeout(() => resolve(rejectedResolved1)); // TypeError: Chaining cycle detected for
promise #<Promise>
});

// 2. Resolving with an object which throws when accessing the `then` property
const rejectedResolved2 = new Promise((resolve) => {
  resolve({
    get then() {
      throw new Error("Can't get then property");
    },
  });
});
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-promise-constructor` |

## Browser compatibility

| | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| Feature | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `Promise()` constructor | 32 | 12 | 29[1] | 19 | 8[2] | 32 | 29[1] | 19 | 8[2] | 2.0 | 4.4.3 | 8[2] | 1.0.0 | 1.0 | 0.12.0[3] |

1. Constructor requires a new operator since version 37.
2. Constructor requires a new operator since version 10.
3. Constructor requires a new operator since version 4.

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Promise.Promise`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Promise` in `core-js`
- Using promises guide
- `Promise.withResolvers()`

---

JavaScript › Reference › Standard built-in objects › Promise

# Promise.all()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/all · MDN page last modified 2026-09-01

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Promise.all()`** static method takes an iterable of promises as input and returns a single `Promise`. This returned promise fulfills when all of the input's promises fulfill (including when an empty iterable is passed), with an array of the fulfillment values. It rejects when any of the input's promises rejects, with this first rejection reason.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Promise.all()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const promise1 = Promise.resolve(3);
const promise2 = 42;
const promise3 = new Promise((resolve, reject) => {
  setTimeout(resolve, 100, "foo");
});

Promise.all([promise1, promise2, promise3]).then((values) => {
  console.log(values);
});
// Expected output: Array [3, 42, "foo"]
```

## Syntax

js

```
Promise.all(iterable)
```

### Parameters

**`iterable`**

An iterable (such as an `Array`) of promises. These values are awaited, so other thenables are also resolved, while non-thenables are returned as-is.

### Return value

A `Promise` that is:

- **Already fulfilled**, if the `iterable` passed is empty.
- **Asynchronously fulfilled**, when all the promises in the given `iterable` fulfill. The fulfillment value is an array of fulfillment values, in the order of the promises passed, regardless of completion order. If the `iterable` passed is non-empty but contains no pending promises, the returned promise is still asynchronously (instead of synchronously) fulfilled.
- **Asynchronously rejected**, when any of the promises in the given `iterable` rejects. The rejection reason is the rejection reason of the first promise that was rejected.

## Description

The `Promise.all()` method is one of the promise concurrency methods. It can be useful for aggregating the results of multiple promises. It is typically used when there are multiple related asynchronous tasks that the overall code relies on to work successfully — all of whom we want to fulfill before the code execution continues.

`Promise.all()` will reject immediately upon **any** of the input promises rejecting. In comparison, the promise returned by `Promise.allSettled()` will wait for all input promises to complete, regardless of whether or not one rejects. Use `allSettled()` if you need the final result of every promise in the input iterable.

Like other promise combinators, `Promise.all()` immediately marks all promises as "handled" when it is called (by calling their `.then()` methods). Subsequent rejections after the first rejection will be ignored, and will not trigger any `unhandledrejection` events.

Rejecting the returned promise does not cancel the remaining operations or unsubscribe the handlers attached to their promises. Repeatedly passing a long-lived pending promise to `Promise.all()` can accumulate handlers on that promise even when another input rejects each time.

## Examples

### Using Promise.all()

`Promise.all` waits for all fulfillments (or the first rejection).

```js
const p1 = Promise.resolve(3);
const p2 = 1337;
const p3 = new Promise((resolve, reject) => {
  setTimeout(() => {
    resolve("foo");
  }, 100);
});

Promise.all([p1, p2, p3]).then((values) => {
  console.log(values); // [3, 1337, "foo"]
});
```

If the `iterable` contains non-promise values, they will be ignored, but still counted in the returned promise array value (if the promise is fulfilled):

```js
// All values are non-promises, so the returned promise gets fulfilled
const p = Promise.all([1, 2, 3]);
// The only input promise is already fulfilled,
// so the returned promise gets fulfilled
const p2 = Promise.all([1, 2, 3, Promise.resolve(444)]);
// One (and the only) input promise is rejected,
// so the returned promise gets rejected
const p3 = Promise.all([1, 2, 3, Promise.reject(new Error("bad"))]);

// Using setTimeout, we can execute code after the queue is empty
setTimeout(() => {
  console.log(p);
  console.log(p2);
  console.log(p3);
});

// Logs:
// Promise { <state>: "fulfilled", <value>: Array[3] }
// Promise { <state>: "fulfilled", <value>: Array[4] }
// Promise { <state>: "rejected", <reason>: Error: bad }
```

### Destructuring the result

You will find destructuring very useful if you are batching together a known number of tasks.

```js
// With then()
Promise.all([p1, p2, p3]).then(([a, b, c]) => {
  console.log(a, b, c); // 3 1337 "foo"
});

// With await
const [a, b, c] = await Promise.all([p1, p2, p3]);
```

Be careful: if the original promises and the result variables' order don't match, you may run into subtle bugs. The `Promise.allKeyed()` method solves exactly this problem.

### Asynchronicity or synchronicity of Promise.all

This following example demonstrates the asynchronicity of `Promise.all` when a non-empty `iterable` is passed:

```js
// Passing an array of promises that are already resolved,
// to trigger Promise.all as soon as possible
const resolvedPromisesArray = [Promise.resolve(33), Promise.resolve(44)];

const p = Promise.all(resolvedPromisesArray);
// Immediately logging the value of p
console.log(p);

// Using setTimeout, we can execute code after the queue is empty
setTimeout(() => {
  console.log("the queue is now empty");
  console.log(p);
});

// Logs, in order:
// Promise { <state>: "pending" }
// the queue is now empty
// Promise { <state>: "fulfilled", <value>: Array[2] }
```

The same thing happens if `Promise.all` rejects:

```js
const mixedPromisesArray = [
  Promise.resolve(33),
  Promise.reject(new Error("bad")),
];
const p = Promise.all(mixedPromisesArray);
console.log(p);
setTimeout(() => {
  console.log("the queue is now empty");
  console.log(p);
});

// Logs:
// Promise { <state>: "pending" }
// the queue is now empty
// Promise { <state>: "rejected", <reason>: Error: bad }
```

`Promise.all` resolves synchronously if and only if the `iterable` passed is empty:

```js
const p = Promise.all([]); // Will be immediately resolved
const p2 = Promise.all([1337, "hi"]); // Non-promise values are ignored, but the evaluation
is done asynchronously
console.log(p);
console.log(p2);
setTimeout(() => {
  console.log("the queue is now empty");
  console.log(p2);
});

// Logs:
// Promise { <state>: "fulfilled", <value>: Array[0] }
// Promise { <state>: "pending" }
// the queue is now empty
// Promise { <state>: "fulfilled", <value>: Array[2] }
```

### Using Promise.all() with async functions

Within async functions, it's very common to "over-await" your code. For example, given the following functions:

```js
function promptForDishChoice() {
  return new Promise((resolve, reject) => {
    const dialog = document.createElement("dialog");
    dialog.innerHTML = `
<form method="dialog">
  <p>What would you like to eat?</p>
  <select>
    <option value="pizza">Pizza</option>
    <option value="pasta">Pasta</option>
    <option value="salad">Salad</option>
  </select>
  <menu>
    <li><button value="cancel">Cancel</button></li>
    <li><button type="submit" value="ok">OK</button></li>
  </menu>
</form>
    `;
    dialog.addEventListener("close", () => {
      if (dialog.returnValue === "ok") {
        resolve(dialog.querySelector("select").value);
      } else {
        reject(new Error("User canceled dialog"));
      }
    });
    document.body.appendChild(dialog);
    dialog.showModal();
  });
}

async function fetchPrices() {
  const response = await fetch("/prices");
  return await response.json();
}
```

You may write a function like this:

js · ✗ bad example

```
async function getPrice() {
  const choice = await promptForDishChoice();
  const prices = await fetchPrices();
  return prices[choice];
}
```

However, note that the execution of `promptForDishChoice` and `fetchPrices` don't depend on the result of each other. While the user is choosing their dish, it's fine for the prices to be fetched in the background, but in the code above, the `await` operator causes the async function to pause until the choice is made, and then again until the prices are fetched. We can use `Promise.all` to run them concurrently, so that the user doesn't have to wait for the prices to be fetched before the result is given:

js · ✓ good example

```
async function getPrice() {
  const [choice, prices] = await Promise.all([
    promptForDishChoice(),
    fetchPrices(),
  ]);
  return prices[choice];
}
```

`Promise.all` is the best choice of concurrency method here, because error handling is intuitive — if any of the promises reject, the result is no longer available, so the whole `await` expression throws.

`Promise.all` accepts an iterable of promises, so if you are using it to run several async functions concurrently, you need to call the async functions and use the returned promises. Directly passing the functions to `Promise.all` does not work, since they are not promises.

js · ✗ bad example

```
async function getPrice() {
  const [choice, prices] = await Promise.all([
    promptForDishChoice,
    fetchPrices,
  ]);
  // `choice` and `prices` are still the original async functions;
  // Promise.all() does nothing to non-promises
}
```

Note that the following, despite other undesirable aspects (like creating more variables and bug-prone error handling), also achieves concurrency:

js

```
async function getPrice() {
  // Fire all async operations upfront
  const choicePromise = promptForDishChoice();
  const pricesPromise = fetchPrices();

  // Wait for each promise (all in-progress async operations keep running)
  const choice = await choicePromise;
  const prices = await pricesPromise;
}
```

### Promise.all fail-fast behavior

`Promise.all` is rejected if any of the elements are rejected. For example, if you pass in four promises that resolve after a timeout and one promise that rejects immediately, then `Promise.all` will reject

immediately.

```js
const p1 = new Promise((resolve, reject) => {
  setTimeout(() => resolve("one"), 1000);
});
const p2 = new Promise((resolve, reject) => {
  setTimeout(() => resolve("two"), 2000);
});
const p3 = new Promise((resolve, reject) => {
  setTimeout(() => resolve("three"), 3000);
});
const p4 = new Promise((resolve, reject) => {
  setTimeout(() => resolve("four"), 4000);
});
const p5 = new Promise((resolve, reject) => {
  reject(new Error("reject"));
});

// Using .catch:
Promise.all([p1, p2, p3, p4, p5])
  .then((values) => {
    console.log(values);
  })
  .catch((error) => {
    console.error(error.message);
  });

// Logs:
// "reject"
```

It is possible to change this behavior by handling possible rejections:

```js
const p1 = new Promise((resolve, reject) => {
  setTimeout(() => resolve("p1_delayed_resolution"), 1000);
});

const p2 = new Promise((resolve, reject) => {
  reject(new Error("p2_immediate_rejection"));
});

Promise.all([p1.catch((error) => error), p2.catch((error) => error)]).then(
  (values) => {
    console.log(values[0]); // "p1_delayed_resolution"
    console.error(values[1]); // "Error: p2_immediate_rejection"
  },
);
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-promise.all |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `all()` | 32 | 12 | 29 | 19 | 8 | 32 | 29 | 19 | 8 | 2.0 | 4.4.3 | 8 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Promise.all`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Promise`
- `Promise.allKeyed()`
- `Promise.allSettled()`
- `Promise.any()`
- `Promise.race()`

---

JavaScript › Reference › Standard built-in objects › Promise

# Promise.allKeyed()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/allKeyed · MDN page last modified 2026-09-18

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`Promise.allKeyed()`** static method is like `Promise.all()`, except that instead of using arrays/iterables as input/output, it uses objects. It takes an object where each own key is associated with a promise, and returns a single `Promise`. This returned promise fulfills when all of the input's promises fulfill, with an object of the same keys mapped to the corresponding fulfillment values. It rejects when any of the input's promises rejects, with this first rejection reason.

Compared to `Promise.all()`, `Promise.allKeyed()` allows you to associate results with semantically meaningful keys, instead of arbitrary array ordering which can be difficult to maintain.

## Syntax

```
Promise.allKeyed(object)
```

### Parameters

**`object`**

An object. All of its own enumerable properties, whether the key is a string or a symbol, should have `Promise` values. These values are awaited, so other thenables are also resolved, while non-thenables are returned as-is.

### Return value

A `Promise` that is:

- **Already fulfilled**, if the `object` passed has no own enumerable properties.
- **Asynchronously fulfilled**, when all the promises in the given `object` fulfill. The fulfillment value is an object of fulfillment values, with the same keys in the same order as the given `object`, regardless of completion order. If the `object` passed is non-empty but contains no pending promises, the returned promise is still asynchronously (instead of synchronously) fulfilled.
- **Asynchronously rejected**, when any of the promises in the given `object` rejects. The rejection reason is the rejection reason of the first promise that was rejected.

## Description

The `Promise.allKeyed()` method is one of the promise concurrency methods. It performs the same kind of task as `Promise.all()`. However, a lot of the times, you don't already have an array of promises and instead just have a few ad-hoc operations to batch up, so you put them in an array and then immediately destructure:

```js
const [resultA, resultB, resultC] = await Promise.all([getA(), getB(), getC()]);
```

The problem with this is that you need to maintain the consistency of ordering on both sides: if you accidentally write `[resultA, resultC, resultB]`, your code will break.

The keyed method mitigates the problem by associating each async operation with a semantic key:

```js
const {
  a: resultA,
  b: resultB,
  c: resultC,
} = await Promise.allKeyed({
  a: getA(),
  b: getB(),
  c: getC(),
});
```

This way, the ordering no longer matters, and any name mismatch is local: accidentally writing `b: resultC` is now much easier to catch!

## Examples

### Using Promise.allKeyed()

The `Promise.allKeyed()` method takes an object and processes all of its own enumerable properties.

```js
function delayed(value, timeout) {
  return new Promise((res) => setTimeout(() => res(value), timeout));
}

const sym = Symbol("example");
```

*(code continues on the next page)*

(code continued from the previous page)

```
const promises = {
  a: delayed("a", 500),
  // Symbol properties are processed
  [sym]: delayed("symbol", 300),
  // Nested properties are not processed; this whole object is treated as
  // an already-resolved value and returned as-is
  nested: {
    b: delayed("b", 100),
  },
};

const result = await Promise.allKeyed(promises);
console.log(result);
// {
//   a: "a",
//   [sym]: "symbol",
//   nested: {
//     b: <Promise>,
//   },
// }
```

For more examples related to concurrency behavior common to `Promise.all()` and `Promise.allKeyed()`, see `Promise.all()`.

## Specifications

| Specification |
| --- |
| Await Dictionary<br>`https://tc39.es/proposal-await-dictionary/#sec-promise.allkeyed` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Nodejs |
| `allKeyed` Experimental | **No** | **No** | 155 | **No** | **No** | **No** | 155 | **No** | **No** | **No** | **No** | **No** | ? | ? | ? |

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Promise.allKeyed`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Promise.allKeyed` in `core-js`
- es-shims polyfill of `Promise.allKeyed`
- `Promise`
- `Promise.allSettledKeyed()`
- `Promise.all()`

JavaScript › Reference › Standard built-in objects › Promise

# Promise.allSettled()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/allSettled · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since July 2020.

The **`Promise.allSettled()`** static method takes an iterable of promises as input and returns a single `Promise`. This returned promise fulfills when all of the input's promises settle (including when an empty iterable is passed), with an array of objects that describe the outcome of each promise.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Promise.allSettled()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const promise1 = Promise.resolve(3);
const promise2 = new Promise((resolve, reject) =>
  setTimeout(reject, 100, "foo"),
);
const promises = [promise1, promise2];

Promise.allSettled(promises).then((results) =>
  results.forEach((result) => console.log(result.status)),
);

// Expected output:
// "fulfilled"
// "rejected"
```

## Syntax

js

```
Promise.allSettled(iterable)
```

### Parameters

**`iterable`**

An iterable (such as an `Array`) of promises. These values are awaited, so other thenables are also resolved, while non-thenables are returned as-is.

### Return value

A `Promise` that is:

- **Already fulfilled**, if the `iterable` passed is empty.
- **Asynchronously fulfilled**, when all promises in the given `iterable` have settled (either fulfilled or rejected). The fulfillment value is an array of objects, each describing the outcome of one promise in the `iterable`, in the order of the promises passed, regardless of completion order. Each outcome object has the following properties:

  **`status`**

  A string, either `"fulfilled"` or `"rejected"`, indicating the eventual state of the promise.

**`value`**

Only present if `status` is `"fulfilled"`. The value that the promise was fulfilled with.

**`reason`**

Only present if `status` is `"rejected"`. The reason that the promise was rejected with.

If the `iterable` passed is non-empty but contains no pending promises, the returned promise is still asynchronously (instead of synchronously) fulfilled.

## Description

The `Promise.allSettled()` method is one of the promise concurrency methods. `Promise.allSettled()` is typically used when you have multiple asynchronous tasks that are not dependent on one another to complete successfully, or you'd always like to know the result of each promise.

In comparison, the Promise returned by `Promise.all()` may be more appropriate if the tasks are dependent on each other, or if you'd like to immediately reject upon any of them rejecting.

## Examples

### Using Promise.allSettled()

```js
Promise.allSettled([
  Promise.resolve(33),
  new Promise((resolve) => setTimeout(() => resolve(66), 0)),
  99,
  Promise.reject(new Error("an error")),
]).then((values) => console.log(values));

// [
//   { status: 'fulfilled', value: 33 },
//   { status: 'fulfilled', value: 66 },
//   { status: 'fulfilled', value: 99 },
//   { status: 'rejected', reason: Error: an error }
// ]
```

You may also destructure the results to have one variable per settling outcome.

```js
const [status1, status2] = await Promise.allSettled([
  Promise.resolve(33),
  new Promise((resolve) => setTimeout(() => resolve(66), 0)),
]);

// status1 = { status: 'fulfilled', value: 33 }
// status2 = { status: 'fulfilled', value: 66 }
```

If you destructure the result, you must keep the result variables in the same order as the input promises; a mismatch may cause subtle bugs. The `Promise.allSettledKeyed()` method avoids this problem by associating each input and output with a key.

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-promise.allsettled` |

## Browser compatibility

| | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| Feature | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `allSettled()` | 76 | 79 | 71 | 63 | 13 | 76 | 79 | 54 | 13 | 12.0 | 76 | 13 | 1.0.0 | 1.0 | 12.9.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Promise.allSettled`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Promise.allSettled` in `core-js`
- es-shims polyfill of `Promise.allSettled`
- Using promises guide
- Graceful asynchronous programming with promises
- `Promise`
- `Promise.all()`
- `Promise.allSettledKeyed()`
- `Promise.any()`
- `Promise.race()`

---

JavaScript › Reference › Standard built-in objects › Promise

# Promise.allSettledKeyed()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/allSettledKeyed · MDN page last modified 2026-09-18

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`Promise.allSettledKeyed()`** static method is like `Promise.allSettled()`, except that instead of using arrays/iterables as input/output, it uses objects. It takes an object where each own key is associated with a promise, and returns a single `Promise`. This returned promise fulfills when all of the input's promises settle, with an object of the same keys mapped to objects that describe the outcome of the corresponding promise.

Compared to `Promise.allSettled()`, `Promise.allSettledKeyed()` allows you to associate results with semantically meaningful keys, instead of arbitrary array ordering which can be difficult to maintain.

## Syntax

```js
Promise.allSettledKeyed(object)
```

### Parameters

**`object`**

An object. All of its own enumerable properties, whether the key is a string or a symbol, should have `Promise` values. These values are awaited, so other thenables are also resolved, while non-thenables are returned as-is.

### Return value

A `Promise` that is:

- **Already fulfilled**, if the `object` passed has no own enumerable properties.
- **Asynchronously fulfilled**, when all promises in the given `object` have settled (either fulfilled or rejected). The fulfillment value is an object, with the same keys in the same order as the given `object`, and each property's value being an object describing the outcome of the corresponding promise in `object`, regardless of completion order. Each outcome object has the following properties:

  **`status`**

  A string, either `"fulfilled"` or `"rejected"`, indicating the eventual state of the promise.

  **`value`**

  Only present if `status` is `"fulfilled"`. The value that the promise was fulfilled with.

  **`reason`**

  Only present if `status` is `"rejected"`. The reason that the promise was rejected with.

  If the `object` passed is non-empty but contains no pending promises, the returned promise is still asynchronously (instead of synchronously) fulfilled.

## Description

The `Promise.allSettledKeyed()` method is one of the promise concurrency methods. It performs the same kind of task as `Promise.allSettled()`. `Promise.allSettledKeyed()` is preferred when you don't already have an array of promises and/or you are immediately destructuring the results; see `Promise.allKeyed()` for more description.

## Examples

### Using Promise.allSettledKeyed()

The `Promise.allSettledKeyed()` method takes an object and processes all of its own enumerable properties.

```js
function delayed(value, timeout) {
  return new Promise((res) => setTimeout(() => res(value), timeout));
}

const sym = Symbol("example");

const promises = {
  a: delayed("a", 500),
  // Symbol properties are processed
  [sym]: delayed("symbol", 300),
  // Nested properties are not processed; this whole object is treated as
  // an already-resolved value and returned as-is
  nested: {
    b: delayed("b", 100),
  },
};

const result = await Promise.allSettledKeyed(promises);
console.log(result);
// {
//   a: { status: "fulfilled", value: "a" },
//   [sym]: { status: "fulfilled", value: "symbol" },
//   nested: { status: "fulfilled", value: { b: <Promise> } },
// }
```

For more examples related to concurrency behavior common to `Promise.allSettled()` and `Promise.allSettledKeyed()`, see `Promise.allSettled()`.

## Specifications

| Specification |
| --- |
| Await Dictionary<br>`https://tc39.es/proposal-await-dictionary/#sec-promise.allsettledkeyed` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `allSettledKeyed` Experimental | **No** | **No** | 155 | **No** | **No** | **No** | 155 | **No** | **No** | **No** | **No** | **No** | ? | ? | ? |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Promise.allSettledKeyed`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Promise.allSettledKeyed` in `core-js`
- es-shims polyfill of `Promise.allSettledKeyed`
- `Promise`
- `Promise.allKeyed()`
- `Promise.allSettled()`

JavaScript › Reference › Standard built-in objects › Promise

# Promise.any()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/any · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since September 2020.

The **`Promise.any()`** static method takes an iterable of promises as input and returns a single `Promise`. This returned promise fulfills when any of the input's promises fulfills, with this first fulfillment value. It rejects when all of the input's promises reject (including when an empty iterable is passed), with an `AggregateError` containing an array of rejection reasons.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Promise.any()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const promise1 = Promise.reject(new Error("error"));
const promise2 = new Promise((resolve) => setTimeout(resolve, 100, "quick"));
const promise3 = new Promise((resolve) => setTimeout(resolve, 500, "slow"));

const promises = [promise1, promise2, promise3];

Promise.any(promises).then((value) => console.log(value));

// Expected output: "quick"
```

## Syntax

js

```
Promise.any(iterable)
```

### Parameters

**`iterable`**

An iterable (such as an `Array`) of promises. These values are awaited, so other thenables are also resolved, while non-thenables are returned as-is.

### Return value

A `Promise` that is:

- **Already rejected**, if the `iterable` passed is empty.
- **Asynchronously fulfilled**, when any of the promises in the given `iterable` fulfills. The fulfillment value is the fulfillment value of the first promise that was fulfilled.
- **Asynchronously rejected**, when all of the promises in the given `iterable` reject. The rejection reason is an `AggregateError` containing an array of rejection reasons in its `errors` property. The errors are in the order of the promises passed, regardless of completion order. If the `iterable` passed is non-empty but contains no pending promises, the returned promise is still asynchronously (instead of synchronously) rejected.

## Description

The `Promise.any()` method is one of the promise concurrency methods. This method is useful for returning the first promise that fulfills. It short-circuits after a promise fulfills, so it does not wait for the other promises to complete once it finds one.

Unlike `Promise.all()`, which returns an *array* of fulfillment values, we only get one fulfillment value (assuming at least one promise fulfills). This can be beneficial if we need only one promise to fulfill but we do not care which one does. Note another difference: this method rejects upon receiving an *empty iterable*, since, truthfully, the iterable contains no items that fulfill. You may compare `Promise.any()` and `Promise.all()` with `Array.prototype.some()` and `Array.prototype.every()`.

Also, unlike `Promise.race()`, which returns the first *settled* value (either fulfillment or rejection), this method returns the first *fulfilled* value. This method ignores all rejected promises up until the first promise that fulfills.

Like other promise combinators, `Promise.any()` immediately marks all promises as "handled" when it is called (by calling their `.then()` methods). Subsequent rejections after the first fulfillment will be ignored, and will not trigger any `unhandledrejection` events.

Fulfilling the returned promise does not cancel the remaining operations or unsubscribe the handlers attached to their promises. Repeatedly passing a long-lived pending promise to `Promise.any()` can accumulate handlers on that promise even when another input fulfills each time.

## Examples

### Using Promise.any()

`Promise.any()` fulfills with the first promise to fulfill, even if a promise rejects first. This is in contrast to `Promise.race()`, which fulfills or rejects with the first promise to settle.

js

```
const pErr = new Promise((resolve, reject) => {
  reject(new Error("Always fails"));
});

const pSlow = new Promise((resolve, reject) => {
  setTimeout(resolve, 500, "Done eventually");
});

const pFast = new Promise((resolve, reject) => {
  setTimeout(resolve, 100, "Done quick");
});

Promise.any([pErr, pSlow, pFast]).then((value) => {
  console.log(value);
  // pFast fulfills first
});
// Logs:
// Done quick
```

### Rejections with AggregateError

`Promise.any()` rejects with an `AggregateError` if no promise fulfills.

```js
const failure = new Promise((resolve, reject) => {
  reject(new Error("Always fails"));
});

Promise.any([failure]).catch((err) => {
  console.log(err);
});
// AggregateError: No Promise in Promise.any was resolved
```

### Displaying the first image loaded

In this example, we have a function that fetches an image and returns a blob. We use `Promise.any()` to fetch a couple of images and display the first one available (i.e., whose promise has resolved).

```js
async function fetchAndDecode(url, description) {
  const res = await fetch(url);
  if (!res.ok) {
    throw new Error(`HTTP error! status: ${res.status}`);
  }
  const data = await res.blob();
  return [data, description];
}

const coffee = fetchAndDecode("coffee.jpg", "Coffee");
const tea = fetchAndDecode("tea.jpg", "Tea");

Promise.any([coffee, tea])
  .then(([blob, description]) => {
    const objectURL = URL.createObjectURL(blob);
    const image = document.createElement("img");
    image.src = objectURL;
    image.alt = description;
    document.body.appendChild(image);
  })
  .catch((e) => {
    console.error(e);
  });
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-promise.any |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| any | 85 | 85 | 79 | 71 | 14 | 85 | 79 | 60 | 14 | 14.0 | 85 | 14 | 1.0.0 | 1.2 | 15.0.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Promise.any`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Promise.any` in `core-js`
- es-shims polyfill of `Promise.any`
- `Promise`
- `Promise.all()`
- `Promise.allSettled()`
- `Promise.race()`

---

JavaScript › Reference › Standard built-in objects › Promise

# Promise.race()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/race · MDN page last modified 2026-09-01

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Promise.race()`** static method takes an iterable of promises as input and returns a single `Promise`. This returned promise settles with the eventual state of the first promise that settles.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Promise.race()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const promise1 = new Promise((resolve, reject) => {
  setTimeout(resolve, 500, "one");
});

const promise2 = new Promise((resolve, reject) => {
  setTimeout(resolve, 100, "two");
});

Promise.race([promise1, promise2]).then((value) => {
  console.log(value);
  // Both resolve, but promise2 is faster
});
// Expected output: "two"
```

## Syntax

js

```
Promise.race(iterable)
```

### Parameters

**`iterable`**

An iterable (such as an `Array`) of promises. These values are awaited, so other thenables are also resolved, while non-thenables are returned as-is.

### Return value

A `Promise` that **asynchronously settles** with the eventual state of the first promise in the `iterable` to settle. In other words, it fulfills if the first promise to settle is fulfilled, and rejects if the first promise to settle is rejected. The returned promise remains pending forever if the `iterable` passed is empty. If the `iterable` passed is non-empty but contains no pending promises, the returned promise is still asynchronously (instead of synchronously) settled.

## Description

The `Promise.race()` method is one of the promise concurrency methods. It's useful when you want the first async task to complete, but do not care about its eventual state (i.e., it can either succeed or fail).

If the iterable contains one or more non-promise values and/or an already settled promise, then `Promise.race()` will settle to the first of these values found in the iterable.

Like other promise combinators, `Promise.race()` immediately marks all promises as "handled" when it is called (by calling their `.then()` methods). Subsequent rejections after the first settlement will be ignored, and will not trigger any `unhandledrejection` events.

Settling the returned promise does not cancel the losing operations or unsubscribe the handlers attached to their promises. If you repeatedly race a long-lived pending promise against short-lived promises, handlers can accumulate on the pending promise even after each race settles.

## Examples

### Using Promise.race()

This example shows how `Promise.race()` can be used to race several timers implemented with `setTimeout()`. The timer with the shortest time always wins the race and becomes the resulting promise's state.

js

```
function sleep(time, value, state) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (state === "fulfill") {
        resolve(value);
      } else {
        reject(new Error(value));
      }
    }, time);
  });
}

const p1 = sleep(500, "one", "fulfill");
const p2 = sleep(100, "two", "fulfill");

Promise.race([p1, p2]).then((value) => {
  console.log(value); // "two"
  // Both fulfill, but p2 is faster
});

const p3 = sleep(100, "three", "fulfill");
const p4 = sleep(500, "four", "reject");
```

(code continues on the next page)

*(code continued from the previous page)*

```
Promise.race([p3, p4]).then(
  (value) => {
    console.log(value); // "three"
    // p3 is faster, so it fulfills
  },
  (error) => {
    // Not called
  },
);

const p5 = sleep(500, "five", "fulfill");
const p6 = sleep(100, "six", "reject");

Promise.race([p5, p6]).then(
  (value) => {
    // Not called
  },
  (error) => {
    console.error(error.message); // "six"
    // p6 is faster, so it rejects
  },
);
```

### Asynchronicity of Promise.race

This following example demonstrates the asynchronicity of `Promise.race`. Unlike other promise concurrency methods, `Promise.race` is always asynchronous: it never settles synchronously, even when the `iterable` is empty.

js

```
// Passing an array of promises that are already resolved,
// to trigger Promise.race as soon as possible
const resolvedPromisesArray = [Promise.resolve(33), Promise.resolve(44)];

const p = Promise.race(resolvedPromisesArray);
// Immediately logging the value of p
console.log(p);

// Using setTimeout, we can execute code after the stack is empty
setTimeout(() => {
  console.log("the stack is now empty");
  console.log(p);
});

// Logs, in order:
// Promise { <state>: "pending" }
// the stack is now empty
// Promise { <state>: "fulfilled", <value>: 33 }
```

An empty iterable causes the returned promise to be forever pending:

```js
const foreverPendingPromise = Promise.race([]);
console.log(foreverPendingPromise);
setTimeout(() => {
  console.log("the stack is now empty");
  console.log(foreverPendingPromise);
});

// Logs, in order:
// Promise { <state>: "pending" }
// the stack is now empty
// Promise { <state>: "pending" }
```

If the iterable contains one or more non-promise value and/or an already settled promise, then `Promise.race` will settle to the first of these values found in the array:

```js
const foreverPendingPromise = Promise.race([]);
const alreadyFulfilledProm = Promise.resolve(100);

const arr = [foreverPendingPromise, alreadyFulfilledProm, "non-Promise value"];
const arr2 = [foreverPendingPromise, "non-Promise value", Promise.resolve(100)];
const p = Promise.race(arr);
const p2 = Promise.race(arr2);

console.log(p);
console.log(p2);
setTimeout(() => {
  console.log("the stack is now empty");
  console.log(p);
  console.log(p2);
});

// Logs, in order:
// Promise { <state>: "pending" }
// Promise { <state>: "pending" }
// the stack is now empty
// Promise { <state>: "fulfilled", <value>: 100 }
// Promise { <state>: "fulfilled", <value>: "non-Promise value" }
```

### Using Promise.race() to implement request timeout

You can race a potentially long-lasting request with a timer that rejects, so that when the time limit has elapsed, the resulting promise automatically rejects.

```js
const data = Promise.race([
  fetch("/api"),
  new Promise((resolve, reject) => {
    // Reject after 5 seconds
    setTimeout(() => reject(new Error("Request timed out")), 5000);
  }),
])
  .then((res) => res.json())
  .catch((err) => displayError(err));
```

If the `data` promise fulfills, it will contain the data fetched from `/api`. `Promise.race` will capture and discard the settlement results of the losing promises, so the `"Request timed out"` rejection will not

bubble up as unhandled. Otherwise, if `fetch` remains pending for 5 seconds and loses the race with the `setTimeout` timer, the final promise will reject.

One promise finishing does not automatically cancel the other; the other's result is simply ignored. This does not pose problems in this small example, but it keeps resources such as network connections and timers alive for longer than necessary. To release resources early, abort the fetch if the timeout wins, or clear the timeout if the fetch wins. Whenever possible—including fetch—prefer using the `AbortController` API instead.

### Using Promise.race() to detect the status of a promise

Because `Promise.race()` resolves to the first non-pending promise in the iterable, we can check a promise's state, including if it's pending. This example is adapted from `promise-status-async`.

```js
function promiseState(promise) {
  const pendingState = { status: "pending" };

  return Promise.race([promise, pendingState]).then(
    (value) =>
      value === pendingState ? value : { status: "fulfilled", value },
    (reason) => ({ status: "rejected", reason }),
  );
}
```

In this function, if `promise` is pending, the second value, `pendingState`, which is a non-promise, becomes the result of the race; otherwise, if `promise` is already settled, we may know its state through the `onFulfilled` and `onRejected` handlers. For example:

```js
const p1 = new Promise((res) => setTimeout(() => res(100), 100));
const p2 = new Promise((res) => setTimeout(() => res(200), 200));
const p3 = new Promise((res, rej) =>
  setTimeout(() => rej(new Error("failed")), 100),
);

async function getStates() {
  console.log(await promiseState(p1));
  console.log(await promiseState(p2));
  console.log(await promiseState(p3));
}

console.log("Immediately after initiation:");
getStates();
setTimeout(() => {
  console.log("After waiting for 100ms:");
  getStates();
}, 100);

// Logs:
// Immediately after initiation:
// { status: 'pending' }
// { status: 'pending' }
// { status: 'pending' }
// After waiting for 100ms:
// { status: 'fulfilled', value: 100 }
// { status: 'pending' }
// { status: 'rejected', reason: Error: failed }
```

> **Note:** The `promiseState` function still runs asynchronously, because there is no way to synchronously get a promise's value (i.e., without `then()` or `await`), even when it is already settled. However, `promiseState()` always fulfills within one tick and never actually waits for any promise's settlement.

### Comparison with Promise.any()

`Promise.race` takes the first settled `Promise`.

```js
const promise1 = new Promise((resolve, reject) => {
  setTimeout(resolve, 500, "one");
});

const promise2 = new Promise((resolve, reject) => {
  setTimeout(reject, 100, "two");
});

Promise.race([promise1, promise2])
  .then((value) => {
    console.log("succeeded with value:", value);
  })
  .catch((reason) => {
    // Only promise1 is fulfilled, but promise2 is faster
    console.error("failed with reason:", reason);
  });
// failed with reason: two
```

`Promise.any` takes the first fulfilled `Promise`.

```js
const promise1 = new Promise((resolve, reject) => {
  setTimeout(resolve, 500, "one");
});

const promise2 = new Promise((resolve, reject) => {
  setTimeout(reject, 100, "two");
});

Promise.any([promise1, promise2])
  .then((value) => {
    // Only promise1 is fulfilled, even though promise2 settled sooner
    console.log("succeeded with value:", value);
  })
  .catch((reason) => {
    console.error("failed with reason:", reason);
  });
// succeeded with value: one
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-promise.race |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `race()` | 32 | 12 | 29 | 19 | 8 | 32 | 29 | 19 | 8 | 2.0 | 4.4.3 | 8 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Promise.race`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Promise`
- `Promise.all()`
- `Promise.allSettled()`
- `Promise.any()`

---

JavaScript › Reference › Standard built-in objects › Promise

# Promise.reject()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/reject · MDN page last modified 2026-09-14

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Promise.reject()`** static method returns a `Promise` object that is rejected with a given reason.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Promise.reject()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function resolved(result) {
  console.log("Resolved");
}

function rejected(result) {
  console.error(result);
}

Promise.reject(new Error("fail")).then(resolved, rejected);
// Expected output: Error: fail
```

## Syntax

js

```
Promise.reject(reason)
```

### Parameters

**`reason`**

Reason why this `Promise` rejected.

### Return value

A `Promise` that is rejected with the given reason.

## Description

The static `Promise.reject` function returns a `Promise` that is rejected. For debugging purposes and selective error catching, it is useful to make `reason` an `instanceof` `Error`.

`Promise.reject()` is generic and supports subclassing, which means it can be called on subclasses of `Promise`, and the result will be a promise of the subclass type. To do so, the subclass's constructor must implement the same signature as the `Promise()` constructor — accepting a single `executor` function that can be called with the `resolve` and `reject` callbacks as parameters. `Promise.reject()` is essentially a shorthand for `new Promise((resolve, reject) => reject(reason))`.

Unlike `Promise.resolve()`, `Promise.reject()` always wraps `reason` in a new `Promise` object, even when `reason` is already a `Promise`.

## Examples

### Using the static Promise.reject() method

```js
Promise.reject(new Error("fail")).then(
  () => {
    // not called
  },
  (error) => {
    console.error(error); // Stacktrace
  },
);
```

### Rejecting with a promise

Unlike `Promise.resolve`, the `Promise.reject` method does not reuse existing `Promise` instances. It always returns a new `Promise` instance that wraps `reason`.

```js
const p = Promise.resolve(1);
const rejected = Promise.reject(p);
console.log(rejected === p); // false
rejected.catch((v) => {
  console.log(v === p); // true
});
```

### Calling reject() on a non-Promise constructor

`Promise.reject()` is a generic method. It can be called on any constructor that implements the same signature as the `Promise()` constructor. For example, we can call it on a constructor that passes it `console.log` as `reject`:

```js
class NotPromise {
  constructor(executor) {
    // The "resolve" and "reject" functions behave nothing like the
    // native promise's, but Promise.reject() calls them in the same way.
    executor(
      (value) => console.log("Resolved", value),
      (reason) => console.log("Rejected", reason),
    );
  }

  static reject = Promise.reject;
}

const p = NotPromise.reject("foo"); // Logs "Rejected foo"
// p is a NotPromise instance
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-promise.reject |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Nodejs |
| `reject()` | 32 | 12 | 29 | 19 | 8 | 32 | 29 | 19 | 8 | 2.0 | 4.4.3 | 8 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Promise.reject`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Promise`

---



# Promise.resolve()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/resolve · MDN page last modified 2026-09-14

**Baseline: Widely available** — available across major browsers since July 2015.

The **`Promise.resolve()`** static method "resolves" a given value to a `Promise`. If the value is a promise, that promise is returned; if the value is a thenable, `Promise.resolve()` will call the `then()` method with two callbacks it prepared; otherwise the returned promise will be fulfilled with the value.

This function flattens nested layers of promise-like objects (e.g., a promise that fulfills to a promise that fulfills to something) into a single layer — a promise that fulfills to a non-thenable value.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Promise.resolve()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const promise1 = Promise.resolve(123);

promise1.then((value) => {
  console.log(value);
  // Expected output: 123
});
```

## Syntax

js

```
Promise.resolve(value)
```

### Parameters

**`value`**

Argument to be resolved by this `Promise`. Can also be a `Promise` or a thenable to resolve.

### Return value

A `Promise` that is resolved with the given value, or the promise passed as value, if the value was a promise object. A resolved promise can be in any of the states — fulfilled, rejected, or pending. For example, resolving a rejected promise will still result in a rejected promise.

## Description

`Promise.resolve()` *resolves* a promise, which is not the same as fulfilling or rejecting the promise. See Promise description for definitions of the terminology. In brief, `Promise.resolve()` returns a promise whose eventual state depends on another promise, thenable object, or other value.

> **Note:** If evaluating the `value` expression may synchronously throw an error, this error won't be caught and wrapped in a rejected promise by `Promise.resolve()`. Consider using `Promise.try(() => value)` in this case.

`Promise.resolve()` is generic and supports subclassing, which means it can be called on subclasses of `Promise`, and the result will be a promise of the subclass type. To do so, the subclass's constructor must implement the same signature as the `Promise()` constructor — accepting a single `executor` function that can be called with the `resolve` and `reject` callbacks as parameters.

`Promise.resolve()` special-cases native `Promise` instances. If `value` belongs to `Promise` or a subclass, and `value.constructor === Promise`, then `value` is directly returned by `Promise.resolve()`, without creating a new `Promise` instance. Otherwise, `Promise.resolve()` is essentially a shorthand for `new Promise((resolve) => resolve(value))`.

The bulk of the resolving logic is actually implemented by the `resolve` function passed by the `Promise()` constructor. In summary:

- If a non-thenable value is passed, the returned promise is already fulfilled with that value.

- If a thenable is passed, the returned promise will adopt the state of that thenable by calling the `then` method and passing a pair of resolving functions as arguments. (But because native promises directly pass through `Promise.resolve()` without creating a wrapper, the `then` method is not called on native promises.) If the `resolve` function receives another thenable object, it will be resolved again, so that the eventual fulfillment value of the promise will never be thenable.

## Examples

### Using the static Promise.resolve method

```js
Promise.resolve("Success").then(
  (value) => {
    console.log(value); // "Success"
  },
  (reason) => {
    // not called
  },
);
```

### Resolving an array

```js
const p = Promise.resolve([1, 2, 3]);
p.then((v) => {
  console.log(v[0]); // 1
});
```

### Resolving another Promise

`Promise.resolve()` reuses existing `Promise` instances. If it's resolving a native promise, it returns the same promise instance without creating a wrapper.

```js
const original = Promise.resolve(33);
const cast = Promise.resolve(original);
cast.then((value) => {
  console.log(`value: ${value}`);
});
console.log(`original === cast ? ${original === cast}`);

// Logs, in order:
// original === cast ? true
// value: 33
```

The inverted order of the logs is due to the fact that the `then` handlers are called asynchronously. See the `then()` reference for more information.

### Resolving thenables and throwing Errors

```js
// Resolving a thenable object
const p1 = Promise.resolve({
  then(onFulfill, onReject) {
    onFulfill("fulfilled!");
  },
});
console.log(p1 instanceof Promise); // true, object casted to a Promise

p1.then(
  (v) => {
    console.log(v); // "fulfilled!"
  },
  (e) => {
    // not called
  },
);

// Thenable throws
// Promise rejects
const p2 = Promise.resolve({
  then() {
    throw new TypeError("Throwing");
  },
});
p2.then(
  (v) => {
    // not called
  },
  (e) => {
    console.error(e); // TypeError: Throwing
  },
);

// Thenable throws after callback
// Promise resolves
const p3 = Promise.resolve({
  then(onFulfilled) {
    onFulfilled("Resolving");
    throw new TypeError("Throwing");
  },
});
p3.then(
  (v) => {
    console.log(v); // "Resolving"
  },
  (e) => {
    // not called
  },
);
```

Nested thenables will be "deeply flattened" to a single promise.

js

```
const thenable = {
  then(onFulfilled, onRejected) {
    onFulfilled({
      // The thenable is fulfilled with another thenable
      then(onFulfilled, onRejected) {
        onFulfilled(42);
      },
    });
  },
};

Promise.resolve(thenable).then((v) => {
  console.log(v); // 42
});
```

> **Warning:** Do not call `Promise.resolve()` on a thenable that resolves to itself. That leads to infinite recursion, because it attempts to flatten an infinitely-nested promise.

**js · ✗ bad example**

```
const thenable = {
  then(onFulfilled, onRejected) {
    onFulfilled(thenable);
  },
};

Promise.resolve(thenable); // Will lead to infinite recursion.
```

### Calling resolve() on a non-Promise constructor

`Promise.resolve()` is a generic method. It can be called on any constructor that implements the same signature as the `Promise()` constructor. For example, we can call it on a constructor that passes it `console.log` as `resolve`:

js

```
class NotPromise {
  constructor(executor) {
    // The "resolve" and "reject" functions behave nothing like the
    // native promise's, but Promise.resolve() calls them in the same way.
    executor(
      (value) => console.log("Resolved", value),
      (reason) => console.log("Rejected", reason),
    );
  }

  static resolve = Promise.resolve;
}

const p = NotPromise.resolve("foo"); // Logs "Resolved foo"
// p is a NotPromise instance
```

The ability to flatten nested thenables is implemented by the `resolve` function of the `Promise()` constructor, so if you call it on another constructor, nested thenables may not be flattened, depending on how that constructor implements its `resolve` function.

```js
const thenable = {
  then(onFulfilled, onRejected) {
    onFulfilled({
      // The thenable is fulfilled with another thenable
      then(onFulfilled, onRejected) {
        onFulfilled(42);
      },
    });
  },
};

const p = NotPromise.resolve(thenable); // Logs "Resolved { then: [Function: then] }"
// p is a NotPromise instance
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-promise.resolve |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| resolve() | 32 | 12 | 29 | 19 | 8 | 32 | 29 | 19 | 8 | 2.0 | 4.4.3 | 8 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Promise.resolve`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Promise`

---

JavaScript › Reference › Standard built-in objects › Promise

# Promise.try()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/try · MDN page last modified 2026-10-26

**Baseline 2025: Newly available** — works across the latest browser versions since January 2025; may not work in older ones.

The **`Promise.try()`** static method takes a callback of any kind (returns or throws, synchronously or asynchronously) and resolves its result to a `Promise`.

## Syntax

```js
Promise.try(func)
Promise.try(func, arg1)
Promise.try(func, arg1, arg2)
Promise.try(func, arg1, arg2, /* …, */ argN)
```

### Parameters

**`func`**

A function that is called synchronously with the arguments provided (`arg1`, `arg2`, ..., `argN`). It can do anything—either return a value, throw an error, or return a promise.

**`arg1, arg2, ..., argN`**

Arguments to pass to `func`.

### Return value

A `Promise` that is:

- Already fulfilled, if `func` synchronously returns a value.
- Already rejected, if `func` synchronously throws an error.
- Asynchronously fulfilled or rejected, if `func` returns a promise. The returned value is resolved to a promise, which means built-in `Promise` objects are returned as-is.

## Description

You may have an API that takes a callback. The callback may be synchronous or asynchronous. You want to handle everything uniformly by wrapping the result in a promise. The most straightforward way might be `Promise.resolve(func())`. The problem is that if `func()` synchronously throws an error, this error would not be caught and turned into a rejected promise.

You can wrap this expression in `try...catch`:

```js
let result;
try {
  result = Promise.resolve(func());
} catch (error) {
  result = Promise.reject(error);
}
```

The problem is that `try...catch` is not an expression, so you can't directly use it in expression positions like passing it to other functions.

Therefore, when lifting a function call result into a promise, fulfilled or rejected, people more commonly do this:

```js
new Promise((resolve) => resolve(func()));
```

For the built-in `Promise()` constructor, errors thrown from the executor are automatically caught and turned into rejections, so this also prevents synchronous errors. The problem is that it unconditionally creates a new `Promise` object, which is unnecessary if `func()` already returns a `Promise`. `Promise.resolve()`, on the other hand, is smart enough to prevent that extra promise wrapping.

`Promise.try()` is almost exactly equivalent to the `try...catch` approach, except that it's shorter and can be used as an expression:

```js
Promise.try(func);
```

Note that `Promise.try()` is *not* equivalent to this, despite being highly similar:

```js
Promise.resolve().then(func);
```

The difference is that the callback passed to `then()` is always called asynchronously, while the executor of the `Promise()` constructor is called synchronously. `Promise.try` also calls the function synchronously, and resolves the promise immediately if possible.

`Promise.try()`, combined with `catch()` and `finally()`, can be used to handle both synchronous and asynchronous errors in a single chain, and make promise error handling appear almost like synchronous error handling.

Like `setTimeout()`, `Promise.try()` accepts extra arguments that are passed to the callback. This means instead of doing this:

```js
Promise.try(() => func(arg1, arg2));
```

You can do this:

```js
Promise.try(func, arg1, arg2);
```

Which are equivalent, but the latter avoids creating an extra closure and is more efficient.

`Promise.try()` is generic and supports subclassing, which means it can be called on subclasses of `Promise`, and the result will contain a promise of the subclass type. To do so, the subclass's constructor must implement the same signature as the `Promise()` constructor — accepting a single `executor` function that can be called with the `resolve` and `reject` callbacks as parameters.

## Examples

### Using Promise.try()

The following example takes a callback, "lifts" it into a promise, handles the result, and does some error handling:

```js
function doSomething(action) {
  return Promise.try(action)
    .then((result) => console.log(result))
    .catch((error) => console.error(error))
    .finally(() => console.log("Done"));
}

doSomething(() => "Sync result");

doSomething(() => {
  throw new Error("Sync error");
});

doSomething(async () => "Async result");

doSomething(async () => {
  throw new Error("Async error");
});
```

In async/await, the same code would look like this:

```js
async function doSomething(action) {
  try {
    const result = await action();
    console.log(result);
  } catch (error) {
    console.error(error);
  } finally {
    console.log("Done");
  }
}
```

### Calling try() on a non-Promise constructor

`Promise.try()` is a generic method. It can be called on any constructor that implements the same signature as the `Promise()` constructor.

The following is a slightly more faithful approximation of the actual `Promise.try()` (although it should still not be used as a polyfill):

```js
Promise.try = function (func, ...args) {
  let result;
  try {
    result = func(...args);
  } catch (error) {
    return Promise.reject.call(this, error);
  }
  return Promise.resolve.call(this, result);
};
```

`Promise.try()` delegates to `Promise.resolve()` and `Promise.reject()` to create the return value, and both of these functions are generic.

For example, we can call it on a constructor that passes `console.log` as the `resolve` and `reject` functions to `executor`:

```js
class NotPromise {
  constructor(executor) {
    // The "resolve" and "reject" functions behave nothing like the native
    // promise's, but Promise.try() just calls resolve
    executor(
      (value) => console.log("Resolved", value),
      (reason) => console.log("Rejected", reason),
    );
  }

  static try = Promise.try;
}

const p = NotPromise.try(() => "hello");
// Logs: Resolved hello
// p is a NotPromise instance
```

*(code continues on the next page)*

*(code continued from the previous page)*

```
const p2 = NotPromise.try(() => {
  throw new Error("oops");
});
// Logs: Rejected Error: oops
// p2 is a NotPromise instance
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-promise.try |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `try` | 128 | 128 | 134 | 114 | 18.2 | 128 | 134 | 85 | 18.2 | 28.0 | 128 | 18.2 | 1.1.22 | 1.46 | 23.0.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Promise.try`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Promise.try` in `core-js`
- es-shims polyfill of `Promise.try`
- Using promises guide
- `Promise`
- `Promise()` constructor

---

JavaScript › Reference › Standard built-in objects › Promise

# Promise.withResolvers()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/withResolvers · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since March 2024.

The **`Promise.withResolvers()`** static method returns an object containing a new `Promise` object and two functions to resolve or reject it, corresponding to the two parameters passed to the executor of the `Promise()` constructor.

## Syntax

js

```
Promise.withResolvers()
```

### Parameters

None.

### Return value

A plain object containing the following properties:

**`promise`**

A `Promise` object.

**`resolve`**

A function that resolves the promise. For its semantics, see the `Promise()` constructor reference.

**`reject`**

A function that rejects the promise. For its semantics, see the `Promise()` constructor reference.

## Description

`Promise.withResolvers()` is exactly equivalent to the following code:

```js
let resolve, reject;
const promise = new Promise((res, rej) => {
  resolve = res;
  reject = rej;
});
```

Except that it is more concise and does not require the use of `let`.

The key difference when using `Promise.withResolvers()` is that the resolution and rejection functions now live in the same scope as the promise itself, instead of being created and used once within the executor. This may enable some more advanced use cases, such as when reusing them for recurring events, particularly with streams and queues. This also generally results in less nesting than wrapping a lot of logic within the executor.

`Promise.withResolvers()` is generic and supports subclassing, which means it can be called on subclasses of `Promise`, and the result will contain a promise of the subclass type. To do so, the subclass's constructor must implement the same signature as the `Promise()` constructor — accepting a single `executor` function that can be called with the `resolve` and `reject` callbacks as parameters.

## Examples

### Transforming a stream to an async iterable

The use case of `Promise.withResolvers()` is when you have a promise that should be resolved or rejected by some event listener that cannot be wrapped inside the promise executor. The following example transforms a Node.js readable stream to an async iterable. Each `promise` here represents a single batch of data available, and each time the current batch is read, a new promise is created for the next batch. Note how the event listeners are only attached once, but actually call a different version of the `resolve` and `reject` functions each time.

```js
async function* readableToAsyncIterable(stream) {
  let { promise, resolve, reject } = Promise.withResolvers();
  stream.on("error", (error) => reject(error));
  stream.on("end", () => resolve());
  stream.on("readable", () => resolve());

  while (stream.readable) {
    await promise;
    let chunk;
    while ((chunk = stream.read())) {
      yield chunk;
    }
    ({ promise, resolve, reject } = Promise.withResolvers());
  }
}
```

### Calling withResolvers() on a non-Promise constructor

`Promise.withResolvers()` is a generic method. It can be called on any constructor that implements the same signature as the `Promise()` constructor. For example, we can call it on a constructor that passes `console.log` as the `resolve` and `reject` functions to `executor`:

```js
class NotPromise {
  constructor(executor) {
    // The "resolve" and "reject" functions behave nothing like the native
    // promise's, but Promise.withResolvers() just returns them, as is.
    executor(
      (value) => console.log("Resolved", value),
      (reason) => console.log("Rejected", reason),
    );
  }

  static withResolvers = Promise.withResolvers;
}

const { promise, resolve, reject } = NotPromise.withResolvers();
resolve("hello");
// Logs: Resolved hello
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-promise.withResolvers |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `withResolvers` | 119 | 119 | 121 | 105 | 17.4 | 119 | 121 | 79 | 17.4 | 25.0 | 119 | 17.4 | 1.0.0 | 1.38 | 22.0.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Promise.withResolvers`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Promise.withResolvers` in `core-js`
- es-shims polyfill of `Promise.withResolvers`
- Using promises guide
- `Promise`
- `Promise()` constructor

---



# Promise[Symbol.species]

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/Symbol.species · MDN page last modified 2025-07-10

**Baseline: Widely available** — available across major browsers since January 2020.

The **`Promise[Symbol.species]`** static accessor property returns the constructor used to construct return values from promise methods.

> **Warning:** The existence of `[Symbol.species]` allows execution of arbitrary code and may create security vulnerabilities. It also makes certain optimizations much harder. Engine implementers are investigating whether to remove this feature. Avoid relying on it if possible.

## Syntax

```js
Promise[Symbol.species]
```

### Return value

The value of the constructor (`this`) on which `get [Symbol.species]` was called. The return value is used to construct return values from promise chaining methods that create new promises.

## Description

The `[Symbol.species]` accessor property returns the default constructor for `Promise` objects. Subclass constructors may override it to change the constructor assignment. The default implementation is basically:

```js
// Hypothetical underlying implementation for illustration
class Promise {
  static get [Symbol.species]() {
    return this;
  }
}
```

Because of this polymorphic implementation, `[Symbol.species]` of derived subclasses would also return the constructor itself by default.

```js
class SubPromise extends Promise {}
SubPromise[Symbol.species] === SubPromise; // true
```

Promise chaining methods — `then()`, `catch()`, and `finally()` — return new promise objects. They get the constructor to construct the new promise through `this.constructor[Symbol.species]`. If `this.constructor` is `undefined`, or if `this.constructor[Symbol.species]` is `undefined` or `null`, the default `Promise()` constructor is used. Otherwise, the constructor returned by `this.constructor[Symbol.species]` is used to construct the new promise object.

## Examples

### Species in ordinary objects

The `Symbol.species` property returns the default constructor function, which is the `Promise` constructor for `Promise`.

```js
Promise[Symbol.species]; // [Function: Promise]
```

### Species in derived objects

In an instance of a custom `Promise` subclass, such as `MyPromise`, the `MyPromise` species is the `MyPromise` constructor. However, you might want to override this, in order to return parent `Promise` objects in your derived class methods.

```js
class MyPromise extends Promise {
  // Override MyPromise species to the parent Promise constructor
  static get [Symbol.species]() {
    return Promise;
  }
}
```

By default, promise methods would return promises with the type of the subclass.

```js
class MyPromise extends Promise {
  someValue = 1;
}

console.log(MyPromise.resolve(1).then(() => {}).someValue); // 1
```

By overriding `[Symbol.species]`, the promise methods will return the base `Promise` type.

```js
class MyPromise extends Promise {
  someValue = 1;
  static get [Symbol.species]() {
    return Promise;
  }
}

console.log(MyPromise.resolve(1).then(() => {}).someValue); // undefined
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-get-promise-%symbol.species% |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| [Symbol.species] | 51 | 79 | 48 | 38 | 10 | 51 | 48 | 41 | 10 | 5.0 | 51 | 10 | 1.0.0 | 1.0 | 6.5.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Promise.@@species`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Promise`
- `Symbol.species`

---

JavaScript › Reference › Standard built-in objects › Promise

# Promise.prototype.catch()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/catch · MDN page last modified 2025-07-20

**Baseline: Widely available** — available across major browsers since July 2015.

The **`catch()`** method of `Promise` instances schedules a function to be called when the promise is rejected. It immediately returns another `Promise` object, allowing you to chain calls to other promise methods. It is a shortcut for `then(undefined, onRejected)`.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Promise.prototype.catch()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const promise = new Promise((resolve, reject) => {
  throw new Error("Uh-oh!");
});

promise.catch((error) => {
  console.error(error);
});
// Expected output: Error: Uh-oh!
```

## Syntax

js

```
promiseInstance.catch(onRejected)
```

### Parameters

**`onRejected`**

A function to asynchronously execute when this promise becomes rejected. Its return value becomes the fulfillment value of the promise returned by `catch()`. The function is called with the following arguments:

**`reason`**

The value that the promise was rejected with.

### Return value

Returns a new `Promise`. This new promise is always pending when returned, regardless of the current promise's status. If `onRejected` is called, the returned promise will resolve based on the return value of this call, or reject with the thrown error from this call. If the current promise fulfills, `onRejected` is not called and the returned promise fulfills to the same value.

## Description

The `catch` method is used for error handling in promise composition. Since it returns a `Promise`, it can be chained in the same way as its sister method, `then()`.

If a promise becomes rejected, and there are no rejection handlers to call (a handler can be attached through any of `then()`, `catch()`, or `finally()`), then the rejection event is surfaced by the host. In the browser, this results in an `unhandledrejection` event. If a handler is attached to a rejected promise whose rejection has already caused an unhandled rejection event, then another `rejectionhandled` event is fired.

`catch()` internally calls `then()` on the object upon which it was called, passing `undefined` and `onRejected` as arguments. The value of that call is directly returned. This is observable if you wrap the methods.

```js
// overriding original Promise.prototype.then/catch just to add some logs
((Promise) => {
  const originalThen = Promise.prototype.then;
  const originalCatch = Promise.prototype.catch;

  Promise.prototype.then = function (...args) {
    console.log("Called .then on %o with arguments: %o", this, args);
    return originalThen.apply(this, args);
  };
  Promise.prototype.catch = function (...args) {
    console.error("Called .catch on %o with arguments: %o", this, args);
    return originalCatch.apply(this, args);
  };
})(Promise);

// calling catch on an already resolved promise
Promise.resolve().catch(function XXX() {});

// Logs:
// Called .catch on Promise{} with arguments: Arguments{1} [0: function XXX()]
// Called .then on Promise{} with arguments: Arguments{2} [0: undefined, 1: function XXX()]
```

This means that passing `undefined` still causes the returned promise to be rejected, and you have to pass a function to prevent the final promise from being rejected.

Because `catch()` just calls `then()`, it supports subclassing.

> **Note:** The examples below are throwing instances of `Error`. As with synchronous `throw` statements, this is considered a good practice; otherwise, the part doing the catching would have to perform checks to see if the argument was a string or an error, and you might lose valuable information such as stack traces.

## Examples

### Using and chaining the catch() method

```js
const p1 = new Promise((resolve, reject) => {
  resolve("Success");
});

p1.then((value) => {
  console.log(value); // "Success!"
  throw new Error("oh, no!");
})
  .catch((e) => {
    console.error(e.message); // "oh, no!"
  })
  .then(
    () => console.log("after a catch the chain is restored"), // "after a catch the chain is
restored"
    () => console.log("Not fired due to the catch"),
  );
```

*(code continues on the next page)*

(code continued from the previous page)

```
// The following behaves the same as above
p1.then((value) => {
  console.log(value); // "Success!"
  return Promise.reject(new Error("oh, no!"));
})
  .catch((e) => {
    console.error(e); // Error: oh, no!
  })
  .then(
    () => console.log("after a catch the chain is restored"), // "after a catch the chain is
restored"
    () => console.log("Not fired due to the catch"),
  );
```

### Gotchas when throwing errors

Throwing an error will call the `catch()` method most of the time:

js

```
const p1 = new Promise((resolve, reject) => {
  throw new Error("Uh-oh!");
});

p1.catch((e) => {
  console.error(e); // "Uh-oh!"
});
```

Errors thrown inside asynchronous functions will act like uncaught errors:

js

```
const p2 = new Promise((resolve, reject) => {
  setTimeout(() => {
    throw new Error("Uncaught Exception!");
  }, 1000);
});

p2.catch((e) => {
  console.error(e); // This is never called
});
```

Errors thrown after `resolve` is called will be silenced:

js

```
const p3 = new Promise((resolve, reject) => {
  resolve();
  throw new Error("Silenced Exception!");
});

p3.catch((e) => {
  console.error(e); // This is never called
});
```

### catch() is not called if the promise is fulfilled

js

```
// Create a promise which would not call onReject
const p1 = Promise.resolve("calling next");
```

(code continues on the next page)

*(code continued from the previous page)*

```
const p2 = p1.catch((reason) => {
  // This is never called
  console.error("catch p1!");
  console.error(reason);
});

p2.then(
  (value) => {
    console.log("next promise's onFulfilled");
    console.log(value); // calling next
  },
  (reason) => {
    console.log("next promise's onRejected");
    console.log(reason);
  },
);
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-promise.prototype.catch |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `catch()` | 32 | 12 | 29 | 19 | 8 | 32 | 29 | 19 | 8 | 2.0 | 4.4.3 | 8 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Promise.catch`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Promise`
- `Promise.prototype.then()`
- `Promise.prototype.finally()`

---

JavaScript › Reference › Standard built-in objects › Promise

# Promise.prototype.finally()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/finally · MDN page last modified 2026-10-26

**Baseline: Widely available** — available across major browsers since October 2018.

The **`finally()`** method of `Promise` instances schedules a function to be called when the promise is settled (either fulfilled or rejected). It immediately returns another `Promise` object, allowing you to chain calls to other promise methods.

Like the `finally` block, this method is usually intended for cleanup actions, regardless of the promise's outcome. It lets you avoid duplicating code in both the promise's `then()` and `catch()` handlers.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: Promise.prototype.finally()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function checkMail() {
  return new Promise((resolve, reject) => {
    if (Math.random() > 0.5) {
      resolve("Mail has arrived");
    } else {
      reject(new Error("Failed to arrive"));
    }
  });
}

checkMail()
  .then((mail) => {
    console.log(mail);
  })
  .catch((err) => {
    console.error(err);
  })
  .finally(() => {
    console.log("Experiment completed");
  });
```

## Syntax

js

```
promiseInstance.finally(onFinally)
```

### Parameters

**`onFinally`**

A function to asynchronously execute when this promise becomes settled. If the function returns a promise, the resulting promise will wait for that promise to settle before continuing. If the returned promise is rejected, the resulting promise is rejected with the same reason. Any other returned value, or the fulfilled value of the returned promise, is ignored.

### Return value

Returns a new `Promise` immediately. This new promise is always pending when returned, regardless of the current promise's status. If `onFinally` throws an error or returns a rejected promise, the new promise will reject with that value. Otherwise, the new promise will settle with the same state as the current promise.

## Description

The `finally()` method can be useful if you want to do some processing or cleanup once the promise is settled, regardless of its outcome.

The `finally()` method is very similar to calling `then(onFinally, onFinally)`. However, there are a couple of differences:

- When creating a function inline, you can pass it once, instead of being forced to either declare it twice, or create a variable for it.
- The `onFinally` callback does not receive any argument. This use case is for precisely when you *do not care* about the rejection reason or the fulfillment value, and so there's no need to provide it.
- A `finally()` call is usually transparent and reflects the eventual state of the original promise. So for example:
  - Unlike `Promise.resolve(2).then(() => 77, () => 77)`, which returns a promise eventually fulfilled with the value `77`, `Promise.resolve(2).finally(() => 77)` returns a promise eventually fulfilled with the value `2`.
  - Similarly, unlike `Promise.reject(3).then(() => 88, () => 88)`, which returns a promise eventually fulfilled with the value `88`, `Promise.reject(3).finally(() => 88)` returns a promise eventually rejected with the reason `3`.

> **Note:** A `throw` (or returning a rejected promise) in the `finally` callback still rejects the returned promise. For example, both `Promise.reject(3).finally(() => { throw 99; })` and `Promise.reject(3).finally(() => Promise.reject(99))` reject the returned promise with the reason `99`.

Like `catch()`, `finally()` internally calls the `then` method on the object upon which it was called. If `onFinally` is not a function, `then()` is called with `onFinally` as both arguments — which, for `Promise.prototype.then()`, means that no useful handler is attached. Otherwise, `then()` is called with two internally created functions, which behave like the following:

> **Warning:** This is only for demonstration purposes and is not a polyfill.

js

```
promise.then(
  (value) => Promise.resolve(onFinally()).then(() => value),
  (reason) =>
    Promise.resolve(onFinally()).then(() => {
      throw reason;
    }),
);
```

Because `finally()` calls `then()`, it supports subclassing. Moreover, notice the `Promise.resolve()` call above — in reality, `onFinally()`'s return value is resolved using the same algorithm as `Promise.resolve()`, but the actual constructor used to construct the resolved promise will be the subclass. `finally()` gets this constructor through `promise.constructor[Symbol.species]`.

## Examples

### Using finally()

js

```
let isLoading = true;

fetch(myRequest)
  .then((response) => {
    const contentType = response.headers.get("content-type");
    if (contentType && contentType.includes("application/json")) {
      return response.json();
    }
```

*(code continues on the next page)*

*(code continued from the previous page)*

```
    throw new TypeError("Oops, we haven't got JSON!");
  })
  .then((json) => {
    /* process your JSON further */
  })
  .catch((error) => {
    console.error(error); // this line can also throw, e.g. when console = {}
  })
  .finally(() => {
    isLoading = false;
  });
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-promise.prototype.finally` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `finally()` | 63 | 18 | 58 | 50 | 11.1 | 63 | 58 | 46 | 11.3 | 8.0 | 63 | 11.3 | 1.0.0 | 1.0 | 10.0.0 |

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Promise.finally`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Polyfill of `Promise.prototype.finally` in `core-js`
- es-shims polyfill of `Promise.prototype.finally`
- `Promise`
- `Promise.prototype.then()`
- `Promise.prototype.catch()`

---

JavaScript › Reference › Standard built-in objects › Promise

# Promise.prototype.then()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/then · MDN page last modified 2026-09-01

**Baseline: Widely available** — available across major browsers since July 2015.

The **`then()`** method of `Promise` instances takes up to two arguments: callback functions for the fulfilled and rejected cases of the `Promise`. It stores the callbacks within the promise it is called on and immediately returns another `Promise` object, allowing you to chain calls to other promise methods.

## Try it

*▷ Interactive example on MDN: "JavaScript Demo: Promise.prototype.then()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const promise1 = new Promise((resolve, reject) => {
  resolve("Success!");
});

promise1.then((value) => {
  console.log(value);
  // Expected output: "Success!"
});
```

## Syntax

js

```
then(onFulfilled)
then(onFulfilled, onRejected)
```

### Parameters

**`onFulfilled`**

A function to asynchronously execute when this promise becomes fulfilled. Its return value becomes the fulfillment value of the promise returned by `then()`. The function is called with the following arguments:

**`value`**

The value that the promise was fulfilled with.

If it is not a function, it is internally replaced with an *identity* function (`(x) => x`) which simply passes the fulfillment value forward.

**`onRejected`** Optional

A function to asynchronously execute when this promise becomes rejected. Its return value becomes the fulfillment value of the promise returned by `then()`. The function is called with the following arguments:

**`reason`**

The value that the promise was rejected with.

If it is not a function, it is internally replaced with a *thrower* function (`(x) => { throw x; }`) which throws the rejection reason it received.

### Return value

Returns a new `Promise` immediately. This returned promise is always pending when returned, regardless of the current promise's status.

One of the `onFulfilled` and `onRejected` handlers will be executed to handle the current promise's fulfillment or rejection. The call always happens asynchronously, even when the current promise is already settled. The behavior of the promise returned by `then()` (referred to as `p` in the following list) depends on the handler's execution result, following a specific set of rules. If the handler function:

- returns a value: `p` gets fulfilled with the returned value as its value.

- doesn't return anything: `p` gets fulfilled with `undefined` as its value.
- throws an error: `p` gets rejected with the thrown error as its value.
- returns an already fulfilled promise: `p` gets fulfilled with that promise's value as its value.
- returns an already rejected promise: `p` gets rejected with that promise's value as its value.
- returns another pending promise: `p` is pending and becomes fulfilled/rejected with that promise's value as its value immediately after that promise becomes fulfilled/rejected.

## Description

The `then()` method schedules callback functions for the eventual completion of a Promise — either fulfillment or rejection. It is the primitive method of promises: the thenable protocol expects all promise-like objects to expose a `then()` method, and the `catch()` and `finally()` methods both work by invoking the object's `then()` method.

For more information about the `onRejected` handler, see the `catch()` reference.

`then()` returns a new promise object but mutates the promise object it's called on, appending the handlers to an internal list if the promise is pending. Therefore the handler is retained by the original promise for as long as that promise remains pending and reachable. For example, the following example will eventually run out of memory even though the returned promise is not retained:

```js
const pendingPromise = new Promise(() => {});
while (true) {
  pendingPromise.then(doSomething);
}
```

If you call the `then()` method twice on the same promise object (instead of chaining), then this promise object will have two pairs of settlement handlers. All handlers attached to the same promise object are always called in the order they were added. Moreover, the two promises returned by each call of `then()` start separate chains and do not wait for each other's settlement.

Thenable objects that arise along the `then()` chain are always resolved — the `onFulfilled` handler never receives a thenable object, and any thenable returned by either handler are always resolved before being passed to the next handler. This is because when constructing the new promise, the `resolve` and `reject` functions passed by the `executor` are saved, and when the current promise settles, the respective function will be called with the fulfillment value or rejection reason. The resolving logic comes from the `resolve` function passed by the `Promise()` constructor.

`then()` supports subclassing, which means it can be called on instances of subclasses of `Promise`, and the result will be a promise of the subclass type. You can customize the type of the return value through the `[Symbol.species]` property.

## Examples

### Using the then() method

```js
const p1 = new Promise((resolve, reject) => {
  resolve("Success!");
  // or
  // reject(new Error("Error!"));
});

p1.then(
  (value) => {
    console.log(value); // Success!
  },
  (reason) => {
    console.error(reason); // Error!
  },
);
```

### Having a non-function as either parameter

```js
Promise.resolve(1).then(2).then(console.log); // 1
Promise.reject(new Error("failed")).then(2, 2).then(console.log, console.log); // Error:
failed
```

### Chaining

The `then` method returns a new `Promise`, which allows for method chaining.

If the function passed as handler to `then` returns a `Promise`, an equivalent `Promise` will be exposed to the subsequent `then` in the method chain. The below snippet simulates asynchronous code with the `setTimeout` function.

```js
Promise.resolve("foo")
  // 1. Receive "foo", concatenate "bar" to it, and resolve that to the next then
  .then(
    (string) =>
      new Promise((resolve, reject) => {
        setTimeout(() => {
          string += "bar";
          resolve(string);
        }, 1);
      }),
  )
  // 2. receive "foobar", register a callback function to work on that string
  // and print it to the console, but not before returning the unworked on
  // string to the next then
  .then((string) => {
    setTimeout(() => {
      string += "baz";
      console.log(string); // foobarbaz
    }, 1);
    return string;
  })
  // 3. print helpful messages about how the code in this section will be run
  // before the string is actually processed by the mocked asynchronous code in the
  // previous then block.
  .then((string) => {
```

*(code continues on the next page)*

(code continued from the previous page)

```
    console.log(
      "Last Then: oops... didn't bother to instantiate and return a promise in the prior
then so the sequence may be a bit surprising",
    );

    // Note that `string` will not have the 'baz' bit of it at this point. This
    // is because we mocked that to happen asynchronously with a setTimeout function
    console.log(string); // foobar
  });

// Logs, in order:
// Last Then: oops... didn't bother to instantiate and return a promise in the prior then so
the sequence may be a bit surprising
// foobar
// foobarbaz
```

The value returned from `then()` is resolved in the same way as `Promise.resolve()`. This means thenable objects are supported, and if the return value is not a promise, it's implicitly wrapped in a `Promise` and then resolved.

js

```
const p2 = new Promise((resolve, reject) => {
  resolve(1);
});

p2.then((value) => {
  console.log(value); // 1
  return value + 1;
}).then((value) => {
  console.log(value, "- A synchronous value works"); // 2 - A synchronous value works
});

p2.then((value) => {
  console.log(value); // 1
});
```

A `then` call returns a promise that eventually rejects if the function throws an error or returns a rejected Promise.

js

```
Promise.resolve()
  .then(() => {
    // Makes .then() return a rejected promise
    throw new Error("Oh no!");
  })
  .then(
    () => {
      console.log("Not called.");
    },
    (error) => {
      console.error(`onRejected function called: ${error.message}`);
    },
  );
```

In practice, it is often desirable to `catch()` rejected promises rather than `then()`'s two-case syntax, as demonstrated below.

```js
Promise.resolve()
  .then(() => {
    // Makes .then() return a rejected promise
    throw new Error("Oh no!");
  })
  .catch((error) => {
    console.error(`onRejected function called: ${error.message}`);
  })
  .then(() => {
    console.log("I am always called even if the prior then's promise rejects");
  });
```

In all other cases, the returned promise eventually fulfills. In the following example, the first `then()` returns `42` wrapped in a fulfilled Promise, even though the previous Promise in the chain was rejected.

```js
Promise.reject(new Error("Oh no!"))
  .then(
    () => 99,
    () => 42,
  ) // onRejected returns 42 which is wrapped in a fulfilled Promise
  .then((solution) => console.log(`Resolved with ${solution}`)); // Fulfilled with 42
```

If `onFulfilled` returns a promise, the return value of `then` will be fulfilled/rejected based on the eventual state of that promise.

```js
function resolveLater(resolve, reject) {
  setTimeout(() => {
    resolve(10);
  }, 1000);
}
function rejectLater(resolve, reject) {
  setTimeout(() => {
    reject(new Error("Error"));
  }, 1000);
}

const p1 = Promise.resolve("foo");
// Return promise here, that will be resolved to 10 after 1 second
const p2 = p1.then(() => new Promise(resolveLater));
p2.then(
  (v) => {
    console.log("resolved", v); // "resolved", 10
  },
  (e) => {
    // not called
    console.error("rejected", e);
  },
);

// Return promise here, that will be rejected with 'Error' after 1 second
const p3 = p1.then(() => new Promise(rejectLater));
p3.then(
  (v) => {
    // not called
    console.log("resolved", v);
  },
```

*(code continues on the next page)*

(code continued from the previous page)

```
  (e) => {
    console.error("rejected", e); // "rejected", 'Error'
  },
);
```

You can use chaining to implement one function with a Promise-based API on top of another such function.

```js
function fetchCurrentData() {
  // The fetch() API returns a Promise. This function
  // exposes a similar API, except the fulfillment
  // value of this function's Promise has had more
  // work done on it.
  return fetch("current-data.json").then((response) => {
    if (response.headers.get("content-type") !== "application/json") {
      throw new TypeError();
    }
    const j = response.json();
    // maybe do something with j

    // fulfillment value given to user of
    // fetchCurrentData().then()
    return j;
  });
}
```

## Asynchronicity of then()

The following is an example to demonstrate the asynchronicity of the `then` method.

```js
// Using a resolved promise 'resolvedProm' for example,
// the function call 'resolvedProm.then(...)' returns a new promise immediately,
// but its handler '(value) => {...}' will get called asynchronously as demonstrated by the
console.logs.
// the new promise is assigned to 'thenProm',
// and thenProm will be resolved with the value returned by handler
const resolvedProm = Promise.resolve(33);
console.log(resolvedProm);

const thenProm = resolvedProm.then((value) => {
  console.log(
    `this gets called after the end of the main stack. the value received is: ${value}, the
value returned is: ${
      value + 1
    }`,
  );
  return value + 1;
});
console.log(thenProm);

// Using setTimeout, we can postpone the execution of a function to the moment the stack is
empty
setTimeout(() => {
  console.log(thenProm);
});
```

(code continues on the next page)

*(code continued from the previous page)*

```
// Logs, in order:
// Promise {[[PromiseStatus]]: "resolved", [[PromiseResult]]: 33}
// Promise {[[PromiseStatus]]: "pending", [[PromiseResult]]: undefined}
// "this gets called after the end of the main stack. the value received is: 33, the value
returned is: 34"
// Promise {[[PromiseStatus]]: "resolved", [[PromiseResult]]: 34}
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-promise.prototype.then` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `then()` | 32 | 12 | 29 | 19 | 8 | 32 | 29 | 19 | 8 | 2.0 | 4.4.3 | 8 | 1.0.0 | 1.0 | 0.12.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Promise.then`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Promise`
- `Promise.prototype.catch()`

JavaScript › Reference › Standard built-in objects

# Proxy

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Proxy · MDN page last modified 2025-08-19

**Baseline: Widely available** — available across major browsers since September 2016.

The **`Proxy`** object enables you to create a proxy for another object, which can intercept and redefine fundamental operations for that object.

## Description

The `Proxy` object allows you to create an object that can be used in place of the original object, but which may redefine fundamental `Object` operations like getting, setting, and defining properties. Proxy objects are commonly used to log property accesses, validate, format, or sanitize inputs, and so on.

You create a `Proxy` with two parameters:

- `target`: the original object which you want to proxy
- `handler`: an object that defines which operations will be intercepted and how to redefine intercepted operations.

For example, this code creates a proxy for the `target` object.

```js
const target = {
  message1: "hello",
  message2: "everyone",
};

const handler1 = {};

const proxy1 = new Proxy(target, handler1);
```

Because the handler is empty, this proxy behaves just like the original target:

```js
console.log(proxy1.message1); // hello
console.log(proxy1.message2); // everyone
```

To customize the proxy, we define functions on the handler object:

```js
const target = {
  message1: "hello",
  message2: "everyone",
};

const handler2 = {
  get(target, prop, receiver) {
    return "world";
  },
};

const proxy2 = new Proxy(target, handler2);
```

Here we've provided an implementation of the `get()` handler, which intercepts attempts to access properties in the target.

Handler functions are sometimes called *traps*, presumably because they trap calls to the target object. The trap in `handler2` above redefines all property accessors:

```js
console.log(proxy2.message1); // world
console.log(proxy2.message2); // world
```

Proxies are often used with the `Reflect` object, which provides some methods with the same names as the `Proxy` traps. The `Reflect` methods provide the reflective semantics for invoking the corresponding object internal methods. For example, we can call `Reflect.get` if we don't wish to redefine the object's behavior:

```js
const target = {
  message1: "hello",
  message2: "everyone",
};

const handler3 = {
  get(target, prop, receiver) {
    if (prop === "message2") {
      return "world";
    }
    return Reflect.get(...arguments);
  },
};

const proxy3 = new Proxy(target, handler3);

console.log(proxy3.message1); // hello
console.log(proxy3.message2); // world
```

The `Reflect` method still interacts with the object through object internal methods — it doesn't "de-proxify" the proxy if it's invoked on a proxy. If you use `Reflect` methods within a proxy trap, and the `Reflect` method call gets intercepted by the trap again, there may be infinite recursion.

### Terminology

The following terms are used when talking about the functionality of proxies.

**handler**

The object passed as the second argument to the `Proxy` constructor. It contains the traps which define the behavior of the proxy.

**trap**

The function that defines the behavior for the corresponding object internal method. (This is analogous to the concept of *traps* in operating systems.)

**target**

Object which the proxy virtualizes. It is often used as storage backend for the proxy. Invariants (semantics that remain unchanged) regarding object non-extensibility or non-configurable properties are verified against the target.

**invariants**

Semantics that remain unchanged when implementing custom operations. If your trap implementation violates the invariants of a handler, a `TypeError` will be thrown.

### Object internal methods

Objects are collections of properties. However, the language doesn't provide any machinery to *directly* manipulate data stored in the object — rather, the object defines some internal methods specifying how it can be interacted with. For example, when you read `obj.x`, you may expect the following to happen:

- The x property is searched up the prototype chain until it is found.
- If x is a data property, the property descriptor's `value` attribute is returned.
- If x is an accessor property, the getter is invoked, and the return value of the getter is returned.

There isn't anything special about this process in the language — it's just because ordinary objects, by default, have a `[[Get]]` internal method that is defined with this behavior. The `obj.x` property access syntax simply invokes the `[[Get]]` method on the object, and the object uses its own internal method implementation to determine what to return.

As another example, arrays differ from normal objects, because they have a magic `length` property that, when modified, automatically allocates empty slots or removes elements from the array. Similarly, adding array elements automatically changes the `length` property. This is because arrays have a `[[DefineOwnProperty]]` internal method that knows to update `length` when an integer index is written to, or update the array contents when `length` is written to. Such objects whose internal methods have different implementations from ordinary objects are called *exotic objects*. `Proxy` enable developers to define their own exotic objects with full capacity.

All objects have the following internal methods:

| Internal method | Corresponding trap |
|---|---|
| `[[GetPrototypeOf]]` | `getPrototypeOf()` |
| `[[SetPrototypeOf]]` | `setPrototypeOf()` |
| `[[IsExtensible]]` | `isExtensible()` |
| `[[PreventExtensions]]` | `preventExtensions()` |
| `[[GetOwnProperty]]` | `getOwnPropertyDescriptor()` |
| `[[DefineOwnProperty]]` | `defineProperty()` |
| `[[HasProperty]]` | `has()` |
| `[[Get]]` | `get()` |
| `[[Set]]` | `set()` |
| `[[Delete]]` | `deleteProperty()` |
| `[[OwnPropertyKeys]]` | `ownKeys()` |

Function objects also have the following internal methods:

| Internal method | Corresponding trap |
|---|---|
| `[[Call]]` | `apply()` |
| `[[Construct]]` | `construct()` |

It's important to realize that all interactions with an object eventually boils down to the invocation of one of these internal methods, and that they are all customizable through proxies. This means almost no behavior (except certain critical invariants) is guaranteed in the language — everything is defined by the object itself. When you run `delete obj.x`, there's no guarantee that `"x" in obj` returns `false` afterwards — it depends on the object's implementations of `[[Delete]]` and `[[HasProperty]]`. A

`delete obj.x` may log things to the console, modify some global state, or even define a new property instead of deleting the existing one, although these semantics should be avoided in your own code.

All internal methods are called by the language itself, and are not directly accessible in JavaScript code. The `Reflect` namespace offers methods that do little more than call the internal methods, besides some input normalization/validation. In each trap's page, we list several typical situations when the trap is invoked, but these internal methods are called in *a lot* of places. For example, array methods read and write to array through these internal methods, so methods like `push()` would also invoke `get()` and `set()` traps.

Most of the internal methods are straightforward in what they do. The only two that may be confusable are `[[Set]]` and `[[DefineOwnProperty]]`. For normal objects, the former invokes setters; the latter doesn't. (And `[[Set]]` calls `[[DefineOwnProperty]]` internally if there's no existing property or the property is a data property.) While you may know that the `obj.x = 1` syntax uses `[[Set]]`, and `Object.defineProperty()` uses `[[DefineOwnProperty]]`, it's not immediately apparent what semantics other built-in methods and syntaxes use. For example, class fields use the `[[DefineOwnProperty]]` semantic, which is why setters defined in the superclass are not invoked when a field is declared on the derived class.

## Constructor

**`Proxy()`**
: Creates a new `Proxy` object.

> **Note:** There's no `Proxy.prototype` property, so `Proxy` instances do not have any special properties or methods.

## Static methods

**`Proxy.revocable()`**
: Creates a revocable `Proxy` object.

## Examples

### Basic example

In this example, the number `37` gets returned as the default value when the property name is not in the object. It is using the `get()` handler.

```js
const handler = {
  get(obj, prop) {
    return prop in obj ? obj[prop] : 37;
  },
};

const p = new Proxy({}, handler);
p.a = 1;
p.b = undefined;

console.log(p.a, p.b); // 1, undefined

console.log("c" in p, p.c); // false, 37
```

### No-op forwarding proxy

In this example, we are using a native JavaScript object to which our proxy will forward all operations that are applied to it.

```js
const target = {};
const p = new Proxy(target, {});

p.a = 37; // Operation forwarded to the target

console.log(target.a); // 37 (The operation has been properly forwarded!)
```

Note that while this "no-op" works for plain JavaScript objects, it does not work for native objects, such as DOM elements, `Map` objects, or anything that has internal slots. See no private field forwarding for more information.

### No private field forwarding

A proxy is still another object with a different identity — it's a *proxy* that operates between the wrapped object and the outside. As such, the proxy does not have direct access to the original object's private elements.

```js
class Secret {
  #secret;
  constructor(secret) {
    this.#secret = secret;
  }
  get secret() {
    return this.#secret.replace(/\d+/, "[REDACTED]");
  }
}

const secret = new Secret("123456");
console.log(secret.secret); // [REDACTED]
// Looks like a no-op forwarding...
const proxy = new Proxy(secret, {});
console.log(proxy.secret); // TypeError: Cannot read private member #secret from an object
whose class did not declare it
```

This is because when the proxy's `get` trap is invoked, the `this` value is the `proxy` instead of the original `secret`, so `#secret` is not accessible. To fix this, use the original `secret` as `this`:

```js
const proxy = new Proxy(secret, {
  get(target, prop, receiver) {
    // By default, it looks like Reflect.get(target, prop, receiver)
    // which has a different value of `this`
    return target[prop];
  },
});
console.log(proxy.secret);
```

For methods, this means you have to redirect the method's `this` value to the original object as well:

```js
class Secret {
  #x = 1;
  x() {
    return this.#x;
  }
}

const secret = new Secret();
const proxy = new Proxy(secret, {
  get(target, prop, receiver) {
    const value = target[prop];
    if (value instanceof Function) {
      return function (...args) {
        return value.apply(this === receiver ? target : this, args);
      };
    }
    return value;
  },
});
console.log(proxy.x());
```

Some native JavaScript objects have properties called *internal slots*, which are not accessible from JavaScript code. For example, `Map` objects have an internal slot called `[[MapData]]`, which stores the key-value pairs of the map. As such, you cannot trivially create a forwarding proxy for a map:

```js
const proxy = new Proxy(new Map(), {});
console.log(proxy.size); // TypeError: get size method called on incompatible Proxy
```

You have to use the "`this`-recovering" proxy illustrated above to work around this.

### Validation

With a `Proxy`, you can easily validate the passed value for an object. This example uses the `set()` handler.

```js
const validator = {
  set(obj, prop, value) {
    if (prop === "age") {
      if (!Number.isInteger(value)) {
        throw new TypeError("The age is not an integer");
      }
      if (value > 200) {
        throw new RangeError("The age seems invalid");
      }
    }

    // The default behavior to store the value
    obj[prop] = value;

    // Indicate success
    return true;
  },
};
```

*(code continues on the next page)*

(code continued from the previous page)

```
const person = new Proxy({}, validator);

person.age = 100;
console.log(person.age); // 100
person.age = "young"; // Throws an exception
person.age = 300; // Throws an exception
```

### Manipulating DOM nodes

In this example we use `Proxy` to toggle an attribute of two different elements: so when we set the attribute on one element, the attribute is unset on the other one.

We create a `view` object which is a proxy for an object with a `selected` property. The proxy handler defines the `set()` handler.

When we assign an HTML element to `view.selected`, the element's `'aria-selected'` attribute is set to `true`. If we then assign a different element to `view.selected`, this element's `'aria-selected'` attribute is set to `true` and the previous element's `'aria-selected'` attribute is automatically set to `false`.

js

```
const view = new Proxy(
  {
    selected: null,
  },
  {
    set(obj, prop, newVal) {
      const oldVal = obj[prop];

      if (prop === "selected") {
        if (oldVal) {
          oldVal.setAttribute("aria-selected", "false");
        }
        if (newVal) {
          newVal.setAttribute("aria-selected", "true");
        }
      }

      // The default behavior to store the value
      obj[prop] = newVal;

      // Indicate success
      return true;
    },
  },
);

const item1 = document.getElementById("item-1");
const item2 = document.getElementById("item-2");

// select item1:
view.selected = item1;

console.log(`item1: ${item1.getAttribute("aria-selected")}`);
// item1: true

// selecting item2 de-selects item1:
view.selected = item2;
```

(code continues on the next page)

*(code continued from the previous page)*

```
console.log(`item1: ${item1.getAttribute("aria-selected")}`);
// item1: false

console.log(`item2: ${item2.getAttribute("aria-selected")}`);
// item2: true
```

### Value correction and an extra property

The `products` proxy object evaluates the passed value and converts it to an array if needed. The object also supports an extra property called `latestBrowser` both as a getter and a setter.

js

```
const products = new Proxy(
  {
    browsers: ["Firefox", "Chrome"],
  },
  {
    get(obj, prop) {
      // An extra property
      if (prop === "latestBrowser") {
        return obj.browsers[obj.browsers.length - 1];
      }

      // The default behavior to return the value
      return obj[prop];
    },
    set(obj, prop, value) {
      // An extra property
      if (prop === "latestBrowser") {
        obj.browsers.push(value);
        return true;
      }

      // Convert the value if it is not an array
      if (typeof value === "string") {
        value = [value];
      }

      // The default behavior to store the value
      obj[prop] = value;

      // Indicate success
      return true;
    },
  },
);

console.log(products.browsers);
//  ['Firefox', 'Chrome']

products.browsers = "Safari";
//  pass a string (by mistake)
```

*(code continues on the next page)*

(code continued from the previous page)

```
console.log(products.browsers);
//  ['Safari'] <- no problem, the value is an array

products.latestBrowser = "Edge";

console.log(products.browsers);
//  ['Safari', 'Edge']

console.log(products.latestBrowser);
//  'Edge'
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/reflection.html#sec-proxy-objects` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `Proxy` | 49 | 12 | 18 | 36 | 10 | 49 | 18 | 36 | 10 | 5.0 | 49 | 10 | 1.0.0 | 1.0 | 6.0.0 |
| `Proxy()` constructor | 49 | 12 | 18 | 36 | 10 | 49 | 18 | 36 | 10 | 5.0 | 49 | 10 | 1.0.0 | 1.0 | 6.0.0 |
| `apply` | 49 | 12 | 18 | 36 | 10 | 49 | 18 | 36 | 10 | 5.0 | 49 | 10 | 1.0.0 | 1.0 | 6.0.0 |
| `construct` | 49 | 12 | 18 | 36 | 10 | 49 | 18 | 36 | 10 | 5.0 | 49 | 10 | 1.0.0 | 1.0 | 6.0.0 |
| `defineProperty` | 49 | 12 | 18 | 36 | 10 | 49 | 18 | 36 | 10 | 5.0 | 49 | 10 | 1.0.0 | 1.0 | 6.0.0 |
| `deleteProperty` | 49 | 12 | 18 | 36 | 10 | 49 | 18 | 36 | 10 | 5.0 | 49 | 10 | 1.0.0 | 1.0 | 6.0.0 |
| `get` | 49 | 12 | 18 | 36 | 10 | 49 | 18 | 36 | 10 | 5.0 | 49 | 10 | 1.0.0 | 1.0 | 6.0.0 |
| `getOwnPropertyDescriptor` | 49 | 12 | 18 | 36 | 10 | 49 | 18 | 36 | 10 | 5.0 | 49 | 10 | 1.0.0 | 1.0 | 6.0.0 |
| `getPrototypeOf` | 49 | 79 | 49 | 36 | 10 | 49 | 49 | 36 | 10 | 5.0 | 49 | 10 | 1.0.0 | 1.0 | 6.0.0 |
| `has` | 49 | 12 | 18 | 36 | 10 | 49 | 18 | 36 | 10 | 5.0 | 49 | 10 | 1.0.0 | 1.0 | 6.0.0 |
| `isExtensible` | 49 | 12 | 31 | 36 | 10 | 49 | 31 | 36 | 10 | 5.0 | 49 | 10 | 1.0.0 | 1.0 | 6.0.0 |
| `ownKeys` | 49 | 12 | 18[1] | 36 | 10 | 49 | 18[2] | 36 | 10 | 5.0 | 49 | 10 | 1.0.0 | 1.0 | 6.0.0 |
| `preventExtensions` | 49 | 12 | 22 | 36 | 10 | 49 | 22 | 36 | 10 | 5.0 | 49 | 10 | 1.0.0 | 1.0 | 6.0.0 |
| `set` | 49 | 12 | 18 | 36 | 10 | 49 | 18 | 36 | 10 | 5.0 | 49 | 10 | 1.0.0 | 1.0 | 6.0.0 |
| `setPrototypeOf` | 49 | 12 | 49 | 36 | 10 | 49 | 49 | 36 | 10 | 5.0 | 49 | 10 | 1.0.0 | 1.0 | 6.0.0 |
| `revocable` | 49 | 12 | 34 | 36 | 10 | 49 | 34 | 36 | 10 | 5.0 | 49 | 10 | 1.0.0 | 1.0 | 6.0.0 |

1. In Firefox 42, the implementation got updated to reflect the final ES2015 specification: The result is now checked if it is an array and if the array elements are either of type string or of type symbol. Enumerating duplicate own property names is not a failure anymore.
2. In Firefox for Android 42, the implementation got updated to reflect the final ES2015 specification: The result is now checked if it is an array and if the array elements are either of type string or of type symbol. Enumerating duplicate own property names is not a failure anymore.

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Proxy`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Proxies are awesome presentation by Brendan Eich at JSConf (2014)

---

JavaScript › Reference › Standard built-in objects › Proxy

# Proxy() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Proxy/Proxy · MDN page last modified 2026-05-22

**Baseline: Widely available** — available across major browsers since September 2016.

The **`Proxy()`** constructor creates `Proxy` objects.

## Syntax

```js
new Proxy(target, handler)
```

> **Note:** `Proxy()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

### Parameters

**`target`**

A target object to wrap with `Proxy`. It can be any sort of object, including a native array, a function, or even another proxy.

**`handler`**

An object whose properties are functions that define the behavior of the proxy when an operation is performed on it.

## Description

Use the `Proxy()` constructor to create a new `Proxy` object. This constructor takes two mandatory arguments:

- `target` is the object for which you want to create the proxy
- `handler` is the object that defines the custom behavior of the proxy.

An empty handler will create a proxy that behaves, in almost all respects, exactly like the target. By defining any of a set group of functions on the `handler` object, you can customize specific aspects of the proxy's behavior. For example, by defining `get()` you can provide a customized version of the target's property accessor.

### Handler functions

This section lists all the handler functions you can define. Handler functions are sometimes called *traps*, because they trap calls to the underlying target object.

**`handler.apply()`**

A trap for a function call.

**`handler.construct()`**

A trap for the `new` operator.

**`handler.defineProperty()`**

A trap for `Object.defineProperty`.

**`handler.deleteProperty()`**

A trap for the `delete` operator.

**`handler.get()`**

A trap for getting property values.

**`handler.getOwnPropertyDescriptor()`**

A trap for `Object.getOwnPropertyDescriptor`.

**`handler.getPrototypeOf()`**

A trap for `Object.getPrototypeOf`.

**`handler.has()`**

A trap for the `in` operator.

**`handler.isExtensible()`**

A trap for `Object.isExtensible`.

**`handler.ownKeys()`**

A trap for `Object.getOwnPropertyNames` and `Object.getOwnPropertySymbols`.

**`handler.preventExtensions()`**

A trap for `Object.preventExtensions`.

**`handler.set()`**

A trap for setting property values.

**`handler.setPrototypeOf()`**

A trap for `Object.setPrototypeOf`.

## Examples

### Selectively proxy property accessors

In this example the target has two properties, `notProxied` and `proxied`. We define a handler that returns a different value for `proxied`, and lets any other accesses through to the target.

```js
const target = {
  notProxied: "original value",
  proxied: "original value",
};

const handler = {
  get(target, prop, receiver) {
    if (prop === "proxied") {
      return "replaced value";
    }
    return Reflect.get(...arguments);
  },
};

const proxy = new Proxy(target, handler);

console.log(proxy.notProxied); // "original value"
console.log(proxy.proxied); // "replaced value"
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/reflection.html#sec-proxy-constructor` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `Proxy()` constructor | 49 | 12 | 18 | 36 | 10 | 49 | 18 | 36 | 10 | 5.0 | 49 | 10 | 1.0.0 | 1.0 | 6.0.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Proxy.Proxy`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Meta programming guide
- `Reflect`

---



# handler.apply()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Proxy/Proxy/apply · MDN page last modified 2025-07-20

**Baseline: Widely available** — available across major browsers since September 2016.

The **`handler.apply()`** method is a trap for the `[[Call]]` object internal method, which is used by operations such as function calls.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: handler.apply()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function sum(a, b) {
  return a + b;
}

const handler = {
  apply(target, thisArg, argumentsList) {
    console.log(`Calculate sum: ${argumentsList}`);
    // Expected output: "Calculate sum: 1,2"

    return target(argumentsList[0], argumentsList[1]) * 10;
  },
};

const proxy = new Proxy(sum, handler);
```

*(code continues on the next page)*

(code continued from the previous page)

```
console.log(sum(1, 2));
// Expected output: 3
console.log(proxy(1, 2));
// Expected output: 30
```

## Syntax

js

```
new Proxy(target, {
  apply(target, thisArg, argumentsList) {
  }
})
```

### Parameters

The following parameters are passed to the `apply()` method. `this` is bound to the handler.

**`target`**
The target callable object.

**`thisArg`**
The `this` argument for the call.

**`argumentsList`**
An `Array` containing the arguments passed to the function.

### Return value

The `apply()` method can return any value, representing the return value of the function call.

## Description

### Interceptions

This trap can intercept these operations:

- Function call: `proxy(...args)`
- `Function.prototype.apply()` and `Function.prototype.call()`
- `Reflect.apply()`

Or any other operation that invokes the `[[Call]]` internal method.

### Invariants

The proxy's `[[Call]]` internal method throws a `TypeError` if the handler definition violates one of the following invariants:

- The `target` must be a callable itself. That is, it must be a function object.

## Examples

### Trapping a function call

The following code traps a function call.

```js
const p = new Proxy(function () {}, {
  apply(target, thisArg, argumentsList) {
    console.log(`called: ${argumentsList}`);
    return argumentsList[0] + argumentsList[1] + argumentsList[2];
  },
});

console.log(p(1, 2, 3)); // "called: 1,2,3"
// 6
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/ordinary-and-exotic-objects-behaviours.html#sec-proxy-object-internal-methods-and-internal-slots-call-thisargument-argumentslist |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| apply | 49 | 12 | 18 | 36 | 10 | 49 | 18 | 36 | 10 | 5.0 | 49 | 10 | 1.0.0 | 1.0 | 6.0.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Proxy.handler.apply`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Proxy`
- `Proxy()` constructor
- `Function.prototype.apply()`
- `Function.prototype.call()`
- `Reflect.apply()`

---



# handler.construct()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Proxy/Proxy/construct · MDN page last modified 2026-10-09

**Baseline: Widely available** — available across major browsers since September 2016.

The **`handler.construct()`** method is a trap for the `[[Construct]]` object internal method, which is used by operations such as the `new` operator. In order for the new operation to be valid on the resulting Proxy object, the target used to initialize the proxy must itself be a valid constructor.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: handler.construct()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function Monster(disposition) {
  this.disposition = disposition;
}

const handler = {
  construct(target, args) {
    console.log(`Creating a ${target.name}`);
    // Expected output: "Creating a Monster"

    return new target(...args);
  },
};

const ProxiedMonster = new Proxy(Monster, handler);

console.log(new ProxiedMonster("fierce").disposition);
// Expected output: "fierce"
```

## Syntax

js

```
new Proxy(target, {
  construct(target, argumentsList, newTarget) {
  }
})
```

### Parameters

The following parameters are passed to the `construct()` method. `this` is bound to the handler.

**`target`**

The target constructor object.

**`argumentsList`**

An `Array` containing the arguments passed to the constructor.

**`newTarget`**

The constructor that was originally called.

### Return value

The `construct()` method must return an object, representing the newly created object.

## Description

### Interceptions

This trap can intercept these operations:

- The `new` operator: `new myFunction(...args)`
- `Reflect.construct()`

Or any other operation that invokes the `[[Construct]]` internal method.

### Invariants

The proxy's `[[Construct]]` internal method throws a `TypeError` if the handler definition violates one of the following invariants:

- The `target` must be a constructor itself.
- The result must be an `Object`.

## Examples

### Trapping the new operator

The following code traps the `new` operator.

```js
const p = new Proxy(function () {}, {
  construct(target, argumentsList, newTarget) {
    console.log(`called: ${argumentsList}`);
    return { value: argumentsList[0] * 10 };
  },
});

console.log(new p(1).value); // "called: 1"
// 10
```

The following code violates the invariant.

```js · ✗ bad example
const p = new Proxy(function () {}, {
  construct(target, argumentsList, newTarget) {
    return 1;
  },
});

new p(); // TypeError is thrown
```

The following code improperly initializes the proxy. The `target` in Proxy initialization must itself be a valid constructor for the `new` operator.

```js · ✗ bad example
const p = new Proxy(
  {},
  {
    construct(target, argumentsList, newTarget) {
      return {};
    },
  },
);

new p(); // TypeError is thrown, "p" is not a constructor
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/ordinary-and-exotic-objects-behaviours.html#sec-proxy-object-internal-methods-and-internal-slots-construct-argumentslist-newtarget |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| construct | 49 | 12 | 18 | 36 | 10 | 49 | 18 | 36 | 10 | 5.0 | 49 | 10 | 1.0.0 | 1.0 | 6.0.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Proxy.handler.construct`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Proxy`
- `Proxy()` constructor
- `new`
- `Reflect.construct()`

---



# handler.defineProperty()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Proxy/Proxy/defineProperty · MDN page last modified 2025-07-20

**Baseline: Widely available** — available across major browsers since September 2016.

The **`handler.defineProperty()`** method is a trap for the `[[DefineOwnProperty]]` object internal method, which is used by operations such as `Object.defineProperty()`.

## Try it

*▷ Interactive example on MDN: "JavaScript Demo: handler.defineProperty()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const handler = {
  defineProperty(target, key, descriptor) {
    invariant(key, "define");
    return true;
  },
};
```

*(code continues on the next page)*

(code continued from the previous page)

```
function invariant(key, action) {
  if (key[0] === "_") {
    throw new Error(`Invalid attempt to ${action} private "${key}" property`);
  }
}

const monster = {};
const proxy = new Proxy(monster, handler);

console.log((proxy._secret = "easily scared"));
// Expected output: Error: Invalid attempt to define private "_secret" property
```

## Syntax

js

```
new Proxy(target, {
  defineProperty(target, property, descriptor) {
  }
})
```

### Parameters

The following parameters are passed to the `defineProperty()` method. `this` is bound to the handler.

**`target`**

The target object.

**`property`**

A string or `Symbol` representing the property name.

**`descriptor`**

The descriptor for the property being defined or modified.

### Return value

The `defineProperty()` method must return a `Boolean` indicating whether or not the property has been successfully defined. Other values are coerced to booleans.

Many operations, including `Object.defineProperty()` and `Object.defineProperties()`, throw a `TypeError` if the `[[DefineOwnProperty]]` internal method returns `false`.

## Description

### Interceptions

This trap can intercept these operations:

- `Object.defineProperty()`, `Object.defineProperties()`
- `Reflect.defineProperty()`

Or any other operation that invokes the `[[DefineOwnProperty]]` internal method.

### Invariants

The proxy's `[[DefineOwnProperty]]` internal method throws a `TypeError` if the handler definition violates one of the following invariants:

- A property cannot be added, if the target object is not extensible. That is, if `Reflect.isExtensible()` returns `false` on `target`, and `Reflect.getOwnPropertyDescriptor()` returns `undefined` for the property on `target`, then the trap must return a falsy value.
- A property cannot be non-configurable, unless there exists a corresponding non-configurable own property of the target object. That is, if `Reflect.getOwnPropertyDescriptor()` returns `undefined` or `configurable: true` for the property on `target`, and `descriptor.configurable` is `false`, then the trap must return a falsy value.
- A non-configurable property cannot be non-writable, unless there exists a corresponding non-configurable, non-writable own property of the target object. That is, if `Reflect.getOwnPropertyDescriptor()` returns `configurable: false, writable: true` for the property on `target`, and `descriptor.writable` is `false`, then the trap must return a falsy value.
- If a property has a corresponding property on the target object, then the target object property's descriptor must be compatible with `descriptor`. That is, pretending `target` is an ordinary object, and `Object.defineProperty(target, property, descriptor)` would throw an error, then the trap must return a falsy value. The `Object.defineProperty()` reference contains more information, but to summarize, when the target property is non-configurable, the following must hold:
  - `configurable`, `enumerable`, `get`, and `set` cannot be changed
  - the property cannot be switched between data and accessor
  - the `writable` attribute can only be changed from `true` to `false`
  - the `value` attribute can only be changed if `writable` is `true`

## Examples

### Trapping of defineProperty

The following code traps `Object.defineProperty()`.

```js
const p = new Proxy(
  {},
  {
    defineProperty(target, prop, descriptor) {
      console.log(`called: ${prop}`);
      return true;
    },
  },
);

const desc = { configurable: true, enumerable: true, value: 10 };
Object.defineProperty(p, "a", desc); // "called: a"
```

When calling `Object.defineProperty()` or `Reflect.defineProperty()`, the `descriptor` passed to `defineProperty()` trap has one restriction—only following properties are usable (non-standard properties will be ignored):

- `enumerable`
- `configurable`
- `writable`
- `value`
- `get`
- `set`

```js
const p = new Proxy(
  {},
  {
    defineProperty(target, prop, descriptor) {
      console.log(descriptor);
      return Reflect.defineProperty(target, prop, descriptor);
    },
  },
);

Object.defineProperty(p, "name", {
  value: "proxy",
  type: "custom",
}); // { value: 'proxy' }
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/ordinary-and-exotic-objects-behaviours.html#sec-proxy-object-internal-methods-and-internal-slots-defineownproperty-p-desc |

## Browser compatibility

| | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| Feature | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| defineProperty | 49 | 12 | 18 | 36 | 10 | 49 | 18 | 36 | 10 | 5.0 | 49 | 10 | 1.0.0 | 1.0 | 6.0.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Proxy.handler.defineProperty`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Proxy`
- `Proxy()` constructor
- `Object.defineProperty()`
- `Reflect.defineProperty()`

JavaScript › Reference › Standard built-in objects › Proxy › Proxy()

# handler.deleteProperty()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Proxy/Proxy/deleteProperty · MDN page last modified 2026-05-22

**Baseline: Widely available** — available across major browsers since September 2016.

The **`handler.deleteProperty()`** method is a trap for the `[[Delete]]` object internal method, which is used by operations such as the `delete` operator.

## Try it

*▷ Interactive example on MDN: "JavaScript Demo: handler.deleteProperty()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const monster = {
  texture: "scaly",
};

const handler = {
  deleteProperty(target, prop) {
    if (prop in target) {
      delete target[prop];
      console.log(`property removed: ${prop}`);
      // Expected output: "property removed: texture"
    }
  },
};

console.log(monster.texture);
// Expected output: "scaly"

const proxy = new Proxy(monster, handler);
delete proxy.texture;

console.log(monster.texture);
// Expected output: undefined
```

## Syntax

js

```
new Proxy(target, {
  deleteProperty(target, property) {
  }
})
```

### Parameters

The following parameters are passed to the `deleteProperty()` method. `this` is bound to the handler.

**`target`**

The target object.

**`property`**

A string or `Symbol` representing the property name.

### Return value

The `deleteProperty()` method must return a `Boolean` indicating whether or not the property has been successfully deleted. Other values are coerced to booleans.

Many operations, including the `delete` operator when in strict mode, throw a `TypeError` if the `[[Delete]]` internal method returns `false`.

## Description

### Interceptions

This trap can intercept these operations:

- The `delete` operator: `delete proxy[foo]` and `delete proxy.foo`
- `Reflect.deleteProperty()`

Or any other operation that invokes the `[[Delete]]` internal method.

### Invariants

The proxy's `[[Delete]]` internal method throws a `TypeError` if the handler definition violates one of the following invariants:

- A property cannot be reported as deleted, if it exists as a non-configurable own property of the target object. That is, if `Reflect.getOwnPropertyDescriptor()` returns `configurable: false` for the property on `target`, then the trap must return a falsy value.
- A property cannot be reported as deleted, if it exists as an own property of the target object and the target object is non-extensible. That is, if `Reflect.isExtensible()` returns `false` on `target`, and `Reflect.getOwnPropertyDescriptor()` returns a property descriptor for the property on `target`, then the trap must return a falsy value.

## Examples

### Trapping the delete operator

The following code traps the `delete` operator.

```js
const p = new Proxy(
  {},
  {
    deleteProperty(target, prop) {
      if (!(prop in target)) {
        console.log(`property not found: ${prop}`);
        return false;
      }
      delete target[prop];
      console.log(`property removed: ${prop}`);
      return true;
    },
  },
);
```

*(code continues on the next page)*

*(code continued from the previous page)*

```
p.a = 10;
console.log("a" in p); // true

const result1 = delete p.a; // "property removed: a"
console.log(result1); // true
console.log("a" in p); // false

const result2 = delete p.a; // "property not found: a"
console.log(result2); // false
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/ordinary-and-exotic-objects-behaviours.html#sec-proxy-object-internal-methods-and-internal-slots-delete-p |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| deleteProperty | 49 | 12 | 18 | 36 | 10 | 49 | 18 | 36 | 10 | 5.0 | 49 | 10 | 1.0.0 | 1.0 | 6.0.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Proxy.handler.deleteProperty`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Proxy`
- `Proxy()` constructor
- `delete`
- `Reflect.deleteProperty()`

---



# handler.get()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Proxy/Proxy/get · MDN page last modified 2025-07-20

**Baseline: Widely available** — available across major browsers since September 2016.

The **`handler.get()`** method is a trap for the `[[Get]]` object internal method, which is used by operations such as property accessors.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: handler.get()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const monster = {
  secret: "easily scared",
  eyeCount: 4,
};

const handler = {
  get(target, prop, receiver) {
    if (prop === "secret") {
      return `${target.secret.substring(0, 4)} ... shhhh!`;
    }
    return Reflect.get(...arguments);
  },
};

const proxy = new Proxy(monster, handler);

console.log(proxy.eyeCount);
// Expected output: 4

console.log(proxy.secret);
// Expected output: "easi ... shhhh!"
```

## Syntax

js

```
new Proxy(target, {
  get(target, property, receiver) {
  }
})
```

### Parameters

The following parameters are passed to the `get()` method. `this` is bound to the handler.

**`target`**

The target object.

**`property`**

A string or `Symbol` representing the property name.

**`receiver`**

The `this` value for getters; see `Reflect.get()`. This is usually either the proxy itself or an object that inherits from the proxy.

### Return value

The `get()` method can return any value, representing the property value.

## Description

### Interceptions

This trap can intercept these operations:

- Property access: `proxy[foo]` and `proxy.bar`
- `Reflect.get()`

Or any other operation that invokes the `[[Get]]` internal method.

### Invariants

The proxy's `[[Get]]` internal method throws a `TypeError` if the handler definition violates one of the following invariants:

- The value reported for a property must be the same as the value of the corresponding target object property, if the target object property is a non-writable, non-configurable own data property. That is, if `Reflect.getOwnPropertyDescriptor()` returns `configurable: false, writable: false` for the property on `target`, then the trap must return the same value as the `value` attribute in the `target`'s property descriptor.
- The value reported for a property must be `undefined`, if the corresponding target object property is a non-configurable own accessor property that has an undefined getter. That is, if `Reflect.getOwnPropertyDescriptor()` returns `configurable: false, get: undefined` for the property on `target`, then the trap must return `undefined`.

## Examples

### Trap for getting a property value

The following code traps getting a property value.

```js
const p = new Proxy(
  {},
  {
    get(target, property, receiver) {
      console.log(`called: ${property}`);
      return 10;
    },
  },
);

console.log(p.a);
// "called: a"
// 10
```

The following code violates an invariant.

```js
const obj = {};
Object.defineProperty(obj, "a", {
  configurable: false,
  enumerable: false,
  value: 10,
  writable: false,
});

const p = new Proxy(obj, {
  get(target, property) {
    return 20;
  },
});

p.a; // TypeError is thrown
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/ordinary-and-exotic-objects-behaviours.html#sec-proxy-object-internal-methods-and-internal-slots-get-p-receiver |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| get | 49 | 12 | 18 | 36 | 10 | 49 | 18 | 36 | 10 | 5.0 | 49 | 10 | 1.0.0 | 1.0 | 6.0.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Proxy.handler.get`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Proxy`
- `Proxy()` constructor
- `Reflect.get()`

JavaScript › Reference › Standard built-in objects › Proxy › Proxy()

# handler.getOwnPropertyDescriptor()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Proxy/Proxy/getOwnPropertyDescriptor · MDN page last modified 2025-07-20

**Baseline: Widely available** — available across major browsers since September 2016.

The **`handler.getOwnPropertyDescriptor()`** method is a trap for the `[[GetOwnProperty]]` object internal method, which is used by operations such as `Object.getOwnPropertyDescriptor()`.

## Try it

*▷ Interactive example on MDN: "JavaScript Demo: handler.getOwnPropertyDescriptor()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const monster = {
  eyeCount: 4,
};

const handler = {
  getOwnPropertyDescriptor(target, prop) {
    console.log(`called: ${prop}`);
    // Expected output: "called: eyeCount"

    return { configurable: true, enumerable: true, value: 5 };
  },
};

const proxy = new Proxy(monster, handler);

console.log(Object.getOwnPropertyDescriptor(proxy, "eyeCount").value);
// Expected output: 5
```

## Syntax

js

```
new Proxy(target, {
  getOwnPropertyDescriptor(target, property) {
  }
})
```

### Parameters

The following parameters are passed to the `getOwnPropertyDescriptor()` method. `this` is bound to the handler.

**`target`**

The target object.

**`property`**

A string or `Symbol` representing the property name.

### Return value

The `getOwnPropertyDescriptor()` method must return an object or `undefined`, representing the property descriptor. Missing attributes are normalized in the same way as `Object.defineProperty()`.

## Description

### Interceptions

This trap can intercept these operations:

- `Object.getOwnPropertyDescriptor()`
- `Reflect.getOwnPropertyDescriptor()`

Or any other operation that invokes the `[[GetOwnProperty]]` internal method.

### Invariants

The proxy's `[[GetOwnProperty]]` internal method throws a `TypeError` if the handler definition violates one of the following invariants:

- The result must be either an `Object` or `undefined`.
- A property cannot be reported as non-existent, if it exists as a non-configurable own property of the target object. That is, if `Reflect.getOwnPropertyDescriptor()` returns `configurable: false` for the property on `target`, then the trap must not return `undefined`.
- A property cannot be reported as non-existent, if it exists as an own property of a non-extensible target object. That is, if `Reflect.isExtensible()` returns `false` for the target object, then the trap must not return `undefined`.
- A property cannot be reported as existent, if it does not exist as an own property of the target object and the target object is not extensible. That is, if `Reflect.isExtensible()` returns `false` for the target object, and `Reflect.getOwnPropertyDescriptor()` returns `undefined` for the property on `target`, then the trap must return `undefined`.
- A property cannot be reported as non-configurable, unless it exists as a non-configurable own property of the target object. That is, if `Reflect.getOwnPropertyDescriptor()` returns `undefined` or `configurable: true` for the property on `target`, then the trap must not return `configurable: false`.
- A property cannot be reported as both non-configurable and non-writable, unless it exists as a non-configurable, non-writable own property of the target object. That is, in addition to the previous invariant, if `Reflect.getOwnPropertyDescriptor()` returns `configurable: false, writable: true` for the property on `target`, then the trap must not return `configurable: false, writable: false`.
- If a property has a corresponding property on the target object, then the target object property's descriptor must be compatible with `descriptor`. That is, pretending `target` is an ordinary object, then `Object.defineProperty(target, property, resultObject)` must not throw an error. The `Object.defineProperty()` reference contains more information, but to summarize, when the target property is non-configurable, the following must hold:
  - `configurable`, `enumerable`, `get`, and `set` must be the same as original. `writable` must also be the original by virtue of the previous invariant.
  - the property must stay as data or accessor
  - the `value` attribute can only be changed if `writable` is `true`

## Examples

### Trapping of getOwnPropertyDescriptor

The following code traps `Object.getOwnPropertyDescriptor()`.

```js
const p = new Proxy(
  { a: 20 },
  {
    getOwnPropertyDescriptor(target, prop) {
      console.log(`called: ${prop}`);
      return { configurable: true, enumerable: true, value: 10 };
    },
  },
);

console.log(Object.getOwnPropertyDescriptor(p, "a").value);
// "called: a"
// 10
```

The following code violates an invariant.

**js · ✗ bad example**

```js
const obj = { a: 10 };
Object.preventExtensions(obj);
const p = new Proxy(obj, {
  getOwnPropertyDescriptor(target, prop) {
    return undefined;
  },
});

Object.getOwnPropertyDescriptor(p, "a"); // TypeError is thrown
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/ordinary-and-exotic-objects-behaviours.html#sec-proxy-object-internal-methods-and-internal-slots-getownproperty-p |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| getOwnPropertyDescriptor | 49 | 12 | 18 | 36 | 10 | 49 | 18 | 36 | 10 | 5.0 | 49 | 10 | 1.0.0 | 1.0 | 6.0.0 |

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Proxy.handler.getOwnPropertyDescriptor`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Proxy`
- `Proxy()` constructor
- `Object.getOwnPropertyDescriptor()`
- `Reflect.getOwnPropertyDescriptor()`

---

JavaScript › Reference › Standard built-in objects › Proxy › Proxy()

# handler.getPrototypeOf()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Proxy/Proxy/getPrototypeOf · MDN page last modified 2026-05-22

**Baseline: Widely available** — available across major browsers since January 2020.

The **`handler.getPrototypeOf()`** method is a trap for the `[[GetPrototypeOf]]` object internal method, which is used by operations such as `Object.getPrototypeOf()`.

## Try it

*▷ Interactive example on MDN: "JavaScript Demo: handler.getPrototypeOf()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const monster = {
  eyeCount: 4,
};

const monsterPrototype = {
  eyeCount: 2,
};

const handler = {
  getPrototypeOf(target) {
    return monsterPrototype;
  },
};

const proxy = new Proxy(monster, handler);

console.log(Object.getPrototypeOf(proxy) === monsterPrototype);
// Expected output: true

console.log(Object.getPrototypeOf(proxy).eyeCount);
// Expected output: 2
```

## Syntax

js

```
new Proxy(target, {
  getPrototypeOf(target) {
  }
})
```

### Parameters

The following parameter is passed to the `getPrototypeOf()` method. `this` is bound to the handler.

**`target`**
The target object.

### Return value

The `getPrototypeOf()` method must return an object or `null`, representing the prototype of the target object.

## Description

### Interceptions

This trap can intercept these operations:

- `Object.getPrototypeOf()`
- `Reflect.getPrototypeOf()`
- `__proto__`
- `Object.prototype.isPrototypeOf()`
- `instanceof`

Or any other operation that invokes the `[[GetPrototypeOf]]` internal method.

### Invariants

The proxy's `[[GetPrototypeOf]]` internal method throws a `TypeError` if the handler definition violates one of the following invariants:

- The result must be either an `Object` or `null`.
- If the target object is not extensible (that is, `Reflect.isExtensible()` returns `false` on `target`), the result must be the same as the result of `Reflect.getPrototypeOf(target)`.

## Examples

### Basic usage

```js
const obj = {};
const proto = {};
const handler = {
  getPrototypeOf(target) {
    console.log(target === obj); // true
    console.log(this === handler); // true
    return proto;
  },
};

const p = new Proxy(obj, handler);
console.log(Object.getPrototypeOf(p) === proto); // true
```

### Five ways to trigger the getPrototypeOf trap

js

```
const obj = {};
const p = new Proxy(obj, {
  getPrototypeOf(target) {
    return Array.prototype;
  },
});
console.log(
  Object.getPrototypeOf(p) === Array.prototype, // true
  Reflect.getPrototypeOf(p) === Array.prototype, // true
  p.__proto__ === Array.prototype, // true
  Array.prototype.isPrototypeOf(p), // true
  p instanceof Array, // true
);
```

### Two kinds of exceptions

js · ✗ bad example

```
const obj = {};
const p = new Proxy(obj, {
  getPrototypeOf(target) {
    return "foo";
  },
});
Object.getPrototypeOf(p); // TypeError: "foo" is not an object or null

const obj2 = Object.preventExtensions({});
const p2 = new Proxy(obj2, {
  getPrototypeOf(target) {
    return {};
  },
});
Object.getPrototypeOf(p2); // TypeError: expected same prototype value
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/ordinary-and-exotic-objects-behaviours.html#sec-proxy-object-internal-methods-and-internal-slots-getprototypeof` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `getPrototypeOf` | 49 | 79 | 49 | 36 | 10 | 49 | 49 | 36 | 10 | 5.0 | 49 | 10 | 1.0.0 | 1.0 | 6.0.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Proxy.handler.getPrototypeOf`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Proxy`
- `Proxy()` constructor
- `Object.getPrototypeOf()`
- `Reflect.getPrototypeOf()`

---



# handler.has()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Proxy/Proxy/has · MDN page last modified 2025-07-20

**Baseline: Widely available** — available across major browsers since September 2016.

The **`handler.has()`** method is a trap for the `[[HasProperty]]` object internal method, which is used by operations such as the `in` operator.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: handler.has()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const handler = {
  has(target, key) {
    if (key[0] === "_") {
      return false;
    }
    return key in target;
  },
};

const monster = {
  _secret: "easily scared",
  eyeCount: 4,
};

const proxy = new Proxy(monster, handler);
console.log("eyeCount" in proxy);
// Expected output: true

console.log("_secret" in proxy);
// Expected output: false

console.log("_secret" in monster);
// Expected output: true
```

## Syntax

js

```
new Proxy(target, {
  has(target, property) {
  }
})
```

### Parameters

The following parameters are passed to `has()` method. `this` is bound to the handler.

**`target`**
The target object.

**`property`**
A string or `Symbol` representing the property name.

### Return value

The `has()` method must return a `Boolean` indicating whether or not the property exists. Other values are coerced to booleans.

## Description

### Interceptions

This trap can intercept these operations:

- The `in` operator: `foo in proxy`
- `with` check: `with(proxy) { (foo); }`
- `Reflect.has()`

Or any other operation that invokes the `[[HasProperty]]` internal method.

### Invariants

The proxy's `[[HasProperty]]` internal method throws a `TypeError` if the handler definition violates one of the following invariants:

- A property cannot be reported as non-existent, if it exists as a non-configurable own property of the target object. That is, if `Reflect.getOwnPropertyDescriptor()` returns `configurable: false` for the property on `target`, the trap must return `true`.
- A property cannot be reported as non-existent, if it exists as an own property of the target object and the target object is not extensible. That is, if `Reflect.isExtensible()` returns `false` on `target`, and `Reflect.getOwnPropertyDescriptor()` returns a property descriptor for the property on `target`, the trap must return `true`.

## Examples

### Trapping the in operator

The following code traps the `in` operator.

```js
const p = new Proxy(
  {},
  {
    has(target, prop) {
      console.log(`called: ${prop}`);
      return true;
    },
  },
);

console.log("a" in p);
// "called: a"
// true
```

The following code violates an invariant.

**js · ✗ bad example**

```js
const obj = { a: 10 };
Object.preventExtensions(obj);

const p = new Proxy(obj, {
  has(target, prop) {
    return false;
  },
});

"a" in p; // TypeError is thrown
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/ordinary-and-exotic-objects-behaviours.html#sec-proxy-object-internal-methods-and-internal-slots-hasproperty-p |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| has | 49 | 12 | 18 | 36 | 10 | 49 | 18 | 36 | 10 | 5.0 | 49 | 10 | 1.0.0 | 1.0 | 6.0.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Proxy.handler.has`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Proxy`
- `Proxy()` constructor
- `in`
- `Reflect.has()`

JavaScript › Reference › Standard built-in objects › Proxy › Proxy()

# handler.isExtensible()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Proxy/Proxy/isExtensible · MDN page last modified 2025-07-20

**Baseline: Widely available** — available across major browsers since September 2016.

The **`handler.isExtensible()`** method is a trap for the `[[IsExtensible]]` object internal method, which is used by operations such as `Object.isExtensible()`.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: handler.isExtensible()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const monster = {
  canEvolve: true,
};

const handler = {
  isExtensible(target) {
    return Reflect.isExtensible(target);
  },
  preventExtensions(target) {
    target.canEvolve = false;
    return Reflect.preventExtensions(target);
  },
};

const proxy = new Proxy(monster, handler);

console.log(Object.isExtensible(proxy));
// Expected output: true

console.log(monster.canEvolve);
// Expected output: true

Object.preventExtensions(proxy);

console.log(Object.isExtensible(proxy));
// Expected output: false

console.log(monster.canEvolve);
// Expected output: false
```

## Syntax

js

```
new Proxy(target, {
  isExtensible(target) {
  }
})
```

### Parameters

The following parameter is passed to the `isExtensible()` method. `this` is bound to the handler.

**`target`**
The target object.

### Return value

The `isExtensible()` method must return a `Boolean` indicating whether or not the target object is extensible. Other values are coerced to booleans.

## Description

### Interceptions

This trap can intercept these operations:

- `Object.isExtensible()`
- `Reflect.isExtensible()`

Or any other operation that invokes the `[[IsExtensible]]` internal method.

### Invariants

The proxy's `[[IsExtensible]]` internal method throws a `TypeError` if the handler definition violates one of the following invariants:

- The result must be the same as `Reflect.isExtensible()` on the target object.

## Examples

### Trapping of isExtensible

The following code traps `Object.isExtensible()`.

```js
const p = new Proxy(
  {},
  {
    isExtensible(target) {
      console.log("called");
      return true;
    },
  },
);

console.log(Object.isExtensible(p));
// "called"
// true
```

The following code violates the invariant.

js · ✗ bad example

```
const p = new Proxy(
  {},
  {
    isExtensible(target) {
      return false;
    },
  },
);

Object.isExtensible(p); // TypeError is thrown
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/ordinary-and-exotic-objects-behaviours.html#sec-proxy-object-internal-methods-and-internal-slots-isextensible |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| isExtensible | 49 | 12 | 31 | 36 | 10 | 49 | 31 | 36 | 10 | 5.0 | 49 | 10 | 1.0.0 | 1.0 | 6.0.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Proxy.handler.isExtensible`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Proxy`
- `Proxy()` constructor
- `Object.isExtensible()`
- `Reflect.isExtensible()`
- `Reflect.preventExtensions()`

---



# handler.ownKeys()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Proxy/Proxy/ownKeys · MDN page last modified 2025-07-20

**Baseline: Widely available** — available across major browsers since September 2016.

The **`handler.ownKeys()`** method is a trap for the `[[OwnPropertyKeys]]` object internal method, which is used by operations such as `Object.keys()`, `Reflect.ownKeys()`, etc.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: handler.ownKeys()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const monster = {
  _age: 111,
  [Symbol("secret")]: "I am scared!",
  eyeCount: 4,
};

const handler = {
  ownKeys(target) {
    return Reflect.ownKeys(target);
  },
};

const proxy = new Proxy(monster, handler);

for (const key of Object.keys(proxy)) {
  console.log(key);
  // Expected output: "_age"
  // Expected output: "eyeCount"
}
```

## Syntax

js

```
new Proxy(target, {
  ownKeys(target) {
  }
})
```

### Parameters

The following parameter is passed to the `ownKeys()` method. `this` is bound to the handler.

**`target`**

The target object.

### Return value

The `ownKeys()` method must return an array-like object where each element is either a `String` or a `Symbol` containing no duplicate items.

## Description

### Interceptions

This trap can intercept these operations:

- `Object.getOwnPropertyNames()`
- `Object.getOwnPropertySymbols()`
- `Object.keys()`
- `Reflect.ownKeys()`

Or any other operation that invokes the `[[OwnPropertyKeys]]` internal method.

### Invariants

The proxy's `[[OwnPropertyKeys]]` internal method throws a `TypeError` if the handler definition violates one of the following invariants:

- The result is an `Object`.
- The list of keys contains no duplicate values.
- The type of each key is either a `String` or a `Symbol`.
- The result list must contain the keys of all non-configurable own properties of the target object. That is, for all keys returned by `Reflect.ownKeys()` on the target object, if the key reports `configurable: false` by `Reflect.getOwnPropertyDescriptor()`, then the key must be included in the result List.
- If the target object is not extensible, then the result list must contain all the keys of the own properties of the target object and no other values. That is, if `Reflect.isExtensible()` returns `false` on `target`, then the result list must contain the same values as the result of `Reflect.ownKeys()` on `target`.

## Examples

### Trapping of getOwnPropertyNames

The following code traps `Object.getOwnPropertyNames()`.

js

```
const p = new Proxy(
  {},
  {
    ownKeys(target) {
      console.log("called");
      return ["a", "b", "c"];
    },
  },
);

console.log(Object.getOwnPropertyNames(p));
// "called"
// [ 'a', 'b', 'c' ]
```

The following code violates an invariant.

js · ✗ bad example

```
const obj = {};
Object.defineProperty(obj, "a", {
  configurable: false,
  enumerable: true,
  value: 10,
});
```

(code continues on the next page)

(code continued from the previous page)

```
const p = new Proxy(obj, {
  ownKeys(target) {
    return [123, 12.5, true, false, undefined, null, {}, []];
  },
});

console.log(Object.getOwnPropertyNames(p));

// TypeError: proxy [[OwnPropertyKeys]] must return an array
// with only string and symbol elements
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/ordinary-and-exotic-objects-behaviours.html#sec-proxy-object-internal-methods-and-internal-slots-ownpropertykeys |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| ownKeys | 49 | 12 | 18[1] | 36 | 10 | 49 | 18[2] | 36 | 10 | 5.0 | 49 | 10 | 1.0.0 | 1.0 | 6.0.0 |

1. In Firefox 42, the implementation got updated to reflect the final ES2015 specification: The result is now checked if it is an array and if the array elements are either of type string or of type symbol. Enumerating duplicate own property names is not a failure anymore.
2. In Firefox for Android 42, the implementation got updated to reflect the final ES2015 specification: The result is now checked if it is an array and if the array elements are either of type string or of type symbol. Enumerating duplicate own property names is not a failure anymore.

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Proxy.handler.ownKeys`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Proxy`
- `Proxy()` constructor
- `Object.getOwnPropertyNames()`
- `Reflect.ownKeys()`

---

JavaScript › Reference › Standard built-in objects › Proxy › Proxy()

# handler.preventExtensions()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Proxy/Proxy/preventExtensions · MDN page last modified 2025-07-20

**Baseline: Widely available** — available across major browsers since September 2016.

The **`handler.preventExtensions()`** method is a trap for the `[[PreventExtensions]]` object internal method, which is used by operations such as `Object.preventExtensions()`.

## Try it

*▷ Interactive example on MDN: "JavaScript Demo: handler.preventExtensions()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const monster = {
  canEvolve: true,
};

const handler = {
  preventExtensions(target) {
    target.canEvolve = false;
    Object.preventExtensions(target);
    return true;
  },
};

const proxy = new Proxy(monster, handler);

console.log(monster.canEvolve);
// Expected output: true

Object.preventExtensions(proxy);

console.log(monster.canEvolve);
// Expected output: false
```

## Syntax

js

```
new Proxy(target, {
  preventExtensions(target) {
  }
})
```

### Parameters

The following parameter is passed to the `preventExtensions()` method. `this` is bound to the handler.

**`target`**

The target object.

### Return value

The `preventExtensions()` method must return a `Boolean` indicating whether or not the operation was successful. Other values are coerced to booleans.

Many operations, including `Object.preventExtensions()`, throw a `TypeError` if the `[[PreventExtensions]]` internal method returns `false`.

## Description

### Interceptions

This trap can intercept these operations:

- `Object.preventExtensions()`
- `Reflect.preventExtensions()`
- `Object.seal()`
- `Object.freeze()`

Or any other operation that invokes the `[[PreventExtensions]]` internal method.

### Invariants

The proxy's `[[PreventExtensions]]` internal method throws a `TypeError` if the handler definition violates one of the following invariants:

- The result is only `true` if `Reflect.isExtensible()` on the target object returns `false` after calling `handler.preventExtensions()`.

## Examples

### Trapping of preventExtensions

The following code traps `Object.preventExtensions()`.

js

```
const p = new Proxy(
  {},
  {
    preventExtensions(target) {
      console.log("called");
      Object.preventExtensions(target);
      return true;
    },
  },
);

console.log(Object.preventExtensions(p));
// "called"
// false
```

The following code violates the invariant.

**js · ✗ bad example**

```
const p = new Proxy(
  {},
  {
    preventExtensions(target) {
      return true;
    },
  },
);

Object.preventExtensions(p); // TypeError is thrown
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/ordinary-and-exotic-objects-behaviours.html#sec-proxy-object-internal-methods-and-internal-slots-preventextensions` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `preventExtensions` | 49 | 12 | 22 | 36 | 10 | 49 | 22 | 36 | 10 | 5.0 | 49 | 10 | 1.0.0 | 1.0 | 6.0.0 |

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Proxy.handler.preventExtensions`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Proxy`
- `Proxy()` constructor
- `Object.preventExtensions()`
- `Reflect.preventExtensions()`

---



# handler.set()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Proxy/Proxy/set · MDN page last modified 2025-07-20

**Baseline: Widely available** — available across major browsers since September 2016.

The **`handler.set()`** method is a trap for the `[[Set]]` object internal method, which is used by operations such as using property accessors to set a property's value.

## Try it

▷ *Interactive example on MDN: "JavaScript Demo: handler.set()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const monster = { eyeCount: 4 };

const handler = {
  set(obj, prop, value) {
    if (prop === "eyeCount" && value % 2 !== 0) {
      console.log("Monsters must have an even number of eyes");
    } else {
      return Reflect.set(...arguments);
    }
  },
};
```

*(code continues on the next page)*

(code continued from the previous page)

```
const proxy = new Proxy(monster, handler);

proxy.eyeCount = 1;
// Expected output: "Monsters must have an even number of eyes"

console.log(proxy.eyeCount);
// Expected output: 4

proxy.eyeCount = 2;
console.log(proxy.eyeCount);
// Expected output: 2
```

## Syntax

js

```
new Proxy(target, {
  set(target, property, value, receiver) {
  }
})
```

### Parameters

The following parameters are passed to the `set()` method. `this` is bound to the handler.

**`target`**

The target object.

**`property`**

A string or `Symbol` representing the property name.

**`value`**

The new value of the property to set.

**`receiver`**

The `this` value for setters; see `Reflect.set()`. This is usually either the proxy itself or an object that inherits from the proxy.

### Return value

The `set()` method must return a `Boolean` indicating whether or not the assignment succeeded. Other values are coerced to booleans.

Many operations, including using property accessors in strict mode, throw a `TypeError` if the `[[Set]]` internal method returns `false`.

## Description

### Interceptions

This trap can intercept these operations:

- Property assignment: `proxy[foo] = bar` and `proxy.foo = bar`
- `Reflect.set()`

Or any other operation that invokes the `[[Set]]` internal method.

### Invariants

The proxy's `[[Set]]` internal method throws a `TypeError` if the handler definition violates one of the following invariants:

- Cannot change the value of a property to be different from the value of the corresponding target object property, if the corresponding target object property is a non-writable, non-configurable own data property. That is, if `Reflect.getOwnPropertyDescriptor()` returns `configurable: false, writable: false` for the property on `target`, and `value` is different from the `value` attribute in the `target`'s property descriptor, then the trap must return a falsy value.
- Cannot set the value of a property if the corresponding target object property is a non-configurable own accessor property that has an undefined setter. That is, if `Reflect.getOwnPropertyDescriptor()` returns `configurable: false, set: undefined` for the property on `target`, then the trap must return a falsy value.

## Examples

### Trap setting of a property value

The following code traps setting a property value.

```js
const p = new Proxy(
  {},
  {
    set(target, prop, value, receiver) {
      target[prop] = value;
      console.log(`property set: ${prop} = ${value}`);
      return true;
    },
  },
);

console.log("a" in p); // false

p.a = 10; // "property set: a = 10"
console.log("a" in p); // true
console.log(p.a); // 10
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/ordinary-and-exotic-objects-behaviours.html#sec-proxy-object-internal-methods-and-internal-slots-set-p-v-receiver |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `set` | 49 | 12 | 18 | 36 | 10 | 49 | 18 | 36 | 10 | 5.0 | 49 | 10 | 1.0.0 | 1.0 | 6.0.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Proxy.handler.set`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Proxy`
- `Proxy()` constructor
- `Reflect.set()`

---



# handler.setPrototypeOf()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Proxy/Proxy/setPrototypeOf · MDN page last modified 2025-07-20

**Baseline: Widely available** — available across major browsers since September 2016.

The **`handler.setPrototypeOf()`** method is a trap for the `[[SetPrototypeOf]]` object internal method, which is used by operations such as `Object.setPrototypeOf()`.

## Try it

*▷ Interactive example on MDN: "JavaScript Demo: handler.setPrototypeOf()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const handler = {
  setPrototypeOf(monster, monsterProto) {
    monster.geneticallyModified = true;
    return false;
  },
};

const monsterProto = {};
const monster = {
  geneticallyModified: false,
};

const proxy = new Proxy(monster, handler);
// Object.setPrototypeOf(proxy, monsterProto); // Throws a TypeError

console.log(Reflect.setPrototypeOf(proxy, monsterProto));
// Expected output: false

console.log(monster.geneticallyModified);
// Expected output: true
```

## Syntax

```js
new Proxy(target, {
  setPrototypeOf(target, prototype) {
  }
})
```

### Parameters

The following parameters are passed to the `setPrototypeOf()` method. `this` is bound to the handler.

**`target`**

The target object.

**`prototype`**

The object's new prototype or `null`.

### Return value

The `setPrototypeOf()` method must return a `Boolean` indicating whether or not the prototype was successfully changed. Other values are coerced to booleans.

Many operations, including `Object.setPrototypeOf()`, throw a `TypeError` if the `[[SetPrototypeOf]]` internal method returns `false`.

## Description

### Interceptions

This trap can intercept these operations:

- `Object.setPrototypeOf()`
- `Reflect.setPrototypeOf()`

Or any other operation that invokes the `[[SetPrototypeOf]]` internal method.

### Invariants

The proxy's `[[SetPrototypeOf]]` internal method throws a `TypeError` if the handler definition violates one of the following invariants:

- If the target object is not extensible, the prototype cannot be changed. That is, if `Reflect.isExtensible()` returns `false` on `target`, and `prototype` is not the same as the result of `Reflect.getPrototypeOf(target)`, then the trap must return a falsy value.

## Examples

If you want to disallow setting a new prototype for your object, your handler's `setPrototypeOf()` method can either return `false`, or it can throw an exception.

### Approach 1: Returning false

This approach means that any mutating operation that throws an exception on failure to mutate, must create the exception itself.

For example, `Object.setPrototypeOf()` will create and throw a `TypeError` itself. If the mutation is performed by an operation that *doesn't* ordinarily throw in case of failure, such as `Reflect.setPrototypeOf()`, no exception will be thrown.

```js
const handlerReturnsFalse = {
  setPrototypeOf(target, newProto) {
    return false;
  },
};

const newProto = {},
  target = {};

const p1 = new Proxy(target, handlerReturnsFalse);
Object.setPrototypeOf(p1, newProto); // throws a TypeError
Reflect.setPrototypeOf(p1, newProto); // returns false
```

### Approach 2: Throwing an Exception

The latter approach will cause *any* operation that attempts to mutate, to throw. This approach is best if you want even non-throwing operations to throw on failure, or you want to throw a custom exception value.

```js
const handlerThrows = {
  setPrototypeOf(target, newProto) {
    throw new Error("custom error");
  },
};

const newProto = {},
  target = {};

const p2 = new Proxy(target, handlerThrows);
Object.setPrototypeOf(p2, newProto); // throws new Error("custom error")
Reflect.setPrototypeOf(p2, newProto); // throws new Error("custom error")
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>https://tc39.es/ecma262/multipage/ordinary-and-exotic-objects-behaviours.html#sec-proxy-object-internal-methods-and-internal-slots-setprototypeof-v |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| setPrototypeOf | 49 | 12 | 49 | 36 | 10 | 49 | 49 | 36 | 10 | 5.0 | 49 | 10 | 1.0.0 | 1.0 | 6.0.0 |

*Compatibility data: MDN browser-compat-data 8.1.8,* `javascript.builtins.Proxy.handler.setPrototypeOf`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- `Proxy`
- `Proxy()` constructor
- `Object.setPrototypeOf()`
- `Reflect.setPrototypeOf()`

---

JavaScript › Reference › Standard built-in objects › Proxy

# Proxy.revocable()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Proxy/revocable · MDN page last modified 2025-07-10

**Baseline: Widely available** — available across major browsers since September 2016.

The **`Proxy.revocable()`** static method creates a revocable `Proxy` object.

## Syntax

```
Proxy.revocable(target, handler)
```

### Parameters

**`target`**

A target object to wrap with `Proxy`. It can be any sort of object, including a native array, a function, or even another proxy.

**`handler`**

An object whose properties are functions defining the behavior of `proxy` when an operation is performed on it.

### Return value

A plain object with the following two properties:

**`proxy`**

A Proxy object exactly the same as one created with a `new Proxy(target, handler)` call.

**`revoke`**

A function with no parameters to revoke (switch off) the `proxy`.

## Description

The `Proxy.revocable()` factory function is the same as the `Proxy()` constructor, except that in addition to creating a proxy object, it also creates a `revoke` function that can be called to disable the proxy. The proxy object and the `revoke` function are wrapped in a plain object.

The `revoke` function does not take any parameters, nor does it rely on the `this` value. The created `proxy` object is attached to the `revoke` function as a private field that the `revoke` function accesses on itself when called (the existence of the private field is not observable from the outside, but it has implications on how garbage collection happens). The `proxy` object is *not* captured within the closure of the `revoke` function (which will make garbage collection of `proxy` impossible if `revoke` is still alive).

After the `revoke()` function gets called, the proxy becomes unusable: any trap to a handler throws a `TypeError`. Once a proxy is revoked, it remains revoked, and calling `revoke()` again has no effect — in fact, the call to `revoke()` detaches the `proxy` object from the `revoke` function, so the `revoke` function will not be able to access the proxy again at all. If the proxy is not referenced elsewhere, it will then be eligible for garbage collection. The `revoke` function also detaches `target` and `handler` from the `proxy`, so if `target` is not referenced elsewhere, it will also be eligible for garbage collection, even when its proxy is still alive, since there's no longer a way to meaningfully interact with the target object.

Letting users interact with an object through a revocable proxy allows you to control the lifetime of the object exposed to the user — you can make the object garbage-collectable even when the user is still holding a reference to its proxy.

## Examples

### Using Proxy.revocable()

```js
const revocable = Proxy.revocable(
  {},
  {
    get(target, name) {
      return `[[${name}]]`;
    },
  },
);
const proxy = revocable.proxy;
console.log(proxy.foo); // "[[foo]]"

revocable.revoke();

console.log(proxy.foo); // TypeError is thrown
proxy.foo = 1; // TypeError again
delete proxy.foo; // still TypeError
typeof proxy; // "object", typeof doesn't trigger any trap
```

## Specifications

| Specification |
| --- |
| ECMAScript® 2027 Language Specification<br>`https://tc39.es/ecma262/multipage/reflection.html#sec-proxy.revocable` |

## Browser compatibility

| Feature | Desktop | | | | | Mobile | | | | | | | Server | | |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| | Chrome | Edge | Firefox | Opera | Safari | Chrome Android | Firefox for Android | Opera Android | Safari on iOS | Samsung Browser | WebView Android | WebView on iOS | Bun | Deno | Node.js |
| `revocable` | 49 | 12 | 34 | 36 | 10 | 49 | 34 | 36 | 10 | 5.0 | 49 | 10 | 1.0.0 | 1.0 | 6.0.0 |

*Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Proxy.revocable`. Numbers are the first version with support; see "How to read this book" for the full legend.*

## See also

- Proxy